

PrimeSim™ Design Robustness User Guide

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

New in This Release	13
Related Products, Publications, and Trademarks	13
Conventions	14
Customer Support	14
Statement on Inclusivity and Diversity	15

1. Introducing Design Robustness	16
Overview of Design Robustness	16
Design Robustness Profiler	17
Design Robustness Analyzer	18
Stress Test Analysis	20
High Capacity Variation Analysis	21
HCVA Limitations	21
Advanced Variation Analysis	21
Setting Up the Path to the Executable	22
Setting Up the Configuration File	22
Example 1: Design Robustness Profiler	23
Example 2: Design Robustness Analyzer	24
Example 3: Stress Test Analysis	25
Example 4: Advanced Variation Analysis	25

2. Running Design Robustness	27
Running Design Robustness Simulation	27
Command-Line Options	27
Examples	28
Generating Graphs in DRP Using a Database File	29
DRP Graph Command-Line Options	29
Examples	30
Running High Capacity Variation Analysis	30
HCVA Command-Line Options	31
Examples	31

Using PrimeSim AVA Across Simulators	32
Invoking PrimeSim AVA With PrimeSim HSPICE	32
Invoking PrimeSim AVA With PrimeSim SPICE	32
Invoking PrimeSim AVA With PrimeSim XA	33

3. Output Files	34
Design Robustness Profiler Output Files	34
Histogram	35
Heatmap	36
Design Robustness Analyzer Output File	37
Sensitivity Report File	38
Stress Test Analysis Output Files	39
High Capacity Variation Analysis Output Files	40
HSMC Outputs	40
The *.ava.report File	40
More on the Confidence Interval	41
Sensitivity Table	42
Sensitivity Table FAQ	44
Confidence Interval and Sample Size	46

4. Configuration File Command Reference	47
Command Hierarchy	47
Common Settings	47
Transient Simulation Setup	48
Design Robustness Setup	48
Design Robustness Common Settings	48
Design Robustness Profiler Settings	48
Design Robustness Analyzer Settings	49
Stress Test Analysis Settings	50
Advanced Variation Analysis Settings	51

5. Defining Advanced Variation Analysis Across Simulators and Application ...	52
Introduction to Advanced Variation Analysis	52
Invoking PrimeSim Design Robustness With PrimeSim HSPICE	53
Use Cases for Invoking AVA	53
AVA Simulation Without .json File	53

AVA Using Dialog Box	54
AVA Simulation With .json File	55
AVA Without Using Dialog Box	55
Using Basic and Advanced Options	56
6. Monte Carlo Analysis	58
Introduction to Monte Carlo Analysis	58
Invoking Monte Carlo From PrimeSim AVA	59
Inputs for Monte Carlo	59
Intermediate files	59
Outputs for Monte Carlo	60
Configuring Inputs for MC	62
Creating a Configuration File	63
Displaying Help for the Settings	64
Configuring Basic Settings	65
Configuring Advanced Settings	68
Using General Tab	70
Saving the Configuration	75
Monte Carlo Outputs	76
Nominal Simulation Files	76
Q-Q Plots in PDF, WaveView, and CSV Formats	76
PDF Q-Q Plots	77
WaveView Q-Q Plots	78
CSV Q-Q File	79
The *.ava.report File	80
*.ava.details.json File	81
*.ava.dsvc.json File	81
*.ava.critical File	83
Critical Points	83
*.ava.hm File	85
out.ava.sens.csv	86
out.ava.irv_corr.csv	87
Netlist Examples	88
Understanding Fast Sensitivity	89
Parallel Processing	89
Distributed Processing	89
Postprocessing	90

Introduction to the Postprocessing Utility	91
Invoking Postprocessing Utility	91
Inputs for the Postprocessing Utility	91
Outputs for Postprocessing Utility	92
Configuring Inputs for Postprocessing	93
Configuring Partial Measure for Postprocessing	97
7. High Sigma Monte Carlo and Parametric Yield Estimation	100
Introduction to HSMC	101
Principles of HSMC	102
HSMC — Target Applications	104
Invoking HSMC	105
Configuring Inputs For HSMC	106
Creating a Configuration File for HSMC	107
Configuring Basic Settings For HSMC	107
Single Tail Measure Support	108
Parallel Processing for HSMC	113
Automated Validation of HSMC Results	114
Example	115
HSMC and SRS Monte Carlo — Performance Comparisons	116
Speedup as a Function of Sigma	117
HSMC Recommendations and Limitations	117
HSMC Frequently-Asked Questions	118
Parametric Yield Estimation	120
Introduction to Parametric Yield Estimation	121
Parametric Yield Estimation in AVA	123
Running Parametric Yield Simulation	124
Invoking Parametric Yield Estimation	125
PYE Output	126
10-Sigma Estimation	127
Introduction to 10-Sigma Estimation	127
Invoking 10 Sigma Estimation	128
Configuring Inputs for 10-Sigma Estimation	128
Understanding Outputs	129

8. Multi Corner Analysis	131
Introduction to Multi Corner Analysis	131
Running Multi Corner Analysis	132
Inputs for Multi Corner Analysis	133
Typical Corner File	133
Column Properties	135
Validating the Input corners.csv File	137
Generating corners.csv File	138
DUMMY Values for Manual Edits	139
User provided Netlist	141
Multiple meas_spec.csv file	142
Outputs for Multi Corner Analysis Simulation	143
Output Files	144
<out>.ava.summary.csv	144
<out>.ava.summary.html	144
Supported Statistical Outputs in the CSV Summary	148
Combined QQ Plot (PDF)	150
wv_plt Group File (Interactive)	151
Multiple Sigma HSMC	152
Multi Corner Distributed Processing	153
Dynamic Task Monitoring and Control – CDPL DP Manager	154
Worker Log Files	154
Advanced Features	155
Incremental Simulation	156
Fault Tolerance	157
Terminate and Suspend	158

9. Sigma Amplification	159
Principles of Sigma Amplification	159
Sigma Amplification — Target Applications	160
Running Sigma Amplification	161
Parallel Processing for Sigma Amplification	162
Configuring Inputs for Sigma Amplification	163
Determining the Amplification Factor for Sigma Amplification	164

Sigma Amplification Outputs	164
The *.ava.report File for Sigma Amplification	165
Sigma Amplification Recommendations	166
Application Example for Sigma Amplification	166
Example	167
Block Specific Sigma Amplification(BSSA)	168
Introduction to Block Specific Sigma Amplification	169
Invoking Block Specific Sigma Amplification	169
Configuring Inputs for Block Specific Sigma Amplification	171
Block Specific Sigma Amplification Outputs	174
.mt# File	174
.ava.report File	174
.ava.critical File	174
Sigma Amplification FAQ	175
10. Design Specific Variation Corners Export/Import	177
Principles of Design Specific Variation Corners Export/Import	177
DSVC Export	179
Running DSVC Export Using High Sigma Monte Carlo	180
Configuring Inputs for DSVC Export	180
Outputs	182
DSVC Import	182
Running DSVC Import	182
Configuring Inputs for DSVC Import	184
Outputs	187
11. Advanced Design Exploration With Design Insights	188
Overview of Design Insights	188
Setting up Design Insights	189
Launching Design Insights	190
Creating diPhysical View	192
Device Bias Voltage Definition File	197
Exporting DSPF by Selected Nets	198
Exploring diPhysical view	199
Analyzing Parasitic	203

Fanout Analysis	209
Understanding Resistance Analysis Results	213
Understanding Elmore Delay Analysis Results	220
Modes of Analysis in Design Insights	223
Histograms for Net Details and Analysis Result	225
Cross Highlighting	227
12. Appendix	232
Probability and Statistical Concepts	232
Probability Density Function	234
Area Under the PDF	235
Cumulative Distribution Function	235
Statistical Measures	236
Statistical Measures: Standard Normal Distribution	237
Statistical Measures: Log Normal Distribution	238
Transformations on the CDF	238
Comparing Distributions	240
From Probabilities to Ordinates	242
Probability and Sigma	243
System and Component Reliability	244
Sampling and Monte Carlo Simulation	246
Q-Q Plot - Confidence Intervals	249
Understanding the PrimeSim AVA Q-Q Plots	251
Sigma, Sample Size, and Statistical Uncertainty	253
One- or Two-Sided Specifications	258
JSON Format and Examples	258
What is JSON?	259
JSON Examples for MC	259
Sample Size Mode Using AVA	259
Sigma/Uncertainty Mode Using AVA	259
JSON Examples for HSMC and Sigma Amplification	260
JSON for HSMC Basic Run	260
JSON for HSMC With Measure Specification, Do_Not_Vary Control, and AVA Output Settings	260
JSON for Left, Right, and Both Measure Tail Selection	261
JSON to Update Alert Time to 60 Minutes	261
JSON for HSMC When Reference Results are Specified Manually	262
JSON for Sigma Amplification With AVA Output Settings	262
JSON for PYE - Left, Right, and Both Measure Tail Selection	262

Contents

JSON for Exporting DSVCs While Running High Sigma Monte Carlo	263
JSON for Importing DSVCs	263
JSON for 10-sigma quantile using EVD in HSMC	264
JSON for Block Specific Sigma Amplification	264
JSON Entries	265
JSON Files	265
Application Related JSON Entries	267
Output Format Entries	268
Other Entries	269
</> Marks in AVA Yield Output Reports	270
Command Reference (Alphabetical)	274
active_device_flow	276
check_tol_sign	276
cck_soa_val_ib	277
cck_soa_val_id	277
cck_soa_val_ig	277
cck_soa_val_is	278
database	278
delta_ids_file	278
delta_ids_pct	279
delta_vth_file	280
delta_vth_val	281
device	281
device_model	282
disable_gate_cap	283
dp_config	284
dp_num	286
dr_setup	286
dr_tag	287
dra	287
drp	288
drp_stage	289
enable	289
enable_lpe	290
exclude_device_filter	290
ids_scale	291
idsat_scale	291
include_device_filter	291

Contents

ioff_scale	292
load_lpe_file	293
mc_index	294
mc_sweep	294
meas	295
meas_list_file	296
outdir	298
outprefix	298
search_method	299
set_current_tol_abs	300
set_current_tol_rel	300
set_hier_delimiter	301
set_timing_tol_abs	301
set_timing_tol_rel	302
set_voltage_tol_abs	302
set_voltage_tol_rel	303
sim_cmdline	303
sim_setup	304
stage	304
stress_test	305
temperature	306
top_region	307
meas_type	307
type	307
variation_feature	308

13. References	309
AVA Customer Presentations	309

14. AVA Revision History	311
AVA Change History	311
AVA Output File Extension Updates	311

15. Design Insights Command Reference	313
di::closeResultsCellView	314
di::createResultsCellView	314

Contents

di::createParasiticView	315
di::getCapacitors	316
di::getCapAttr	316
di::getCorners	317
di::getElmoreDelayResult	317
di::getElmoreDelayResultAttr	318
di::getFanoutResult	318
di::getFanoutResultAttr	319
di::getInstAttr	319
di::getInsts	320
di::getLayers	321
di::getModels	321
di::getNetAttr	321
di::getNets	322
di::getNodeAttr	322
di::getResAttr	323
di::getResistanceContributionAttr	324
di::getResistanceResult	324
di::getResistanceResultAttr	325
di::getResistors	325
di::getTermAttr	326
di::getTerms	326
di::getViewType	327
di::listAttrs	327
di::loadResultsCellView	327
di::runElmoreDelayAnalysis	328
di::runFanoutAnalysis	328
di::runResistanceAnalysis	329
Preferences	329

About the PrimeSim Design Robustness User Guide

The *PrimeSim Design Robustness User Guide* describes how to use the Synopsys® PrimeSim Design Robustness tool.

This preface includes the following sections:

- [New in This Release](#)
- [Related Products, Publications, and Trademarks](#)
- [Conventions](#)
- [Customer Support](#)
- [Statement on Inclusivity and Diversity](#)

New in This Release

Information about new features, enhancements, and changes, known limitations, and resolved Synopsys Technical Action Requests (STARs) is available in the *PrimeSim Reliability Analysis Release Notes* on the SolvNetPlus site.

Related Products, Publications, and Trademarks

For additional information about the PrimeSim Design Robustness tool, see the documentation on the Synopsys SolvNetPlus support site at the following address:

<https://solvnetplus.synopsys.com>

You might also want to see the documentation for the following related Synopsys products:

- PrimeWave™ Design Environment
- PrimeSim™ Reliability Analysis

Note:

You must be logged in to SolvNetPlus for the preceding link to connect directly to the article. If you are prompted to log in to SolvNetPlus upon clicking the link to the article, log in, then click the link again to reach the article.

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
Courier	Indicates syntax, such as <code>write_file</code> .
<i>Courier italic</i>	Indicates a user-defined value in syntax, such as <code>write_file design_list</code>
Courier bold	Indicates user input—text you type verbatim—in examples, such as <code>prompt> write_file top</code>
Purple	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as <code>include_enclosing = true false</code>
[]	Denotes optional arguments in syntax, such as <code>write_file [-format fmt]</code>
...	Indicates that arguments can be repeated as many times as needed, such as <code>pin1 pin2 ... pinN</code> .
	Indicates a choice among alternatives, such as <code>low medium high</code>
\	Indicates a continuation of a command line.
/	Indicates levels of directory structure.
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.
Edit > Copy	Indicates a path to a menu command, such as opening the Edit menu and choosing Copy .
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

<https://solvnetplus.synopsys.com>

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to <https://solvnetplus.synopsys.com>.

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Introducing Design Robustness

This chapter introduces the concept of design robustness and the Design Robustness tool.

- [Overview of Design Robustness](#)
- [Setting Up the Path to the Executable](#)
- [Setting Up the Configuration File](#)

Overview of Design Robustness

Variability issues are escalating in advanced technology nodes and impacting the robustness of the design and its Power-Performance-Area(PPA) in both digital and Analog Mixed Signal(AMS) designs. There are stringent requirements for variation robustness across variety of applications hence there is a significant increase in number of corners from one technology node to another. Therefore, ensuring robustness across multiple Process, Voltage and Temperature. Post layout verification becomes significant challenge due to increasing design sizes, parasitics and corners. Therefore, designer often resort to margin which leads to over-design impact Power-Performance and Area (PPA).

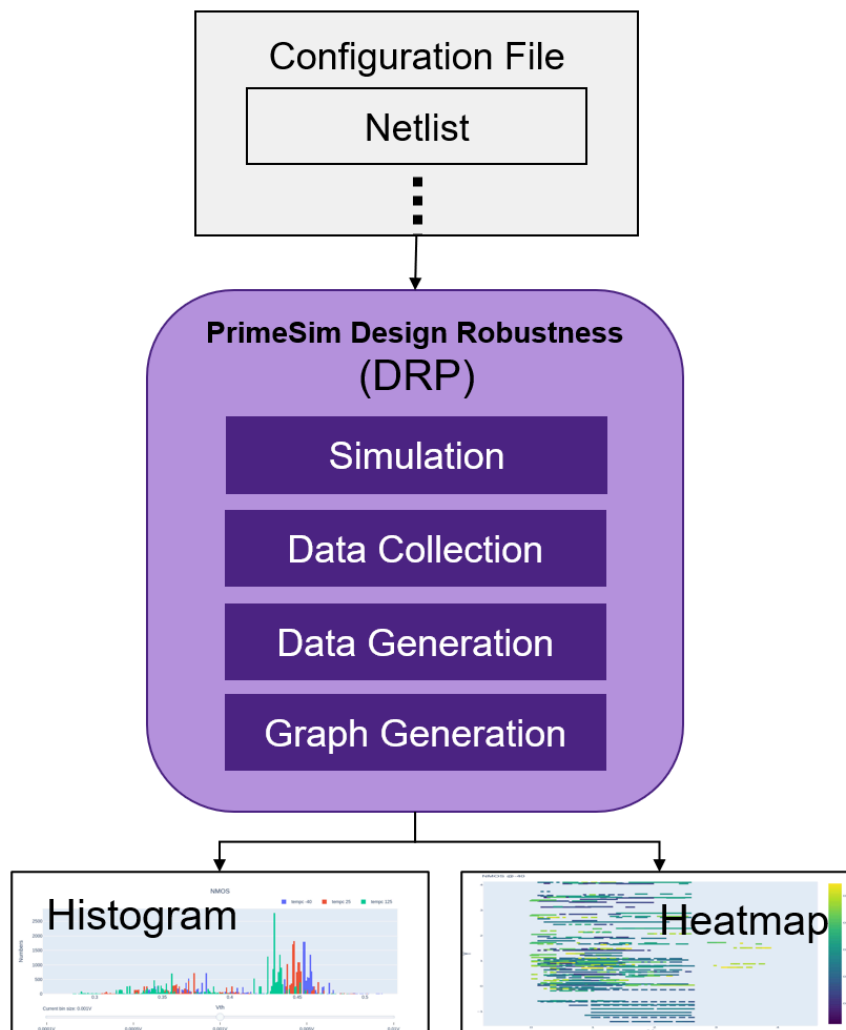
Increasing instance count for core building blocks like standard cells, memory bitcells, and so on increases the probability of failure due to chip-to-chip and on-chip variations. High Sigma validation is necessary on these building blocks to ensure proper functionality of these building blocks with acceptable variation, thereby enabling robust design. For large SRAM, Flash and other AMS designs it is also essential to identify failures and root-cause the same. Traditionally designer relies on expensive Monte Carlo simulations with Spice simulators for variability analysis, PrimeSim Design Robustness provides the ability to perform fast variation profiling, robustness analysis and High-Capacity-Variation Analysis for memories and large AMS designs. The High Sigma MC technology in PrimeSim Design Robustness is also a key enabler for variation library (LVF) generation with PrimeLib. PrimeSim Design Robustness works seamlessly with PrimeSim simulators to offer comprehensive solution for design robustness.

Design Robustness Profiler

The Design Robustness Profiler (DRP) application profiles the following device model data and generates a set of data in graphic form for analysis (see [Figure 1](#)):

- V_{th} – Gate voltage threshold
- I_{dsat} – Drain-Source saturation current
- I_{off} – Drain-Source off-state current
- $R_{ds(on)}$ – Drain-Source on-state resistance
- $Gate_Cap$ – Capacitance on the gate (Default is disable)

Figure 1 Overview of Design Robustness Profiler (DRP)



Note:

For Design Robustness Profiler, the parent performs post processing tasks and all the profiling task is completed by the workers. The default worker number is 1.

Design Robustness Analyzer

The Design Robustness Analyzer (DRA) application uses theory-guided machine learning data to identify sensitive devices in the circuit (see [Figure 2](#)).

Sensitive devices may reduce design margins or cause functional failure. Identifying and optimizing the sensitive devices can help to improve power consumption and area without impacting design margins.

Figure 2 Overview of Design Robustness Analyzer (DRA)

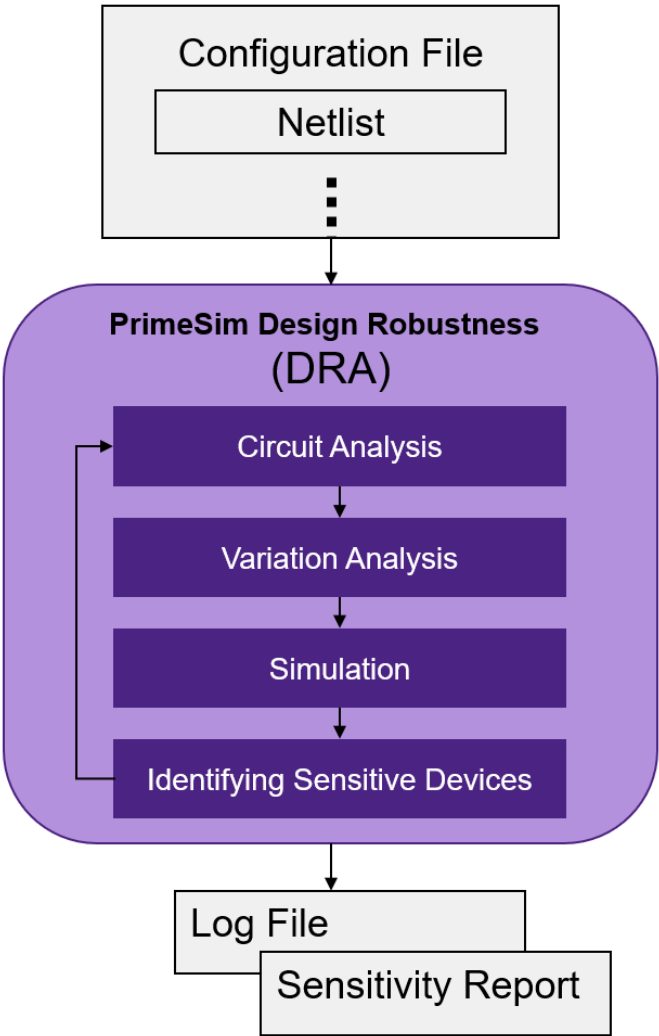


Table 1 shows the list of supported models to run the DRA application:

Table 1 Supported Models for DRA

Model Name	HSPICE Model Level	Eldo Model Level
MOS9	50	59
BSIM3v3	53	53
BSIM4	54	60
PSP	69	75

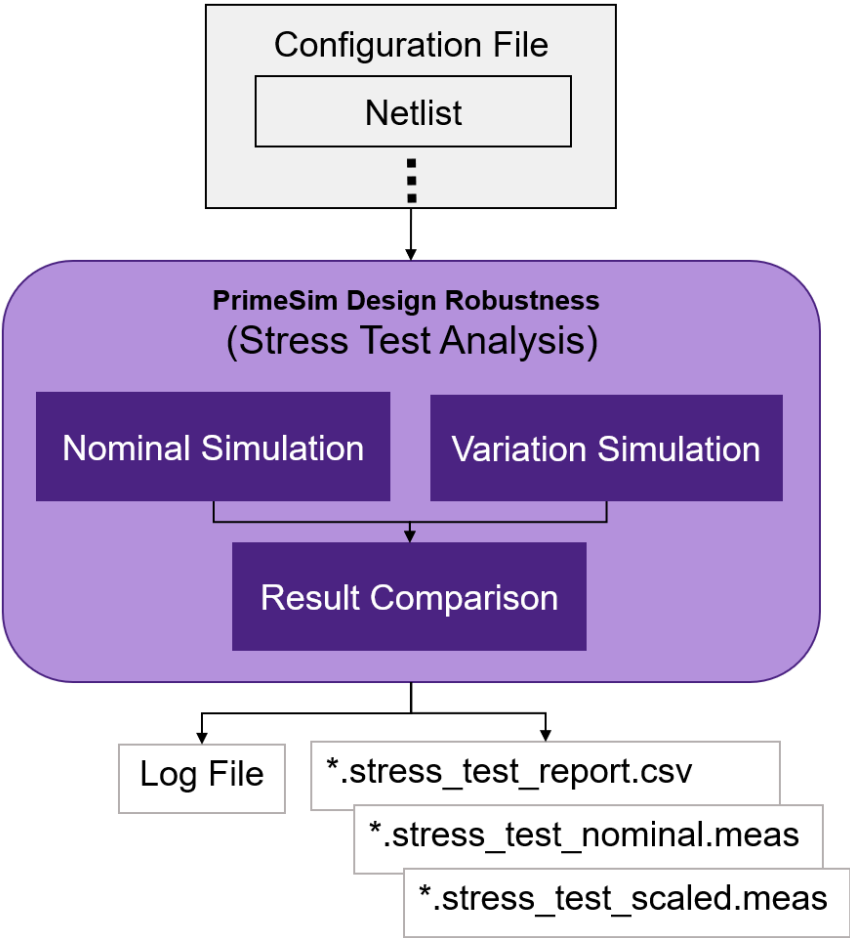
Table 1 Supported Models for DRA (Continued)

Model Name	HSPICE Model Level	Eldo Model Level
BSIM-CMG	72	77
LETI-UTSOI	76	80/84
BSIM-IMG	78	81

Stress Test Analysis

The stress test analysis enables you to perform what-if analysis to run a nominal simulation and variation simulation to compare the measurement results (see [Figure 3](#)).

Figure 3 Overview of Stress Test Analysis



High Capacity Variation Analysis

Monte Carlo analysis is a common method used to understand the effect of random process variations on circuit performance. The challenge of the traditional Monte Carlo method has been on slow performance and limited capacity. With circuit sizes getting larger through advancing technologies, traditional Monte Carlo does not scale well to the largest designs. With the higher sigma requirement and tighter margin criteria, traditional Monte Carlo does not meet the sign-off requirement.

The Design Robustness tool supports High Capacity Variation Analysis(HCVA) analysis. It uses theory-guided machine learning technology under the Design Robustness tool to improve runtime and memory footprint to achieve same distribution result as the traditional Monte Carlo analysis. The usage of HCVA is fully compatible with the traditional Monte Carlo analysis supported with SRAM command for SRAM design.

HCVA Limitations

The following features are not currently supported with HCVA:

- Non-SRAM design
- Post-layout back-annotation flow
- External sampling
- Data sweep (`.ALTER`, data sweep, bisection)
- Variation Block
- Aging
- Netlist command base block specific sigma amplification
- Cosimulation
- PrimeSim AVA Monte Carlo (HSMC, BSSA, and DSVC)
- Eldo and other 3rd-party Monte Carlo

Advanced Variation Analysis

The usage model for invocation of AVA config through DR JSON is as follows:

```
primesim_dr -json test.json
```

The following is an example of the JSON file:

```
cat test.json
{ "sim_setup" :
  { "sim_cmd" : "hspice -i input.spi"
```

```
    },
    "dr_setup" : [
    {
        "dr_tag" : "default",
        "stage" : "ava",
        "ava" :
        { "ava_cfg_file": "ava.json"
        }
    }
    ]
}
```

Setting Up the Path to the Executable

The executable name for the Design Robustness tool is `primesim_dr`. The Design Robustness executable is called by the launch script path. The launch script is in the `PrimeSim-DR_INSTALL_DIR/bin` directory. You must set up the shell path to include the path to the Design Robustness launch script using the following syntax:

C-Shell:

```
set path = (PrimeSim-DR_INSTALL_DIR/bin $path)
```

Example

```
set path = (/tools/synopsys/primesim-dr-2022.06/bin $path)
```

Bourne, Korn, or Bash Shells:

```
export PATH=PrimeSim-DR_INSTALL_DIR/bin:$path
```

Example

```
export PATH=/tools/synopsys/primesim-dr-2022.06/bin:$path
```

The Design Robustness tool calls the PrimeSim XA simulator for running transient simulation. Follow the instructions in the PrimeSim XA documentation to set up the path to the executable.

Setting Up the Configuration File

To run the Design Robustness tool, you must create a configuration file in JavaScript Object Notation (JSON) format. The configuration file contains a list of commands to run Design Robustness simulation.

For a details on the supported commands and subcommands, see [Configuration File Command Reference](#).

The following are the basic JSON syntax rules:

- Commands are represented in name/value pairs, separated by a comma.
- A name/value pair consists of a field name (in double quotation marks), followed by a colon, followed by a value.
- A value must be one of the following data types: a string, a number, an object, an array, a Boolean (true or false), or null.
- Curly braces ({ }) hold objects, and each name is followed by colon.
- Square brackets ([]) hold arrays.

The following shows the syntax structure of the configuration file:

```
{  
    command1,  
    command2,  
    ...  
    commandn  
}
```

You can add comments in the configuration file. To add single line comments, use double slash characters (//). Any text written to the right side of the double slash is considered a comment and is ignored during a simulation.

You can add multiline comments by placing /* at the beginning of the comments. Use the characters */ to end the multiline comments.

The following sections show examples of Design Robustness configuration files:

- [Example 1: Design Robustness Profiler](#)
- [Example 2: Design Robustness Analyzer](#)
- [Example 3: Stress Test Analysis](#)
- [Example 4: Advanced Variation Analysis](#)

Example 1: Design Robustness Profiler

The following is an example of the configuration file to run the Design Robustness Profiler (DRP) application using the Design Robustness tool:

```
{  
    "dp_config" : "xhost/xhost_sh_launch",  
    "dp_num"    : 8,  
  
    "sim_setup" :  
    {  
        "sim_cmd" : "xa -mt 2 -hspice main.in"    }  
}
```

```

    },
    "dr_setup" : [
    {
        "dr_tag" : "run_drp_lpe",
        "stage" : "drp",
        "drp" :
        {
            "drp_stage" : "full",
            "enable_lpe" : true,
            "temperature" : [-40, 25, 125]
        }
    } ]
}

```

Example 2: Design Robustness Analyzer

The following is an example of the configuration file to run the Design Robustness Analyzer (DRA) application using the Design Robustness tool:

```

{
    "dp_config" : "xhost/xhost_sh_launch",
    "dp_num" : 8,

    "sim_setup" :
    {
        "sim_cmd" : "xa -mt 2 -hspice main.in"
    },

    "dr_setup" : [
    {
        "dr_tag" : "run_dra_region",
        "stage" : "dra",
        "dra" :
        {
            "variation_feature" : "all",
            "delta_vth_val" : 0.13,
            "delta_ids_pct" : -40
        },
        "meas" :
        {
            "meas_list_file" : "key_meas_file"
        },
        "device" :
        {
            "type" : "all"
        },
        "active_device_flow" : [
        {
            "cck_soa_val_id" : 1e-6,
            "cck_soa_val_ig" : 1e-6,
            "cck_soa_val_is" : 1e-6,

```

```
        "cck_soa_val_ib" : 1e-6
    } ]
} ]
}
```

Example 3: Stress Test Analysis

The following is an example of the configuration file to run a stress test analysis using the Design Robustness tool:

```
{
  "dp_config" : "xhost/xhost_sh_launch",
  "dp_num"    : 1,

  "sim_setup" :
  {
    "sim_cmd" : "xa -mt 2 -hspice main.in"
  },

  "dr_setup" : [
  {
    "dr_tag" : "run_stress_test",
    "stage"  : "stress_test",
    "stress_test" :
    [ {
      "ids_scale" : 0.9,
      "device_model" : ["slvtpfet"]
    },
    {
      "idsat_scale" : 0.9,
      "device_model" : ["slvtnfet"]
    } ]
  } ]
}
```

Example 4: Advanced Variation Analysis

The following is an example of the configuration file to run an Advanced Variation Analysis using the Design Robustness tool:

```
{
  "dp_config" : "xhost/xhost_sh_launch", "dp_num" : 1,
  "sim_setup" :
  {
    "sim_cmd" : "xa -mt 2 -hspice main.in"
  },
  "dr_setup" : [
  {
    "dr_tag" : "run_ava ",
    "stage"  : "ava",

```

```
"ava" :  
{  
  "ava_cfg_file": "file_path"  
}  
] ]  
}
```

See Also

- [JSON Format and Examples](#)

2

Running Design Robustness

This chapter outlines how to run the Design Robustness tool.

- [Running Design Robustness Simulation](#)
- [Generating Graphs in DRP Using a Database File](#)
- [Running High Capacity Variation Analysis](#)
- [Using PrimeSim AVA Across Simulators](#)

Running Design Robustness Simulation

Run the Design Robustness tool with the following command at the UNIX command prompt:

```
$> primesim_dr -json dr_config_file  
[-t tagName]  
[-o outdir/outprefix]  
[-version|-v]  
[-help|-h]
```

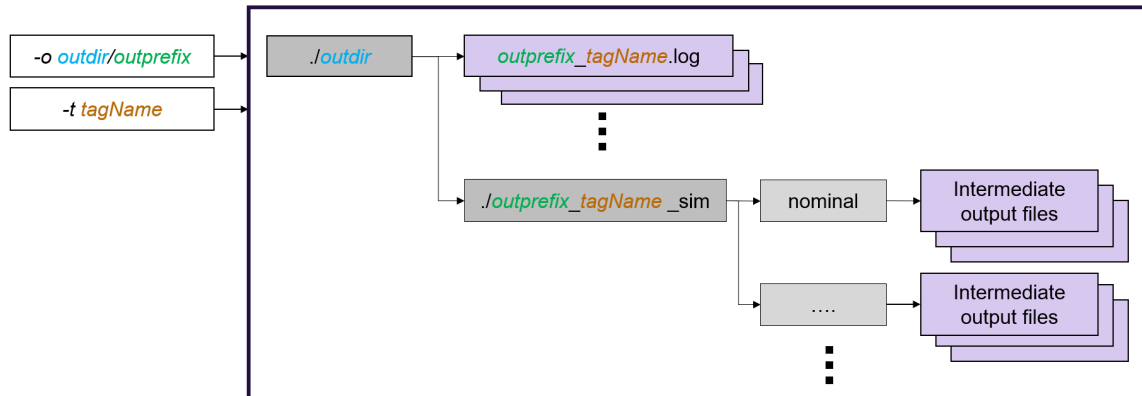
Command-Line Options

The following table describes the Design Robustness simulation command-line options:

Command-Line Option	Description
<code>-json dr_config_file</code>	Specifies the name of the Design Robustness configuration file that contains a list of commands to execute simulation. The configuration file is in JSON format. This is a mandatory command line option.
<code>-t tagName</code>	Specifies the tag name of the simulation. If this option is not specified, it defaults to the tag name, <code>default</code> . It selects the group of commands under the specified tag name in the configuration file for simulation. <code>tagName</code> is also used as part of the output file name. See Figure 4 for details.

Command-Line Option	Description
<code>-o outdir/outprefix</code>	Specifies the name of the output directory and prefix output file name. The final output file name is <code>outprefix_tagName</code> . See Figure 4 for details. <code>outdir</code> is the name of the output directory and it overrides the name specified in the <code>outdir</code> command in the configuration file. <code>outprefix</code> is the prefix name of the output file and the intermediate simulation output directory. It overrides the name specified in the <code>outprefix</code> command in the configuration file. If the names are not specified, the default name for <code>outdir</code> is the name of current working directory and the default name for <code>outprefix</code> is <code>dr</code> .
<code>-version -v</code>	Prints the version information.
<code>-help -h</code>	Prints the usage information.

Figure 4 Simulation Output Structure



Examples

Example 1

If the `dr_cfg.json` file does not contain the `outdir` and `outprefix` commands, the following command-line invocation generates all output files with name `dr_default*` under the current working directory:

```
$> primesim_dr -json dr_cfg.json
```


Example 2

The following command-line invocation generates all output files with the tag name `my_vth_50mv*` under the `dr_root_tmp` output directory:

```
$> primesim_dr -json dr_cfg.json -t vth_50mv -o dr_root_tmp/my
```

Generating Graphs in DRP Using a Database File

When running the Design Robustness Profiler (DRP) application, a database file is generated. The database file can be used to generate a histogram and heatmap for analysis.

Run the Design Robustness tool with the following command to generate DRP outputs:

```
$> primesim_dr -drp drp_option  
-i database_filename1 [... database_filenameN]  
[-o new_database_filename]  
[-comp_different_temp]
```

DRP Graph Command-Line Options

The following table describes the Design Robustness command-line options to generate DRP graphs:

Command-Line Option	Description
<code>-drp drp_option</code>	Specifies the DRP application to execute. It supports the following applications: <code>histogram</code>: Generates a histogram graph. <code>heatmap</code>: Generates a heatmap graph. <code>comparison</code>: Compares database files and merges them into a new database file. By default, it only compares and merges runs of the same temperature. If merging runs with different temperatures, you must specify the <code>-comp_different_temp</code> option.
<code>-i database_filename1 [... database_filenameN]</code>	Specifies the name of the database files. When <code>-drp comparison</code> is set, you must also specify more than one database file.
<code>-o new_database_filename</code>	Specifies the name of the new database file when <code>-drp comparison</code> is set.
<code>-comp_different_temp</code>	Enables merging of many database files with different temperatures. This option is mandatory if the database files were run with different temperatures.

Examples

Example 1

The following command-line invocation enables the DRP application to load the `my_database.db` database file to generate a histogram graph:

```
$> primesim_dr -drp histogram -i my_database.db
```

See [Histogram](#) for more information and an example of the output.

Example 2

The following command-line invocation enables the DRP application to load the `my_database.db` database file to generate a heatmap graph:

```
$> primesim_dr -drp heatmap -i my_database.db
```

See [Heatmap](#) for more information and an example of the output.

Example 3

The following command-line invocation loads the `dr_root_tmp/my_db_25c.db` and `dr_root_tmp/my_db_125c.db` database files for comparison and merges them into a new database file, `my_out_db.db`. Because the two original database files were run with different temperatures, you must also specify the `-comp_different_tempc` option to compare and merge the database files:

```
$> primesim_dr -drp comparison -i dr_root_tmp/my_db_25c.db  
dr_root_tmp/my_db_125c.db -o my_out_db.db -comp_different_tempc
```

The following command-line invocation loads in the new `my_out_db.db` database file from the previous example to generate the histogram graph:

```
$> primesim_dr -drp histogram -i my_out_db.db
```

See [Histogram](#) for more information and an example of the output.

Running High Capacity Variation Analysis

The Design Robustness tool uses the theory-guided machine learning to set up the High Capacity Variation Analysis (HCVA) and calls the PrimeSim XA tool to run transient simulation. You must have the PrimeSim Design Robustness tool and the PrimeSim XA tool to run HCVA analysis.

HCVA supports all the commands and features of Monte Carlo analysis with the PrimeSim XA tool. See the *PrimeSim™ XA User Guide* for the supported options, commands, and features. You must follow the PrimeSim XA tool to set up Monte Carlo analysis.

You can enable HCVA in the Design Robustness tool with the following command-line syntax:

```
$> primesim_dr -flow HCVA
-simulator XA "simulator_command_line"
-dp num_dp -dpconfig dp_config_file
[-o outdir/outprefix]
```

HCVA Command-Line Options

The following table describes the Design Robustness command-line options to run HCVA:

Command-Line Option	Description
-flow HCVA	Enable HCVA analysis.
-simulator XA	Specifies the name of the transient simulator for Monte Carlo simulation. Only XA is supported.
"simulator_command_line"	Specifies the command line to run the PrimeSim XA tool for Monte Carlo simulation.
-dp num_dp	Sets the maximum number of parallel simulations are run. The default is 1.
-dpconfig dp_config_file	Specifies the hosts information file for distributed processing capability. If this is not specified, it runs on a local machine.
-o outdir/outprefix	Specifies the name of the output directory and prefix output file name. <i>outdir</i> is the name of the output directory. <i>outprefix</i> is the prefix name of the output files. If the names are not specified, the default name for <i>outdir</i> is the name of current working directory and the default name for <i>outprefix</i> is xa.

Examples

Example

The following example runs the HCVA simulation using the Design Robustness tool. The Design Robustness tool sets up the HCVA and calls the PrimeSim XA tool to execute the transient simulation in the maximum of four parallel processes. The output files are directed to the HCVA_RESULT directory:

```
$> primesim_dr -flow HCVA -simulator XA "/path/to/bin/xa T@000.sp -c
T@000.tcl" -o HCVA_RESULT/T@000_dp4 -dp 4
```

Using PrimeSim AVA Across Simulators

The PrimeSim AVA tool is supported by all simulators in the PrimeSim Continuum. It aids each simulator to make faster computations for large sample sizes with the help of machine learning techniques.

Invoke each simulator with the `-adv` keyword to create a `.json` file and run the PrimeSim AVA tool. Each simulator supports different AVA applications, based on their general field of utility.

Details for each simulator are described in the following sections:

- [Invoking PrimeSim AVA With PrimeSim HSPICE](#)
- [Invoking PrimeSim AVA With PrimeSim SPICE](#)
- [Invoking PrimeSim AVA With PrimeSim XA](#)

Invoking PrimeSim AVA With PrimeSim HSPICE

To perform the PrimeSim Design Robustness simulator, you must have the following input files:

- Input netlist file [Required]
 - Netlist file contains circuit and analyses statements.
 - `monte=n` keyword is not mandatory in any statement while invoking AVA using `-adv`.
- `<config>.json` file [Optional]
 - Format of the file is `.json`. This helps to set various options within AVA.
 - You can generate this file using the dialog box or use an existing file.
 - If an existing `.json` file is used, the dialog box does not pop- up.

Syntax: `hspice -i <input> -o <out> -adv <config>.json`

Invoking PrimeSim AVA With PrimeSim SPICE

To perform the PrimeSim Design Robustness simulation, you must have the following input files:

- Required Tools
 - PrimeSim SPICE and (Set both tools)

- Setup environment

- `setenv AVA_HOME /path/to/your/latest/primesim_ava/installation`

Note:

PrimeSim Design Robustness binary is available at `:/path/to/your/latest/primesim_ava/installation/bin/ava`.

Syntax: `primesim -i <input> -adv <config>.json -o <output> -spice`

Invoking PrimeSim AVA With PrimeSim XA

To perform the PrimeSim XA AVA simulation, you must have the following input files:

- Required Tools
 - PrimeSim XA and (Set both tools)

- Setup environment

- `setenv AVA_HOME /path/to/your/latest/primesim_ava/installation`

Note:

PrimeSim AVA binary is available at `:/path/to/your/latest/primesim_ava/installation/bin/ava`.

Syntax: `xa -i <input> -adv <config>.json -o <output> -spice`

3

Output Files

This chapter outlines the output files generated by the Design Robustness tool.

The Design Robustness tool generates the following output files:

- Simulation Log File (*.log)

The simulation log file contains machine-executed command line, information, distributed job information, and simulation runtime.

- Intermediate Simulation Directory (*_sim/)

The intermediate simulation directory contains all the intermediate simulation files. It includes the nominal run and all simulation runs with variation. The intermediate files are mainly used for debugging possible issues.

The following sections describe more information on Design Robustness output files:

- [Design Robustness Profiler Output Files](#)
- [Design Robustness Analyzer Output File](#)
- [Stress Test Analysis Output Files](#)
- [High Capacity Variation Analysis Output Files](#)
- [HSMC Outputs](#)

Design Robustness Profiler Output Files

When running the Design Robustness Profiler (DRP) application in interactive mode, the following graphs are displayed:

- [Histogram](#)
- [Heatmap](#)

You can also generate the graphs using a pre-existing database file from the previous DRP run (see [Generating Graphs in DRP Using a Database File](#)).

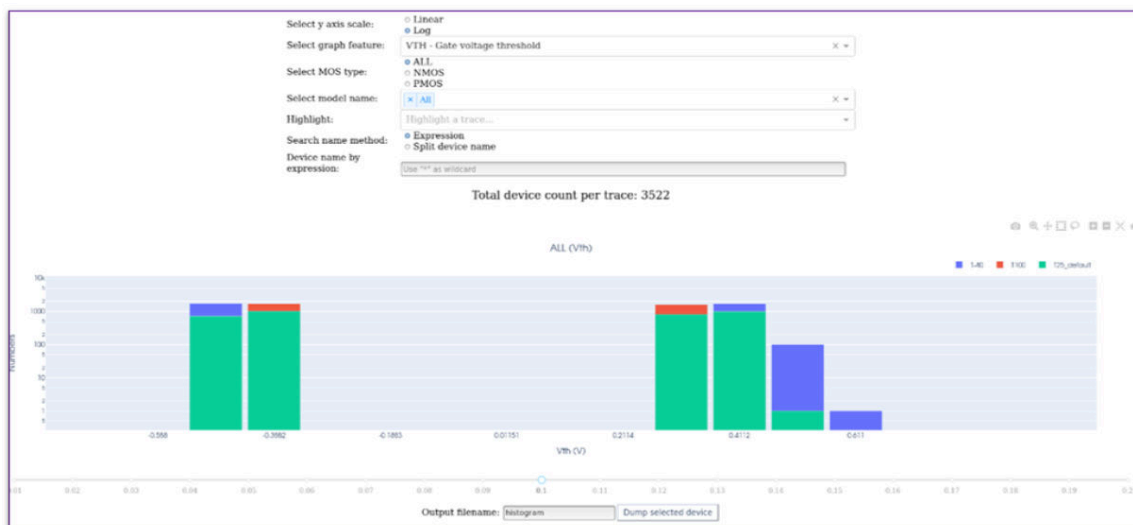
Histogram

When running the DRP application in interactive mode (or [Generating Graphs in DRP Using a Database File](#)), the following message is printed on the screen so you can copy the URL to a supported browser (Chrome or Firefox) to analyze the data:

Running on **<Generated_URL>** (Press CTRL+C to quit)

This histogram (see [Figure 5](#)) is an interactive graph. You can filter, zoom, highlight, and output selected devices into a file for analysis.

Figure 5 Example of DRP Histogram



The available device filtering options are:

- **Expression:** specify the **Device name by expression**.

Figure 6 Expression

Search name method:

☒ Expression
 ☐ Split device name

Device name by expression:

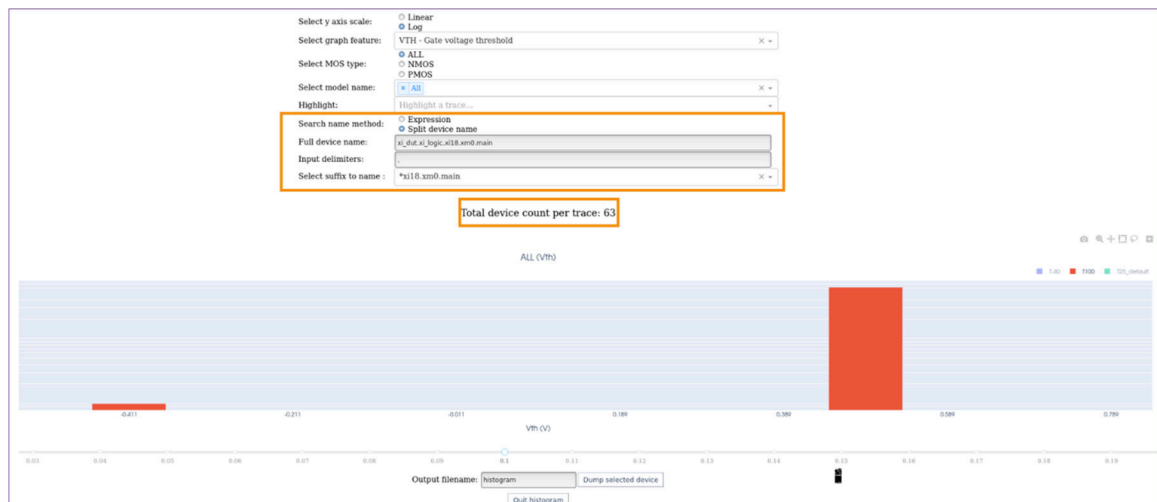
- If no input name is specified, the Histogram includes all devices filtered by other settings.
- If **"*"**: wildcard is specified. For example, **"*im*"** filters the device by **"im"** in the device name.

- **Split device name:** this filter displays the following additional filters:
 1. **Full device name:** specify with device name hierarchy.
 2. **Input delimiters:** specify the input delimiter. This field supports multi-delimiters.
 3. **Select suffix to name:** the drop-down lists the names of each hierarchy level split by delimiter “*” wildcard. If no delimiter is specified, it only shows the full device names.

Figure 7 Example of a single delimiter

Figure 4 shows the histogram of v_{th} under the filter option of split device name.

Figure 8 Histogram



Heatmap

A heatmap is only generated when XY coordinates are presented in the netlist. The following different forms of syntax are supported:

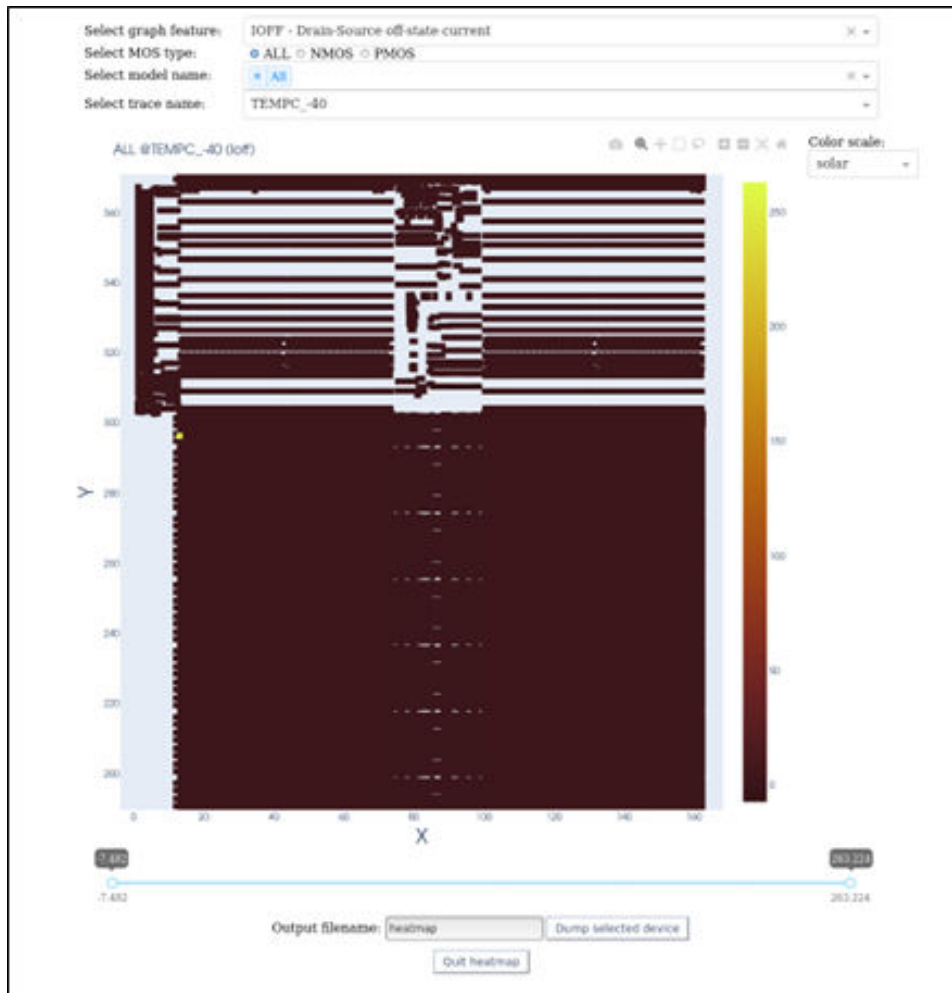
- $x=value\ y=value$
- $\$x=value\ \$y=value$

When CTRL+C is enabled after generating a histogram, the following message is printed on the screen so you can copy the URL to a supported browser to analyze the data:

Running on [Generated_URL](#) (Press CTRL+C to quit)

This heatmap (see [Figure 9](#)) is an interactive graph. You can filter, zoom, highlight, and output selected devices into a file for analysis.

Figure 9 Example of DRP Heatmap



Design Robustness Analyzer Output File

When running the Design Robustness Analyzer (DRA) application, the following output file is generated:

- [Sensitivity Report File](#)

Sensitivity Report File

When running the DRA application, the application generates a sensitivity report file (`*_report.csv`). The sensitivity report includes the following sections:

- **Sensitive Device:** A sensitive device is an individual device that does not meet accuracy requirements after applying the variation only to that device.
- **Sensitive Group:** A sensitive group is a group of devices that does not meet the accuracy requirements when applying variation to all the devices in a group. When applying variation to each individual device in a sensitive group, it may still meet the accuracy requirement.

The sensitivity report contains the following information:

- Device group number (Sensitive Group section only)
- Device name
- Simulation tag name
- Failed measurement name
- Failed measurement type
- Nominal simulation result
- Variation simulation result
- Absolute error
- Relative error in percentage
- Result location/path

If `"search_method" : "region"` is specified, the Design Robustness tool reports the top 10 devices with largest absolute error in that region. By default, the Design Robustness tool only searches one top region with the largest error. You can modify the number of top regions using `"top_region"` command.

If `"search_method" : "error_tol"` is specified, the Design Robustness tool reports all the devices exceeding the error tolerance specified in the command file.

Example

The following is an example of the sensitivity report file:

```
*****
* PRIMESIM_DR linux64 V-2023.12
* Build id: 7984432
*****
//
*== Sensitive Device ==, , , , , , , , , ,
```

```
Device Name,Simulation,Measurement_Name,Measurement_Type,Nominal_Result,Variation
_Result,Error,Rel_Error(%),Result_Location,
xtop/xb/xctrl/xd/xi6/mm1.m1,sim_00_04_00_35,t_ck2q65,timing,1.52579e-09,1.51342e-
09,1.24e-11,0.81,DRA_RESULT/dr_run_dra_region_sim/run_dra_region_mos_00_04_00/sim
_35_p,
xtop/xb/xctrl/xd/mm1.m1,sim_00_04_00_01,t_ck2q126,timing,1.52627e-09,1.51404e-09,
1.22e-11,0.8,DRA_RESULT/dr_run_dra_region_sim/run_dra_region_mos_00_04_00/sim_01
_p,
xtop/xb/xctrl/xd/xi4/mm1.m1,sim_00_04_00_15,t_ck2q126,timing,1.52627e-09,1.51406e
-09,1.22e-11,0.79,DRA_RESULT/dr_run_dra_region_sim/run_dra_region_mos_00_04_00/si
m_15_p,
xtop/xb/xctrl/xd/xi5/mm0.m1,sim_00_04_00_24,t_ck2q126,timing,1.52627e-09,1.51421e
-09,1.21e-11,0.79,DRA_RESULT/dr_run_dra_region_sim/run_dra_region_mos_00_04_00/si
m_24_p,
xtop/xb/xctrl/xd/xi6/mm0.m1,sim_00_04_00_37,t_ck2q62,timing,1.52517e-09,1.51307e-
09,1.21e-11,0.79,DRA_RESULT/dr_run_dra_region_sim/run_dra_region_mos_00_04_00/sim
_37_p,
,,,,,
*==== Sensitive Group ===,,,,,
Group#,Device_Name,Simulation,Measurement_Name,Measurement_Type,Nominal_Result,Va
riation_Result,Error,Rel_Error(%),Result_Location,
1,xtop/xb/xctrl/xd/mm4.m1,sim_00_04_00_04,t_ck2q124,timing,1.51994e-09,1.85368e-0
9,3.34e-10,21.95,DRA_RESULT/dr_run_dra_region_sim/run_dra_region_mos_00_04_00/sim
_04_p,
1,xtop/xb/xctrl/xd/mm4@2.m1,sim_00_04_00_04,t_ck2q124,timing,1.51994e-09,1.85368e
-09,3.34e-10,21.95,DRA_RESULT/dr_run_dra_region_sim/run_dra_region_mos_00_04_00/s
im_04_p,
2,xtop/xb/xctrl/xd/mm3.m1,sim_00_04_00_03,t_ck2q124,timing,1.51994e-09,1.59569e-0
9,7.57e-11,4.98,DRA_RESULT/dr_run_dra_region_sim/run_dra_region_mos_00_04_00/sim
_03_p,
2,xtop/xb/xctrl/xd/mm3@2.m1,sim_00_04_00_03,t_ck2q124,timing,1.51994e-09,1.59569e
-09,7.57e-11,4.98,DRA_RESULT/dr_run_dra_region_sim/run_dra_region_mos_00_04_00/si
m_03_p,
3,xtop/xb/xctrl/xd/mm10.m1,sim_00_04_00_02,t_ck2q126,timing,1.52627e-09,1.5045e-0
9,2.18e-11,1.42,DRA_RESULT/dr_run_dra_region_sim/run_dra_region_mos_00_04_00/sim
_02_p,
3,xtop/xb/xctrl/xd/mm10@2.m1,sim_00_04_00_02,t_ck2q126,timing,1.52627e-09,1.5045e
-09,2.18e-11,1.42,DRA_RESULT/dr_run_dra_region_sim/run_dra_region_mos_00_04_00/si
m_02_p,
```

Stress Test Analysis Output Files

When running the stress test analysis, the following output files are generated:

- Nominal simulation measurement file (*.stress_test_nominal.meas)
- Scaled simulation measurement file (*.stress_test_scaled.meas)
- Measurement comparison report (*.stress_test_report.csv)

Example

The following is an example of the measurement comparison report:

```
Measurement,Absolute Error,Relative Error(%)
v1_avg,2.83e-06,6.06
```

```
ring_fall_delay,1.03e-11,6.04
ring_rise_delay,1e-11,5.91
```

High Capacity Variation Analysis Output Files

The output files generated by High Capacity Variation Analysis (HCVA) are the same as the output files generated by traditional Monte Carlo in the PrimeSim XA tool. See the *PrimeSim™ XA User Guide* for details of each output file.

The output files include:

- *.mc0
- *.mc
- *.mc.csv
- *.meas OR *.mt
- *.mc_params

HSMC Outputs

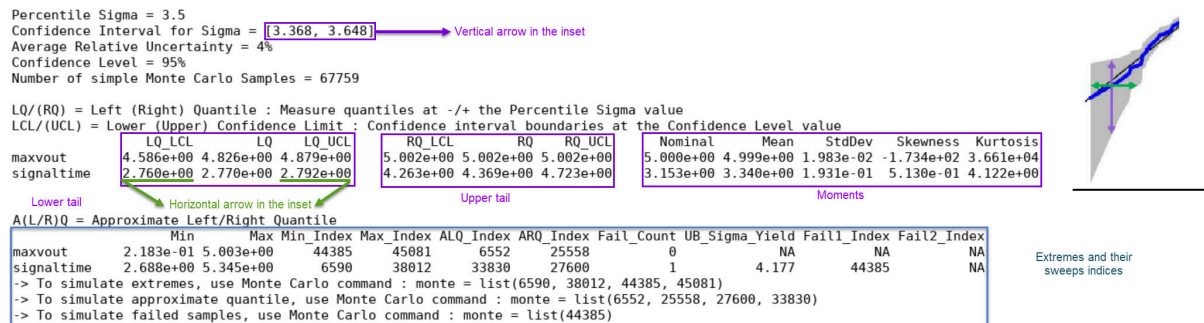
HSMC analysis produces the following outputs:

- [The *.ava.report File](#)

The *.ava.report File

The output *.ava.report file is a HSMC report file with moments, lower and upper quantiles and their confidence intervals, extremes, and sensitivities. [Figure 10](#) and [Figure 12](#) describe the outputs in more detail.

Figure 10 Quantiles, Moments, and Extremes (*.ava.report file)



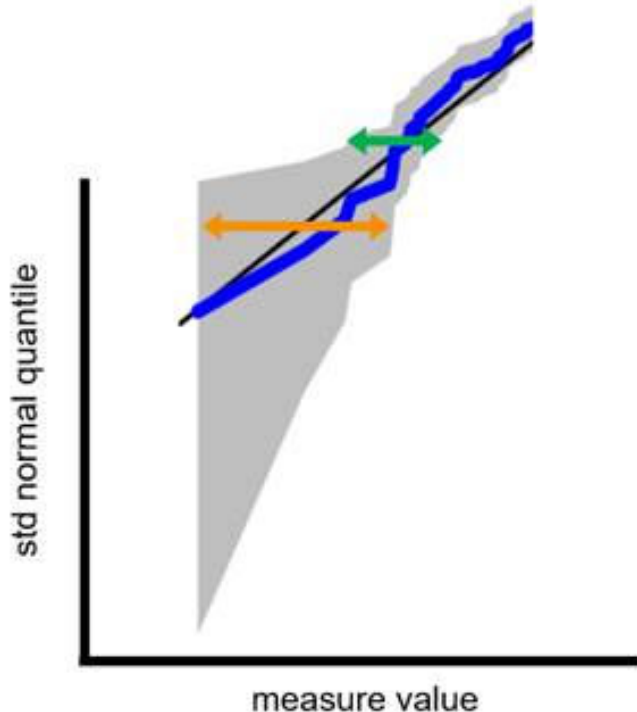
More on the Confidence Interval

There are conditions for which the lower confidence interval value of the left tail or the upper confidence interval value of the right tail are unable to be determined.

Consider [Figure 11](#). The two arrows represent the left tail measure confidence intervals for two different values of Average_Relative_Uncertainty (proportional to the gray region height at the intersection of the given arrow and blue curve).

The green arrow is at the start of tail, that allows to determine both upper and lower confidence interval values. On the other hand, the orange arrow is almost at the end of the tail, where the QQ-plot is truncated at its lowest measure value, which prevents in determining the lower confidence interval value.

Figure 11 *Illustration of Confidence Intervals*



For the green arrow (which has a smaller value of Average_Relative_Uncertainty), both the lower and upper values of the confidence interval are resolvable.

For the orange arrow (which has a larger value of Average_Relative_Uncertainty), the upper confidence interval value is determinable, however the lower value is not - you can only say that it is less than the left-most boundary.

In [Figure 10](#) UB_Sigma_Yield refers to the upper bound of the yield in terms of sigma. This could be either less or more than target sigma. In case of a pass/fail circuit, this can be interpreted in different ways based on failure samples produced.

When using the PrimeSim AVA tool for any variation analysis for a pass/fail circuit check for failed samples, Left/Right quantiles, and UB_sigma_yield in the `.ava.report` file:

UB_Sigma_Yield is unresolved with Zero failures: Design passes the given target sigma. Ensure to check other quantiles.

UB_Sigma_Yield > Target Sigma with few failures and resolved quantiles: There are some errors reported, but the error count is sufficiently small than the target sigma quantile that is able to be calculated in both tails. It is recommended to understand the reason for the failure by co-relating the results back to some analysis and check if it still passes target sigma.

UB_Sigma_Yield < Target Sigma with lot of failures and unresolved Left/Right quantiles: Refers that even before any measure specification criteria are taken into consideration, the measure yield is already limited to Upper bound that is UB_Sigma_Yield. The failures reported might have occurred between the upper bound and target sigma, so the design must be improved. Exception include DC convergence failure where simulation has failed and not measure statement. Other reasons could be:

- Measure definition refinement
- Insufficient tstop value
- Model parameter out of bound
- Improper biasing
- Circuit in need of redesign

Sensitivity Table

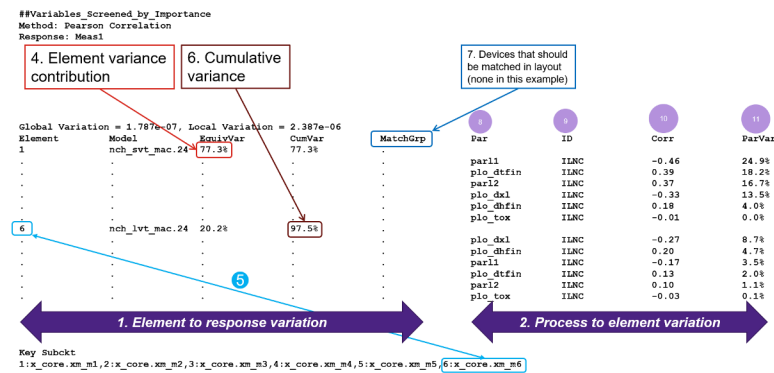
The Sensitivity or the Variation Contribution table (see [Figure 12](#)) serves the following:

- It partitions the response variation amongst the various devices and then to the process variation parameters within devices.
- The contribution table permits designers to identify the dominant variation contributors and, if possible, to adjust the dimensions of these devices to reduce variation.
- The contribution table also provides information about which sets of devices form match groups and should be kept close together in the layout to reduce the influence of spatial variation and thermal gradients.

Note:

A number of simplifying assumptions are made in the generation of the contribution table and you should interpret the numbers in a somewhat qualitative manner.

Figure 12 Sensitivities (*.ava.report file)



- The sensitivity table contains two sets of columns. The left side of the table shows how individual elements contribute to the overall response variation.
- The right side of the table shows how process variables contribute to the variation for a particular element.
- The header record gives the overall contribution from global and local variation across all elements.
- *EquivVar* is the most important column. It gives the variation contribution for a specific element. The rows in the table are sorted on this column such that elements that contribute the most appear towards the top of the table. The table ends when the contribution becomes sufficiently small. In this example, the first element is responsible for 77.3% of the variation.
- The first column in the table, *Element*, is the element identifier. As hierarchical element names can be quite long, an integer is used as an alias. The integer to element name mapping is given at the bottom of the table. Thus element #6 is x_core.xm_m6.
- The *CumVar* column gives the cumulative variance associated with elements in the table up to that row. It increases from the top of the table to the bottom and the table ends when the cumulative variation is sufficiently large. In this example, the first two elements, #1 and #6, account for 97.5% of the variation. Response variation can be reduced by increasing the sizes of these devices.
- Analog circuits are sensitive to mismatch in matched elements. The *MatchGrp* column contains integer tags to identify groups of such elements. It is recommended that

elements in each group be laid out very close to each other, possibly inter-digitated, to reduce the impact of processing and thermal gradients.

Note:

The group identification is based on post-processing the Monte Carlo data rather than on topological pattern matching.

- The *Par* column on the right side of the table gives the names of the global or local independent random variables. These are the parameters defined through Agauss functions.
- The *ID* field is a key defining the type of random variable, see the PrimeSim™ HSPICE® User Guide: Basic Simulation and Analysis for the encoding.
- The *Corr* column is the correlation between the independent random variable and the element variation.
- The *ParVar* column splits the element variation into contributions from the process-level independent random variables. The entries in the *ParVar* column are ordered by size within each element block.

Consider element #6 in the example. It contributes 20.2% of the variation, see column *EquivVar*. This 20.2% is split as: 8.7% from plo_dx1, 4.7% from plo_dhfin, and so on, and the sum of the entries is 20.2%.

Sensitivity Table FAQ

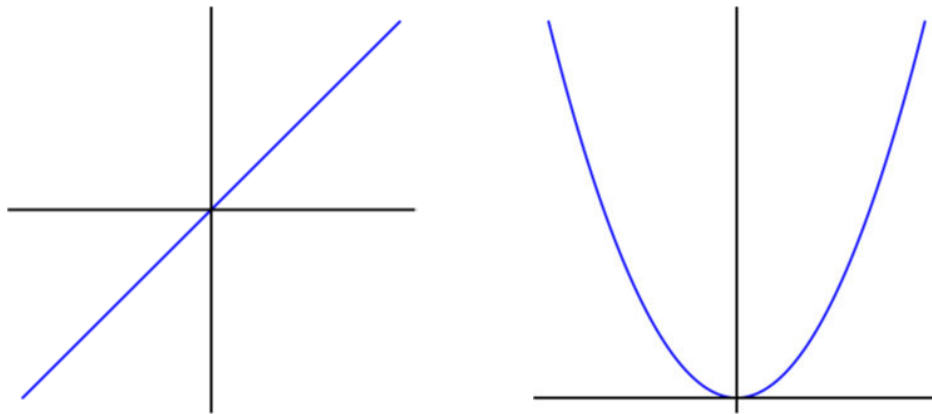
- How are the process level contributions computed?

The process level contributors are the independent random variables (IRVs) that are defined through Agauss style statements. Their coefficient is the correlation between that particular IRV and the response. Implicitly, this assumes a linear relation between the IRV and the response. This assumption is often valid unless the response is strongly nonlinear and can be justified by considering the first order terms in a Taylor series expansion of the response. The use of the correlation is justified because the random variables are independent.

- Are there situations where the linear assumption can be misleading?

Consider [Figure 13](#) where the y-axis represents the response and the x-axis, the IRV.

Figure 13 Linear and Quadratic IRV Response



A linear response is shown on the left and the assumptions hold here. The quadratic response is on the right; it has a derivative of zero at the nominal (origin) and the correlation coefficient will be close to zero. However, the response clearly depends on the IRV.

- How are the element level contributions computed?

As the process level random variables are independent, the element level contribution is the square root of the sum of squares of the process level IRVs associated with the element.

- Do the numbers in the table depend on sample size?

Definitely. As with all statistical methods, the correlation coefficient also has a sampling error which decreases roughly as $1/(\sqrt{N})$ where N is the sample size. Mathematically, the sampling error is quantified by the confidence interval. If the sample correlation coefficient is r , the 95% confidence interval for it is given by the following equation:

$$CI = \tanh \left(\operatorname{atanh} (r) \pm 1.96 / \left(\sqrt{N-3} \right) \right)$$

Consider the example where $r = 0.1$ and $N = 200$. Then, the 95% confidence interval for r is $[-0.04, 0.24]$. One can thus not rule out the hypothesis that r could perhaps be zero.

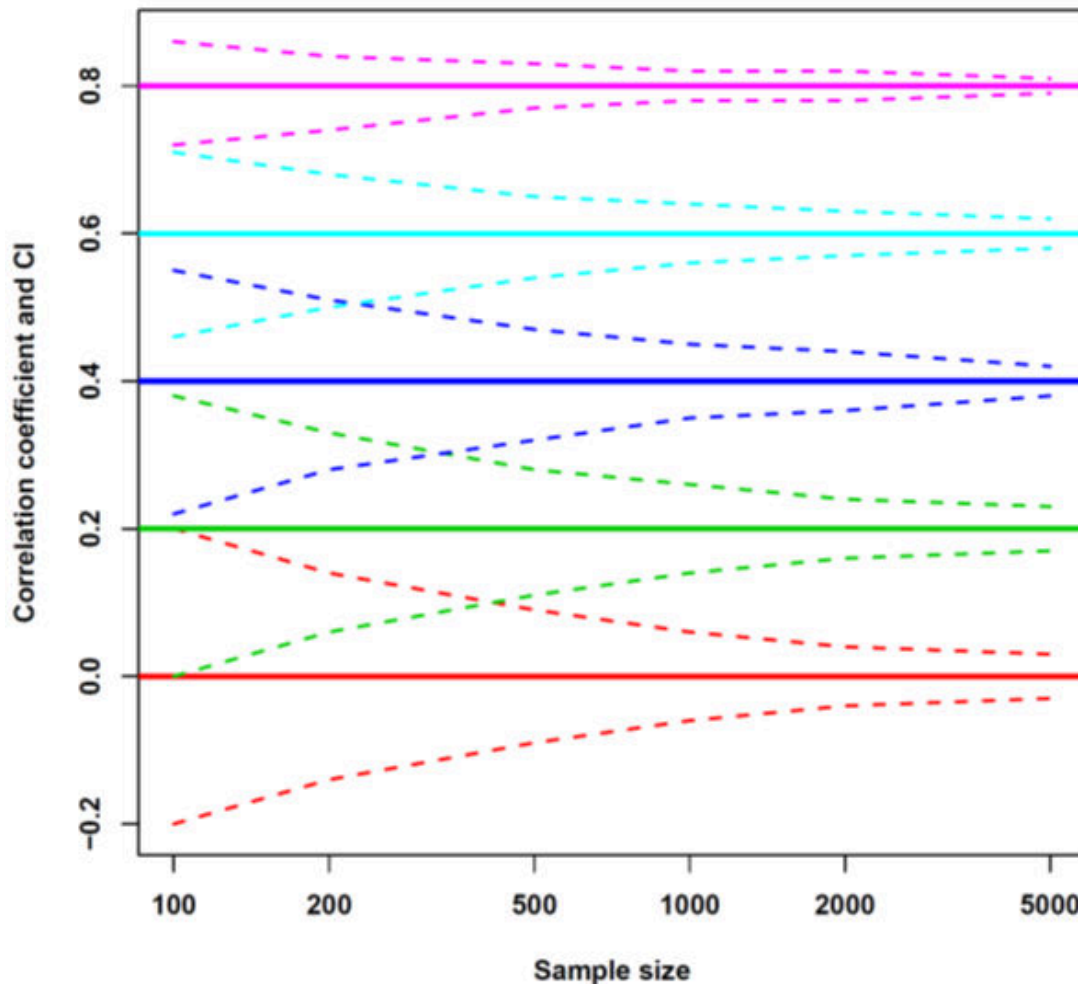
- What is the meaning of Qxxx in the SA *.ava.report file?

This xxx number represents the percentile. For example, the percentile of the 3-sigma standard normal quantile is 99.865%, which is written as Q99.865 in the report.

Confidence Interval and Sample Size

Figure 14 shows the confidence bands as a function of the sample correlation coefficient and sample size. Notice that the confidence intervals overlap at small sample sizes even when the correlation coefficients are separated by 0.2.

Figure 14 Correlation Coefficient and the Confidence Interval



The implication is that it is not possible to distinguish between the ranking of the independent variables or the element contributions when the sample size is small and the correlation coefficients are close together. Similarly, contributors that theoretically have zero correlation can show up in the sensitivity table with small sample sizes.

4

Configuration File Command Reference

This chapter details the Design Robustness configuration file commands and subcommands.

The commands in the Design Robustness tool are grouped into the following categories:

- [Command Hierarchy](#)

Command Hierarchy

The following sections illustrate the command hierarchy of the Design Robustness tool:

- [Common Settings](#)
- [Transient Simulation Setup](#)
- [Design Robustness Setup](#)

Common Settings

Use the following commands to enable common settings in design robustness simulation, such as output directory name, output file prefix name, and so on:

- `dp_config` – Specifies the hosts information file for distributed parallel processing capability.
- `dp_num` – Sets the maximum number of parallel simulations that are run.
- `outdir` – Specifies the name of output directory to store all the intermediate files and simulation results.
- `outprefix` – Specifies the prefix name used as part of the output file name and the intermediate simulation directory name.
- `set_hier_delimiter` – Specifies the character used as a hierarchical delimiter in the netlist and the character used as a hierarchical delimiter in the instances during extraction.

Transient Simulation Setup

Use the following command and subcommand to control all the necessary settings to run transient simulation:

- `sim_setup` – Contains all the transient-related setup commands:
 - `sim_cmdline` – Specifies the command line to run transient simulation.

Design Robustness Setup

Use the following command and subcommands to control design robustness simulation:

- `dr_setup` – Contains all the design robustness related setup commands:
 - [Design Robustness Common Settings](#)
 - [Design Robustness Profiler Settings](#)
 - [Design Robustness Analyzer Settings](#)
 - [Stress Test Analysis Settings](#)

Design Robustness Common Settings

Use the following commands to enable common settings in design robustness simulation:

- `dr_tag` – Specifies the tag name for the group of the commands.
- `stage` – Selects the type of application to execute.

Design Robustness Profiler Settings

Use the following command and subcommands to set up Design Robustness Profiler (DRP) simulation:

- `drp` – Contains all the commands related to the Design Robustness Profiler (DRP) application:
 - `database` – Specifies the name of the database to load for generating graphs.
 - `drp_stage` – Selects the stages to execute in Design Robustness Profiler (DRP) simulation.
 - `enable_lpe` – Enables the Design Robustness tool to process a list of standard layout parameters in Design Robustness Profiler (DRP) simulation.
 - `load_lpe_file` – Enables the Design Robustness tool to load a file containing a list of layout parameters to process in Design Robustness Profiler (DRP) simulation.

- `mc_index` – Specifies a Monte Carlo index to run in Design Robustness Profiler (DRP) simulation.
- `mc_sweep` – Specifies a Sweep Monte Carlo samples to run in Design Robustness Profiler (DRP) simulation.
- `temperature` – Specifies a list of temperatures in Celsius to run in Design Robustness Profiler (DRP) simulation.

Design Robustness Analyzer Settings

Use the following commands and subcommands to set up Design Robustness Analyzer (DRA) simulation:

- `dra` – Contains all the commands related to the Design Robustness Analyzer (DRA) application:
 - `delta_ids_file` – Provides a file to manually override the percentage change in `Ids` for each device for simulation.
 - `delta_ids_pct` – Specifies the percentage change in `Ids` value for variation.
 - `delta_vth_file` – Provides a file to manually override the change in `vth` value for each device for simulation.
 - `delta_vth_val` – Specifies the change in `vth` value for variation in volts.
 - `search_method` – Specifies the method to search for sensitive devices.
 - `variation_feature` – Specifies the device model data used for variation.
- `meas` – Contains all the commands related to accuracy checking criteria for the Design Robustness Analyzer (DRA) application:
 - `meas_list_file` – Provides a file to list all the key measurements and local tolerance values for each measurement to check for accuracy.
 - `check_tol_sign` – Specifies the direction of error tolerance for a DRA search. The error tolerance is defined as the difference between variation result to nominal result.
 - `set_current_tol_abs` – Specifies the global absolute tolerance value for accuracy checking of the current measurement result.
 - `set_current_tol_rel` – Specifies the global relative tolerance value for accuracy checking of the current measurement result.
 - `set_timing_tol_abs` – Specifies the global absolute tolerance value for accuracy checking of the timing measurement result.

- `set_timing_tol_rel` – Specifies the global relative tolerance value for accuracy checking of the timing measurement result.
- `set_voltage_tol_abs` – Specifies the global absolute tolerance value for accuracy checking of the voltage measurement result.
- `set_voltage_tol_rel` – Specifies the global relative tolerance value for accuracy checking of the voltage measurement result.
- `top_region` – Specifies the number of regions with the largest error to search for sensitive devices.
- `device` – Contains all the commands relating to scoping. It enables you to apply variation to the circuit scope:
 - `exclude_device_filter` – Provides a file to list all the devices names to exclude from variation for simulation.
 - `include_device_filter` – Provides a file to list all the device names to apply the variation for simulation.
 - `type` – Scopes the variation to the specified device type.
- `active_device_flow` – Contains all the commands relating to enabling active device flow:
 - `cck_soa_val_ib` – Specifies the current value of terminal bulk (I_b) of a MOSFET.
 - `cck_soa_val_id` – Specifies the current value of terminal drain (I_d) of a MOSFET.
 - `cck_soa_val_ig` – Specifies the current value of terminal gate (I_g) of a MOSFET.
 - `cck_soa_val_is` – Specifies the current value of terminal source (I_s) of a MOSFET.
 - `enable` – Enables or disables the active device flow.

Stress Test Analysis Settings

Use the following command and subcommands to set up Stress Test Analysis:

- `stress_test` – Contains all the commands related to enabling a stress test:
 - `device_model` – Lists the device model to apply scaling of the specified model data for a stress test.
 - `ids_scale` – Scales the value of drain-source current (I_{ds}) of a MOSFET.

- `idsat_scale` – Scales the value of drain current (I_d) and drain-source current (I_{ds}) of a MOSFET in the linear and saturation region.
- `ioff_scale` – Scales the value of drain current (I_d) and drain-source current (I_{ds}) of a MOSFET in the cutoff region.

Advanced Variation Analysis Settings

The following is an example of the configuration file to run an Advanced Variation Analysis using the Design Robustness tool:

```
"dp_config" : "xhost/xhost_sh_launch", "dp_num" : 1,
"sim_setup" :
{
  "sim_cmd" : "xa -mt 2 -hspice main.in"
},
"dr_setup" : [
{
  "dr_tag" : "run_ava ",
  "stage" : "ava",
  "ava" :
  {
    "ava_cfg_file": "hsmc.json"
  }
} ]
}
```

The following is the hsmc.json file:

```
JSON for HSMC Basic Run
{
  "Active_Application": "High_Sigma_Monte_Carlo",
  "High_Sigma_Monte_Carlo": {
    "Chip_Sigma": 3.5,
    "Component_Count": 1,
    "Average_Relative_Uncertainty": 4.0
  }
}
{
  "Active_Application": "High_Sigma_Monte_Carlo",
  "High_Sigma_Monte_Carlo": {
    "Chip_Sigma": 3.5,
    "Component_Count": 1,
    "N_SRS_Samples": 67759
  }
}
```

5

Defining Advanced Variation Analysis Across Simulators and Application

This chapter describes the invoking of advanced variation analysis and the associated use cases across simulators and application.

This chapter covers the following topics:

- [Introduction to Advanced Variation Analysis](#)
- [Invoking PrimeSim Design Robustness With PrimeSim HSPICE](#)
- [Use Cases for Invoking AVA](#)
- [Using Basic and Advanced Options](#)

Introduction to Advanced Variation Analysis

The PrimeSim AVA tool of PrimeSim addresses the Monte Carlo solution with machine learning techniques. The application in the AVA provides to different problems based on the entities such as amplification factor, quantiles, and parametric yield that must be calculated. It also considers the design-dependent chip level sigma, design-dependent component count and user-defined statistical uncertainty.

Currently supported applications:

- Monte Carlo Analysis(Basic)
- Utility > Post processing
- High Sigma Monte Carlo
- Parametric Yield Estimation
- Sigma Amplification
- DSVC Import

Invoking PrimeSim Design Robustness With PrimeSim HSPICE

To perform the PrimeSim Design Robustness simulator, you must have the following input files:

- Input netlist file [Required]
 - Netlist file contains circuit and analyses statements.
 - `monte=n` keyword is not mandatory in any statement while invoking AVA using `-adv`.
- `<config>.json` file [Optional]
 - Format of the file is `.json`. This helps to set various options within AVA.
 - You can generate this file using the dialog box or use an existing file.
 - If an existing `.json` file is used, the dialog box does not pop-up.

Syntax: `hspice -i <input> -o <out> -adv <config>.json`

Use Cases for Invoking AVA

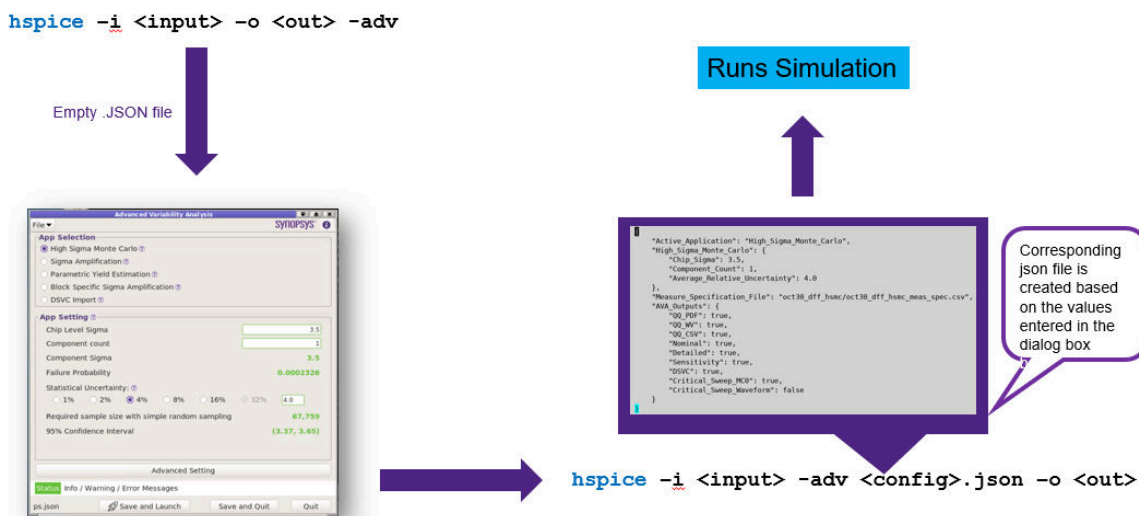
This section describes the various uses cases scenarios that are involved while invoking an AVA application.

- [AVA Simulation Without .json File](#)
- [AVA Simulation With .json File](#)

AVA Simulation Without .json File

You can configure the values of the `.json` file from the application and invoke the simulation. When the `-adv` argument only is available in the syntax, then the dialog box appears. You can enter the required values and the `.json` file is populated as shown in [Figure 15](#). Based on the values available in the `.json` file, the simulation is invoked.

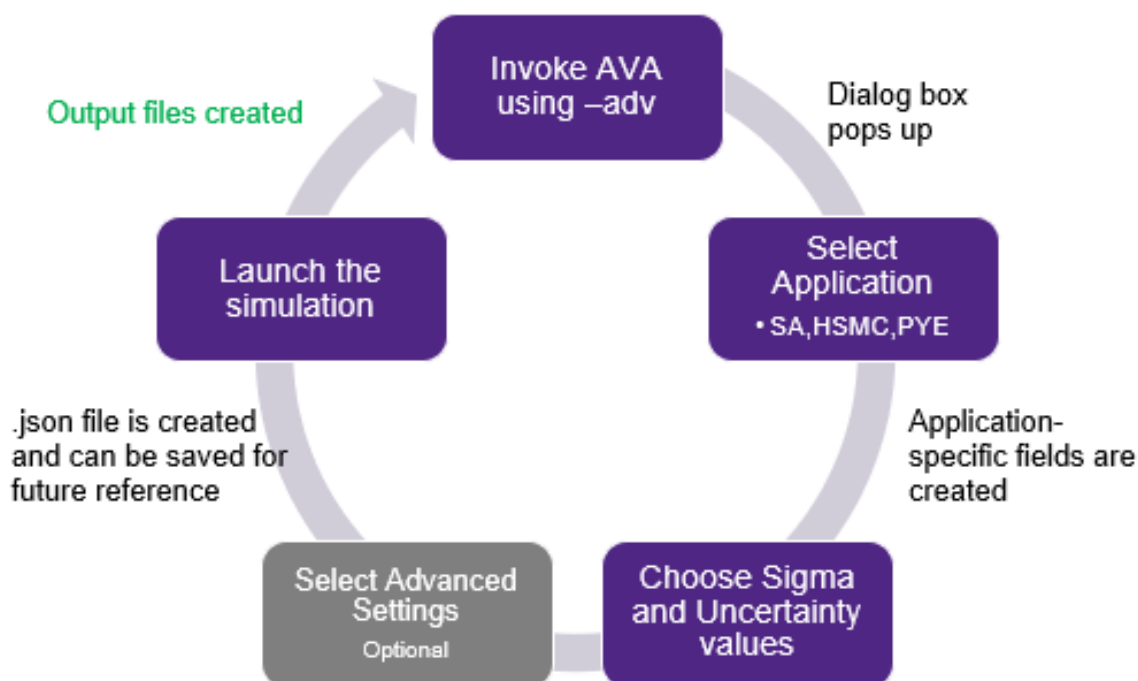
Figure 15 AVA Simulation Without .json File



AVA Using Dialog Box

In this use case, the .json file is created based on the values specified in the dialog box. By default, this option is invoked. Figure 16 explains the various steps when the .json file is not specified.

Figure 16 AVA Using Dialog Box



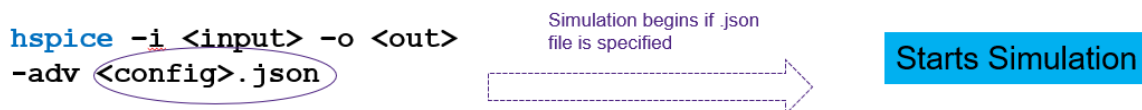
Note:

If an empty `.json` file is provided in the argument, then the dialog box appears. Enter the required fields to configure a `.json` file.

AVA Simulation With .json File

Use the `.json` argument in the syntax to invoke the simulation.

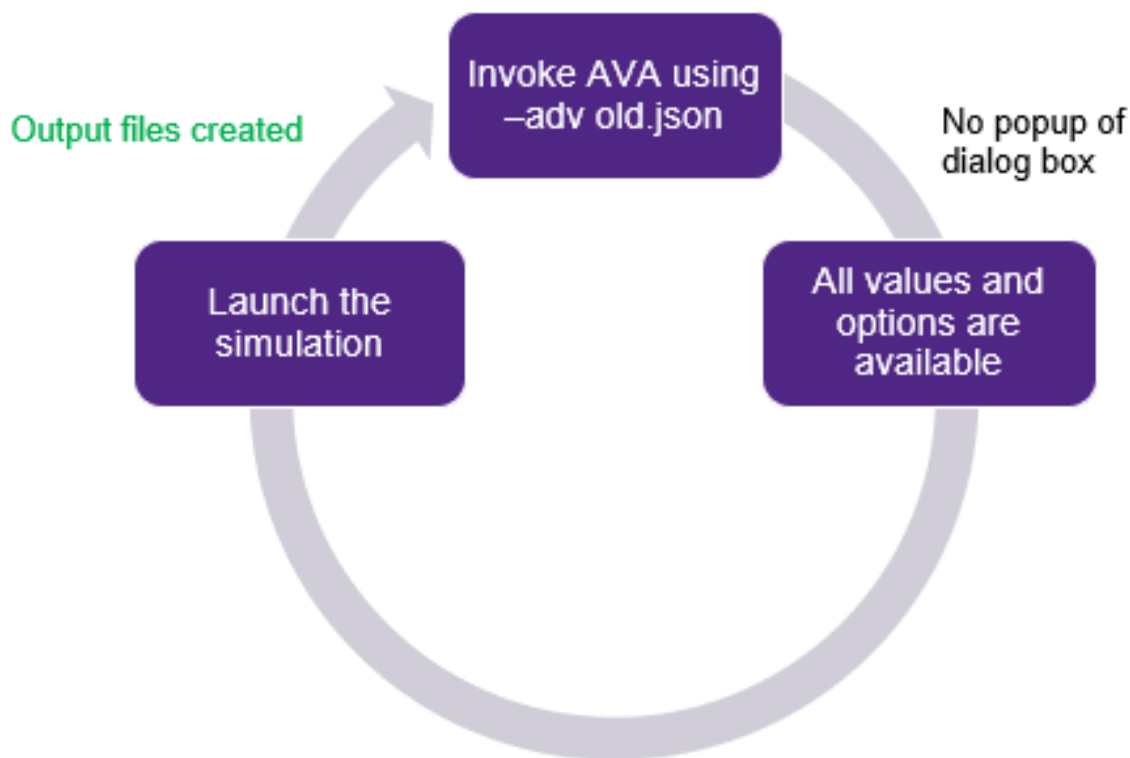
Figure 17 AVA Simulation With .json File



AVA Without Using Dialog Box

This use case is helpful in case of scripted PrimeSim HSPICE simulations. Here, an existing `.json` file is specified in the syntax. Figure 18 explains the various steps when the `.json` file is specified.

Figure 18 AVA Using Dialog Box



Using Basic and Advanced Options

The option in the dialog box has both basic and advanced options.

The basic option settings/inputs are important to decide the criteria such as confidence interval, failure probability, sample size used and so on. For more information on the basic option values, see [Probability and Statistical Concepts](#).

The advanced option settings/inputs define how the variations are applied, how the outputs are saved and any additional info. Advanced option supports the following:

- Simulation-based
 - Measure selections and specifications
 - Selective variation on models/instances
 - Validation with golden results
- Output-based

- File formats
- Save IRVs for critical corners

For more information on the advanced option values, see [Automated Validation of HSMC Results](#).

6

Monte Carlo Analysis

This chapter describes the principles of Monte Carlo(MC) Analysis simulation. It provides information on performing a MC simulation in the PrimeSim AVA tool and its outputs.

This section covers the following topics:

- [Introduction to Monte Carlo Analysis](#)
- [Invoking Monte Carlo From PrimeSim AVA](#)
- [Configuring Inputs for MC](#)
- [Monte Carlo Outputs](#)
- [Understanding Fast Sensitivity](#)
- [Parallel Processing](#)
- [Distributed Processing](#)
- [Postprocessing](#)

Introduction to Monte Carlo Analysis

The Monte Carlo application of the PrimeSim AVA tool brings the basic utilities of Monte Carlo analysis in the PrimeSim Continuum simulators under one umbrella. This application runs each sample individually to create outputs and does not use any machine learning methods. It uses the PrimeSim AVA tool to create the post-processing files. Additionally, the dialog box estimates the quantiles and confidence intervals before running simulation. Most of the features of PrimeSim HSPICE MC are available as different options and settings.

Monte Carlo analysis using the PrimeSim AVA tool is user-centric and visually helpful in choosing correct Monte Carlo options from a myriad of capabilities. Therefore, it provides an additional benefit of creating more customized Monte Carlo reports when run using appropriate options. Specifying selective variation is much easier with help of a visualized design tree. The disk usage can be minimized omitting unnecessary measures and files.

Note:

Throughout this chapter, all command-line invocations are denoted with `$SIM`. For the current chapter Monte Carlo, `$SIM` refers to PrimeSim HSPICE, PrimeSim XA, PrimeSim SPICE and PrimeSim Pro.

Invoking Monte Carlo From PrimeSim AVA

Use the following syntax to invoke the Monte Carlo application through the Advanced Variation Analysis dialog box:

```
hspice -i <input> -adv <user.json> -o <out> -dp <n>
```

You can select **Sample Number** or **Sigma/U** modes for defining a Monte Carlo Analysis.

Inputs for Monte Carlo

Argument	Description
<code>hspice -i <input></code>	Input SPICE netlist.
<code>-adv</code>	After initial parsing and nominal simulation, invokes the PrimeSim AVA advanced Variation analyses.
<code>user.json</code>	Input configuration file. Sample of <code>user.json</code> content for a MC simulation: <pre>{ "Active_Application": "Monte_Carlo", "Monte_Carlo": { "N_Samples": 20000, "Sampling_Method": "SRS", "Normal_Limit": 20, "LSMC": false } }</pre> If this configuration file is not provided, PrimeSim AVA displays a dialog box to help you create one or browse and open an existing file. See Configuring Inputs for MC.
<code>-dp <n></code>	Invokes n number of workers and distributes the simulation. The reports are generated for the entire sample size/sigma selected.

Intermediate files

The `.json` file consists of the information that was populated from the dialog box. You can modify (without format errors) the file and can be used during your subsequent simulation. Such invocation skips the dialog box and launches the simulation directly from

the settings available in the `.json` file. The basic inputs might slightly be different if the Sigma/Uncertainty mode is used.

Figure 19 MC Sample Mode `.json` File

```
{
  "Active_Application": "Monte_Carlo",
  "Monte_Carlo": {
    "N_Samples": 200000,
    "Sampling_Method": "SRS",
  },
  "Measure_Specification_File": "oa_sample/sample_meas_spec.csv",
  "AVA_Outputs": {
    "QQ_PDF": true,
    "QQ_WV": true,
    "QQ_CSV": true,
    "Nominal": true,
    "Detailed": true,
    "Sensitivity": true,
    "Critical_Sweep_Ext": true,
    "Critical_Sweep_Waveform": false,
    "Measure_File": true,
    "Sample_File": false
  },
}
```

Basic Inputs

Advanced Settings

Outputs for Monte Carlo

File	Description	See Also
out.ava.report	Statistic report : moments, sensitivity table, and so on.	The *.ava.report File
*.ava.details.json	Details of the <code>.json</code> file such as the input settings, derived quantities, outputs, and golden reference.	*.ava.details.json File
*.ava.dsvc.json	Output file for DSVC simulation	*.ava.dsvc.json File
*.ava.critical	Contains critical corner sweep indexes and corresponding IRVs in MC0 file format.	*.ava.critical File
*.ava.hm	Health monitor file that lists the resources used and reports the progress of the simulation in real time.	*.ava.hm File

File	Description	See Also
<code>out.ava.annotate</code>	Annotate report	
<code>out.ava.qq.pdf</code> <code>out.ava.qq.csv</code> <code>out.ava.qq.wv_plt</code>	Quantile data and plot in various forms.	Q-Q Plots in PDF, WaveView, and CSV Formats
<code>out.ava.log</code>	Run log: memory, elapsed time, and so on.	
<code>out.ava.sens.csv</code>	csv formatted sensitivity table	out.ava.sens.csv
<code>out.ava.irv_corr.csv</code>	Aggregated sensitivity report	out.ava.irv_corr.csv

Note:

Refer [AVA Revision History](#) for changes in output name formats.

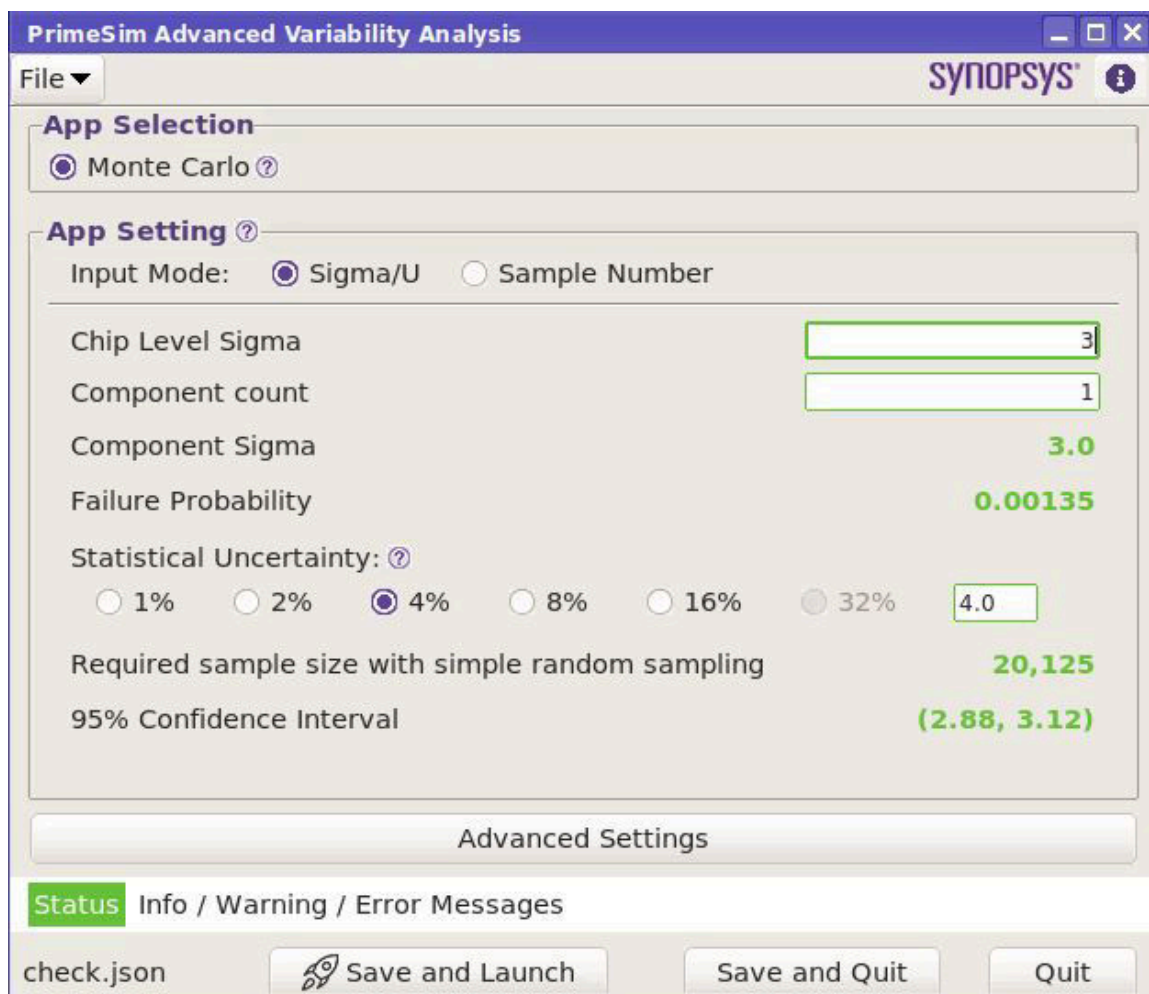
For backward compatibility:

Use `setenv HSP_OMEGA_QQ 1`, to switch and generate `qqplot` files with old file name, until the previous release cycle.

Configuring Inputs for MC

The **PrimeSim Advanced Variation Analysis** dialog box allows you to configure inputs for Monte Carlo.

1. Enter `hspice -i <input> -adv <user.json> -o <out> -dp <n>` syntax the following screen appears:



For more information, see [Inputs for Monte Carlo](#).

2. From the dialog box, you can invoke the Monte Carlo analysis in two possible modes.
 - To open an existing configuration (.json) file, choose **File > Open**, then select the configuration file to open in the file browser interface and click **Open**. For more information of the .json file, refer to [Appendix A](#).
 - To configure new inputs for MC, see [Creating a Configuration File](#).

Creating a Configuration File

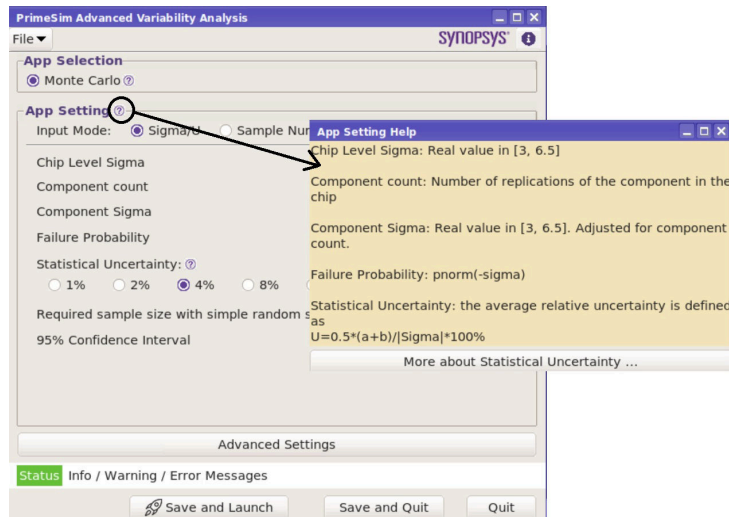
In the **App Setting** area of the MC dialog box, you can configure basic settings for MC.

- [Displaying Help for the Settings](#)
- [Configuring Basic Settings](#)
- [Configuring Advanced Settings](#)
- [Saving the Configuration](#)

Displaying Help for the Settings

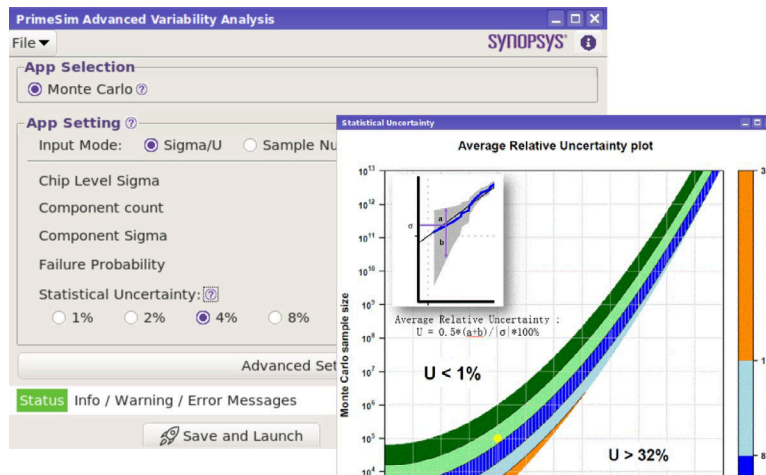
Click  adjacent to **App Setting**, to see a description of the fields and, if applicable, its value range (see [Figure 20](#)):

Figure 20 Help on the Dialog Box



Clicking  adjacent to **Statistical Uncertainty** displays detailed information about this field.

Figure 21 Help for Statistical Uncertainty



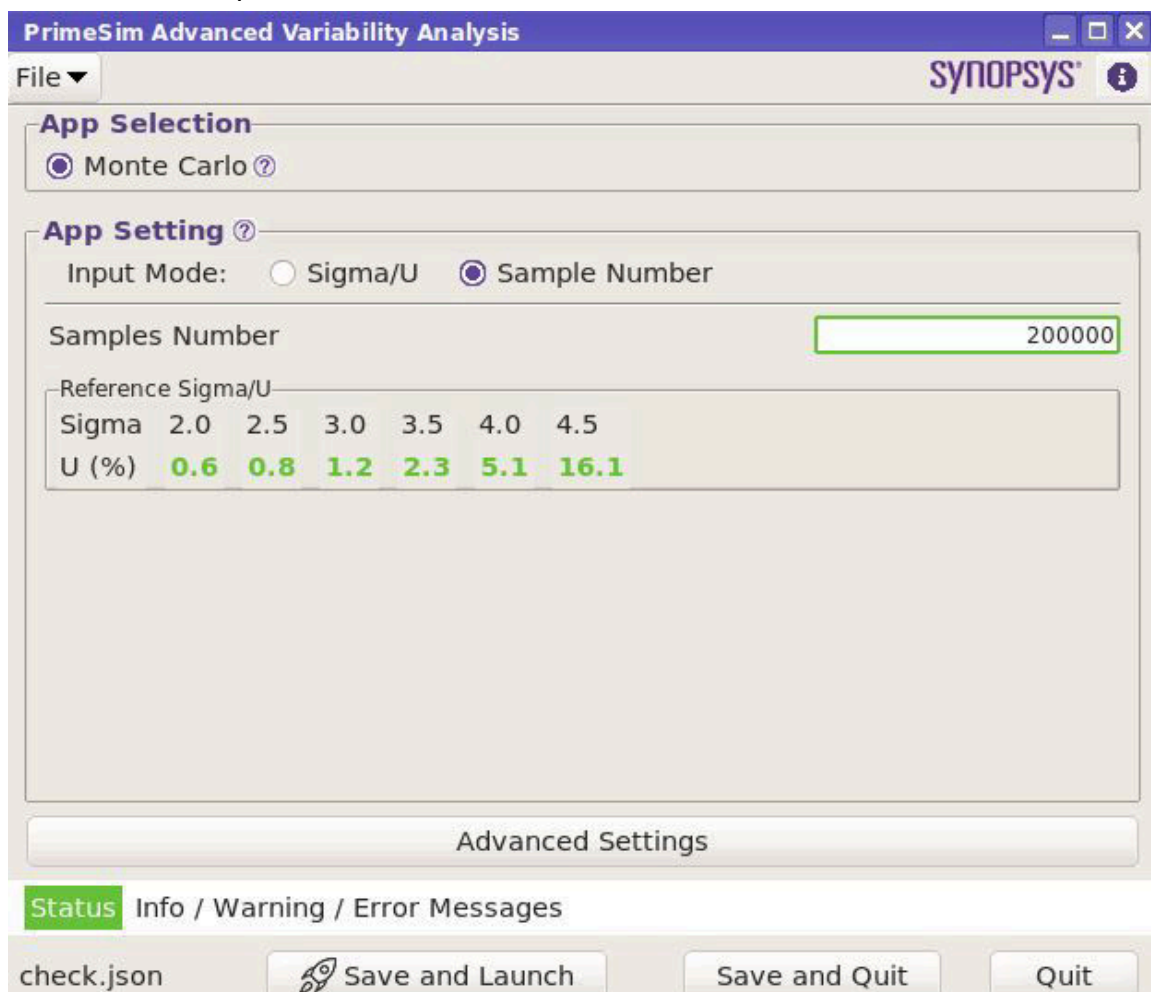
Configuring Basic Settings

Use the following procedure to configure MC basic settings:

1. Choose **Monte Carlo**
2. In the **App Setting**, enter **Sample Number** as **Input Mode**. The Reference values of Sigma and Uncertainty fields appear as shown in [Figure 22](#).
 - This sample size gets reflected in the netlist as `monte=n`.
 - Sample number decides all other parameters such as Target Sigma, Confidence intervals, Failure Probability, and so on.

For detailed information on the parameters, see [Probability and Statistical Concepts](#).

Figure 22 MC: Samples Number



- **Sigma:**
 - In a Gaussian distribution, for a given sample size using an appropriate sampling method, there could be only limited tail samples. These tail samples occur at different distances (measured in terms of sigma for example, at 4-sigma). For a small sample size, tail samples are clustered around the mean and the distance from mean is less. Thus, the sigma it achieves is usually less. For a sample size of 20k, a 3-sigma is achieved. Conversely, to achieve a sigma of 4.5, 2M samples are required.
 - For the sample size entered, the available sigma values are populated.
- **Uncertainty:**
 - For a given sample size, estimating measure values away from the mean comes along with uncertainty. This uncertainty can be reduced if more tail samples are created and thereby resulting in more sample size. For detailed information, see [Sigma, Sample Size, and Statistical Uncertainty](#).

For each sigma value, a reference value of uncertainty is populated.

3. In the **App Setting**, enter **Sigma/U** as **Input Mode**. The Reference values of Component Sigma, Failure Probability, and Statistical Uncertainty fields appear as shown in [Figure 23](#).

Component Sigma and Uncertainty levels together decide the sample size that is used and the confidence interval achieved. This calculated sample size gets reflected in the netlist as `monte=n`.

Figure 23 MC: Sigma/U

The screenshot shows the 'PrimeSim Advanced Variability Analysis' dialog box. The 'App Selection' section has 'Monte Carlo' selected. The 'App Setting' section has 'Input Mode' set to 'Sigma/U'. The 'Chip Level Sigma' is set to 3, and 'Component count' is set to 1. The 'Component Sigma' is 3.0, and 'Failure Probability' is 0.00135. The 'Statistical Uncertainty' is set to 4%, with a custom value of 4.0 entered in the field. The 'Required sample size with simple random sampling' is 20,125, and the '95% Confidence Interval' is (2.88, 3.12). The 'Advanced Settings' button is visible. The 'Status' bar shows 'Info / Warning / Error Messages'. The bottom bar contains 'check.json', 'Save and Launch', 'Save and Quit', and 'Quit' buttons.

Parameter	Value
Chip Level Sigma	3
Component count	1
Component Sigma	3.0
Failure Probability	0.00135
Statistical Uncertainty (%)	4%
Statistical Uncertainty (Custom)	4.0
Required sample size with simple random sampling	20,125
95% Confidence Interval	(2.88, 3.12)

For detailed information, see [Sigma](#), [Sample Size](#), and [Statistical Uncertainty](#).

4. Enter the **Chip Level Sigma** and **Component Count** values. Then select **Statistical Uncertainty** values from the given list or fill a custom value in the blank field on the right. For less uncertainty, the sample size required becomes larger. The **Chip Sigma Value** can range in between 1.0-6.5.

Configuring Advanced Settings

Use the following procedure to configure advanced settings for MC:

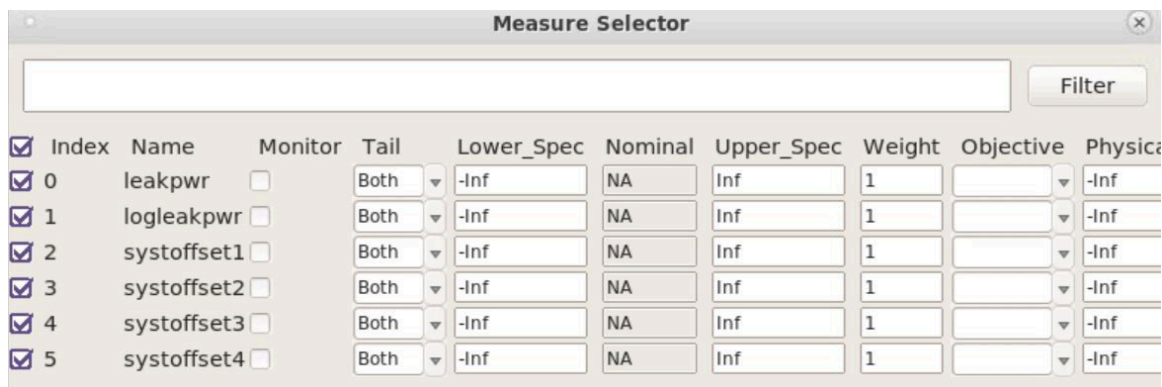
1. Click the **Advanced Settings** button to open the **Advanced Settings** dialog box.

Figure 24 MC Advanced Settings: Measure Selector



Click **Measure Selector**.

Figure 25 Measure Selector



Use steps 2–4 to configure measures for the Monte Carlo analysis.

2. Select/deselect the measures for your analysis by using the check box adjacent to the measures. All the reports and plots generated by MC contain only the selected measures.

Caution:

The runtime of the analysis increases with the increase in the number of selected measures.

3. If you require the quantiles in only the left or the right tail, select **Left Tail** or **Right Tail**, respectively. The default is **Both Tails**.

Selecting a single tail saves some computing time.

For more information, see [Single Tail Measure Support](#).

4. Enter the lower and upper limits for the selected measures in the **Lower Spec** and **Upper Spec** fields, respectively.

Note:

- This step is optional and, if the specification limits are provided, they are displayed on the graphic outputs.
- The **Monitor** and **Objective** fields are not applicable to the current release.

5. Click **Save** to save the settings.

Using General Tab

1. Click **General**, the General settings screen appears. This includes different ways of defining the variations, outputs creation, sampling and sensitivity methods employed during AVA simulation

Figure 26 MC Advanced Settings: General

General

Variation

Ignore Global Variation ☐

Ignore Local Variation ☐

Compare Against Golden Reference Results

Automatic Validation ☐

Output File:

Output Controls

☒ Q-Q plot in pdf format

☒ Q-Q plot in Waveview format

☒ Q-Q plot in CSV format

☒ Nominal simulation files

☒ Details file

☒ Critical Sweep Ext Number of Min/Max Critical Points

☐ Critical Sweep Waveform

Sensitivity Report Options ?

☒ Generate Sensitivity Report

Sensitivity Method : ☒ Classic ☐ Fast

Screening Method : ☒ Pearson ☐ Spearman

Save Cancel

2. Select the **Ignore Global Variation** or **Ignore Local Variation** according to the specified variation definition.
3. In **Output Controls**, you can select/deselect the output files/formats you want the MC simulation to generate. By default, all MC outputs are generated. You can deselect some of these to reduce storage requirements, especially for script-based computations on a large number of circuits.

For more information see, [Monte Carlo Outputs](#).

The the .mc0 file consists of the list of IRVs used for all samples. By default, this is unchecked. Select the **mc0 File** option, to save the file.

Number of Min/Max Critical Points is used to configure number of critical points. It supports values between 1 and 5 (default is 1).

4. In the **App Specified Options (Monte Carlo)**, select the required sampling methods such as **SRS**, **Sobol**.
5. In the **Sensitivity Report Options**, select the required option to enable sensitivity of the .ava.report file. This can be further customized to save in CSV format. Also, in this option you can add or modify specific details.

Use **Classic** or **Fast** Sensitivity methods to perform the sensitivity analysis. If there are composite devices used, this may create separate contributions for each component within. In such case, specify an input the .json file of composite devices. Then, the PrimeSim Design Robustness tool utilizes group based fast sensitivity method and treats the composite device as a single component. For more information, see [Understanding Fast Sensitivity](#).

6. Check the Design Specific Variation Corner box to output the IRVs of tail samples for each measure in the netlist, based on the selected left/right/both tails.

Check the critical sweep mc0 box to output critical corner sweep indices and corresponding IRVs in MC0 file format. Critical corners include min/max measure values or failed measures and other quantiles specified by user. The mc0 file is created with the extension out.ava.critical.mc0. By default, this check box is enabled.

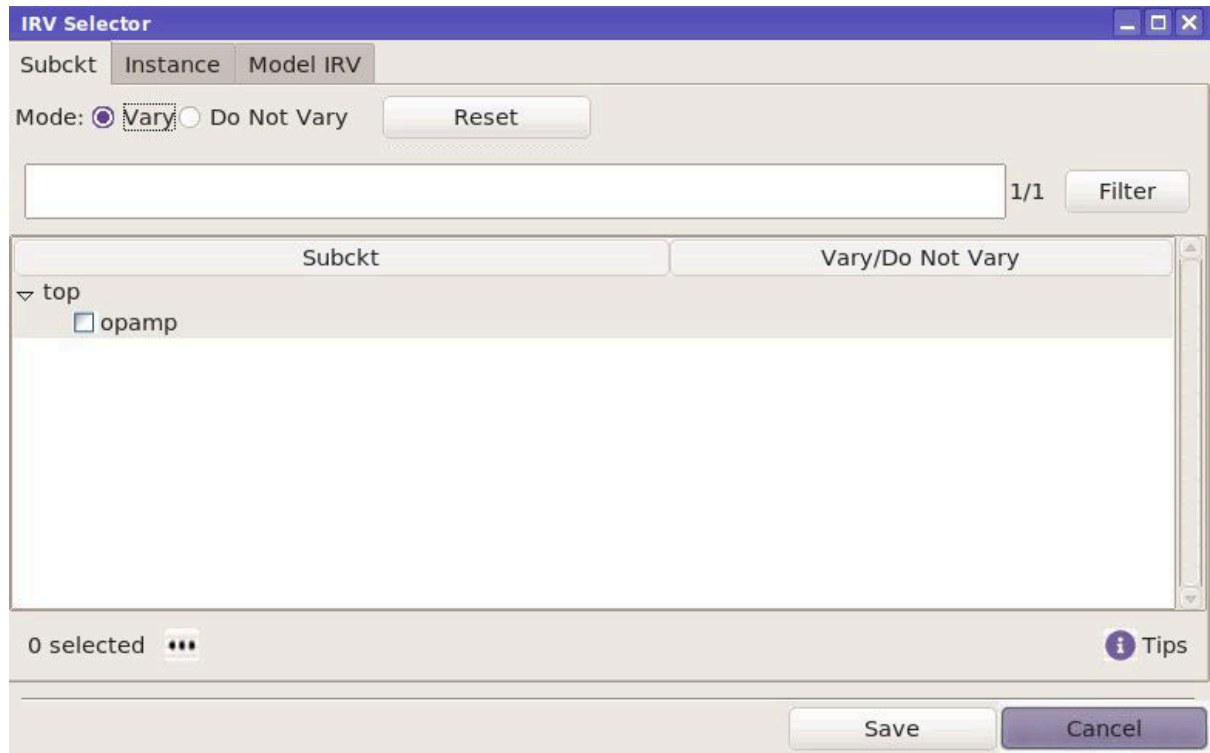
Check the Critical Waveform file box to automatically launch corresponding PrimeSim simulator to simulate the critical samples generated in above out.ava.critical.mc0 file. The waveform file is generated only if the netlist contains probe statements. By default, this option is not enabled.

7. Click **Save** to save the settings.

Using IRV Selector

1. Click **Advanced Settings > IRV Selector** to configure options to limit the variation to certain parts (models, subcircuits, and instances) of the design.
2. In the **Subckt** area (see [Figure 27](#)), select/deselect subcircuits in your design and apply options to vary only or not vary the selected subcircuits.

Figure 27 MC Advanced Settings: IRV Selector > Subckt



3. In the **Instance** area (see [Figure 29](#)), select/deselect instances in your design and apply options to vary only or not vary the selected instances.
4. In the **Model IRV** area (see [Figure 28](#)), select/deselect models in your design and apply options to vary or not vary the selected model random variables.

Figure 28 MC Advanced Settings: IRV Selector > Model IRV

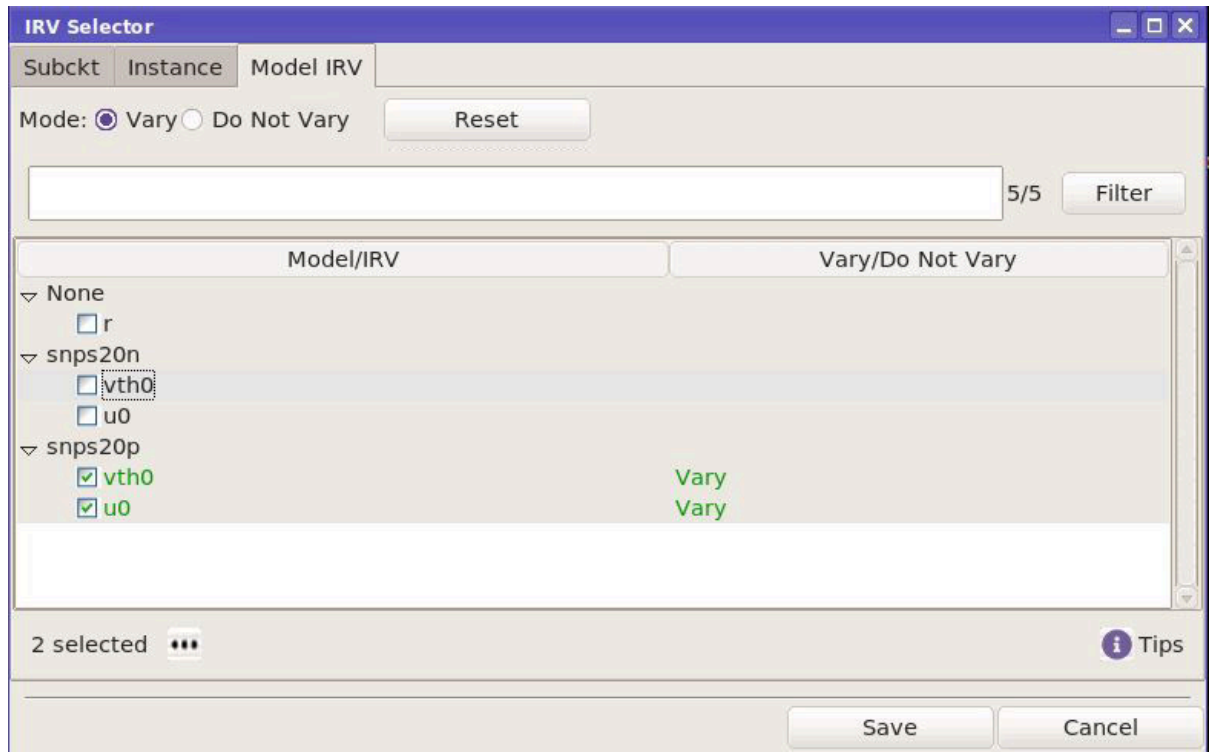


Figure 29 MC Advanced Settings: IRV Selector > Instance

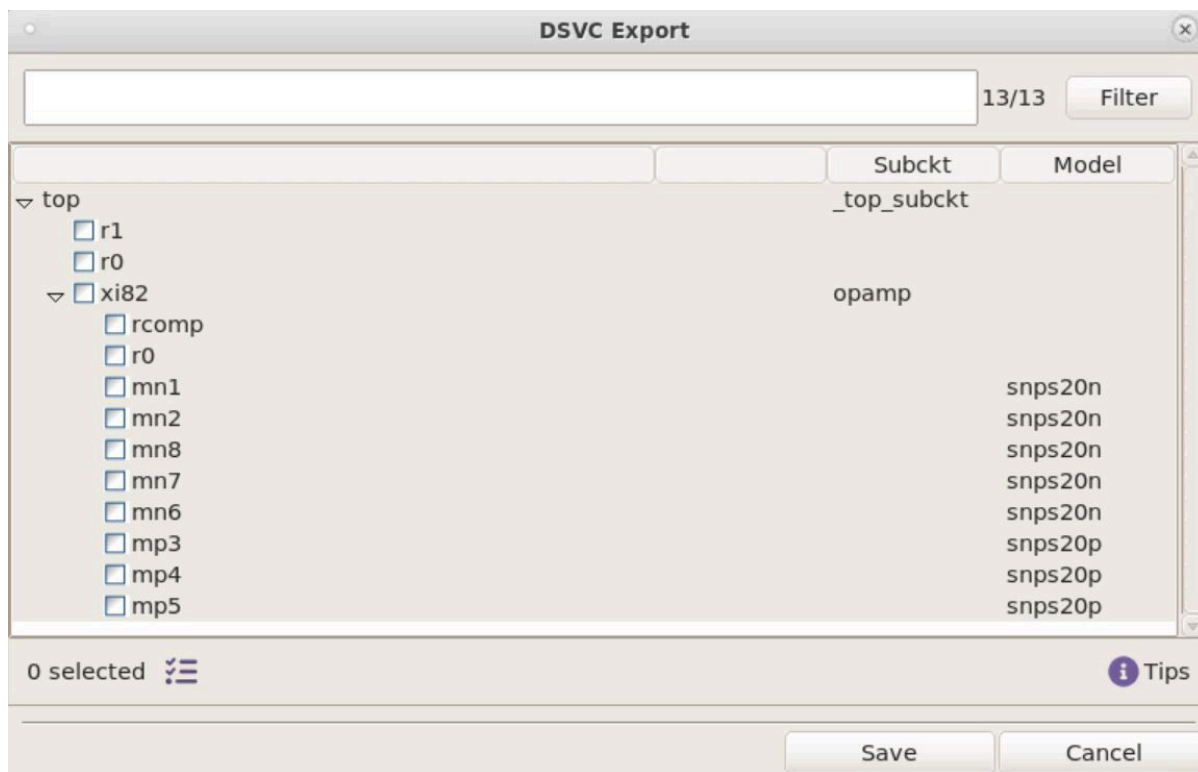


- You can select a specific block for which the IRV information is required using the **DSVC Selector** tab (see [Figure 30](#)).

The outputs are listed under out.ava.dsvc.json file.

For more information on the output file, see [Monte Carlo Outputs](#).

Figure 30 DSVC Selector



- When you are finished configuring the advanced settings, close the **Advanced Settings** dialog box.

Note:

For very large netlists, the design tree parsing might take time. In such cases, **General** tab of the **Advanced Settings** only is enabled and the other tabs remain inactive. Once parsing completes, the remaining tabs are enabled.

Saving the Configuration

To save a newly-created MC configuration information, in the MC dialog box, click **Save and Launch** and specify a file name for the configuration file (extension `.json` is added by default) before launching the simulation.

Save and Quit saves the configuration information and closes the application without running the simulation.

Monte Carlo Outputs

MC analysis produces the following outputs:

- [Nominal Simulation Files](#)
- [Q-Q Plots in PDF, WaveView, and CSV Formats](#)
- [The *.ava.report File](#)
- [*.ava.details.json File](#)
- [*.ava.dsvc.json File](#)
- [*.ava.critical File](#)
- [*.ava.hm File](#)
- [out.ava.sens.csv](#)
- [out.ava.irv_corr.csv](#)
- [Netlist Examples](#)

Nominal Simulation Files

While running PrimeSim AVA simulation, the simulation outputs such as .lis, .log, .ms0 files are also generated. These files contain information relating to invocation, distributed processing, netlist details and other information parsed.

- .lis file: PrimeSim HSPICE log file
- .log file: PrimeSim SPICE/PrimeSim XA log file
- .m(s/t/d)0: Measure file
- .mc0: IRV file

Q-Q Plots in PDF, WaveView, and CSV Formats

The PrimeSim AVA tool generates Q-Q plots in PDF (*.ava.qq.pdf file), WaveView (*.ava.qq.wv_plt file), and ASCII CSV (**.ava.qq.csv).

- [PDF Q-Q Plots](#)
- [WaveView Q-Q Plots](#)
- [CSV Q-Q File](#)

PDF Q-Q Plots

The PDF contains the Q-Q plot for each measure included for the analysis.

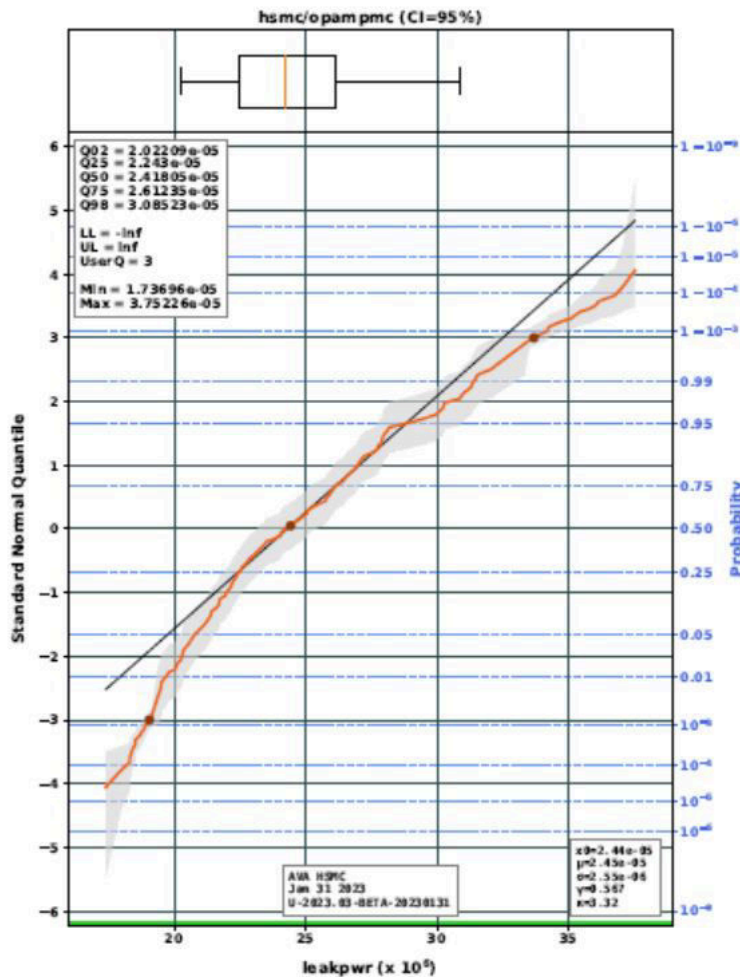
[Figure 31](#) shows a sample PDF Q-Q plot output of MC analysis.

In the Q-Q plot, the X-axis corresponds to a particular measurement. The Q-Q plot displays dual Y-axes. The left Y-axis shows the probabilities in terms of the standard normal quantiles. The right Y-axis shows the corresponding cumulative probabilities.

The orange line is the Q-Q plot curve for the given measure. The grey shaded area in the PDF plot gives the 95% confidence intervals for the Q-Q plot. The diagonal line, called the Q-Q line, is a visual guide showing where data from a normal distribution would lie.

The bottom right box provides the values for the nominal value x_0 , and the first four central moments (mean μ , standard deviation σ , skewness γ , and kurtosis κ).

Figure 31 Example of a PDF Q-Q Plot



If the MC configuration included lower and upper specifications for the measure, they are displayed as vertical lines in the Q-Q plot.

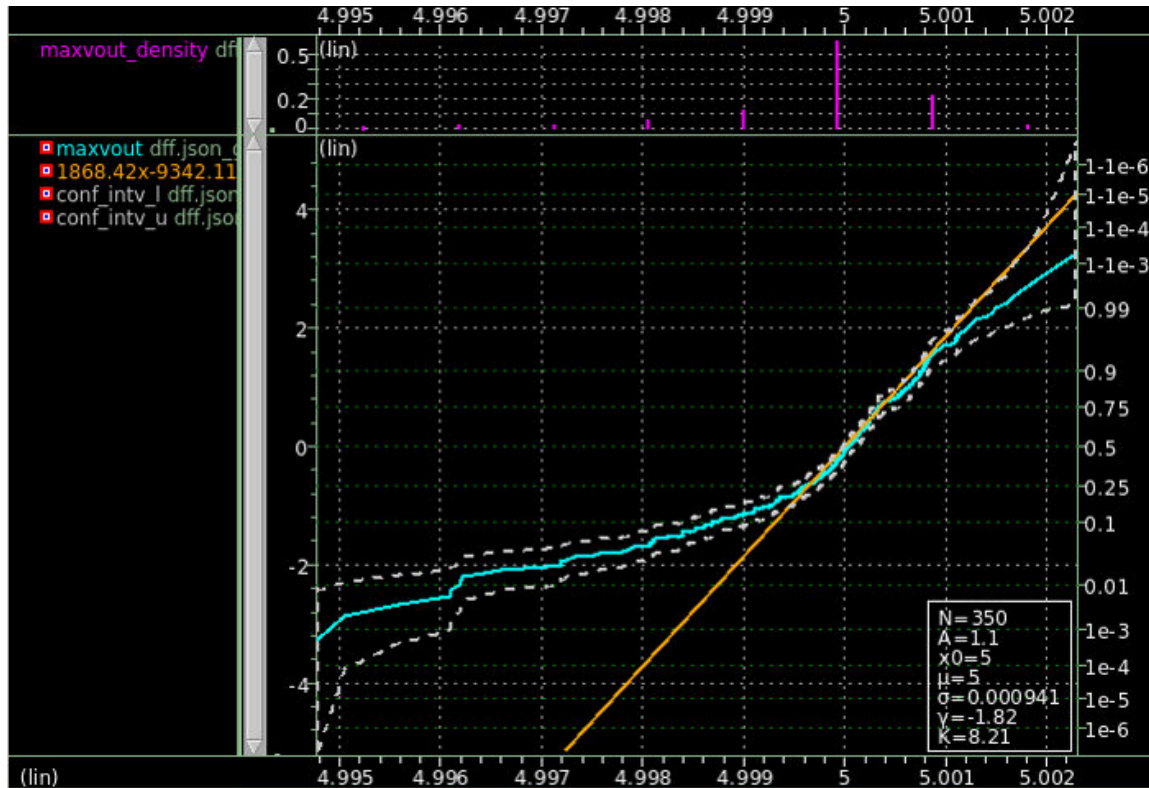
If these limits were not specified during configuration, no limit information is displayed on the Q-Q plot, as in the example shown.

WaveView Q-Q Plots

Figure 32 shows an example of a WaveView Q-Q plot. When working with the `*.ava.qq.wv_plt` file in WaveView, you can interactively select measures for the Q-Q plot display. WaveView supports pan, zoom, and dynamic measurements based on

the cursor location. In addition, appearing above the Q-Q plot panel in WaveView, is the corresponding probability density plot, displayed for reference purposes.

Figure 32 Example of a Q-Q Plot in WaveView



CSV Q-Q File

The CSV format Q-Q file is an ASCII file with information on the data points required to create the Q-Q plot of the measures selected in the run. Each point is the measure quantile plotted against the standard normal quantile. Upper and lower confidence

intervals can also be plotted for any given measure using the last two columns QL and QU. For ease of use, this file is generated in the CSV format.

Figure 33 An Example of a CSV Format Q-Q File

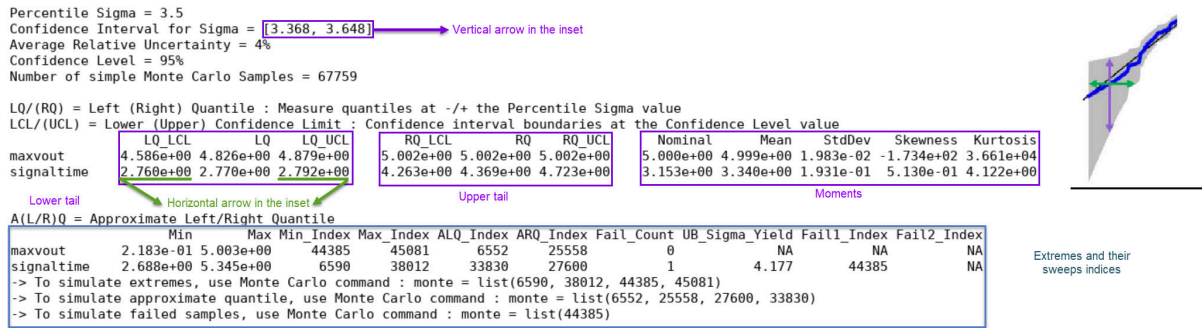
```
#QQnorm plots from data
# Not suitable for quantitative analysis
#Number of plot points
81
#Number of Measures
6
#Measure moments
Statistic,leakpwr,logleakpwr,systoffset1,systoffset2,systoffset3,systoffset4
Nominal,2.441e-05,-10.621,0.0013997,0.0010863,0.00090467,0.00077089
Mean_CI(2.5%),2.43e-05,-10.63,0.0013098,0.0010719,0.00090503,0.00076918
Mean,2.4459e-05,-10.625,0.0013818,0.0011077,0.00093057,0.00078702
Mean_CI(97.5%),2.4613e-05,-10.618,0.0014569,0.0011407,0.00095225,0.000805
Median_CI(2.5%),2.4036e-05,-10.64,0.0012919,0.0010684,0.00089811,0.00075778
Median,2.4181e-05,-10.631,0.001324,0.0011212,0.00094672,0.00077089
Median_CI(97.5%),2.4508e-05,-10.627,0.001433,0.0011372,0.00095849,0.00079982
StdDev_CI(2.5%),2.4322e-06,0.092983,0.0010866,0.00053814,0.00036423,0.00026102
StdDev,2.5509e-06,0.097316,0.0011397,0.00056743,0.00037983,0.00027531
StdDev_CI(97.5%),2.6658e-06,0.10197,0.0011952,0.00059099,0.00039702,0.00028838
MAD_CI(2.5%),NA,NA,NA,NA,NA,NA
MAD,NA,NA,NA,NA,NA,NA
MAD_CI(97.5%),NA,NA,NA,NA,NA,NA
Skewness_CI(2.5%),0.38287,0.25616,-0.19406,-0.097343,-0.11838,-0.11782
Skewness,0.56713,0.39507,-0.007669,0.060131,0.055325,0.041457
Skewness_CI(97.5%),0.72355,0.54259,0.16363,0.21041,0.21838,0.2672
Kurtosis_CI(2.5%),2.8196,2.8686,2.8459,2.8388,2.7248,2.9718
Kurtosis,3.3172,3.183,3.154,3.1381,3.0279,3.3897
Kurtosis_CI(97.5%),4.0093,3.5816,3.5839,3.4937,3.3398,4.3002

# Q/QL/QU refer to the Quantile and its Lower/Upper confidence limits in terms of sigma
# LCL/(UCL) = Lower (Upper) Confidence Limit : Confidence interval boundaries at the Confidence Level value in
terms of measure
#Measure names
Q,QL,QU,leakpwr,LCL(leakpwr),UCL(leakpwr),logleakpwr,LCL(logleakpwr),UCL(logleakpwr),systoffset1,LCL(systoffset
t1),UCL(systoffset1),systoffset2,LCL(systoffset2),UCL(systoffset2),systoffset3,LCL(systoffset3),UCL(systoffset
3),systoffset4,LCL(systoffset4),UCL(systoffset4)
-4,-5.2282,-3.482,1.7495e-05,<1.737e-05,1.8295e-05,-10.954,<-10.961,-10.909,-0.0034475,<-0.0035963,-0.0027173,
-0.0012661,<-0.0013419,-0.00089887,-0.00064626,<-0.00069909,-0.00040079,-0.00038743,<-0.00042888,-0.00020716
-3.9,-4.8299,-3.4498,1.7714e-05,<1.737e-05,1.8313e-05,-10.941,<-10.961,-10.908,-0.0031868,<-0.0035963,-0.00269
69,-0.0011334,<-0.0013419,-0.00089316,-0.00055372,<-0.00069909,-0.00039735,-0.00031483,<-0.00042888,-0.0002029
```

The *.ava.report File

The output *.ava.report file is a HSMC report file with moments, lower and upper quantiles and their confidence intervals, extremes, and sensitivities. [Figure 10](#) and [Figure 12](#) describe the outputs in more detail.

Figure 34 Quantiles, Moments, and Extremes (*.ava.report file)



*.ava.details.json File

Figure 35 provides a fragment of the output *.ava.details.json file. The input settings and derived quantities are shown in blue, the outputs in green, and the golden reference comparisons in red.

Figure 35 *.ava.details.json* File

```

...
"Measure_Names": ["delay_a1_1h_x_hl", "tout_x_hl"],
"Sigma": 4.5,
"Quantiles": [[4.732765, 5.672219], [2.156925, 2.540678]],
"Quantiles_Error_Percent": [[0.0, 0.0], [0.0, 0.0]],
"Quantiles_Error_Absolute": [[0.0, 0.0], [0.0, 0.0]],
"Moments": [[5.176, 0.1043, 0.08657, 3.016], [2.339, 0.04307, 0.08865, 2.961]],
"Moments_Error_Percent": [[-0.07233, -0.59805, 2.43914, -0.00412], [-0.10655, -0.32141, 1.84272, 1.72104]],
"Moments_Error_Absolute": [[-0.003743567, -0.006237653, 0.002111564, -0.0001242247], [-0.002492201, -0.0001384302, 0.010498857, 0.0509601]],
"N_Simple_Monte_Carlo_Samples": 1918418,
"Average_Relative_Uncertainty": 4.0,
"Left_Tail_Indices": [[1748297, 588089, 350510, 965371, 231850, 1144189, 1125081, 1208155], [1748297, 350510, 588089, 241853, 1125081, 1314659, 1636777, 690157]],
"Left_Tail_Values": [[4.672444, 4.700288, 4.707932, 4.720642, 4.723728, 4.731041, 4.732764, 4.732843], [2.126085, 2.142005, 2.149849, 2.15028, 2.151287, 2.152433, 2.156917, 2.157331]],

```

The left/right tail indices and their corresponding values are listed in this file. If you require additional debugging for a given tail sample, you can use these indices to re-simulate the netlist using a normal MC analysis and the `list()` syntax.

You must choose the indexes according to the measures of interest, since the *.ava.details.json file displays the tail samples of every measure separately. The entries have an ordering from the extreme end of the distribution, inward.

*.ava.dsvc.json File

This is an output file of JSON format, as part of first step of DSVC simulation. On selecting the required instances in the **Advanced Settings > DSVC export** tab, this file is created.

It contains the information of tail indices and Valid Variation corners of each measure. Then, for each such corner, the IRVs of the selected instances are listed out along with their MC index.

This file is used as an input to DSVC Import analysis, where created IRVs are mapped to selected cells.

Figure 36 Example: out.ava.dsvc.json File

```

# ***** Advanced Variation Analysis *****
# Copyright (c) 2018 - 2023 by Synopsys, Inc. All Rights Reserved.
# Notice: This file format and content are subject to change.
{
  "Copyright" : "***** Advanced Variation Analysis *****\nCopyright (c) 2018 - 2023 by Synopsys, Inc. ...
  "Valid_Variation_Corners" : ["maxvout.Min",
    "maxvout.Max",
    "maxvout.QL",
    "maxvout.QR",
    "maxvout.Min2",
    "maxvout.Min3",
    "maxvout.Min4",
    "maxvout.Min5",
    "maxvout.Max2",
    "maxvout.Max3",
    "maxvout.Max4",
    "maxvout.Max5",
    "signaltime.Min",
    "signaltime.Max",
    "signaltime.Max5"],
  "Exported_Instances" : ["xi3.xi61"],
  "DSVC_Data" : [{
    "Variation_Corners" : ["maxvout.Min"], "Index" : 556500, "Local_Variables" : {
      "xi3.xi61.mb2" : {
        "model" : "nmos", "params" : {
          "vth0" : [-0.0728383499977937, "iln"], "u0" : [5.102736190344414, "iln"]
        }
      }, "xi3.xi61.mb1" : {
        "model" : "pmos", "params" : {
          "vth0" : [0.5308826040809711, "iln"], "u0" : [1.1084840973201493, "iln"]
        }
      }
    }
  },
  {
    "Variation_Corners" : ["maxvout.Max"], "Index" : 1108505, "Local_Variables" : {

```


*.ava.critical File

This file contains critical corner sweep indexes and corresponding IRVs in MC0 file format. Critical corners include min/max measure values or failed measures and other quantiles that you specified.

Figure 37 Example: .ava.critical File

```
***** Advanced Variation Analysis *****
Copyright (c) 2018 - 2023 by Synopsys, Inc. All Rights Reserved.

This software and the associated documentation are proprietary
to Synopsys, Inc. This software may only be used in accordance
with the terms and conditions of a written license agreement with
Synopsys, Inc. All other use, reproduction, or distribution of
this software is strictly prohibited.
AVA Version: V-2023.12-VAL-20230613
AVA Build time: Tue Jun 13 12:22:40 2023
*** File format subject to change ***
Option hsp_ava_flow=1
Critical points

Option MeasOut=1 PutMeas=1 Post=1 Probe=1 McBrief=1
Option Sampling_Method=External Set_Missing_Values=Zero Block_Name=cp_data
data cp_data
index xi3.xi61.mb2:@:nmos:@:vth0:@:iln xi3.xi61.mb2:@:nmos:@:u0:@:iln
xi3.xi61.mb1:@:pmos:@:vth0:@:iln xi3.xi61.mb1:@:pmos:@:u0:@:iln
Critical_Point: nominal
1 0 0 0
Critical_Point: maxvout.min4 (sweep index 172489)
2 1.9908 1.9372 1.5456 0.4650
Critical_Point: maxvout.min3 (sweep index 361500)
3 1.1712 2.5343 -0.7811 3.3355
Critical_Point: maxvout.min5 (sweep index 366596)
4 -1.0442 -1.0867 -1.4020 3.6754
Critical_Point: signaltime.max (sweep index 442630)
5 -0.8788 -0.8080 0.5232 -4.3041
Critical_Point: maxvout.max2 (sweep index 483506)
6 -0.2954 -3.0894 -2.8777 2.0444
Critical_Point: signaltime.min2 (sweep index 485465)
7 2.9898 -1.8255 -3.2999 0.3456
Critical_Point: maxvout.max5 (sweep index 540184)
8 0.0199 -0.5670 -4.4595 -1.8623
Critical_Point: maxvout.min (sweep index 556500)
9 -0.0728 5.1027 0.5309 1.1085
Critical_Point: signaltime.min5 (sweep index 589229)
10 -0.3134 2.2447 -1.1330 -4.4068
.....
enddata
```

Critical Points

Critical points are Monte Carlo sweep points or a set of IRV values that are associated with specific response properties. High Sigma Monte Carlo Analysis (HSMC), Parametric yield estimation (PYE), Sigma Amplification (SA), or Multiple Corner analysis (MC)

applications generate external sampling file for critical points. To run MC simulation on specific critical points, simulator needs to use the IRV values of critical points. The .ava.critical and DSVC files contain IRV values for MC simulation.

Critical point notation

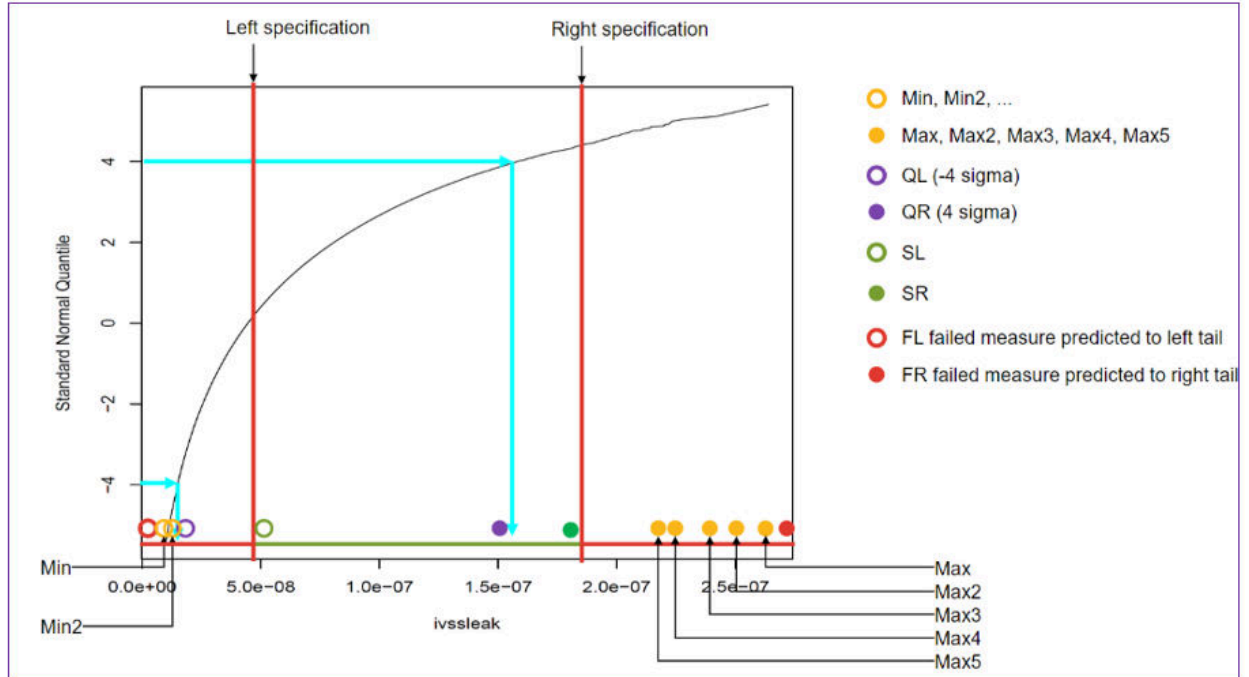
Critical point notation is used in .ava.critical and ava.dsvc.json files.

- Minimum: Min
- Maximum: Max
- Quantile Left: QL
- Quantile Right: QR
- Spec Left: SL
- Spec Right: SR
- Fail Left: FL
- Fail Right: FR
- * 2nd Minimum: Min2
- * 3rd Minimum: Min3
- * 4th Minimum: Min4
- * 5th Minimum: Min5
- * 2nd Maximum: Max2
- * 3rd Maximum: Max3
- * 4th Maximum: Max4
- * 5th Maximum: Max5

The properties marked with * are controlled by hyper-parameter `Number_Min_Max_Critical_Points` in input JSON, the hyper-parameter is used to configure number of critical points. It supports values between 1 and 5 (default is 1).

Critical Points in QQ Plot

Figure 38 Critical Points in QQ Plot



*.ava.hm File

This is a health monitor file that lists the resources used and reports the progress of the simulation in real time.

For the PrimeSim Design Robustness tool, the health monitor file lists the number of Monte Carlo samples completed in real time. This feature is applicable only to Monte Carlo and Sigma Amplification applications. The number of samples are listed under the column: *MC samples done*.

Figure 39 Example: *.ava.hm File

```

Hostname: ne[redacted]
N cores: 20
Model name: I[redacted] (R) CPU E5-2640 v4 @ 2.40GHz
Cache size: 25600 KB
OS kernel: Linux 3.10.0-1127.10.1.el7.x86_64
OS release: CentOS Linux release 7.3.1611 (Core)
CPU load: 0.600000, 0.470000, 0.240000
Virtual memory: Total=503.6G, Available=497.9G, Percent=1.1, Used=4.0G, Free=428.9G,
Active=25.2G, Inactive=44.8G, Buffers=2.5M, Cached=70.7G, Shared=386.1M, Slab=1.5G

```

Chapter 6: Monte Carlo Analysis

Monte Carlo Outputs

```

/tmp file space:
Filesystem      1G-blocks  Used Available Use% Mounted on
/d/             582      12      570    3% /

/SCRATCH file space:
Filesystem      1G-blocks  Used Available Use% Mounted on
/d/             838      1      838    1% /SCRATCH

Output directory /re/ /tests/py/fs_omega_hsmc/_16_test file space:
Filesystem      1G-blocks  Used Available Use% Mounted on
us/4:/HSP/2     1300    1013    288   78% /re/e2

Start time: 2021-08-03 08:03:24 1628003004.4
Total number of Monte Carlo samples to simulate = 350

```

elapsed time (s)	cpu load %	memory available (GB)	/tmp available (GB)	output available (GB)	estimated time_to finish	MC samples done	AVA %cpu	total rss	total vms
1	3.0	497.9	570.0	288.0	NA	NA	0.0	123.6M	1.1G
13	3.7	497.4	570.0	288.0	52s	40	395.5	738.9M	5.2G
26	7.3	497.4	570.0	288.0	28s	139	394.5	739.0M	5.2G
38	9.2	497.4	570.0	288.0	15s	236	394.6	739.0M	5.2G
50	12.0	497.4	570.0	288.0	1s	336	295.5	739.0M	5.2G

Notes:

- The state of resources in a grid environment can be difficult to predict/control. If the resources become extremely limited, the runtime of PrimeSim Design Robustness jobs can be severely impacted and the job has to be restarted on another machine.
- The PrimeSim AVA tool terminates an AVA job automatically when the machine resources meet the following criteria:
 - Free memory becomes less than 2GB for ~6 minutes.
 - Free storage space becomes less than 2GB for ~6 minutes.
 - Load average is more than three times the number of cores on the machine for ~6 minutes.

out.ava.sens.csv

The `out.ava.sens.csv` file contains sensitivity values of each measure with respect to IRVs present in that netlist. Use the **Output sensitivity table in CSV format** option to generate this file. It represents the **Variables Screened by Importance** section of `.ava.report` file in CSV format.

Figure 40 Example: .ava.report File

```
##Variables_Screened_by_Importance
#-----
Method: Pearson Correlation
#Print threshold: 95%
#(elements with EquivVar smaller than 0.05% will not be printed.)
#(parameters with ParVar smaller than 0.01% will not be printed.)

Response: log_pwr
Global Variation = 8.723e-03, Local Variation = 9.972e-02
Element,Model,EquivVar,CumVar,MatchGrp,Par,ID,Corr,ParVar
xi82.r0,r,65.05%,65.05%,,,,,,
,,,,,,r,ILN,-0.80,65.05%
xi82.mn8,snps20n,24.55%,89.60%,,,,,,
,,,,,,vth0,ILN, 0.47,21.77%
,,,,,,u0,ILN,-0.17,2.78%
xi82.mn6,snps20n,5.83%,95.43%,,,,,,
,,,,,,vth0,ILN,-0.23,5.10%
,,,,,,u0,ILN, 0.09,0.73%
```

out.ava.irv_corr.csv

The out.ava.irv_corr.csv file contains the information of aggregated sensitivity values of each instance, device and IRV parameter separately for each measure. The detailed sensitivity report is supported when Sensitivity Method in **Advanced Settings > General** is set to **Fast**. Enable the **Output Detailed Sensitivity File** option to generate the out.ava.irv_corr.csv file.

Figure 41 Example: *out.ava.irv_corr.csv* File

```

### Higher levels of the hierarchy
Instance_Name,log_pwr(Corr),log_pwr(%),offset1(Corr),offset1(%),offset2(Corr),offset2(%),offset3(Corr),offset3(%),offset4(Corr),offset4(%),pwr(Corr),pwr(%),
xi82,0.9884,98.14,1.0245,99.81,1.0250,99.80,1.0251,99.80,1.0252,99.80,0.9839,98.21

### Transistor level aggregation
Instance_Name,log_pwr(Corr),log_pwr(%),log_pwr(Grp),offset1(Corr),offset1(%),offset1(Grp),offset2(Corr),offset2(%),offset2(Grp),offset3(Corr),offset3(%),offset3(Grp),offset4(Corr),offset4(%),offset4(Grp),pwr(Corr),pwr(%),pwr(Grp)
r0,0.0568,0.32,0,-0.0133,0.02,0,-0.0128,0.02,0,-0.0127,0.02,0,-0.0127,0.02,0,0.0577,0.34,0
r1,-0.0877,0.77,0,-0.0226,0.05,0,-0.0223,0.05,0,-0.0223,0.05,0,-0.0222,0.05,0,-0.0824,0.69,0
xi82.mn1,0.0134,0.02,0,0.5273,26.45,1,0.5315,26.83,1,0.5327,26.94,1,0.5335,27.02,1,0.0276,0.08,0
xi82.mn2,0.0134,0.02,0,0.5579,29.60,1,0.5619,29.99,1,0.5632,30.12,1,0.5636,30.16,1,0.0158,0.03,0

### Stack parameter level aggregation
## Composite Devices

## Individual devices
Param_Name,log_pwr(Corr),log_pwr(%),offset1(Corr),offset1(%),offset2(Corr),offset2(%),offset3(Corr),offset3(%),offset4(Corr),offset4(%),pwr(Corr),pwr(%),
xi82:@:r:@:ILN,0.8049,65.09,0.0518,0.25,0.0521,0.26,0.0522,0.26,0.0522,0.26,0.8062,65.93
xi82:@:snps20n:@:u0:@:ILN,0.1985,3.96,0.2379,5.38,0.2370,5.34,0.2374,5.35,0.2374,5.35,0.1944,3.83
xi82:@:snps20n:@:vth0:@:ILN,0.5274,27.95,0.7336,51.18,0.7401,52.03,0.7418,52.26,0.7427,52.38,0.5208,27.52
xi82:@:snps20p:@:u0:@:ILN,0.0712,0.51,0.2076,4.10,0.1996,3.79,0.1974,3.70,0.1963,3.66,0.0613,0.38
xi82:@:snps20p:@:vth0:@:ILN,0.0798,0.64,0.6395,38.90,0.6358,38.40,0.6345,38.23,0.6339,38.15,0.0735,0.55

### IRV Correlation Coefficients

### Global_Variation,0.76%,0.12%,0.14%,0.14%,0.14%,0.76%
IRV_Name,log_pwr(Corr),log_pwr(%),offset1(Corr),offset1(%),offset2(Corr),offset2(%),offset3(Corr),offset3(%),offset4(Corr),offset4(%),pwr(Corr),pwr(%),
snps20n:@:u0:@:IGN,0.0653,0.43,0.0173,0.03,0.0187,0.03,0.0192,0.03,0.0194,0.04,0.0656,0.44
snps20n:@:vth0:@:IGN,-0.0494,0.25,0.0043,0.00,0.0038,0.00,0.0036,0.00,0.0036,0.00,-0.0484,0.24
snps20p:@:u0:@:IGN,0.0284,0.08,0.0069,0.00,0.0064,0.00,0.0061,0.00,0.0061,0.00,0.0290,0.09
snps20p:@:vth0:@:IGN,0.0062,0.00,0.0303,0.09,0.0319,0.10,0.0324,0.10,0.0327,0.10,0.0054,0.00

### Local_Variation,99.24%,99.88%,99.86%,99.86%,99.86%,99.24%
## Composite Devices
## Individual devices
IRV_Name,log_pwr(Corr),log_pwr(%),offset1(Corr),offset1(%),offset2(Corr),offset2(%),offset3(Corr),offset3(%),offset4(Corr),offset4(%),pwr(Corr),pwr(%),
r0:@:r:@:ILN,0.0568,0.32,-0.0133,0.02,-0.0128,0.02,-0.0127,0.02,0.0577,0.34
r1:@:r:@:ILN,-0.0877,0.77,-0.0226,0.05,-0.0223,0.05,-0.0223,0.05,-0.0222,0.05,-0.0824,0.69
xi82.mn1:@:snps20n:@:u0:@:ILN,0.0131,0.02,0.1842,3.23,0.1830,3.18,0.1832,3.19,0.1831,3.18,0.0207,

```

Netlist Examples

The opampmc.sp file is available at `Location:$installdir/demo/hspice/variability` (Operational amplifier for Monte Carlo).

Understanding Fast Sensitivity

For very low sample sizes, the classic sensitivity analysis occasionally falls short in terms of accuracy. In such cases, fast sensitivity analysis helps in creating a more accurate information of the contributions with help of machine learning techniques. This method is also helpful in cases when the simulation time is very long and or has large number of IRVs. Use the **Fast** option in the **Advanced Settings > General** to generate `<out>.irv.corr.csv` report . Fast sensitivity also supports the use of composite devices in the PrimeSim tool.

Parallel Processing

N_Core defines the number of cores on the machine.

Shell variable AVA is the `$Sim ... -adv user.json`

- AVA run in scalar mode
- For example, `hspice -i input.sp -o out -adv user.json`

Invocation Forms	Usage
<code>\$AVA</code>	Scalar mode AVA simulation
<code>\$AVA -master K</code>	Multi-core processing with min(N_Core, K) cores
<code>\$AVA -dp N</code>	Same as <code>-master K</code>
<code>\$AVA -master K -dp N</code>	Same as <code>-master</code> with min(N_Core, max(K, N)) cores
<code>\$AVA -dp N -dpconfig config_file</code>	Simulator based distributed processing with N cores Only Sigma Amplification and AVA based Monte Carlo are currently supported
<code>\$AVA -master K -dp N -dpconfig config_file</code>	Multi-core processing with min(N_Core, K) cores and simulator based distributed processing with N cores Only Sigma Amplification and AVA based Monte Carlo are currently supported and the master flag is ignored

Distributed Processing

Distributed processing supports large number of corners on compute farm (SGE, LSF, SSH, ..). Each corner is submitted to the farm and executed in parallel CDPL based distributed processing and it is compatible with PrimeSim input dpconfig file.

Input examples

- Run DP 200 through LSF farm

```
$SIM -i deck.cir -o output -adv mpvt_hsmc6.json -dp 200 -dpconfig  
lsf.ini
```

Contents of lsf.ini

```
1||-1||/tmp|LSF|bsub -app batch -R "span[hosts=1]  
select[cputype=emt64]" -n 3
```

- Run DP 200 through SGE farm

```
$SIM -i deck.cir -o output -adv mpvt_hsmc6.json -dp 200 -dpconfig  
sge.ini
```

Contents of sge.ini

```
1||-1||/tmp|SGE| qsub -P bnormal -l arch=glinux,os_bit=64,cputype=emt64
```

- Run DP 200 through SSH

```
$SIM -i deck.cir -o output -adv mpvt_hsmc6.json -dp 200 -dpconfig  
ssh.ini
```

Contents of ssh.ini

```
1|host1|-1||/tmp|SSH|ssh  
1|host2|-1||/tmp|SSH|ssh  
1|host3|-1||/tmp|SSH|ssh  
1|host4|-1||/tmp|SSH|ssh
```

- Run DP 16 on local host

```
$ SIM -i deck.cir -o output -adv mpvt_hsmc6.json -dp 16
```

Postprocessing

This chapter describes generating comprehensive reports using existing measure files and the PrimeSim AVA tool.

This section covers the following topics:

- [Introduction to the Postprocessing Utility](#)
- [Invoking Postprocessing Utility](#)
- [Configuring Inputs for Postprocessing](#)
- [Configuring Partial Measure for Postprocessing](#)

Introduction to the Postprocessing Utility

For a Monte Carlo simulation, the data generated is huge and analyzing each sample is impractical. Interpreting it with appropriate properties is the most essential task. The PrimeSim AVA tool creates comprehensive reports and highlights tail samples, quantiles and even provides design specific variation corners. This Monte Carlo report generation is not limited within the PrimeSim AVA tool. This requires the new measurements to be formatted in additional lines. For all such cases, PrimeSim AVA provides a standalone application for performing postprocessing as a utility. This utility gathers information from existing measure files and IRV files to create output files.

This feature is most helpful when you want to create a combined report from several measure files generated over multiple runs. It skips simulation part and performs postprocessing directly. PrimeSim AVA reports contain extensive data including statistical summary, correlated contribution of IRVs on elements, and design environment back-annotation information.

Note:

Throughout this chapter, all command-line invocations are denoted with `$SIM`. For the current chapter Postprocessing, `$SIM` refers to PrimeSim HSPICE, PrimeSim XA, PrimeSim SPICE, and PrimeSim Pro.

Invoking Postprocessing Utility

Use the following syntax to invoke the postprocessing utility through Advanced Variation Analysis dialog box:

```
hspice -adv <user.json> -o outfile
```

Inputs for the Postprocessing Utility

The inputs for using the postprocessing utility is a pointer to existing measure files and IRV files in the form of a `.json` file. This `.json` file can be created with the help of the dialog box or manually by user (with strict adherence to syntax). Within this `.json` file, further customization of reports can be done by specifying output format based options in **Advanced Settings**.

Use the following syntax to invoke the postprocessing utility.

```
hspice -adv user.json -o outfile
```

Argument	Description
-adv	After initial parsing and nominal simulation, invokes PrimeSim AVA advanced variation analyses.

Argument	Description
<code>user.json</code>	[Optional]: Input configuration file. Sample of <code>user.json</code> content for a post processing simulation: <pre>{ "Active_Application": "Utility", "Utility": { "Method": "Post_Processing", "Measure_Files" : ["input.ms0", "input.ms1", ...], "Sample_Files": ["input.mc0", "input.mc1", ...], ... } }</pre> If this configuration file is not provided, the PrimeSim AVA tool displays a dialog box to help you create one. See Configuring Inputs for MC.
<code>-o output</code>	Specifies the output prefix or folder.

Outputs for Postprocessing Utility

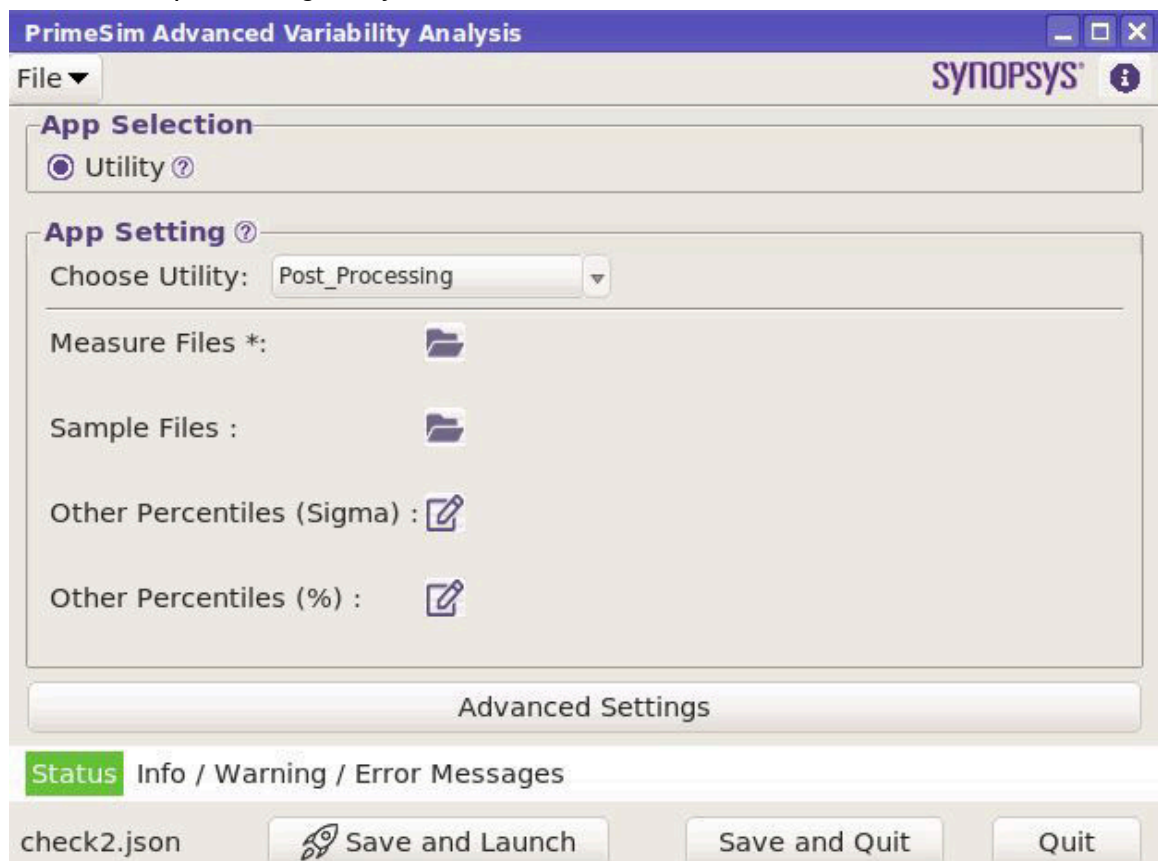
The postprocessing utility generates the following output.

File	Description	See Also
<code>out.ava.report</code>	Statistic report: moments, sensitivity table, and so on.	The *.ava.report File
<code>out.ava.annotate</code>	Annotate report	
<code>out.ava.qq.pdf</code>	Quantile data and plot in various forms.	Q-Q Plots in PDF, WaveView, and CSV Formats
<code>out.ava.log</code>	Run log: memory, elapsed time, and so on.	
<code>out.ava.sens.csv</code>	csv formatted sensitivity table	out.ava.sens.csv
<code>out.ava.irv_corr.csv</code>	Aggregated sensitivity report	out.ava.irv_corr.csv

Configuring Inputs for Postprocessing

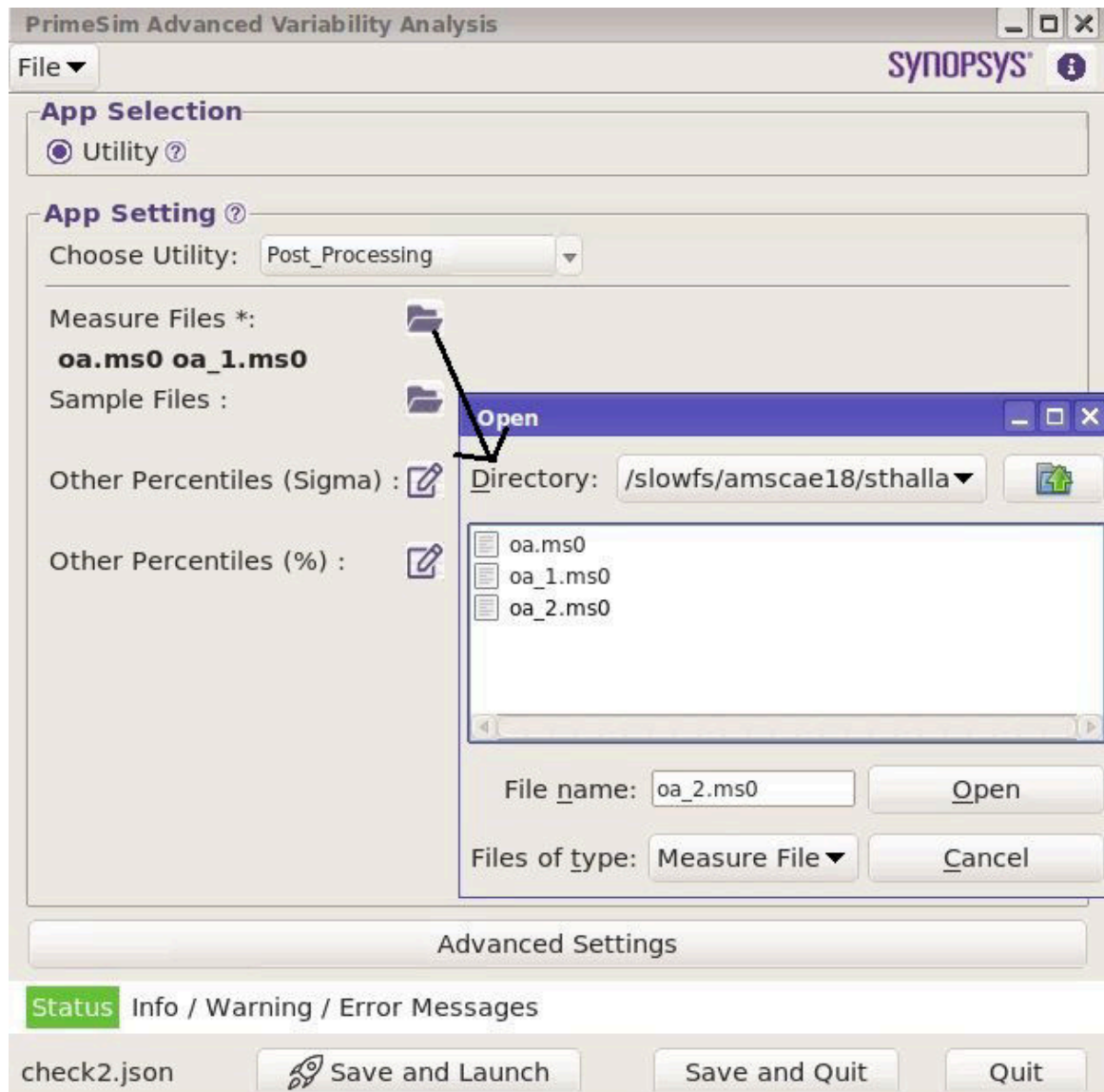
1. Use the `.json` file as an input as measure files for postprocessing. You can create this file using a dialog box.

Figure 42 Postprocessing Utility



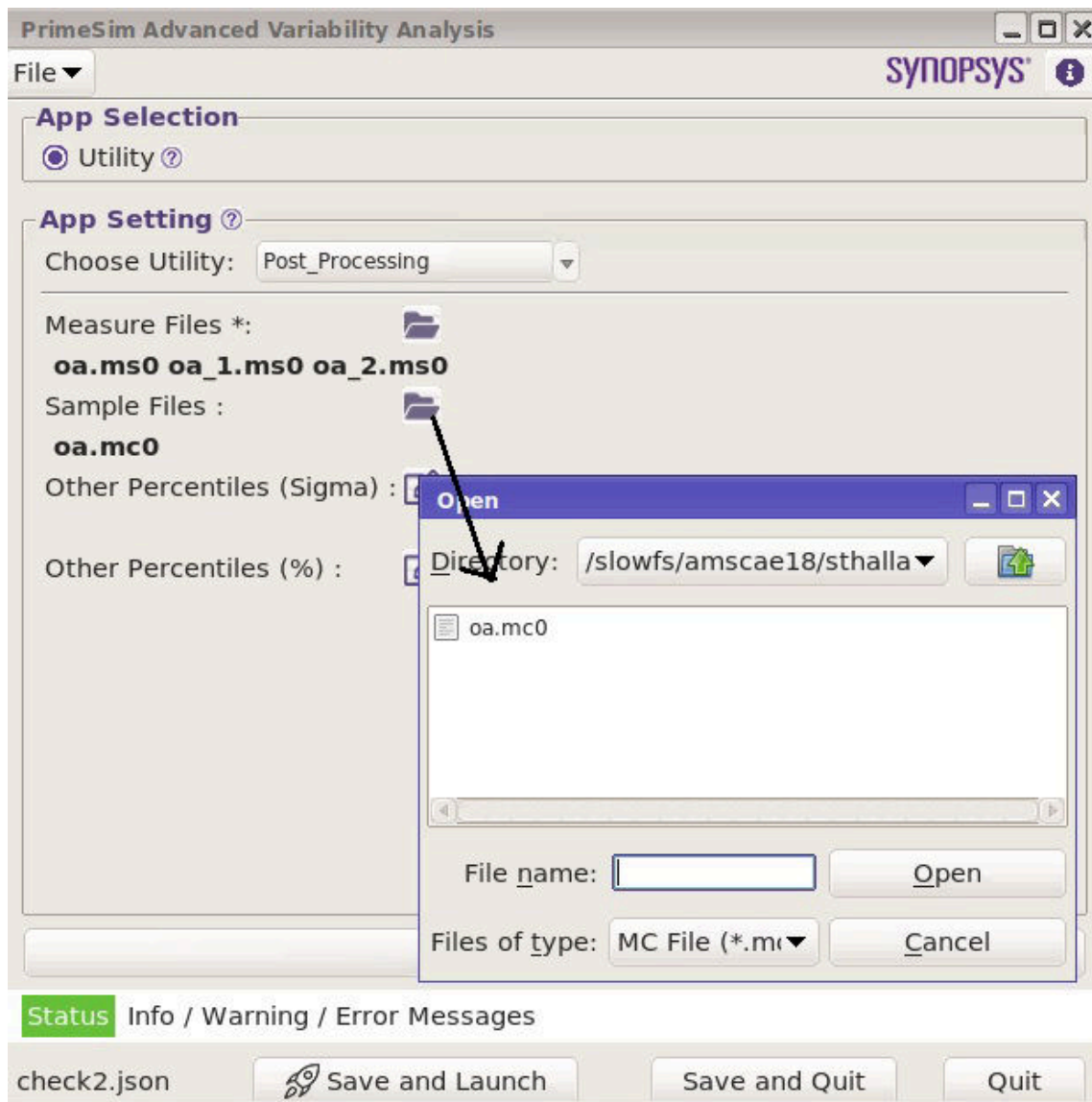
2. Choose the **Utility** option.
3. Select **Post_Processing** from the **Choose Utility** drop-down list.
4. Click the **File Browse** icon to select one or more measure files. Measure files of format `m(s/d/t)0` are supported.

Figure 43 Measure Files: Browser



Sample file facilitates the computation of sensitivity. Sample files refer to the list of samples/IRVs in the `mc0` files. You can generate sample files from other simulators and must be in `mc0` format.

Figure 44 Sample Files : Browser



5. (Optional) Specify the sigma or percentile values of interest as shown in [Figure 45](#) . The default values are shown in [The *.ava.report File](#). Use “+” icon on the top-right corner to add additional values and “-” beside each entry to remove them from the list. Because both quantities are dependent, when the former is filled, the latter is populated in the other column. During the simulation, if any of the user-defined percentiles fall out of range of available measures, they are ignored with a warning.

Figure 45 Other Percentiles (Sigma and %)

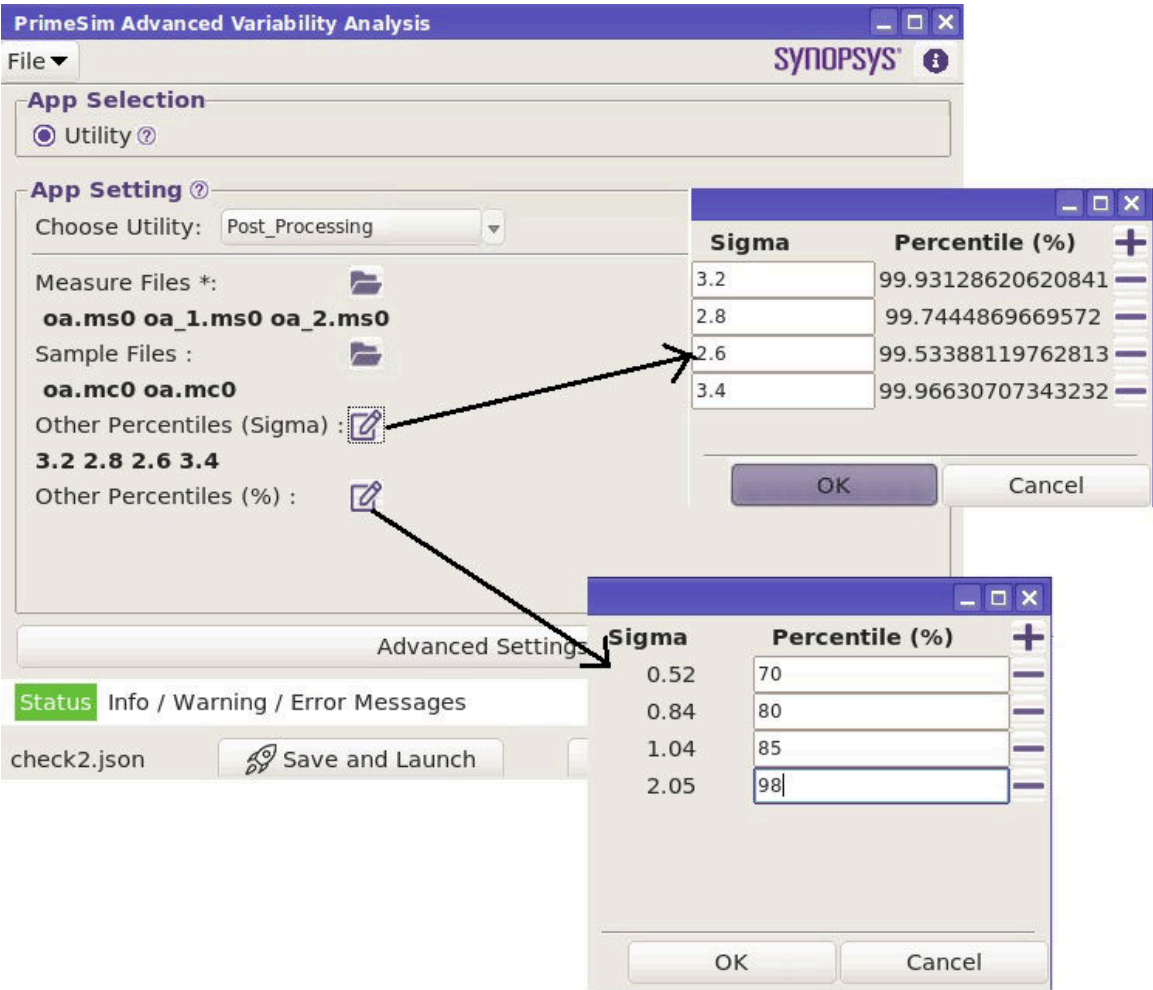
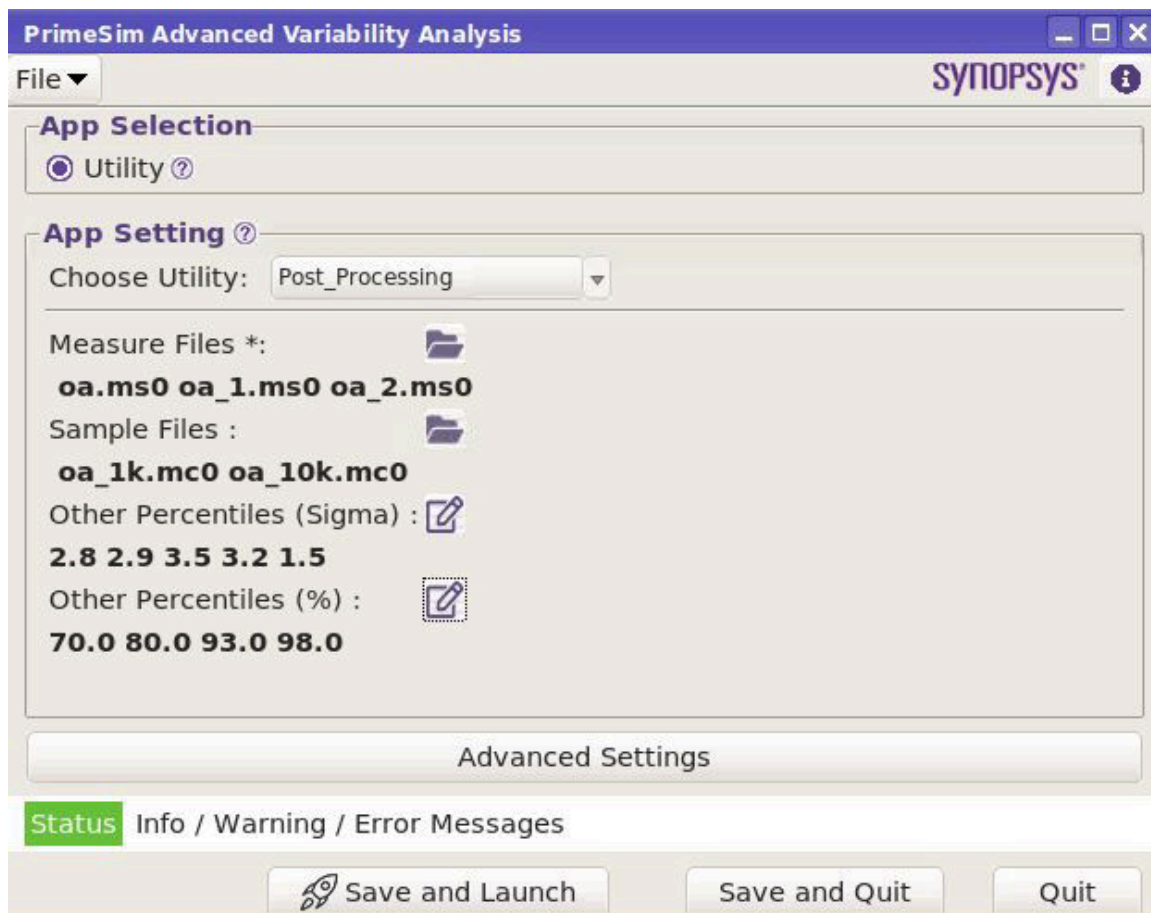


Figure 46 list all the specified inputs.

Figure 46 Inputs of Measure Files



Configuring Partial Measure for Postprocessing

You can select a subset of existing large columns of measure to perform post processing without editing the measure file or editing the original netlist and rerun the simulation.

Example of `user.json` content for a post processing simulation:

```
{
  "Active_Application": "Utility",
  "Utility": {
    "Method": "Post_Processing",
    "Measure_Files": [
      "output.ms0"
    ],
    "Selected_Measures": [ "systoffset", "vout" ]
  }
}
```

```
}  
}
```

Figure 47 Select partial measure

index	vout	vout1	vout2	vout3	vout4	vout5	sysoffset	sysoffset1	sysoffset2	sysoffset3	sysoffset4	sysoffset5	temper alter#
1	1.2486003	1.2486003	1.2486003	1.2486003	1.2486003	1.2486003	1.085698e-03	1.085698e-03	1.085698e-03	1.085698e-03	1.085698e-03	1.085698e-03	25.000000 1
2	1.2503058	1.2503058	1.2503058	1.2503058	1.2503058	1.2503058	2.232452e-04	2.232452e-04	2.232452e-04	2.232452e-04	2.232452e-04	2.232452e-04	25.000000 1
3	1.2486208	1.2486208	1.2486208	1.2486208	1.2486208	1.2486208	1.070163e-03	1.070163e-03	1.070163e-03	1.070163e-03	1.070163e-03	1.070163e-03	25.000000 1
14	1.2492776	1.2492776	1.2492776	1.2492776	1.2492776	1.2492776	7.659187e-04	7.659187e-04	7.659187e-04	7.659187e-04	7.659187e-04	7.659187e-04	25.000000 1
15	1.2488353	1.2488353	1.2488353	1.2488353	1.2488353	1.2488353	1.006762e-03	1.006762e-03	1.006762e-03	1.006762e-03	1.006762e-03	1.006762e-03	25.000000 1

Figure 48 Results of Postprocessing

	sysoffset	vout
Nominal	0.001085698	1.2486
Mean CI (2.5%)	0.001078976	1.248568
Mean	0.001091707	1.248593
Mean CI (97.5%)	0.001105332	1.248619
Median CI (2.5%)	0.001078803	1.248551
Median	0.001095288	1.248589
Median CI (97.5%)	0.001112176	1.248624
StdDev CI (2.5%)	0.0005263705	0.001056231
StdDev	0.0005369432	0.001075285
StdDev CI (97.5%)	0.0005481005	0.001096206
MAD CI (2.5%)	0.0005084025	0.00102507
MAD	0.000527047	0.001054573
MAD CI (97.5%)	0.0005460739	0.001094455
Skewness CI (2.5%)	-0.03489009	-0.08008203
Skewness	0.05478536	-0.02563155
Skewness CI (97.5%)	0.1434076	0.04106418
Kurtosis CI (2.5%)	2.942507	2.923883
Kurtosis	3.219945	3.038434
Kurtosis CI (97.5%)	3.619426	3.144251

You can select the measures for which the postprocessing is needed from **Advanced Settings > Measure Selector**. Use the **Filter** button to sort the measures and it also supports wildcard characters.

Figure 49 Output Controls



7

High Sigma Monte Carlo and Parametric Yield Estimation

This chapter describes the principles of High Sigma Monte Carlo(HSMC) simulation. It provides information on performing an HSMC simulation in the PrimeSim AVA tool and its outputs.

The following sections discuss these topics:

- [Introduction to HSMC](#)
- [Principles of HSMC](#)
- [HSMC — Target Applications](#)
- [Invoking HSMC](#)
- [Configuring Inputs For HSMC](#)
- [Single Tail Measure Support](#)
- [Parallel Processing for HSMC](#)
- [Automated Validation of HSMC Results](#)
- [HSMC and SRS Monte Carlo — Performance Comparisons](#)
- [HSMC Recommendations and Limitations](#)
- [HSMC Frequently-Asked Questions](#)
- [Parametric Yield Estimation](#)
- [10-Sigma Estimation](#)

Note:

Throughout this chapter, all command-line invocations are denoted with `$SIM`. For the current chapter High Sigma Monte Carlo Analysis, `$SIM` refers to PrimeSim HSPICE, PrimeSim SPICE, PrimeSim Pro and PrimeSim XA.

Introduction to HSMC

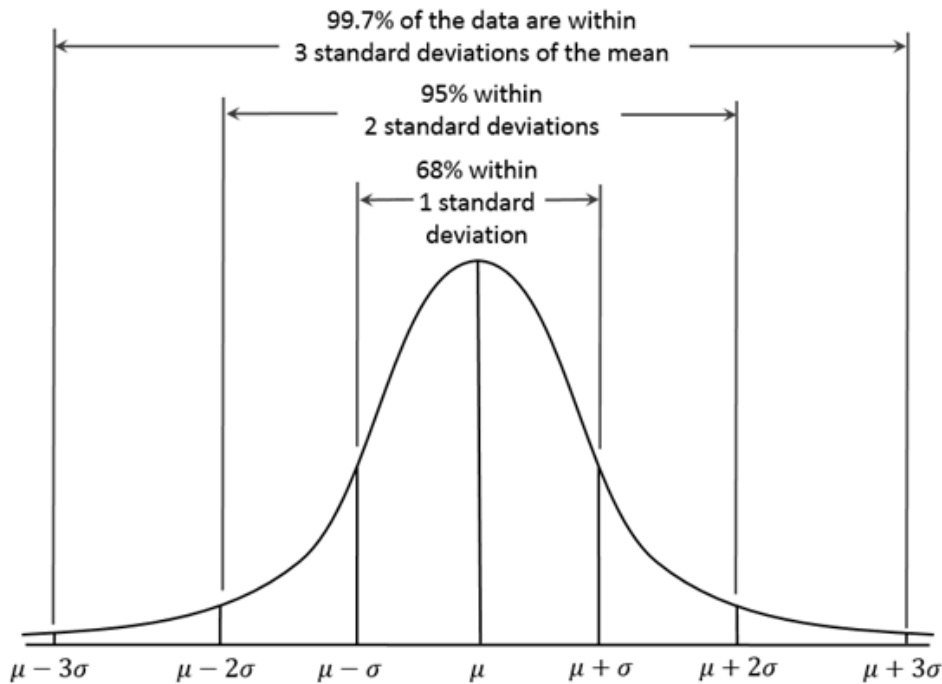
HSMC is a fast and accurate Monte Carlo simulation of small circuits such as standard cells and SRAM bit cells in the $3 - 6.5\sigma$ range (20K – 20B Monte Carlo samples with SRS). HSMC can be used to ensure robust standard cell and memory components at 7nm and 5nm technology nodes, for automotive industry requirements, and for smaller geometry and lower operating voltages.

The advantages of using HSMC include the following:

- Robust technique
- Accurate – provides the same results for quantiles as the Monte Carlo with SRS sampling in PrimeSim HSPICE in almost all cases
- Fast – reduces computation time by a significant factor
- Easy to use
- Reasonable storage and memory requirements
- Identifies critical variation contributors to guide design improvements
- Confidence intervals on results
- Determines statistical corners to speed up design iterations

Principles of HSMC

Figure 50 Principles of HSMC Simulation

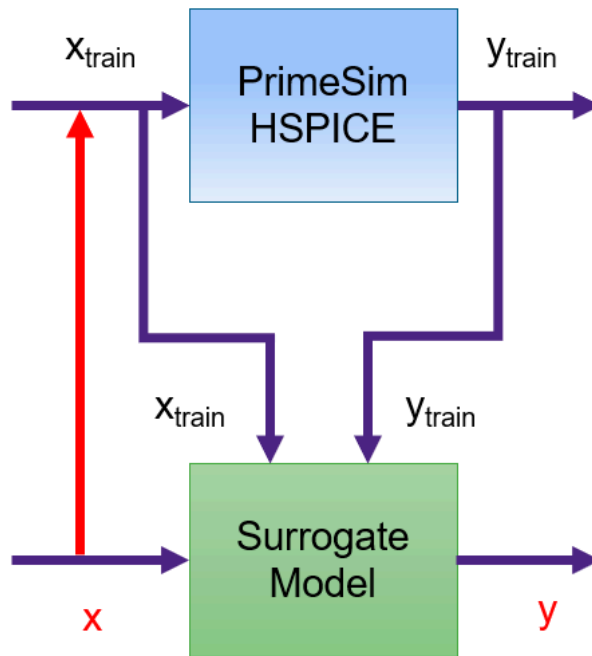


The normal distribution is used for illustration only.
Similar results apply to other unimodal continuous distributions.

Consider the PDF in [Figure 50](#). The samples in the tails are more important for determining failure probability and percentiles. However, most samples are clustered near the mean value. If there is a technique to quickly identify the tail samples, it would be possible to avoid simulating samples in the central region and simulate only the tail samples. HSMC provides a solution to select samples likely to be in tail region.

Figure 51 provides an illustration of the HSMC simulation.

Figure 51 HSMC — An Illustration



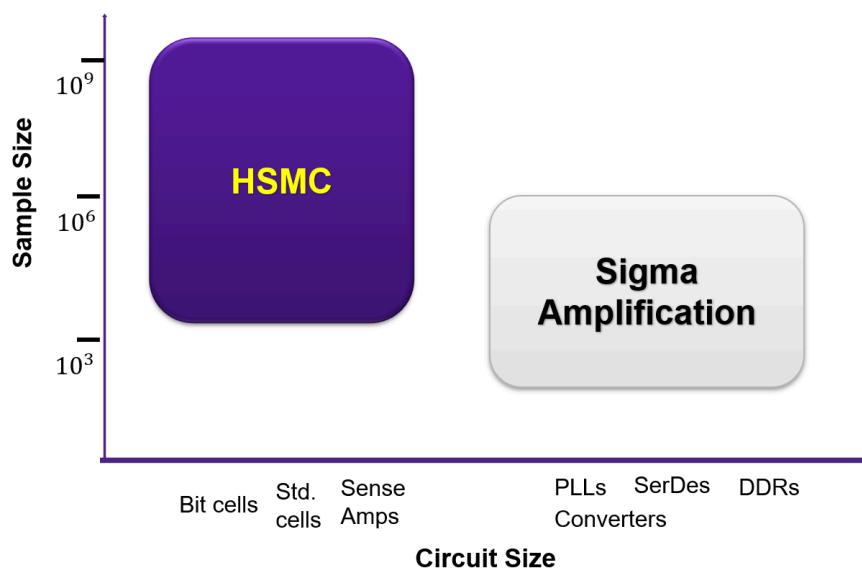
The salient features are:

- Identifies tail samples
- Uses a surrogate model which is a computationally cheap approximation to the true system. The surrogate model selects samples likely to be in the tail region
- Is based on an advanced Machine Learning algorithm
- Provides excellent accuracy, typically within 1% of PrimeSim Design Robustness with traditional MC for large percentiles
- Provides significant runtime improvement

HSMC — Target Applications

As illustrated in [Figure 52](#), HSMC is more suited for smaller circuits such as bit cells, standard cells, and memory components, and highly repetitive IP blocks. In terms of chip level sigma, HSMC is best suited for the 3 - 6.5 σ range, which would require more than 1 billion samples for a traditional Monte Carlo simulation. For such circuits, HSMC can be used to verify the circuit robustness by identifying failing circuits in the tail regions of the sample distribution while bringing down the simulation time from several days to only minutes.

Figure 52 Target Applications for HSMC



For smaller circuits, HSMC helps designers address the following design challenges:

- Design validation or improvement of design robustness
- Certainty of robust standard cell and memory components at advanced technology nodes
- Certainty of design for automotive requirements
- Design for smaller geometry and lower operating voltages

Invoking HSMC

Use the following syntax to invoke HSMC application through the Advanced Variation Analysis dialog box:

```
$SIM -i netlist.sp -o out/ -adv [user.json]
```

Note:

To invoke the specific simulators, use the appropriate syntax as listed below:

- `hspice` for PrimeSim HSPICE
- `primesim` for PrimeSim Pro
- `primesim -spice` for PrimeSim SPICE
- `xa` for PrimeSim XA

PrimeSim HSPICE also supports a combination of distributed and threaded modes for running HSMC. For more information, see [Parallel Processing for HSMC](#).

Inputs

Argument	Description
<code>netlist.sp</code>	Input SPICE netlist.
<code>-adv</code>	After initial parsing and nominal simulation, invokes thePrimeSim AVA tool.
<code>user.json</code>	Input configuration file. Sample of <code>user.json</code> content for a HSMC simulation: <pre>{ "Active_Application": "High_Sigma_Monte_Carlo", "High_Sigma_Monte_Carlo": { "Chip_Sigma": 4.5, "Component_Count": 1, "Average_Relative_Uncertainty": 4 } }</pre> If this configuration file is not provided, PrimeSim AVA displays a dialog box to help you create one or browse and open an existing file. See Configuring Inputs for MC.

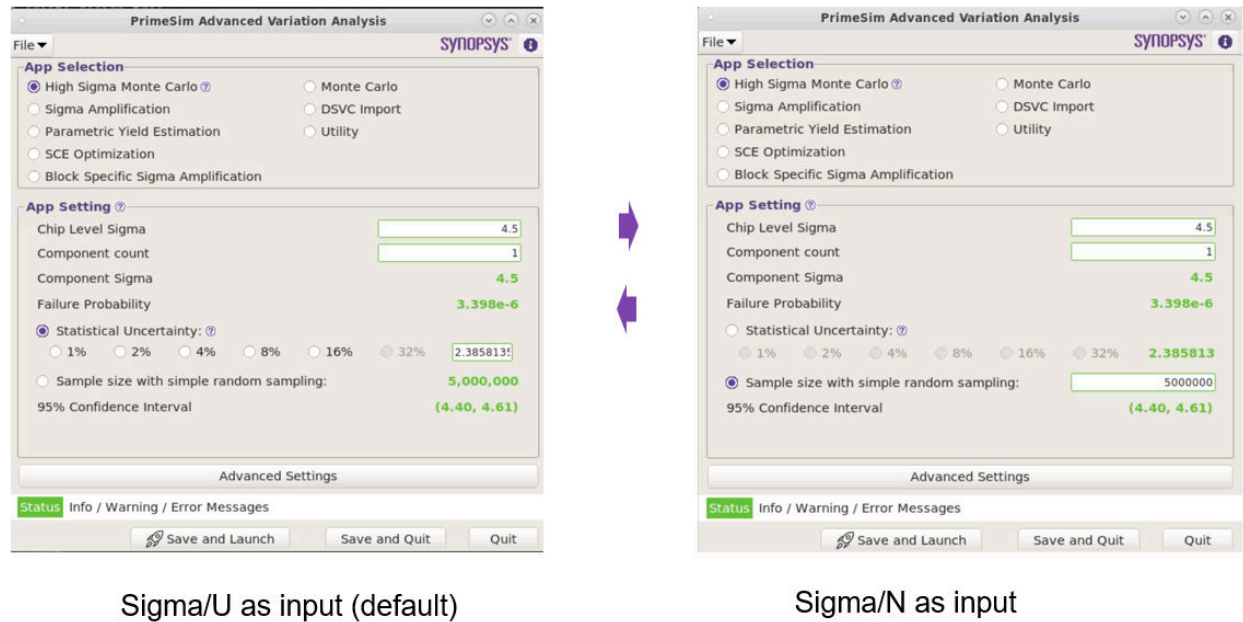
Outputs for HSMC

File	Description	See Also
out.ava.report	Statistic report : moments, sensitivity table, and so on.	The *.ava.report File
out.ava.annotate	Annotate report	
out.ava.qq.pdf	Quantile data and plot in various forms.	Q-Q Plots in PDF, WaveView, and CSV Formats
out.ava.log	Run log: memory, elapsed time, and so on.	
out.ava.sens.csv	csv formatted sensitivity table	out.ava.sens.csv
out.ava.irv_corr.csv	Aggregated sensitivity report	out.ava.irv_corr.csv

Configuring Inputs For HSMC

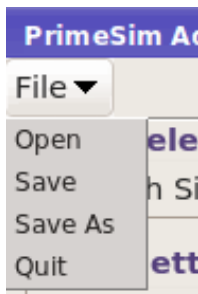
The advanced variation analysis dialog box to configure inputs for HSMC is shown in [Figure 53](#):

Figure 53 Advanced Variability Analysis: HSMC Dialog Box



In the dialog box:

- To open an existing configuration (.json) file, click the **File** menu



and select **Open**. Then select the configuration file to open in the file browser interface and click **Open**. For more information of the JSON file, refer to [Appendix A](#).

- Click the **Browse** button to browse and select a reference standard MC simulation result file, in the **output file** field, for [Automated Validation of HSMC Results on page 114](#).
- To configure new inputs for HSMC, see [Creating a Configuration File](#).

Creating a Configuration File for HSMC

In the App Setting area of the HSMC dialog box, you can configure basic settings for HSMC.

- [Displaying Help for the Settings](#)
- [Configuring Basic Settings](#)
- [Configuring Advanced Settings](#)
- [Saving the Configuration](#)

Configuring Basic Settings For HSMC

Use the following procedure to configure HSMC basic settings:

1. In the **App Setting** area, in the **Chip Level Sigma** field, enter the value for your target chip level sigma.

For this field, the value range is 3 – 6.5.
2. In the **Component Count** field, enter the number of repetition of the component instances in the design. For a **Component Count** value of 1, the **Component Sigma** value will be the same as the **Chip Sigma**.

The HSMC engine computes and auto-populates the **Component Sigma**, **Failure Probability**, and the **Required sample size with simple random sampling** fields.

As mentioned in [System and Component Reliability on page 244](#), for high replication circuits, to satisfy the specified chip level sigma requirement, a higher component level sigma is required. The failure probability also increase with increase in the number of components.

3. In the **Statistical Uncertainty** field, select the acceptable uncertainty range between 1% to 16%.

As discussed in [Sigma, Sample Size, and Statistical Uncertainty](#) average relative uncertainty is a measure of accuracy and determines the confidence interval and the required sample size to use for the specified accuracy.

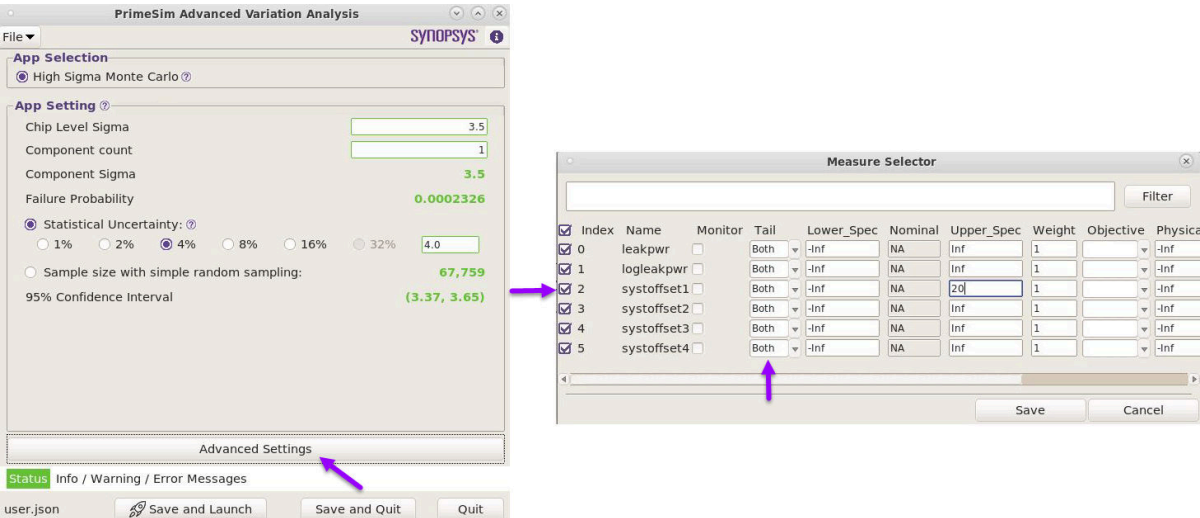
The **Uncertainty Level** can also be defined directly using the sample size option. Click the **Required sample size using SRS** radio button to switch from automatic sample size calculation to user-specified sample size. The range of sample size can be in between (1000,600B)

This option is particularly helpful when comparing the HSMC results against the golden reference results.

Single Tail Measure Support

For HSMC analysis PrimeSim Design Robustness supports the selection of either the left or the right or both tails, independently for each chosen measure. You can specify these in the **Measure Selector** tab of the HSMC Advanced Settings dialog box (see [Figure 54](#)).

Figure 54 Left, Right, and Both Tails Selection, for Each Measure



[Figure 55](#), [Figure 56](#), and [Figure 57](#) show examples of Q-Q plot outputs from HSMC for a single tail selection.

Figure 55 One-Sided Q-Q Plot in PDF

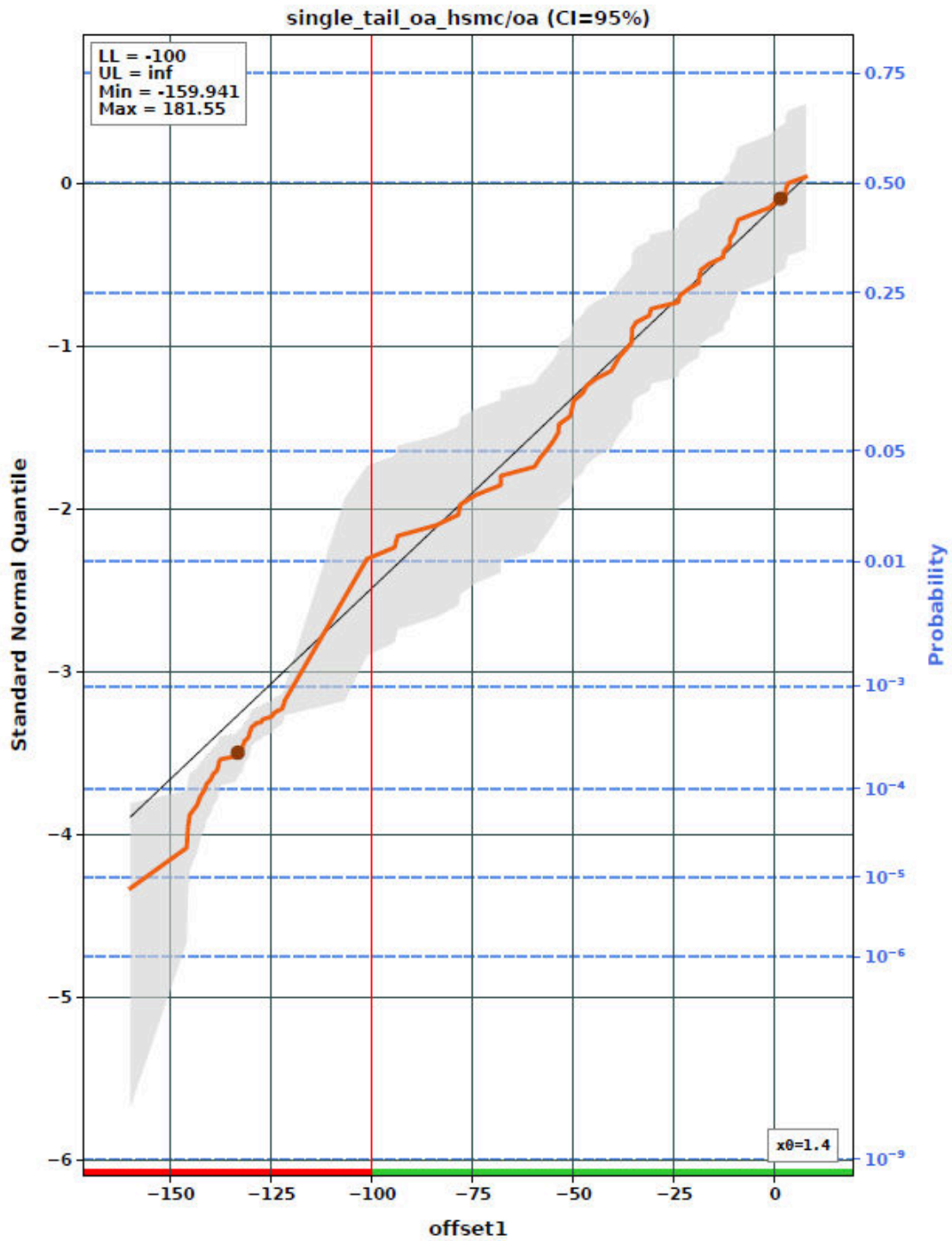


Figure 56 One-Sided Q-Q Plot in WaveView

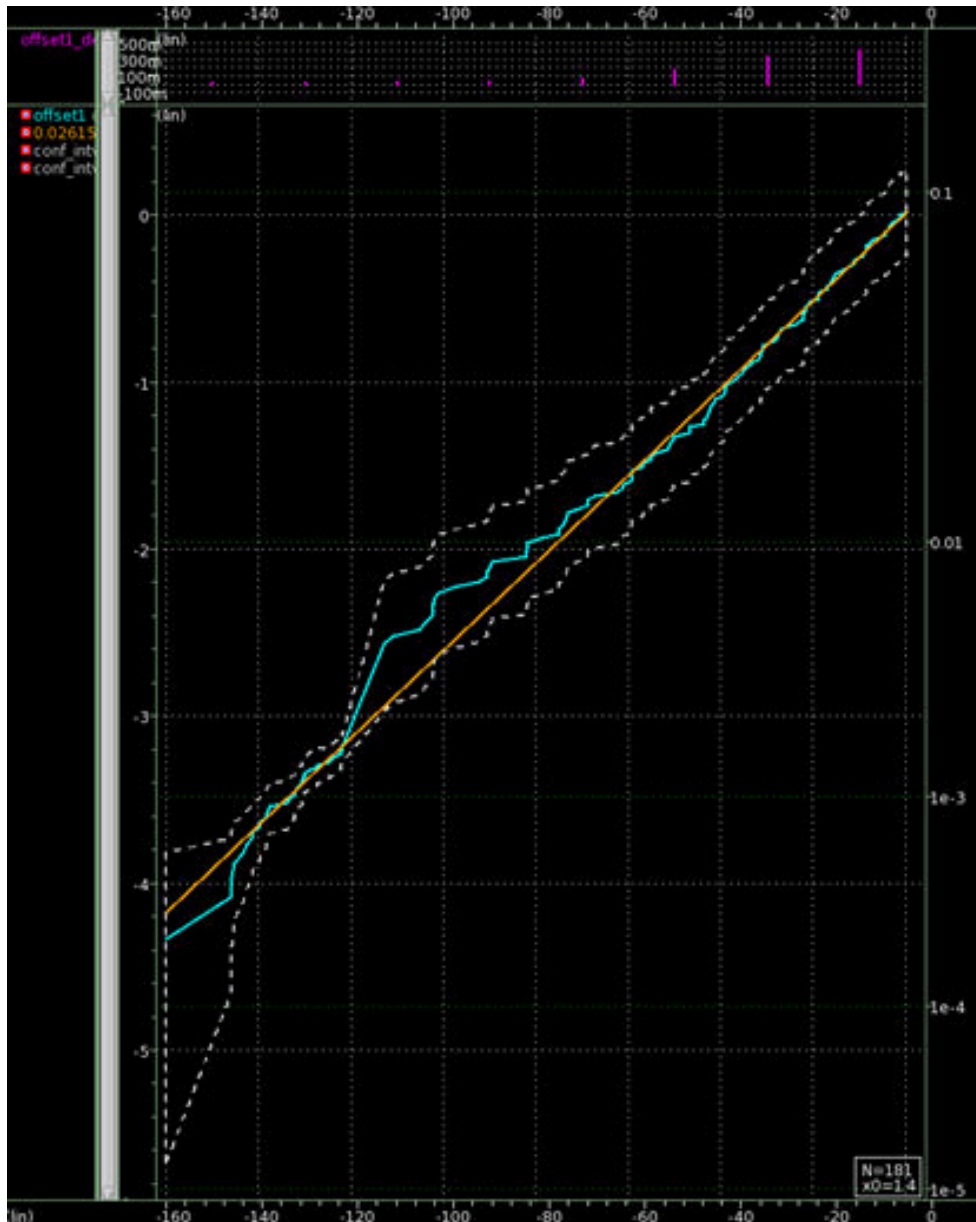


Figure 57 One-Sided *.ava.qq.csv file

```
#QQnorm plots from data
# Not suitable for quantitative analysis
#Number of plot points
87
#Number of Measures
3
#Measure moments
Statistic,offset1,offset2,pwr
Nominal,1.4,1.086,24.41
Mean,NA,NA,NA
StdDev,NA,NA,NA
Skewness,NA,NA,NA
Kurtosis,NA,NA,NA
# Q/QL/QU refer to the Quantile and its Lower/Upper
confidence limits in terms of sigma
#Measure names
offset1,offset2,pwr,Q,QL,QU
-158.1,NA,16.63,-4.3,-5.536,-3.802
-152.5,NA,16.73,-4.2,-5.13,-3.77
-146.9,NA,16.83,-4.1,-4.724,-3.738
-145.8,NA,16.93,-4,-4.459,-3.694
-145.3,NA,17.03,-3.9,-4.249,-3.642
-143.1,NA,17.16,-3.8,-4.074,-3.582
:
:
-10.08,NA,24.2,-0.3,-0.7467,0.1457
-6.435,NA,24.38,-0.2,-0.6454,0.2448
1.113,NA,24.47,-0.1,-0.5446,0.3443
3.322,1.504,24.83,0,-0.4442,0.4442
NA,3.915,24.93,0.1,-0.3443,0.5446
NA,5.639,25.28,0.2,-0.2448,0.6454
NA,8.036,25.65,0.3,-0.1457,0.7467
NA,10.25,25.91,0.4,-0.04701,0.8484
:
:
NA,73.9,40.07,3.9,3.642,4.249
NA,74.63,40.98,4,3.694,4.459
NA,75.68,41.3,4.1,3.738,4.724
NA,81.58,42.05,4.2,3.77,5.13
NA,87.48,42.8,4.3,3.802,5.536
```

Figure 58 and Figure 59 show the *.ava.report and *.ava.details.json outputs for single tail selection.

Percentile Sigma = 3.5
 Confidence Interval for Sigma = [3.368, 3.648]
 Average Relative Uncertainty = 4%
 Confidence Level = 95%
 Number of simple Monte Carlo Samples = 67759

LQ/(RQ) = Left (Right) Quantile : Measure quantiles at +/- the Percentile Sigma value
 LCL/(UCL) = Lower (Upper) Confidence Limit : Confidence interval boundaries at the Confidence Level value

	LQ_LCL	LQ_UCL	RQ_LCL	RQ_UCL	Nominal	Mean	StdDev	Skewness	Kurtosis
offset1	-1.405e+02	-1.333e+02	-1.304e+02	NA	1.400e+00	NA	NA	NA	NA
offset2	NA	NA	NA	6.500e+01	6.776e+01	7.030e+01	1.086e+00	NA	NA
pwr	1.737e+01	1.760e+01	1.776e+01	3.685e+01	3.754e+01	3.834e+01	2.441e+01	2.474e+01	2.315e+00

A(L/R)Q = Approximate Left/Right Quantile

	Min	Max	Min_Index	Max_Index	ALQ_Index	ARQ_Index	Fail_Count	UB_Sigma_Yield	Fail1_Index	Fail2_Index
offset1	-1.599e+02	NA	10894	NA	16921	NA	0	NA	NA	NA
offset2	NA	8.938e+01	NA	5465	NA	50208	0	NA	NA	NA
pwr	1.660e+01	4.304e+01	63579	64227	18780	63218	0	NA	NA	NA

-> To simulate extremes, use Monte Carlo command : monte = list(5465, 10894, 63579, 64227)
 -> To simulate approximate quantile, use Monte Carlo command : monte = list(16921, 18780, 50208, 63218)
 -> To simulate failed samples, use Monte Carlo command : monte = list()

Left tail
 Right tail
 Both tails

Note: Moments have value "NA" whenever either tail is missing

```

:
"Measure Names" : ["offset1","offset2","pwr"],
:
"Sigma" : 3.5,
"Quantiles" : [[-133.3152,null],[null,67.76978],[7.6035,37.54555]],
"Moments" : [[null,null,null,null],[null,null,null],[24.53,2.484,0.3945,3.547]],
"N_Simple Monte Carlo Samples" : 67759,
"Average Relative Uncertainty" : 4.0,
"N_Tail Samples Quantile Calculation" : 16.26,
"N_Reported Tail Samples" : 16,
"N_Reported Interior Samples" : 1,
"Left Tail Indices" :
[[10894,6556,14626,52251,66041,28795,45859,41112,16158,16014,22397,63961,32865,60034,61824,21356,16921],[null,null,null,null,null,null,null,null],[83579,22479,16337,55541,65213,34092,40077,66094,7838,63189,19722,14971,45034,46764,16959,4232,18780]],
"Left Tail Values" : [-159.9065,-145.9682,-145.6817,-145.2327,-143.3864,-142.6361,-141.5686,-141.0481,-139.9031,-139.4612,-138.4202,-138.1197,-138.0196,-137.6215,-133.7799,-133.4714,-132.4248],[null,null,null,null,null,null,null,null],[16.60103,16.85063,16.9702,17.04804,17.0712,17.27144,17.35329,17.35926,17.38699,17.42225,17.49064,17.56441,17.58006,17.58602,17.59176,17.59305,17.63282]],
"Right Tail Indices" :
[[null,null,null,null,null,null,null,null,null,null,null],[5465,61298,62322,11330,10953,65342,19680,18428,3320,53765,1577,12938,10964,12817,37086,53612,50208],[84227,39579,14897,40043,35983,17497,15334,34563,38475,57590,50011,58055,35281,4159,47828,57516,63218]],
"Right Tail Values" :
[[null,null,null,null,null,null,null,null,null,null,null],[88.38372,74.73014,74.59892,73.71444,72.85138,72.70895,71.05651,70.74352,69.86975,69.81898,69.07603,68.96157,68.70102,68.69621,68.4038,68.0422,67.00525],[48.04407,41.17731,40.89393,39.8514,39.72959,38.5484,38.39942,38.36609,38.3245,38.14693,37.99425,37.85671,37.85623,37.74454,37.67677,37.64576,37.26446]],
:
:
:

```

HSMC in PrimeSim Design Robustness supports a combination of distributed and threaded modes. The resource allocation depends on the cost of a single Monte Carlo simulation and on the required sample size with SRS which, in turn, is a function of sigma and uncertainty.

Syntax:

```
$SIM -i input.sp -o out -adv [user.json]  
+ [-dp k1 -dpconfig configfile] [-master k2]
```

This means that, at most $k2$ cores are used on the master and $k1$ single core workers are on the grid. $k1$ depends on how long each Monte Carlo simulation takes, the availability of farm resources, and on how quickly the user wants the results. $k2$ depends on the SRS sample size and a value of 4 or 8 is recommended for large samples.

You can skip the `dp` part of the command for small combinatorial circuits. For more information, see the following table.

Equivalent sample size for simple random sampling		
	Less than 250 million	More than 250 million
SRAM bit cells, small combinatorial cells	No parallel processing required	Use parallel processing with $k2$ set as 4 or 8
Large combinatorial cells and sequential cells	Use parallel processing with $k1$ set to a suitably high number	Use parallel processing: in addition to $k1$, set $k2$ to 4 or 8

Automated Validation of HSMC Results

The PrimeSim Design Robustness tool provides automatic validation of HSMC results with a specified reference standard MC simulation results.

To gain confidence with HSMC, it is recommended that you specify a reference standard MC simulation results for PrimeSim Design Robustness to compare the results from HSMC against the reference, at least for lower sigma values, on a collection of representative circuits.

Generation of reference results for high sigma is practical for a few cases because:

- PrimeSim Design Robustness supports distributed processing on hundreds of cores.
- The simulation automatically switches to Large Scale Monte Carlo above 1M samples, to conserve storage.
- 1B samples can be simulated for SRAM bit cells in under a day using 100 workers.

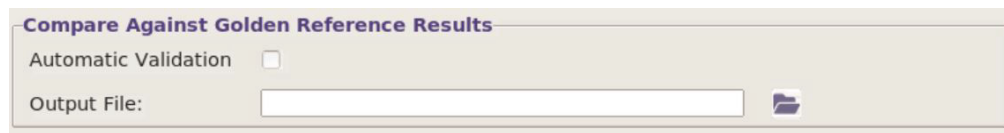
The same circuit can then be simulated for an HSMC run. The Q-Q plots, percentiles, values of tail samples, their Monte Carlo sweep indexes, and the first few moments are automatically compared between the two sets of simulations.

The supported PrimeSim AVA MC output formats for the reference results:

- Measure files (`golden.mt0`, `golden.mt0.gz`)
- Tail files (`golden.tail0.csv`)

Use the **output file** field in the HSMC **Advanced Settings: General interface**, to specify the golden reference file for the HSMC simulation validation (see [Configuring Advanced Settings](#)).

Figure 60



Alternatively, you can point to the reference results by using the "Golden_Prefix" field in the `user.json` file. This prefix serves to automatically match golden files with extensions `.m[ast]0(.gz)` and `*.tail0.csv`. See [JSON for HSMC When Reference Results are Specified Manually](#).

If both these methods of specifying the reference results are used, then the dialog box specification will take precedence.

The reference data is overlaid on the Q-Q plots and the following information is added to the `*.ava.details.json` file:

- Golden reference summary
 - Tail indices / values
 - Quantile and moment values
- HSMC and standard MC Comparison
 - Tail ranks
 - Quantile and moment relative/absolute errors

Example

This is an example of an automated HSMC results validation on a small standard cell (ND2) test case.

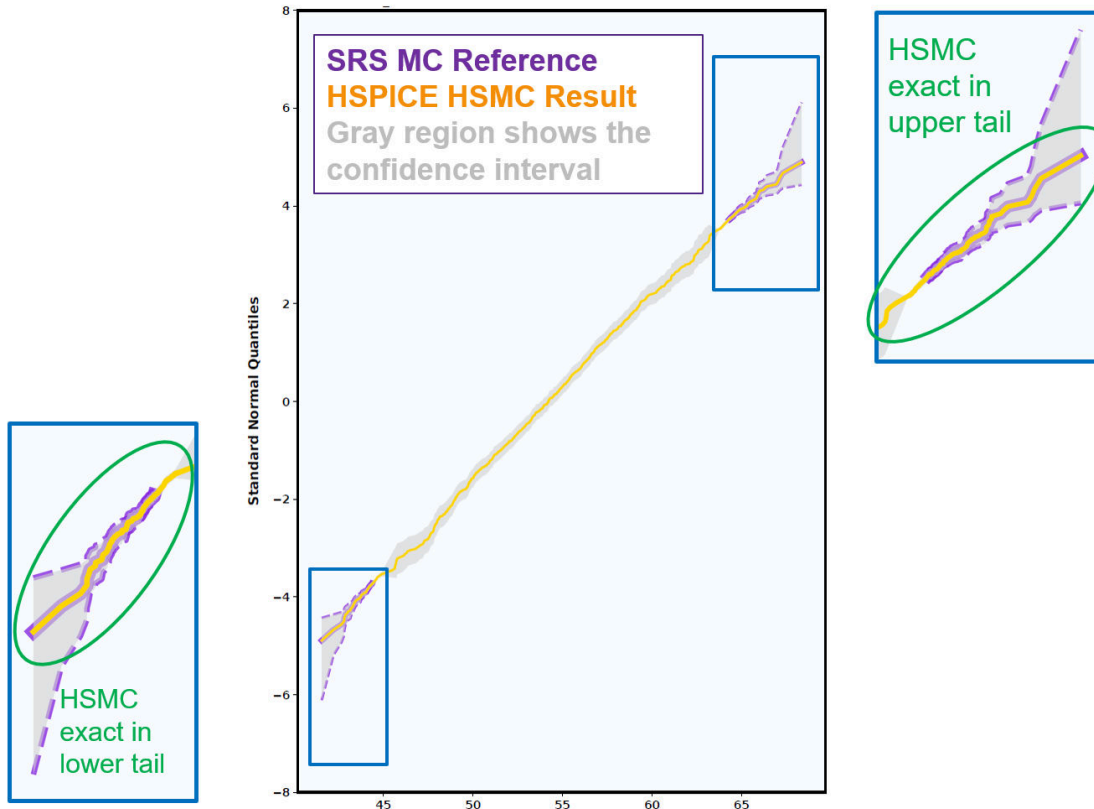
The salient points of the test case:

- Sigma = 4.5, U = 4%
- The required samples with SRS ~2M

- The reference results with simple random sampling took about 1.2 hours to generate with 20 cores.
- The HSMC run took about 7 minutes on a single core (~200X speedup).

The Q-Q plot comparison is shown in [Figure 61](#):

Figure 61 Q-Q Plot Comparison



In [Figure 61](#), the reference MC results are shown in purple and the HSMC results in yellow. The gray region and the dotted purple show 95% confidence intervals for HSMC and the reference results, respectively.

For this case, overlay is exact in the tail regions where the quantiles are calculated (purple outlines yellow) and HSMC and reference quantiles showed an excellent match in the tails.

HSMC and SRS Monte Carlo — Performance Comparisons

For a 5nm technology, 6 MOS, 65 R, 152 C SRAM bit cell test case, SRS Monte Carlo with 100M samples took 39 mins on 600 cores (= 16 days on 1 core).

This sample size corresponds to ~5.3 sigma.

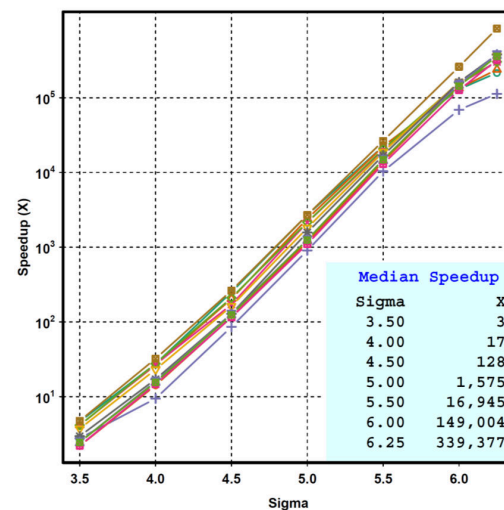
In contrast, a 6 sigma HSMC simulation takes only a few minutes; the equivalent SRS Monte Carlo would require 2.5B samples (not practical with SRS, would require ~5 months on 1 core). Also, the quantile accuracy of HSMC is better than 1%.

Speedup as a Function of Sigma

For the test cases described in the table in [Figure 62](#), the increase in the HSMC simulation speedup with increase in sigma are shown in the graphs on the right.

Figure 62 Speedup as a Function of Sigma for Various Test Cases

Case (Tech)	# MOS	# R	# C	Comment
INV (16)	2	116	184	Standard Cell
INV (7)	6	151	259	Standard Cell
SRAM (7)	6	0	0	Bit Cell
OR (16)	8	195	604	Standard Cell
Nand (16)	11	219	435	Standard Cell
Majority gate (16)	13	252	629	Standard Cell
Latch (16)	14	309	715	Standard Cell (S)
AOI (7)	22	481	1,109	Standard Cell
Ring Osc (7)	26	0	0	Ring Osc
MUX (16)	76	1,136	2,288	Standard Cell
AOI (16)	80	1,064	2,600	Standard Cell
OAI (16)	84	1,090	2,995	Standard Cell
Clock gate (16)	122	1,879	3,578	Standard Cell



The median speedup X as a function of sigma is highlighted in blue.

The reference simulations used DP on 20 cores with 2M – 18M samples while the HSMC simulations used a single core.

Also, with HSMC, there was virtually zero error in the percentile computations, where comparison against SRS was possible.

HSMC Recommendations and Limitations

Recommendations:

- Circuit size of 15K transistors or less.
- Component sigma range is 3–6.5.
- Number of measures is less than or equal to 5.

- The measure distributions must be continuous (not pass/fail) and reasonably close to normal. This includes delay, slope, and other constraints for standard cells and most SRAM bitcell and sense amplifier measures.
- Some measures may be used as intermediate ones as the HSMC and PYE runtime is affected by the number of measures being characterized.
- Add `PRINT = 0` to intermediate measure statements and use as few measures as possible when running HSMC and PYE.

Limitations:

- Does not support ALTERs or sweeps (other than Monte Carlo).
- Supports only the default MCG random number generator.
- RF functions and MOSRA are not supported.
- At times, there can be jumps in the Q-Q plots and this is an artifact of the algorithms used.
- The definitions for measures with functions like min, max, abs, and ternary operators are not suitable for the current AVA flows. These non-smooth functions often create strange QQ plots. It is recommended to avoid non-smooth functions/operators in measure definitions.

Warning in AVA:

`WARNING! Measurements of meas1, meas2 contain "ternary" function. This can lead to non-smooth QQ plots.`

HSMC Frequently-Asked Questions

- What algorithms are used for HSMC?

This information is Synopsys proprietary.

- Does HSMC depend on the technology node?

No, it is independent of the technology node.

- Does the HSMC performance depend on the number of measures?

Performance speedup is greatest when the number of measures is small (that is, fewer than 5).

- Does HSMC work with all measures?

Is extensively tested for delay/slope/constraint type measures.

It does not work for discrete measures like pass/fail; use continuous measures.

- Does HSMC depend on circuit size?

It is designed to work best for small circuits where the number of transistors is less than 400. Larger circuits take longer to simulate and the results may not be as reliable.

- How is the confidence interval in the `*.ava.report` file and the Q-Q plots created?

The confidence interval is based on the relation between order statistics and the beta distribution and is exact.

- Does the confidence interval depend on the technology node and circuit type?

The confidence interval depends only on the specified sigma value and the uncertainty U.

- What are different sets of samples from MC runs that can be simulated after the HSMC run?

The extreme samples listed in the `*.ava.report` file and the tail samples displayed in the `*.ava.details.json` file.

Consider the following example:

```
"Left_Tail_Indices" : [[263136,1288394,218434,566645,1087441,1021485,1577739,1819198],[1845297,766439,655366,565335,70191,1465778,1058626,486005]],
"Left_Tail_Values" : [[2.833312,2.992934,3.010356,3.01943,3.078372,3.215435,3.411222,3.442644],[2.676798e-09,2.687665e-09,2.690666e-09,2.694173e-09,2.696843e-09,2.697385e-09,2.698253e-09,2.698316e-09]],
"Right_Tail_Indices" : [[534134,605205,1003481,440823,840377,602547,69525,427735],[263136,566645,1087441,218434,1288394,1021485,201066,1819198]],
"Right_Tail_Values" : [[5.002942,5.002768,5.002739,5.002737,5.00271,5.002684,5.002683,5.002642],[5.46027e-09,5.436918e-09,5.431874e-09,5.427474e-09,5.414701e-09,5.399348e-09,5.371355e-09,5.357435e-09]],
```

The example content represents the left/right tail indexes and values. The entries have an ordering from the extreme end of the distribution, inward. In the example, there are two such lists for each dictionary key as there are two measures.

- If `Vary/Do_Not_Vary` is turned on for an HSMC run, do they have to be manually turned on in the standalone run using these MC indices to recreate accurate results?

Yes, you must align these options in the standalone and HSMC runs.

- Can the measures from the MC simulations be viewed?

The identity (sample index) and measure values of the selected samples are available. If waveforms are needed, you should run the MC simulation with the `monte=list(index1,index2,...)` syntax to produce the waveforms.

- Can the actual model parameters be viewed, say from one of these “extreme” simulations, or do I need to display them manually?

The `mc0` file contains the values of the independent random variables and can be produced with the `monte=list()` syntax with the appropriate tail indices.

- Can `Vary_Only` and `Do_Not_Vary` both be used in the same simulation?

Yes, you can include both these options in the same simulation.

- While “pass/fail” criteria cannot be used in HSMC, are there any indicators that a HSMC simulation run into a problem?

In the AVA scenario, pass/fail measures are meant to refer to measures which only have two distinct values. These are constructed when you create a measure which applies a specification to a raw measurement directly in the netlist, for example, `vsat > 5.3`. Such measures are automatically dropped from the HSMC analysis and, if no measures remain, then the HSMC run will abort with an appropriate error message.

- What is the difference between High Sigma Monte Carlo and Sigma Amplification if both of them focus on taking more samples at the end of the distribution?

HSMC does not change the distribution. Rather, it skips PrimeSim HSPICE or other PrimeSim simulators evaluation of samples that are likely to be in the middle of the response distribution by evaluating all samples in the surrogate. Sigma amplification (SA) distorts the input distributions, does not construct a surrogate, and simulates all samples in the corresponding stimulator. The two methods are complementary – HSMC works for smaller circuits and SA for larger ones. The accuracy of HSMC is generally better than SA and it has a tighter confidence band.

- How does PrimeSim Design Robustness decide if a given sample is tail sample?

The algorithms used to classify any given sample as tail sample in both HSMC and SA are Synopsys proprietary.

Parametric Yield Estimation

This chapter describes the principles of Parametric Yield Estimation (PYE). It provides information on performing a PYE simulation in PrimeSim AVA and details of its outputs.

The following sections discuss these topics:

- [Introduction to Parametric Yield Estimation](#)
- [Parametric Yield Estimation in AVA](#)
- [Running Parametric Yield Simulation](#)
- [PYE Output](#)

Note:

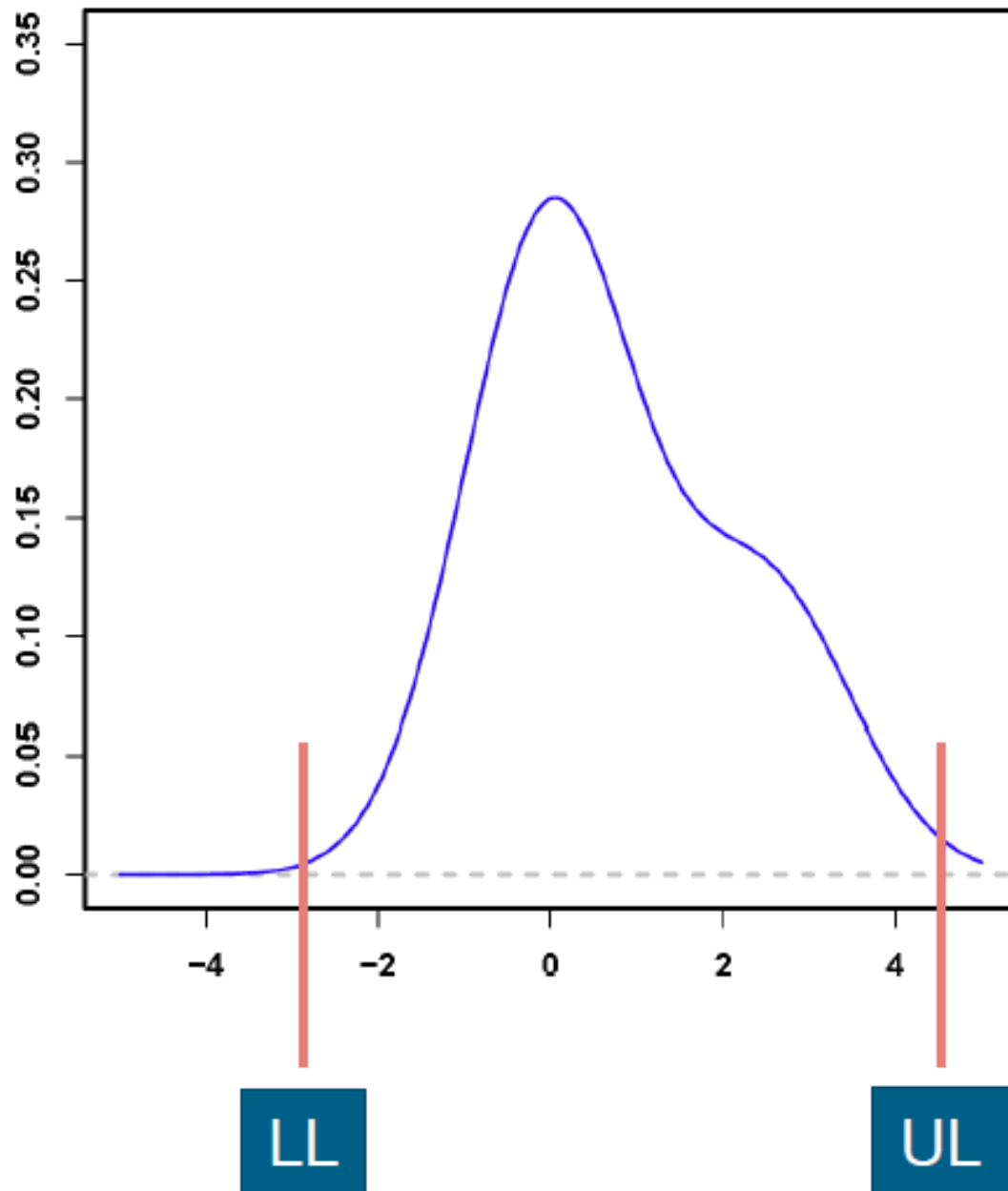
Throughout this chapter, all command-line invocations are denoted with `$SIM`. For the current chapter Parametric Yield Estimation, `$SIM` refers to PrimeSim HSPICE, PrimeSim SPICE and PrimeSim Pro.

Introduction to Parametric Yield Estimation

Parametric yield estimation is defined as the probability that the given netlist fulfills the specifications given for certain measures.

Consider a measure whose probability density function (PDF) is as shown in [Figure 63](#). For the circuit to function accurately, you must expect it to fall within a given range defined by the lower limit and upper limit. This range is called the specification. For measures where the specification is defined as less than a given number x , the upper limit for such a measure is x and the lower limit is $-\infty$. Similarly, for measures that are defined as greater than a given number, the lower limit is x and the upper limit is $+\infty$. Circuits with measure values within the specification are considered as pass. Parametric yield is defined as the probability of passing all specifications that is the area under the density curve between the lower and upper limit.

Figure 63 Probability Density Function With Upper and Lower Limits Marked



The probability density function of a measure output can be determined by using Monte Carlo Analysis. After the density function is determined, PYE measures the probability of that measure being within the specifications. For a system with n identical components,

assuming each component has the yield of probability p functioning accurately, then the yield of the system is p^n .

For greater design insight, the failure probability corresponding to each individual specification is reported. A measure has two failure modes when both lower limit and upper limit are finite.

- Pf_L is the area under the probability curve from $-\infty$ to lower limit
- Pf_U is the area under the probability curve from upper limit to $+\infty$

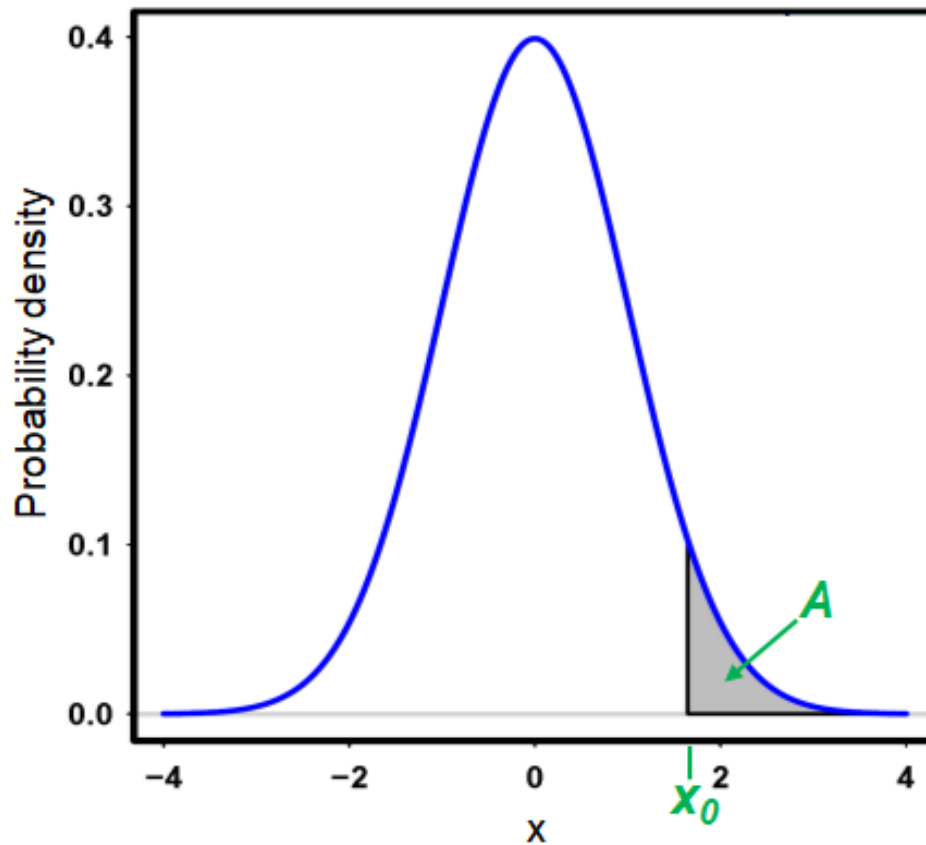
If the system consists of n identical components, the system failure probability is related to the component failure probability f by $1 - (1 - f)^n$.

Parametric Yield Estimation in AVA

To evaluate a failure probability, running a general Monte Carlo simulation might result in consuming high disk space and take more time. PYE provides the advantage of machine learning methods similar to High Sigma Monte Carlo simulation to evaluate the failure probability. This failure probability provides an estimate of yield to the designer.

A probability (area A in [Figure 64](#)) is given as input for HSMC and the corresponding measure values (quantiles or x_0 in the figure) are the output. Whereas for PYE, specifications on measures (x_0 - one on each side that act as upper and lower limits of the specification) are given as input and the failure probability (or yield) is the output. PYE and HSMC share the same probability and statistical concepts (such as uncertainty, confidence interval, Q-Q plot, and so on).

Figure 64



However, PYE has its own input and output usage to set lower/upper limit specification, Monte Carlo number, and result display.

Running Parametric Yield Simulation

Use the following syntax to invoke parametric yield estimation.

Syntax

```
$SIM -adv usr.json -i input.sp -o output
```

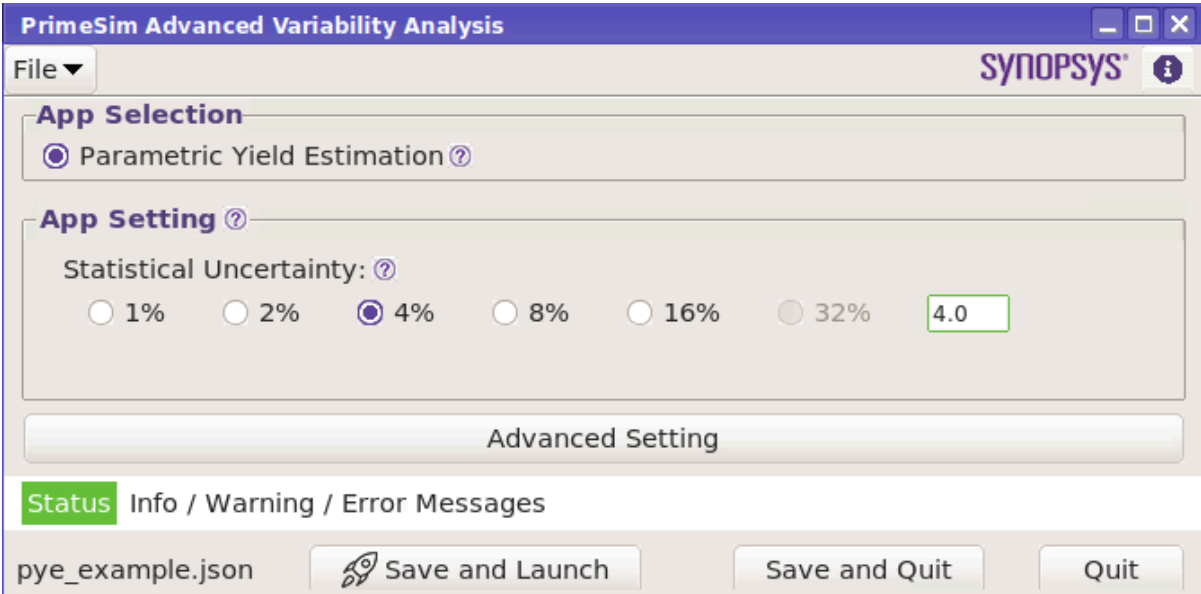
Note:

Use `hspice` in place of `$SIM` to invoke PrimeSim HSPICE. For PrimeSim SPICE, PrimeSim Pro and PrimeSim XA, refer to the respective chapters.

Invoking Parametric Yield Estimation

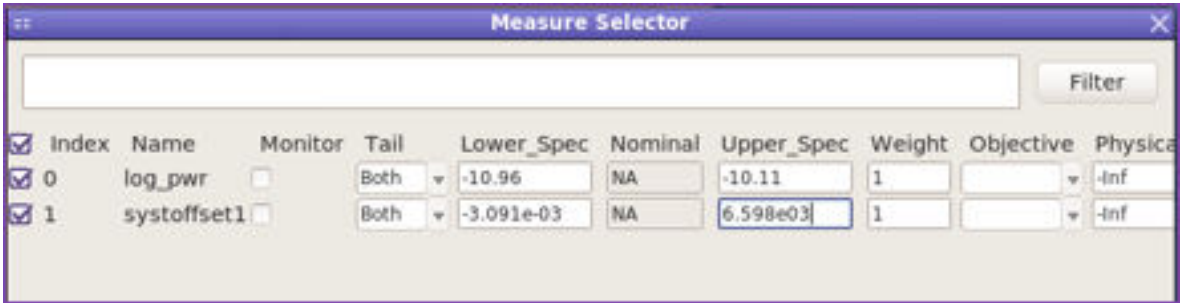
The syntax invokes the Advanced Variation Analysis dialog box as shown in [Figure 65](#).

Figure 65 Parametric Yield Estimation Interface



1. Click the **Advanced Settings** option to setup the specifications for a specific measure.
2. Select **Measure Selector** tab. Enter the required measures in the **Lower_Spec** and **Upper_Spec** columns as shown in [Figure 66](#).

Figure 66 Parametric Yield Estimation Interface - Limit Specification



PYE Output

PYE analysis produces the following outputs:

- output.ava.report
- output.ava.details.json
- out.qqt(s/a)0.pdf

The results of P_{fL} and P_{fU} with confidence interval and traditional variance contribution are displayed in output*.ava.report file as shown as follows.

Figure 67 Example of the PYE output

```
Average Relative Uncertainty = 4%
Confidence Level = 95%
Number of simple Monte Carlo Samples = 830637
Note: Number of simple Monte Carlo samples is based on preliminary estimate of yield
Note: Quantile confidence intervals computed by the beta distribution for order statistics

P_f{L,R,M,A} = [P]robability of [f]ailure
CI_{L,R,M,A}{L,U} = [C]onfidence [I]nterval limit
Final Yield = aggregate yield of all tail specifications for all measures as well as
measurement failures; corresponds to R syntax : qnorm(1-Overall_P_fA/100)
SVS_{LR} = [S]igma [V]alue at [S]pecification
{L,R,M,A} = [L]eft/[R]ight tail / [M]easurement failures / [A]ll failures (L/R/M combined)
{L,U} = [L]ower/[U]pper

Failure Report in Percentage
```

	Left_Spec	CI_LL%	P_fL%	CI_LU%	Right_Spec	CI_RL%	P_fR%
	CI_RU%	CI_ML%	P_fM%	CI_MU%	CI_AL%	P_fA%	CI_AU%
v1	6	0.00233819	0.0034913	0.00487187	14	0.00165985	0.00264857
	0.00386457	<3.048e-06	<0.00012039	<0.000444102	0.00457157	0.00613987	0.00793586
v2	6	0.00165985	0.00264857	0.00386457	14	0.00273475	0.00397285
	0.00543849	<3.048e-06	<0.00012039	<0.000444102	0.00498821	0.00662142	0.00848237
Overall	.	.	.	.	.	.	.
	<3.048e-06	<0.00012039	<0.000444102	.	0.010448	0.0127613	0.0153024

```
*Final Yield = 3.657 sigma

Sigma Value at Specification
```

	Left_Spec	CI_LL	SVS_L	CI_LU	Right_Spec	CI_RL	SVS_R	CI_RU
v1	6	-4.071	-3.977	-3.897	14	3.953	4.042	4.150
v2	6	-4.150	-4.042	-3.953	14	3.870	3.946	4.035

```
WS(L/R)E = Within Specification Left/Right Extreme
Min Max Min_Index Max_Index WSLE_Index WSRE_Index Fail_Count UB_Sigma_Yield
Fail1_Index Fail2_Index
v1 5.236e+00
1.474e+01 491907 317308 240837 695888 0 NA NA
NA
v2 4.641e+00
1.553e+01 321116 801398 605536 757566 0 NA NA
NA
-> To simulate extremes, use Monte Carlo command : monte = list(317308, 321116, 491907,
801398)
-> To simulate within specification extremes, use Monte Carlo command : monte = list(240837,
605536, 695888, 757566)
-> To simulate failed samples, use Monte Carlo command : monte = list()
```

For PYE, the tail information and Q-Q plot are saved in the output files of output_ava.details.json and output_...ava.qq.pdf respectively.

See Also

- [</> Marks in AVA Yield Output Reports](#)

10-Sigma Estimation

This chapter describes the principles of 10-Sigma Estimation. This integration provides the resampling and reevaluation of the netlist that are omitted to make the computation at very high sigmas time efficient.

- [Introduction to 10-Sigma Estimation](#)
- [Invoking 10 Sigma Estimation](#)
- [Configuring Inputs for 10-Sigma Estimation](#)
- [Understanding Outputs](#)

Throughout this chapter, all command-line invocations are denoted with `$SIM`. For the current, `$SIM` refers to PrimeSim HSPICE, PrimeSim SPICE, and PrimeSim Pro.

Introduction to 10-Sigma Estimation

As the device density within memory and standard cell applications keeps rising, the characteristic behavior at higher sigmas is evident. Computation of these samples using traditional Monte Carlo is resource intensive and employs Machine Learning (ML) techniques. The High Sigma Monte Carlo (HSMC) application targets on estimating the quantiles, tail samples, and from 3.5 to 6.50 sigma. Beyond this, the training models fall short to estimate the quantiles. Extreme Value Distribution (EVD) is used to estimate the quantile estimates and confidence intervals. This computation is integrated within the High Sigma Monte Carlo application of PrimeSim Design Robustness. Due to this integration, the resampling and reevaluation of the netlist are omitted to make the computation at very high sigmas time efficient. The total simulation time is thus optimized.

The PrimeSim AVA tool estimates 10 sigma quantiles using the High Sigma Monte Carlo (HSMC) application and the command syntax remains the same; for more information , see [Invoking HSMC](#).

For details on using this integration through the PrimeSim Advanced Variation Analysis dialog box, see [Configuring Inputs For HSMC](#).

Invoking 10 Sigma Estimation

The PrimeSim AVA tool estimates 10 sigma quantiles using the High Sigma Monte Carlo (HSMC) application and the command syntax remains the same; see [High Sigma Monte Carlo and Parametric Yield Estimation > Invoking HSMC](#).

For details on using this integration through the PrimeSim Advanced Variation Analysis dialog box, see [Configuring Inputs for 10-Sigma Estimation](#).

Configuring Inputs for 10-Sigma Estimation

Use the **Advanced Variation Analysis** dialog box to configure inputs for quantile estimation at very large sigma using HSMC.

1. Use the `$SIM -i netlist.sp -o out/ -adv [user.json]` syntax to invoke the following screen

Figure 68 PrimeSim Advanced Variation Analysis:High Sigma Monte Carlo

The screenshot shows the 'PrimeSim Advanced Variation Analysis' dialog box with the 'High Sigma Monte Carlo' application selected. The 'App Setting' section displays various parameters for the simulation.

Parameter	Value
Chip Level Sigma	8
Component count	1
Component Sigma	8.0
Failure Probability	6.221e-16
Statistical Uncertainty: ?	4.0
Required sample size with simple random sampling	>> 50e9
95% Confidence Interval	(na, na)

The dialog box also includes a 'Status' bar at the bottom with the file name '8sig_2.json' and buttons for 'Save and Launch', 'Save and Quit', and 'Quit'.

2. Choose **High Sigma Monte Carlo**.
3. In the **App Setting**, enter the value for your target chip level sigma in the **Chip Level Sigma** field. The usual sigma value range is 3.5-6.5. The sigma value range to invoke very high sigma distribution is from 6.5-10.
4. In the **Component count** field, enter the number of repetitions of the component instances in the design. For a **Component count** value of 1, the **Component Sigma** value is the same as the **Chip Level Sigma**.

The HSMC engine computes and auto-populates the **Component Sigma** and **Failure Probability**.

Note:

The SRS sample size and confidence interval cannot be estimated.

5. In the **Statistical Uncertainty** field, select the acceptable uncertainty level as per your requirement.
6. Click **Save and Launch**.

The simulation first runs for sigma up to 6.5 using HSMC methods. After that, EVD methods are invoked and a separate section is created within PrimeSim Design Robustness reports denoting the quantiles separately.

Extreme Value Distribution (EVD) is not enabled by default, you need enable it through the Enable EVD hyper-parameter.

Edit the user.json and add the following to extend the sigma to 10:

- a. Update Chip_Sigma to 10.
- b. Add "Enable_EVD": true.

The .json file generated for EVD is denoted in example # in the [Appendix A](#).

Understanding Outputs

The .ava.report file contains the additional section 10.0 Sigma Quantile Estimates. It contains the information of basics moments and right/left quantile values across the confidence interval.

Figure 69 10.0 Sigma Quantile Estimates - .ava.report

```
EVD Quantile Estimates:
Notice: the confidence intervals are calculated by bootstrap.
Notice: the numbers followed by "*" violate the physical bound.

log_pwr:
          LQ_LCL      LQ      LQ_UCL      RQ_LCL      RQ      RQ_UCL
9.000e+00 sigma 2.606e+00 2.650e+00 2.684e+00 4.072e+00 4.192e+00 4.408e+00

offset1:
          LQ_LCL      LQ      LQ_UCL      RQ_LCL      RQ      RQ_UCL
9.000e+00 sigma -2.661e+03 -4.329e+02 -2.358e+02 4.820e+02 4.991e+02 4.365e+03

offset2:
          LQ_LCL      LQ      LQ_UCL      RQ_LCL      RQ      RQ_UCL
9.000e+00 sigma -3.106e+02 -1.393e+02 -1.190e+02 1.278e+02 1.559e+02 3.211e+02
```

The outputs generated by using EVD have the following limitations:

- If you find a questionable quantile, PrimeSim AVA cannot return the Monte index to reproduce the corner.
- The QQ plots generated by PrimeSim AVA include data only until 6.5 sigma. The data beyond that is not displayed in form of QQ plots.

8

Multi Corner Analysis

This chapter describes the principles of Multi Corner Analysis. It provides information on performing a Multi Corner Analysis in PrimeSim AVA and details of its outputs.

The following sections discuss these topics:

- [Introduction to Multi Corner Analysis](#)
- [Running Multi Corner Analysis](#)
- [Inputs for Multi Corner Analysis](#)
- [Validating the Input corners.csv File](#)
- [Generating corners.csv File](#)
- [DUMMY Values for Manual Edits](#)
- [User provided Netlist](#)
- [Multiple meas_spec.csv file](#)
- [Outputs for Multi Corner Analysis Simulation](#)
- [Multiple Sigma HSMC](#)
- [Multi Corner Distributed Processing](#)
- [Dynamic Task Monitoring and Control – CDPL DP Manager](#)
- [Worker Log Files](#)
- [Advanced Features](#)

Introduction to Multi Corner Analysis

The variation analysis is done on multiple PVT and parameter settings in many applications to view the design's robustness. Such combination of PVT/parameters is usually called as a *corner*. You can currently perform such simulations with a shell loop in which parts of the netlist are modified. These modified corner netlists are used to initiate

a PrimeSim AVA simulation. This process is performed over multiple iterations and the results are merged across the corners, which is tedious and error prone.

The objective of the PrimeSim AVA Multi Corner analysis is to simplify the use model, run the PrimeSim AVA simulations and collect the results in a unified output file. During this automation, machine learning methods in the PrimeSim AVA uses information across corners to reduce total run time.

Running Multi Corner Analysis

Multi corner analysis can be performed for a HSMC simulation. The `user.json` file contains additional Corners field. To run a Multi Corner analysis across below corners:

Corners definitions:

- Temperature (3) : -40, 25, 125
- vdd (4) : 2.0, 2.5, 3.0, 3.5

Command to run the Multi Corner analysis:

```
$SIM -i input.sp -o out/ -adv user.json  
user.json:
```

```
{  
  "Active_Application": "High_Sigma_Monte_Carlo",  
  "High_Sigma_Monte_Carlo": {  
    "Chip_Sigma": 3.5,  
    "Component_Count": 1,  
    "Average_Relative_Uncertainty": 4.0  
  },  
  "Measure_Specification_File": "mpvt_meas_spec.csv",  
  "Corners": {  
    "Properties": {  
      "T": {  
        "Attribute": "Temperature",  
        "Values": [-40, 25, 125]  
      },  
      "vdd": {  
        "Attribute": "Voltage",  
        "Values": [2.0, 2.5, 3.0, 3.5]  
      }  
    },  
    "Generate_Corners": "T*vdd"  
  }  
}
```


On executing the command 12 corners are generated in the `.csv` file as:

Index	Name	Include	T	vdd
1	C_1	1	-40	2
2	C_2	1	-40	2.5
3	C_3	1	-40	3
4	C_4	1	-40	3.5
5	C_5	1	25	2
6	C_6	1	25	2.5
7	C_7	1	25	3
8	C_8	1	25	3.5
9	C_9	1	125	2
10	C_10	1	125	2.5
11	C_11	1	125	3
12	C_12	1	125	3.5

Inputs for Multi Corner Analysis

For running Multi Corner analysis, you must specify an additional file along with the usual HSMC inputs. This file consists of the details of Process, Voltage and Temperature variations that you can explore in the `.csv` format. Throughout this chapter, this file is referred as `corners.csv`. Alternatively, PrimeSim AVA has a mechanism that can help generate the `corners.csv` for few general templates when provided with respective values.

Typical Corner File

A typical `corners.csv` file uses comma as delimiter, the first three columns are implicitly defined. The function and label are reserved.

Example 1

[Table 1](#) list the details of the typical corner file

Index	Name	Include	V	T	P
1	corner1	1	0.5	-40	ss_file
2	corner2	1	0.5	-40	ff_file
3	corner3	1	0.5	125	ss_file
4	corner4	1	0.5	125	ff_file
5	corner5	1	0.7	-40	ss_file
6	corner6	1	0.7	-40	ff_file
7	corner7	1	0.7	125	ss_file
8	corner8	1	0.7	125	ff_file

Where, ss_file

```
.lib "svt.lib" ss
.param flag = 0
.lib diode.lib ss
```

- **Index**, non negative unique integers, default is 1, 2, ...
- **Name**, unique alphanumeric labels (default is C_ \$Index)
- **Include**, 0/1 values for row selection, default is 1

Note:

PrimeSim AVA adds these columns implicitly even if they are missing in the `corners.csv` file

The labels for the remaining columns are arbitrary and can be alphanumeric ([a-z,A-Z][a-z,A-Z,0-9_]*) labels. For example, the file has three labels – V, T and P. Column properties section in the `user.json` file provides their interpretation.

Once all the columns are populated, the topmost row, that is header row, consists of all labels. And, each row in the `csv` file corresponds to the PrimeSim AVA simulation. For example, if the original netlist with parametrized voltage sources and without the `.temp` or `.Lib` statements.

```
...
Vdd 1 0 'V'      $ Parametrized voltage source
...
```

Then, the PrimeSim AVA simulation creates a netlist appending the following lines for corner1:

```
.param V = 0.5
.temp -40
.include ss_file      $ ss_file contains appropriate .Lib statements
  ss_file contents:
.lib "svt.lib" ss
.param flag = 0
.lib diode.lib ss
```

Entries in the Include column can be set to 0 for those corners you do not need to simulate. By default, all rows in the table are simulated. In below example:

Index	Name	Include	V	T	P
1	corner1	1	0.5	-40	ss_file
2	corner2	0	0.5	-40	ff_file
3	corner3	1	0.5	125	ss_file
4	corner4	0	0.5	125	ff_file
5	corner5	1	0.7	-40	ss_file
6	corner6	1	0.7	-40	ff_file
7	corner7	1	0.7	125	ss_file
8	corner8	1	0.7	125	ff_file

Entries in rows 2 and 4 are skipped in the previous table. This feature allows to simulate only a subset of corners and avoid having to delete rows that might be needed later.

Column Properties

Properties of all columns (except Index, Name, and Include) in the `corners.csv` file are specified in the properties section.

The properties section consists of various keys. The attribute key is required. Min, Max, Set, Alias, and Values keys are optional. Columns with the *File* attribute have an additional property referred as *Type*. Definitions that do not match labels in the `corners.csv` file are ignored.

Example 2

In this example, the attribute *File* and its *Type* is specified to read the `corners.csv` file in [Example 1](#).

```
{
  "Active_Application": "High_Sigma_Monte_Carlo",
  "High_Sigma_Monte_Carlo": {
    "Chip_Sigma": 5.0,
    "Component_Count": 1,
    "Average_Relative_Uncertainty": 4.0
  },
  "Corners": {
    "Corners_CSV": "corners.csv",
    "Properties": {
      "V": {"Attribute": "Voltage"},
      "T": {"Attribute": "Temperature"},
      "P": {"Attribute": "File", "Type": "Process"}
    }
  }
}
```

Example 3

The column properties can be further elaborated using range function. If range (Min, Max) or Set membership are specified for any column, PrimeSim AVA ensures that these conditions are satisfied for all entries in that column. This is an ease-of-use feature that helps avoid trivial mistakes in case the `corners.csv` file is created or edited manually. In [Example 1](#), you can specify range to ensure all rows fall within the expected corner limits.

```
"Corners": {
  "Corners_CSV": "corners.csv",
  "Properties": {
    "V": {"Attribute": "Voltage", "Min": -2, "Max": 2},
    "T": {"Attribute": "Temperature", "Min": -273.15, "Max": 200},
    "P": {"Attribute": "File", "Type": "Process", "Set": ["ss_file",
"ff_file"]}
  }
}
```

Example 4

Alias key allows long strings to be aliased to short and descriptive strings which can then be used in the `corners.csv` file. This is an ease-of-use feature, mainly for substituting long file names by short generic equivalents:

Index	Name	Include	V	T	P
1	corner1	1	0.5	-40	ss
2	corner2	1	0.5	-40	ff

Index	Name	Include	V	T	P
3	corner3	1	0.5	125	ss
4	corner4	1	0.5	125	ff
5	corner5	1	0.7	-40	ss
6	corner6	1	0.7	-40	ff
7	corner7	1	0.7	125	ss
8	corner8	1	0.7	125	ff

Where, SS -> ss_file

```
.lib "svt.lib" ss
.param flag = 0
.lib diode.lib ss

"Corners": {
  "Corners_CSV": "corners.csv",
  "Properties": {
    "V": {"Attribute": "Voltage"},
    "T": {"Attribute": "Temperature"},
    "P": {
      "Attribute": "File",
      "Type" : "Process",
      "Alias": {
        "SS": "ss_file",
        "FF": "ff_file"
      }
    }
  }
}
```

Validating the Input corners.csv File

PrimeSim AVA validates all inputs of the `corner.csv` file before any simulations are performed. For example, in certain scenarios, you may want to execute only the validation step.

To execute only the validation step, perform the following:

1. Create the `corners.csv` file as a template for further editing
2. Verify that the netlist setup is correct
3. Specify the `Dry Run` option in the netlist.

```
{
  "Properties": {
    "V": {"Attribute": "Voltage", "Min": 0, "Max": 1, "Values": [0.5,
0.7, 0.9]},
    "T": {"Attribute": "Temperature", "Min": -50, "Max": 150, "Values":
[-40, 125]},
    "P": {
      "Attribute": "File",
      "Type" : "Process",
      "Set": ["SS", "FF"],
      "Alias": {
        "SS": "ss_file",
        "FF": "ff_file"
      },
      "Values": ["SS", "FF"]
    }
  },
  "Generate_Corners": "V*T*P",
  "Dryrun": true
}
```

The `corners.csv` file is created under the `<output>_ava_corners/` directory. The Min, Max, and Set values for the columns of the `corners.csv` file are verified. Existence and permissions of all files and directories are checked. Final netlist syntax for each corner is checked. The `user.json` and measure spec files are checked. But, the final simulations are not performed

Generating corners.csv File

A common usage scenario is one in which the `corners.csv` file is constructed by all possible combinations of a few short vectors (cartesian product). PrimeSim AVA has a simple method to generate a template `corners.csv` file for such cases. The template file can be edited to match the requirements.

```
"Corners": {
  "Properties": {
    "V": {"Attribute": "Voltage", "Values": [0.5, 0.7, 0.9]},
    "T": {"Attribute": "Temperature", "Values": [-40, 125]},
    "P": {
      "Attribute": "File",
      "Type" : "Process",
      "Alias": {
        "SS": "ss_file",
        "FF": "ff_file"
      },
      "Values": ["SS", "FF"]
    }
  },
  "Generate_Corners": "V*T*P"
}
```

The `Generate_Corners` command creates corners on the fly and performs multi corner simulation. A `corners.csv` file is generated under the `<output>_ava_corners/` directory to review and modify as a template. The `Generate_Corners` command is ignored if the `corners.csv` file already exists. The `.JSON` file generates the following `corners.csv` file.

Index	Name	Include	V	T	P
1	C_1	1	0.5	-40	SS
2	C_2	1	0.5	-40	FF
3	C_3	1	0.5	125	SS
4	C_4	1	0.5	125	FF
5	C_5	1	0.7	-40	SS
6	C_6	1	0.7	-40	FF
7	C_7	1	0.7	125	SS
8	C_8	1	0.7	125	FF
9	C_9	1	0.9	-40	FF
10	C_10	1	0.9	-40	FF
11	C_11	1	0.9	-125	SS
12	C_12	1	0.9	125	FF

DUMMY Values for Manual Edits

In certain scenario, values in a column are hard to describe as a cartesian product and you may have to edit it later. If you want to use `Generate_Corners` for the other column, specify the special value *DUMMY* in the Values field and generate the `corners.csv` file as a Cartesian Product for the other columns. The `Dryrun` option is automatically set to true.

You can replaces the *DUMMY* entries in the `corners.csv` file by their desired values. *DUMMY* is a special token that bypasses the value and file existence checks.

```
...
    "Driver": {
        "Attribute": "File",
        "Type": "Include",
        "Values": ["DUMMY"]
    },
    "Generate_Corners": "P*VDD*T*DUT*Driver"
```

This generates a `corners.csv` as follows. Here, *DUMMY* must be replaced manually in the Driver column.

Index	Name	Include	V	T	P	Driver
1	C_1	1	0.5	-40	SS	DUMMY
2	C_2	1	0.5	-40	FF	DUMMY
3	C_3	1	0.5	125	SS	DUMMY
4	C_4	1	0.5	125	FF	DUMMY
5	C_5	1	0.7	-40	SS	DUMMY
6	C_6	1	0.7	-40	FF	DUMMY
7	C_7	1	0.7	125	SS	DUMMY

Table 2 lists Attributes, Implied Properties and PrimeSim AVA actions:

Table 2 Attributes, Implied Properties and PrimeSim AVA Actions

Attribute	Implied Properties	Assumptions, Actions, and Notes
Index (and col name)	Sequential integers starting at 1	AVA generated column
Name (and col name)	Unique strings [a-zA-Z][a-zA-Z0-9_]*	Identify each corner by name, default "C_\$Index". Serves as the default output directory name.
Include (and col name)	1 or 0	Set to 0 to skip simulating corner. Default is all 1.
File	Input file with suitable permissions	Further qualified through "Type" property, see next slide
Voltage	Real variable with finite range	Netlist should be parametrized with respect to the variable. PrimeSim AVA adds a <code>.Param</code> statement.
Temperature	Real variable with finite range	PrimeSim AVA adds a <code>.Temp</code> statement
Parameter	Real with finite range	PrimeSim AVA adds a <code>.Param</code> statement

Table 2 Attributes, Implied Properties and PrimeSim AVA Actions (Continued)

Attribute	Implied Properties	Assumptions, Actions, and Notes
Information	Arbitrary text treated as a string	Generally ignored by PrimeSim AVA (may use in subset selection). Example is a Comments column.
Output_Directory	Directory	Output directory rather than the default given in the Name column (limited usage expected).

Type Property of Columns Associated with Files

Type	Description
Process	An include file with appropriate the .Lib statements.
Netlist	Self contained netlists for different testbenches
DUT	Device under test, typically a subcircuit definition
IC	Initial conditions to be used as Nodeset
Include	Include file in netlist (Not to be confused with the column in corners.csv)
User_JSON	The user.json file associated with the corner.

User provided Netlist

You can define the netlist name with the `File` attribute and `Netlist` type in the `corners.csv` file. It is used as the input netlist for each corner.

Since the netlist is provided in the `corners.csv` file, there is no need to provide `-i <input.sp>` in the run command.

An example of the `corners.csv` file is as follows:

```
"Corners": {
  "Properties": {
    "case": {
      "Attribute": "File",
      "Type": "Netlist"
    }
  }
}
```

```
}
}
```

An example of CSV file that contains netlist path is as follows:

case
cell1/cell1_0.5_-40_ss.sp
cell1/cell1_0.5_-40_ff.sp
cell1/cell1_0.5_-40_tt.sp
cell2/cell2_0.5_-40_ss.sp
cell2/cell2_0.5_-40_ff.sp
cell2/cell2_0.5_-40_tt.sp
cell3/cell3_0.5_-40_ss.sp
cell3/cell3_0.5_-40_ff.sp
cell3/cell3_0.5_-40_tt.sp

Multiple meas_spec.csv file

The Measure_Specification file defines the measure spec csv file for each corner.

These files can have the same measurements or different measurements. If the measurements are not common, the corners.csv file takes a union of all corners and fill missing entries by value "---".

An example of the meas_spec.csv file is as follows:

```
"Corners": {
  "Corners_CSV": "corners.csv",
  "Properties": {
    "V": {"Attribute": "Voltage"},
    "T": {"Attribute": "Temperature"},
    "cell": {
      "Attribute": "Information"
    },
    "spec": {"Attribute": "File", "Type": "Measure_Specification" }
  }
}
```

An example of corners.csv file is as follows:

V	T	cell	spec
0.5	-40	A	a.csv

V	T	cell	spec
0.5	-40	B	b.csv
0.5	125	A	a.csv
0.5	125	B	b.csv
0.7	-40	A	a.csv
0.7	-40	B	b.csv
0.7	125	A	a.csv
0.7	125	B	b.csv

Outputs for Multi Corner Analysis Simulation

Table 3 lists the Multi Corner Analysis simulation outputs.

Table 3 Output for Multi Corner Analysis Simulation

File Name	Description
<output>.ava.log	Log file
<output>.ava.hm	Histogram file
<output>.ava.summary.csv	# Consolidated summary csv file
<output>.ava.summary.html	# Consolidated summary html file
<output>.ava.summary.qq.pdf	# Combined QQ plot PDF file
<output>.ava.summary.qq.wv_plt.grp	# WaveView QQ plot group file
<output>_ava_store/	# Directory for intermediate and ML data
<output>_ava_corners/	# The output directory for each corner simulation
corners.csv	# Generated csv file if "Generate_Corners" is set
C_1/ # Output directory of Corner 1	corner.inc # Include file output.ava.report output.ava.details.json output.ava.critical output.ava.qq.pdf output.ava.qqt.csv output.ava.qqt.wv_plt

Table 3 Output for Multi Corner Analysis Simulation (Continued)

File Name	Description
C_2/	

Output Files

This section covers the following topics:

- [<out>.ava.summary.csv](#)
- [<out>.ava.summary.html](#)
- [Supported Statistical Outputs in the CSV Summary](#)
- [Combined QQ Plot \(PDF\)](#)
- [wv_plt Group File \(Interactive\)](#)

<out>.ava.summary.csv

<out>.ava.summary.csv: The CSV Summary file contains the results for the corners (row by row). It consists of three parts: Inputs, Information, and Statistical Outputs. The statistical outputs contain metrics for each measure: Nominal, quantiles, moments, and confidence intervals.

Figure 70 Multi-PVT Summary CSV

Index	Name	Include	V	T	Rundir	ExitStatus	Nominal (meas1)	LQ(leakage)	RQ(leakage)	Mean(leakage)	StdDev(leakage)	...	Nominal (offset)	...
1	C_1	1	2.0	-40	.\out_corners\C_1	0	1.48e-5	1.118e-5	2.233e-5	1.53e-5	1.55e-6		3.139	
2	C_2	1	2.5	-40	.\out_corners\C_2	0	2.46e-5	1.857e-5	3.646e-5	2.47e-5	2.568e-6		3.257	
3	C_3	1	3.0	-40	.\out_corners\C_3	0	3.68e-5	2.791e-5	5.431e-5	3.69e-5	3.879e-6		2.013	
4	C_4	1	3.5	-40	.\out_corners\C_4	0	5.18e-5	3.944e-5	7.646e-5	5.144e-5	5.144e-6		3.571	

Inputs Information Statistical Outputs

<out>.ava.summary.html

The same information in the csv file can be shared in the form of a HTML page.

Statistical data for all corners

JWA Multi Corner																
file:///C:/Users/45454545/OneDrive/Desktop/test_topamp_ava_summary.html																
Show 15 entries																
Index	Name	Include	T	VDD	RunDir	ExitStatus	Nominal(leakpwr)	LQ(leakpwr)	RQ(leakpwr)	LQ_LCL(leakpwr)	LQ_UCL(leakpwr)	RQ_LCL(leakpwr)	RQ_UCL(leakpwr)	Mean(leakpwr)	StdDev(leakpwr)	Skewness(leakpwr)
1	C_1	1	-40	2.0	RunDir OOPlot	PASS	1.498e-05	1.119e-05	2.234e-05	1.101e-05	1.126e-05	2.206e-05	2.234e-05	0.000015	0.000002	0.5004
2	C_2	1	-40	2.4	RunDir OOPlot	PASS	2.250e-05	1.694e-05	3.334e-05	1.671e-05	1.705e-05	3.301e-05	3.437e-05	0.000023	0.000002	0.6120
3	C_3	1	-40	2.5	RunDir OOPlot	FAIL(code 1)	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
4	C_4	1	-40	2.6	RunDir OOPlot	FAIL(code 1)	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
5	C_5	1	-40	2.7	RunDir OOPlot	PASS	2.922e-05	2.208e-05	4.314e-05	2.183e-05	2.226e-05	4.279e-05	4.452e-05	0.000029	0.000003	0.7941
6	C_6	1	-40	2.8	RunDir OOPlot	PASS	3.167e-05	2.397e-05	4.669e-05	2.367e-05	2.415e-05	4.635e-05	4.822e-05	0.000032	0.000003	0.8132
7	C_7	1	-40	3.0	RunDir OOPlot	PASS	3.689e-05	2.793e-05	5.433e-05	2.764e-05	2.818e-05	5.390e-05	5.610e-05	0.000037	0.000004	0.8270
8	C_8	1	0	2.0	RunDir OOPlot	PASS	1.495e-05	1.119e-05	2.226e-05	1.101e-05	1.126e-05	2.198e-05	2.290e-05	0.000015	0.000002	0.4890
9	C_9	1	0	2.4	RunDir OOPlot	PENDING	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
10	C_10	1	0	2.5	RunDir OOPlot	PENDING	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
Showing 1 to 10 of 35 entries																
														Previous 1 2 3 4 Next		

Results summary

The status of the simulation and results directories are updated dynamically as each corner is completed. Clicking the links navigates to the corresponding output files.

Index ▴	Name ▴	Include ▴	T ▴	VDD ▴	Rundir ▴	ExitStatus ▴
▼	▼	▼	▼	▼	▼	▼
1	C_1	1	-40	2.0	Rundir QQPlot	PASS
2	C_2	1	-40	2.4	Rundir QQPlot	PASS
3	C_3	1	-40	2.5	Rundir QQPlot	FAIL(code 1)
4	C_4	1	-40	2.6	Rundir QQPlot	FAIL(code 1)
5	C_5	1	-40	2.7	Rundir QQPlot	PASS
6	C_6	1	-40	2.8	Rundir QQPlot	PASS
7	C_7	1	-40	3.0	Rundir QQPlot	PASS
8	C_8	1	0	2.0	Rundir QQPlot	PASS
9	C_9	1	0	2.4	Rundir QQPlot	PENDING
10	C_10	1	0	2.5	Rundir QQPlot	PENDING

Index	Name	Include	T	VDD	Status	Rundir	Nominal(leakpwr)	Min(leakpwr)	Max(leakpwr)	LQ(leakpwr)
8	C_8	1	0	2.5		Rundir QQPlot	1.495373e-5	9.470686e-6	NA	9.931173e-6
1	C_1	1	-40	2.5		Rundir QQPlot	1.498378e-5	9.769507e-6	NA	9.954151e-6
15	C_15	1	25	2.5		Rundir QQPlot	1.49444e-5	9.332323e-6	NA	9.971291e-6
22	C_22	1	75	2	PASS	Rundir QQPlot	1.494125e-5	9.817325e-6	NA	1.002278e-5
29	C_29	1	125	2	PASS	Rundir QQPlot	1.495265e-5	9.845615e-6	NA	1.005577e-5
23	C_23	1	75	2.4	PASS	Rundir QQPlot	2.224752e-5	1.480783e-5	NA	1.514932e-5
30	C_30	1	125	2.4	PASS	Rundir QQPlot	2.22072e-5	1.480894e-5	NA	1.516976e-5

Use the filter and sort buttons to navigate through the reports. Use **Columns Visibility Control** to show or hide the columns in the report. It allows you to hide/show columns that have NA values. It also allows to show or hide the columns visibility by groups, measure names, and statistical metrics.

Showing 1 to 10 of 35 entries Columns Visibility Control										
Index	Name	Include	T	VDD	Status	Rundir	Nominal(leakpwr)	Min(leakpwr)	Max(leakpwr)	LQ(leakpwr)
8	C_8	1	0	2.5		Rundir QQPlot	1.495373e-5	9.470686e-6	NA	9.931173e-6
1	C_1	1	-40	2.5		Rundir QQPlot	1.498378e-5	9.769507e-6	NA	9.954151e-6
15	C_15	1	25	2.5		Rundir QQPlot	1.49444e-5	9.332323e-6	NA	9.971291e-6
22	C_22	1	75	2	PASS	Rundir QQPlot	1.494125e-5	9.817325e-6	NA	1.002278e-5
29	C_29	1	125	2	PASS	Rundir QQPlot	1.495265e-5	9.845615e-6	NA	1.005577e-5
23	C_23	1	75	2.4	PASS	Rundir QQPlot	2.224752e-5	1.480783e-5	NA	1.514932e-5
30	C_30	1	125	2.4	PASS	Rundir QQPlot	2.22072e-5	1.480894e-5	NA	1.516976e-5

Use the **Hide empty (---) columns** check box to hide the columns where all cell values are “---” or “NA”.

In a column if all the cells have “---” value, you can hide this column to focus on other columns and reduce the table width. This check box helps in dynamically filtering the rows, if there are multiple measure spec files.

Cells with “---” values indicate that there should be no value, this happens in any of the following scenarios:

- One side tail analysis where the other tail related columns and moments related columns does not have a value and “---” is displayed.
- Multiple measure spec file and different corner can have different measure names. The measure related columns are not available in this corner and there value is shown as “---”.

Cells with “NA” values indicate the data is not collected due to some reason, but it is not expected.

Click the **Export CSV** button to export the summary details to a CSV file.

Showing 1 to 35 of 35 entries Export CSV Columns Visibility Control ☒ Hide empty(---) columns								
Index	Name	Include	T	VDD	Status	Rundir	Nominal(leakpwr)	Min(leakpwr)
1	C_1	1	-40	2	PASS	Rundir QOPlot	1.498378e-5	9.769507e-6
2	C_2	1	-40	2.4	PASS	Rundir QOPlot	2.250044e-5	1.487007e-5
3	C_3	1	-40	2.5	PASS	Rundir QOPlot	2.46353e-5	1.632306e-5
4	C_4	1	-40	2.6	PASS	Rundir QOPlot	2.687425e-5	1.784842e-5
5	C_5	1	-40	2.7	PASS	Rundir QOPlot	2.92182e-5	1.944678e-5
6	C_6	1	-40	2.8	PASS	Rundir QOPlot	3.166803e-5	2.111876e-5
7	C_7	1	-40	3	PASS	Rundir QOPlot	3.688893e-5	2.468605e-5

Supported Statistical Outputs in the CSV Summary

Table 4 *Statistical Outputs and Associated Applications*

Statistical Outputs	Application	Comments
Nominal	ALL	Nominal
LQ	HSMC, SA, MC	Left Quantile
RQ	HSMC, SA, MC	Right Quantile
LQ_LCL	HSMC, SA, MC	Left Quantile Lower Confidence Limit
LQ_UCL	HSMC, SA, MC	Left Quantile Upper Confidence Limit
RQ_LCL	HSMC, SA, MC	Right Quantile Lower Confidence Limit
RQ_UCL	HSMC, SA, MC	Right Quantile Upper Confidence Limit
Mean	HSMC, SA, MC	Moments – Mean
StdDev	HSMC, SA, MC	Moments – Standard deviation
Skewness	HSMC, SA, MC	Moments – Skewness
Kurtosis	HSMC, SA, MC	Moments – Kurtosis
Left_Spec	PYE, MC	Measure Left Specification
Right_Spec	PYE, MC	Measure Right Specification
CI_LL	PYE, MC	Lower Confidence Interval of Left spec failure
P_fL	PYE, MC	Failure Probability of Left tail beyond spec
CI_LU	PYE, MC	Upper Confidence Interval of Left spec failure
CI_RL	PYE, MC	Lower Confidence Interval of Right spec failure
P_fR	PYE, MC	Failure Probability of Right tail beyond spec
CI_RU	PYE, MC	Upper Confidence Interval of Right spec failure
CI_ML	PYE, MC	Lower Confidence Interval of measure failure
P_fM	PYE, MC	Failure Probability of measure
CI_MU	PYE, MC	Upper Confidence Interval of measure failure
CI_AL	PYE, MC	Lower Confidence Interval of All failures
P_fA	PYE, MC	Failure Probability across All measures (L/R/M combined)

Table 4 Statistical Outputs and Associated Applications (Continued)

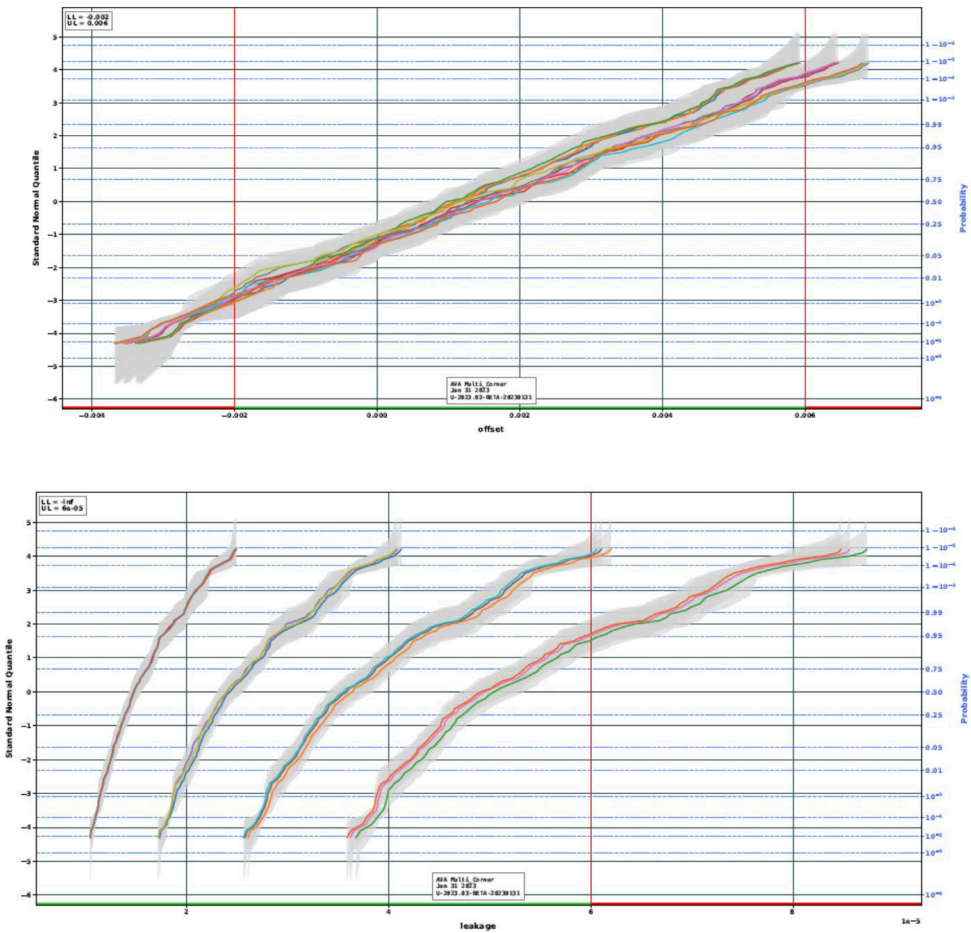
Statistical Outputs	Application	Comments
CI_AU	PYE, MC	Upper Confidence Interval of All failures

Combined QQ Plot (PDF)

QQ lines across all corners are merged with the confidence bands shown in grey shade.

- Useful as an overview of the distributions across all corners.
- Two vertical lines are added to help check if the results across all corners are within range when you provide a measure specification file.

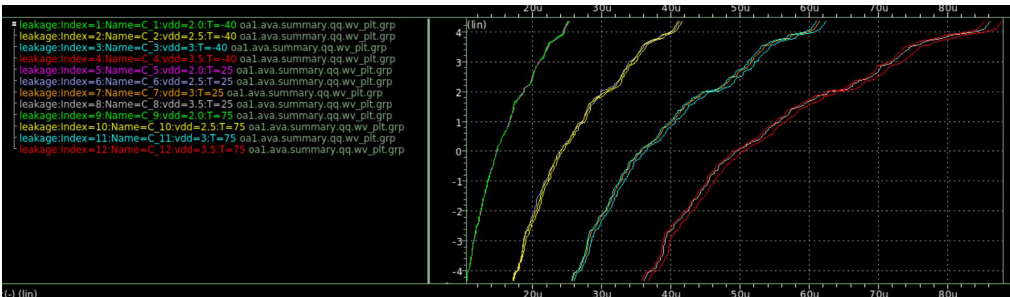
Figure 71 Combined QQ Plot (PDF)



wv_plt Group File (Interactive)

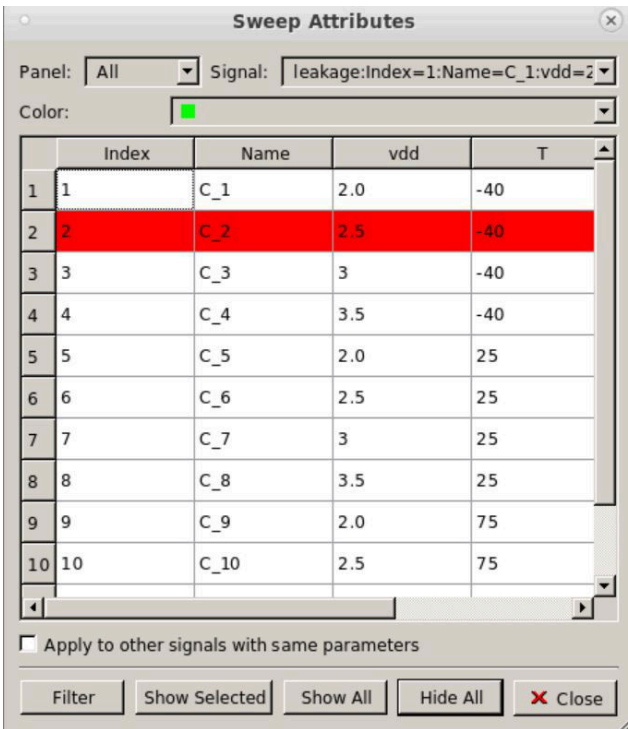
WaveView supports a group file (.grp), which can link multiple waveforms together. Move the cursor to highlight a specific QQ line and check the corner information on the left panel.

Figure 72 wv_plt Group File (Interactive)

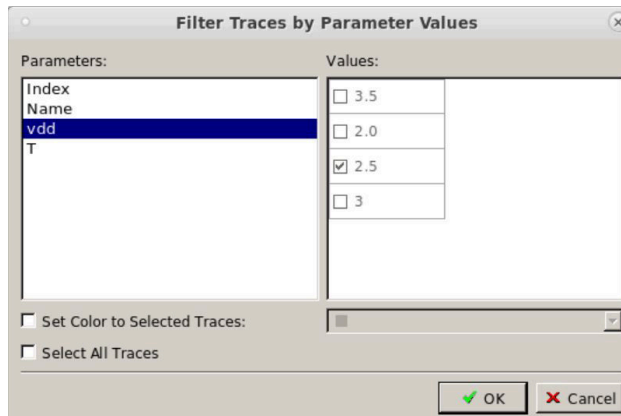


1. Right click on the QQ waveform and select **Sweep Attributes** to select specific corner waveform.

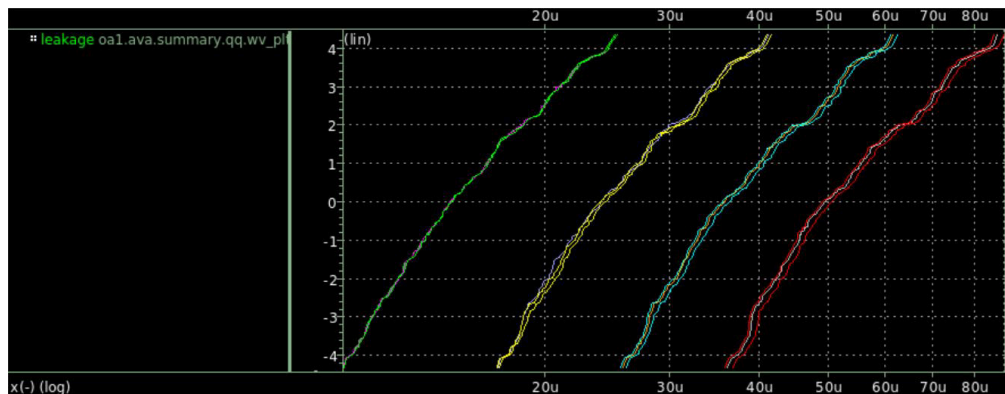
Figure 73 Sweep Attributes



2. WaveView provides interactive capability to filter specific corners. Click **Filter** on bottom-left corner in Sweep Attributes form. display shows all corners with $VDD = 2.5$.



3. Measures with log-normal distributions are often better viewed on a log scale.
 - Right click on the X-axis, a pop-up menu allows transformation from linear to log scale.
 - Examples of such measures include delay at low voltages and leakage power.



Multiple Sigma HSMC

Input

Chip_Sigma argument allows a list of sigma values as input . A warning is displayed if there are duplicate sigma entries. If any sigma value is outside the accepted range, an error is displayed.

```
User.json
{
  "Active_Application": "High_Sigma_Monte_Carlo",
  "High_Sigma_Monte_Carlo": {
    "Chip_Sigma": [3, 5],
    "Component_Count": 1,

```

```

    "Average_Relative_Uncertainty": 4.0
  },
  "Corners": {
    "Properties": {
      "case": {
        "Attribute": "File",
        "Type": "Netlist"
      }
    }
  }
}

```

Output Summary (CSV/HTML)

The output summary displays all information for a particular testbench in the same row.

Quantiles and Confidence Interval are reported for each sigma value as follows:

- LQ_3 -> Left Quantile at 3 sigma
- LQ_5 -> Left Quantile at 5 sigma
- LQ_LCL_3 -> Left Quantile Lower Confidence Limit at 3 sigma

Nominal and Moments are reported for each testbench.

Index	Name	Include	case	Rundir	ExitStatus	Nominal (leakage)	LQ_3(leakage)	RQ_3(leakage)	LQ_5(leakage)	RQ_5(leakage)	...	Mean(leakage)	StdDev(leakage)	...
1	C_1	1	A.sp	.jout_corner s/C_1	0	1.48e-5	1.118e-5	2.233e-5	1.118e-5	2.233e-5		1.53e-5	1.55e-6	
2	C_2	1	B.sp	.jout_corner s/C_2	0	2.46e-5	1.857e-5	3.646e-5	1.857e-5	3.646e-5		2.47e-5	2.568e-6	
3	C_3	1	C.sp	.jout_corner s/C_3	0	3.68e-5	2.791e-5	5.431e-5	2.791e-5	5.431e-5		3.69e-5	3.879e-6	
4	C_4	1	D.sp	.jout_corner s/C_4	0	5.18e-5	3.944e-5	7.646e-5	3.944e-5	7.646e-5		5.144e-5	5.144e-6	

Multi Corner Distributed Processing

When launching Multi corner analysis across the grid, you can expect resource failure in many forms. AVA addresses these issues and improves usability for such high-volume tasks.

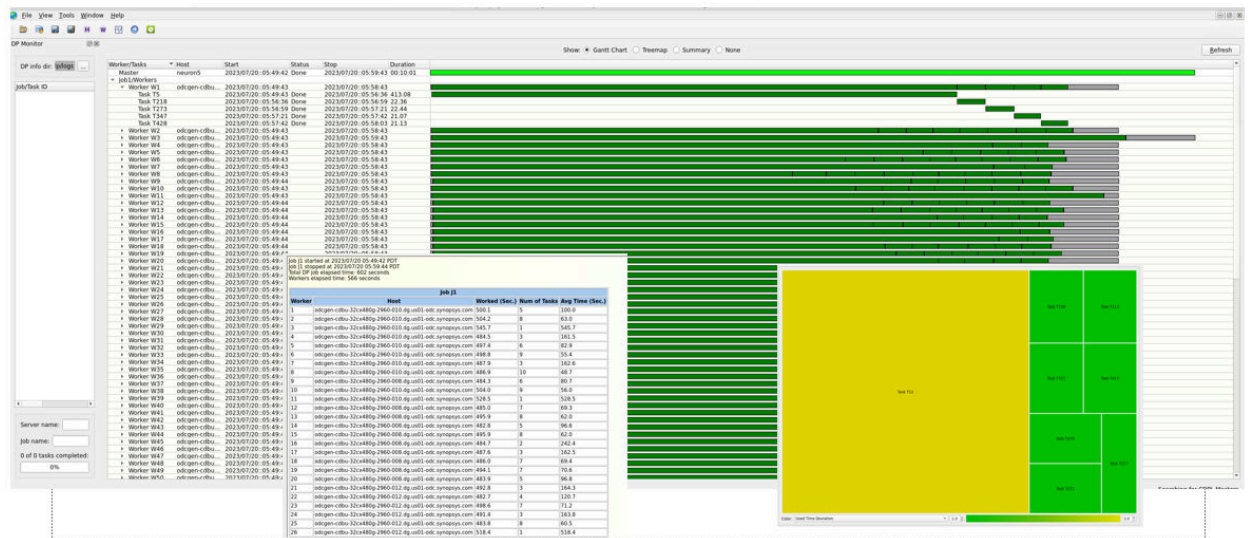
Key features

- Resource Monitoring: Monitor CPU load, memory, and disk usage of each machine.
- Resiliency: Tasks that violate resource limitations error out and are resubmitted to another machine. A machine that reports multiple failures is marked as 'bad' and dropped from the machine pool.

- Auto resubmission: A task that runs for much longer than its expected runtime is killed and resubmitted to another machine. The expected runtime is estimated dynamically from other similar tasks that have already completed.
- Dynamic resource allocation: The machine pool is adjusted dynamically based on the number of remaining tasks to avoid waste of farm resources.

Dynamic Task Monitoring and Control – CDPL DP Manager

The corner simulations submitted to the grid can be monitored using CDPL DP manager. The detailed report including host name, submission time, runtime and the realtime progress bar is available.



Worker Log Files

The information related to workers and pending tasks for each corner is available in the logs as follows:

<output>.ava.log

```
8.52 min: 440 tasks, 187 running, 145 pending, 108 done (0 failed), 0
killed [187 active workers]
COMPLETED task: C_3 T2 successfully
COMPLETED task: C_16 T15 successfully
COMPLETED task: C_25 T24 successfully
COMPLETED task: C_38 T37 successfully
COMPLETED task: C_39 T38 successfully
...
...
```

```
20.90 min: 440 tasks, 3 running, 0 pending, 437 done (0 failed), 0 killed
[9 active workers]
21.06 min: 440 tasks, 3 running, 0 pending, 437 done (0 failed), 0 killed
[9 active workers]
COMPLETED task: C_437 T436 successfully
COMPLETED task: C_438 T437 successfully
21.23 min: 440 tasks, 1 running, 0 pending, 439 done (0 failed), 0 killed
[3 active workers]
21.40 min: 440 tasks, 1 running, 0 pending, 439 done (0 failed), 0 killed
[3 active workers]
COMPLETED task: C_434 T433 successfully
21.57 min: 440 tasks, 0 running, 0 pending, 440 done (0 failed), 0 killed
[3 active workers]
All corners conclude
Multi Corner finished
Program finished successfully. (SUCCESS)
```

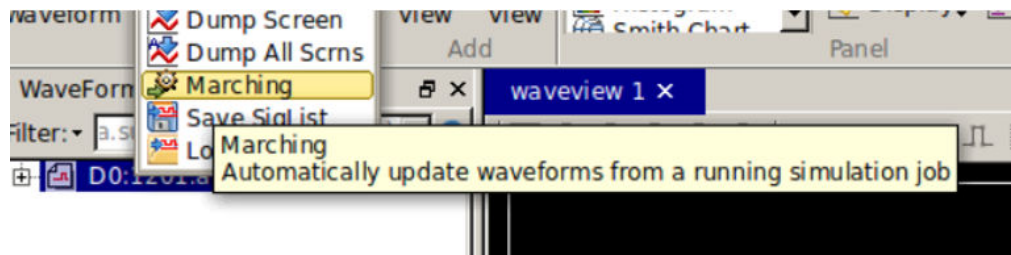
<output>_ava_dp/

```
twscript.sh
logs/
    thinworker.W180.odcgen-ams-360g-803-004.13089.err
    thinworker.W180.odcgen-ams-360g-803-004.13089.log
    ...
    master.neuron5.22234.err
    master.neuron5.22234.log
```

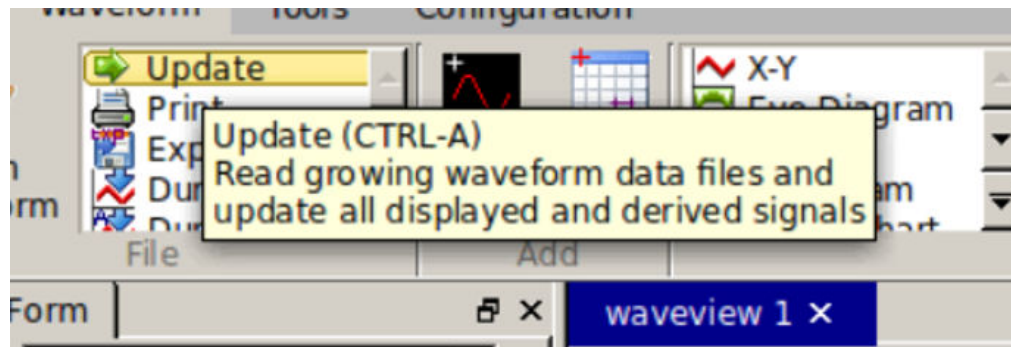
Advanced Features

- Partial summary files are generated when a corner completes and updated as results for more corners become available. You can view the results early and need not wait until all the corners are completed.
 - The full summary csv file is shown and pending is indicated in the exit status column for corners that are not yet complete. In addition, entries in the summary statistics sections are shown as NA for these corners.
 - The combined PDF QQ plot only shows the completed corners.
 - Similarly, the WV group file contains only the completed corners.
- WaveView can update plots as the group file changes.

- Click **Marching** to monitor file changes and update the display automatically.



- Or click **Update** to manually load the data and update the display.



Incremental Simulation

You can run a few corners for a preliminary analysis. Later, run the remaining corners without re-running the previous ones.

Example 5

Step 1:

Suppose the `corners.csv` file has 5 rows and if it initially runs 2 corners successfully. The output structure will be

```
<output>.ava.summary.csv
  <output>.ava.summary.qq.pdf
  <output>.ava.summary.qq.wv_plt.grp
  ...
  <output>_ava_corners/
    C_1/
    C_2/
```


Step 2:

Add 3 more corners for a total of 5 corners. Results for two corners already exist, they are skipped, and results generated for the 3 additional corners. The summary reports are updated to include all corners.

```
<output>.ava.summary.csv
<output>.ava.summary.qq.pdf
<output>.ava.summary.qq.wv_plt.grp
...
<output>_ava_corners/
    C_1/
    C_2/
    C_3/
    C_4/
    C_5/
```

Fault Tolerance

In case of machine failure, or if the simulation is suspended by you or the system, PrimeSim AVA can recover old results and continue from the point where it was interrupted. A second run of a command generally overwrites previous results. However, simulation cost in Multi Corner analysis is high, and the simulation data (intermediate or final) are valuable

For this reason, the subsequent run with the same command in Multi Corner analysis does not delete the previous data. Rather, it reads the existing data and continues from the point at which the run was interrupted.

Note:

Existing files and directories must be deleted manually to start the run from scratch.

Example 6

First run:

Suppose the `corners.csv` file has 5 rows and initially 2 corners run were successful, and during the C_3 corner simulation the process exits due to lack of disk space. The output structure:

```
<output>.ava.summary.csv
<output>.ava.summary.qq.pdf
<output>.ava.summary.qq.wv_plt.grp
<output>_ava_store/
<output>_ava_corners/
    C_1/                                # complete
    C_2/
.....
```

```
.....C_3/                                # interrupted
.....C_4/                                # not yet started
.....C_5
```

Subsequent run:

After disk clean up, if you run the same command to continue the task Corners C_1 and C_2 are already completed and are skipped. Simulation of C_3 is completed and then the remaining corners, C_4 and C_5 are processed. The summary reports are updated to include results for all corners.

```
<output>.ava.summary.csv
<output>.ava.summary.qq.pdf
<output>.ava.summary.qq.wv_plt.grp
<output>_ava_corners/
      C_1/
      C_2/
      C_3/
      C_4/
      C_5/                                # complete
```

Terminate and Suspend

Terminate

- Terminate the job through `Ctrl-C` or use `kill -15 <pid>`
- `kill -9` is not recommended unless the above methods fail to respond, as PrimeSim AVA may not be able to exit correctly and cleanup redundant files.

Suspend

- Use `Ctrl-Z` to suspend the job.
- Type `bg` or `fg` at the terminal to resume the job in background or foreground modes.

9

Sigma Amplification

This chapter describes the principles of the Sigma Amplification simulation. It provides information on performing a Sigma Amplification simulation in PrimeSim AVA and its outputs.

The following sections discuss these topics:

- [Principles of Sigma Amplification](#)
- [Sigma Amplification — Target Applications](#)
- [Running Sigma Amplification](#)
- [Configuring Inputs for Sigma Amplification](#)
- [Sigma Amplification Outputs](#)
- [Sigma Amplification Recommendations](#)
- [Application Example for Sigma Amplification](#)
- [Block Specific Sigma Amplification\(BSSA\)](#)
- [Sigma Amplification FAQ](#)

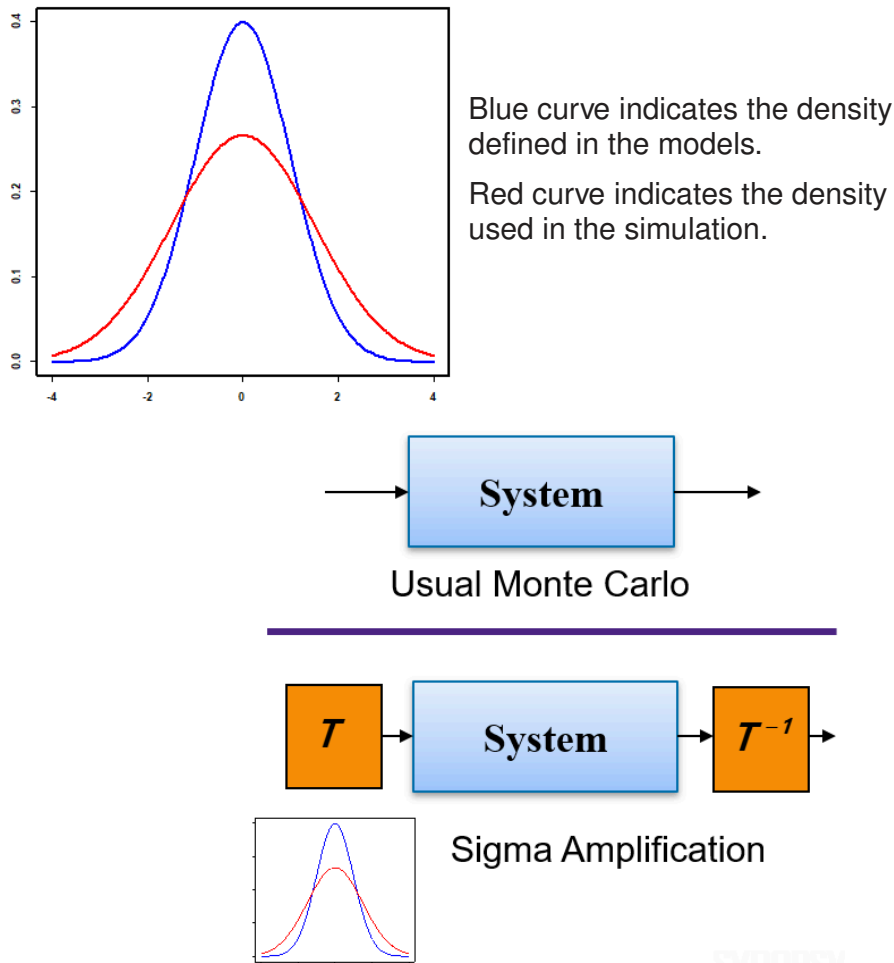
Note:

Throughout this chapter, all command-line invocations are denoted with `$SIM`. For the current chapter Sigma Amplification, `$SIM` refers to PrimeSim HSICE, PrimeSim SPICE, PrimeSim Pro, and PrimeSim XA

Principles of Sigma Amplification

Sigma Amplification is a special form of importance sampling that can be applied to IRVs whose distributions are Gaussian or Normal. This type of sampling amplifies the standard deviations of all IRVs by a common scaling factor to increase the number of samples in the tail regions (see [Figure 74](#)). This enables designers to have a better idea about the circuit behavior in the tails.

Figure 74 Sigma Amplification Principle



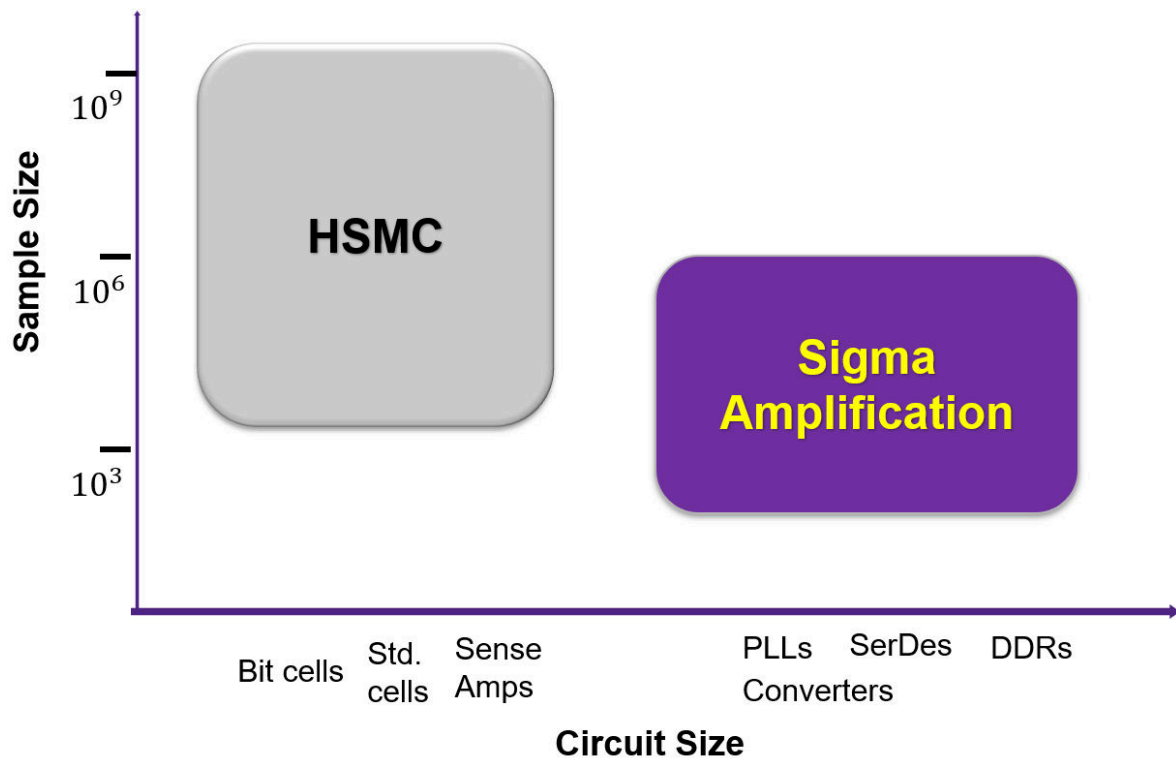
The Sigma Amplification technique is useful for validating the robustness of your circuit design (in terms of finding and fixing the most-likely failures) and for generating design-specific corners (which can be used during design iterations).

After Sigma Amplification, the results are post-processed and scaled back to correspond to the original IRVs for statistical inference.

Sigma Amplification — Target Applications

As illustrated in [Figure 75](#), Sigma Amplification is appropriate for larger circuits such as PLLs, DDRs, SerDes, and converters where a larger number of measures are typically used and for which traditional Monte Carlo techniques would take longer simulation times running into several days. In terms of chip level sigma, Sigma Amplification is more suited for the 2–5 σ range.

Figure 75 Target Applications for Sigma Amplification



Running Sigma Amplification

The syntax to run Sigma Amplification is similar to that for an HSMC simulation. See [Invoking HSMC on page 105](#).

Additionally, you should specify the affordable number of samples. This is to run the Sigma Amplification simulation with this reduced sample size. Once you specify this value, the simulator computes the Amplification Factor for the simulation. The smaller this value, the higher would be the Amplification Factor to meet the **Required sample size with simple random sampling** value. (You can check estimated value of this field in the PrimeSim Advanced Variability Analysis dialog box, with App Selection set to Sigma Amplification, see [Figure 76](#).)

It is suggested to have as large a sample size as you can afford, based on available computational power and simulation runtime, and keep the Amplification Factor low. The minimum acceptable value for the affordable number of samples is 300.

Sample user.json content for Sigma Amplification:

```
{
  "Active_Application": "Sigma_Amplification",
```

```
"Sigma_Amplification": {  
  "Chip_Sigma": 3.5,  
  "Component_Count": 1,  
  "Average_Relative_Uncertainty": 4,  
  "N_Afford_Samples": 1000  
}
```

If this configuration file is not provided, the PrimeSim Design Robustness tool displays a dialog box to help you create one or browse and open an existing file. See [Configuring Inputs for Sigma Amplification](#).

Parallel Processing for Sigma Amplification

Sigma Amplification in PrimeSim AVA supports a combination of distributed and threaded modes. The resource allocation depends on the cost of a single Monte Carlo simulation and on the required sample size with SRS.

The syntax for parallel Sigma Amplification processing is:

Syntax:

```
$SIM -i input.sp -o out -adv [user.json]  
+ [-dp k1 -dpconfig configfile] [-master k2]
```

For a description of the arguments in the preceding syntax, see [Invoking HSMC](#) and [Parallel Processing for HSMC](#).

Configuring Inputs for Sigma Amplification

The Advanced Variation Analysis dialog box to configure inputs for Sigma Amplification is shown in [Figure 76](#):

Figure 76 *Advanced Variation Analysis: Sigma Amplification Dialog Box*

The screenshot shows the 'PrimeSim Advanced Variability Analysis' dialog box. The 'App Selection' section has 'Sigma Amplification' selected. The 'App Setting' section contains the following fields and values:

Setting	Value
Chip Level Sigma	2.5
Component count	1
Component Sigma	2.5
Failure Probability	0.00621
Statistical Uncertainty: ?	
☐ 1% ☐ 2% ☐ 4% ☐ 8% ☒ 16% ☐ 32%	16.0
Required sample size with simple random sampling	618
Affordable number of samples:	350
Amplification factor:	1.09
95% Confidence Interval	(2.08, 3.01)

Below the settings is an 'Advanced Setting' button. At the bottom, there is a 'Status' bar with 'Info / Warning / Error Messages' and a file name 'dff_sa.json'. To the right of the file name are three buttons: 'Save and Launch' (with a rocket icon), 'Save and Quit', and 'Quit'.

For ease of use, the Sigma Amplification interface is designed to be consistent with the HSMC interface. Use the same steps as in [Configuring Inputs for MC](#) to configure basic and advanced settings for Sigma Amplification.

Determining the Amplification Factor for Sigma Amplification

The following factors influence the choice of the amplification factor for Sigma Amplification:

- How many samples can be simulated (computational budget)
- The required/target accuracy in terms of percentiles or sigmas

[Table 5](#) summarizes the links between the basic settings in the Sigma Amplification dialog box and the amplification factor for Sigma Amplification.

Table 5 *Sigma Amplification Basic Settings and Their Significance*

Setting	Significance
Chip Level Sigma	The higher this value, the higher is the amplification factor.
Component count	The higher this number, the higher is the Component Sigma and the amplification factor.
Statistical Uncertainty	The lesser this value, the narrower is the confidence interval and higher is the amplification factor.
Affordable number of samples	The lower this value, the higher is the amplification factor.

Depending on the configured values of these basic settings, the Sigma Amplification engine automatically calculates the amplification factor for the Sigma Amplification simulation.

Sigma Amplification Outputs

Sigma Amplification analysis produces the outputs as described in the following table:

File	Description	See Also
<code>out.ava.report</code>	Report file with moments and percentiles.	The *.ava.report File for Sigma Amplification
<code>out.ava.qq.pdf</code> , <code>out.ava.qq.wv_plt</code> , and <code>out.ava.qq.csv</code>	Q-Q plot files in the PDF (<code>.ava.qq.pdf</code> file), the WaveView (<code>.ava.qq.wv_plt</code> file), and CSV (<code>out.ava.qq.csv</code> file) formats, the same as in HSMC analysis.	Q-Q Plots in PDF, WaveView, and CSV Formats
<code>out.ava.log</code>	Log file.	

The *.ava.report File for Sigma Amplification

For Sigma Amplification, moments and percentiles corresponding to the original measures are printed to the *.ava.report file.

Figure 77 An Example of a *.ava.report File Output of Sigma Amplification

```
##Sample_Moments
#-----
#95% Bootstrap confidence intervals for moments are based on 500 re-samples
```

	log_pwr	offset1	offset2	offset3	offset4	pwr
Nominal	3.194637	1.399701	1.086253	0.9046714	0.7708879	24.41035
Mean_CI(2.5%)	3.18611	-3.987881	-1.36914	-0.8329339	-0.4496008	24.45353
Mean	3.191699	-1.632438	-0.3303901	-0.01644431	0.09082058	24.45819
Mean_CI(97.5%)	3.200413	1.22927	1.189539	1.008445	0.9515791	24.80531
Median_CI(2.5%)	3.185178	-4.348004	-1.997636	-1.215439	-0.7267868	24.25629
Median	3.35317	-20.09594	-8.93909	-5.596132	-4.035277	28.54844
Median_CI(97.5%)	3.202397	2.654732	1.886876	1.517724	1.256244	24.66423
StdDev_CI(2.5%)	0.1164778	43.97592	21.16834	13.86856	10.38605	2.865101
StdDev	0.1027337	38.49994	18.57514	12.25431	9.146361	2.504189
StdDev_CI(97.5%)	0.1263363	48.21631	23.02793	15.24115	11.38712	3.14927
MAD_CI(2.5%)	NaN	NaN	NaN	NaN	NaN	NaN
MAD	NaN	NaN	NaN	NaN	NaN	NaN
MAD_CI(97.5%)	NaN	NaN	NaN	NaN	NaN	NaN
Skewness_CI(2.5%)	-0.03286919	-0.2231984	-0.2115746	-0.1963482	-0.2255123	0.3108564
Skewness	0.02034824	-0.1276104	-0.1201882	-0.1189185	-0.1191466	0.3004066
Skewness_CI(97.5%)	0.2272422	0.08117058	0.08651983	0.08869908	0.1006173	0.5776409
Kurtosis_CI(2.5%)	2.71285	2.879286	2.869637	2.819584	2.828633	2.901553
Kurtosis	2.782066	3.326176	3.308697	3.302134	3.299168	3.019484
Kurtosis_CI(97.5%)	3.189391	3.413544	3.388854	3.363105	3.377304	3.543318

```
##Quantiles
#-----
# Percentile row header example : 'Q99.99683 ( 4)' means the 99.99683% percentile which is equivalent to 4 sigma
# Row with (2.5%) and (97.5%) are lower/upper Confidence Intervals
# Prefix with < or > indicate the confidence interval value is less or more than this boundary
```

		log_pwr	offset1	offset2	offset3	offset4	pwr
Q1e-03	(2.5%)	<2.754995	<-159.9807	<-76.31588	<-50.08933	<-37.284	<15.65633
Q1e-03	(-4.265)	2.760422	-159.3617	-76.0817	-49.94346	-37.17613	15.75507
Q1e-03	(97.5%)	2.867346	-146.1587	-68.06599	-44.84798	-33.41473	17.51974
Q3.17e-03	(2.5%)	<2.754995	<-159.9807	<-76.31588	<-50.08933	<-37.284	<15.65633
Q3.17e-03	(-4)	2.808013	-153.9334	-74.02788	-48.66414	-36.23001	16.62111
Q3.17e-03	(97.5%)	2.880105	-140.6845	-66.89793	-44.05094	-32.84835	17.63428
Q1e-02	(2.5%)	<2.754995	<-159.9807	<-76.31588	<-50.08933	<-37.284	<15.65633
Q1e-02	(-3.719)	2.850248	-149.2813	-71.31499	-46.9234	-34.93706	17.36138
Q1e-02	(97.5%)	2.892262	-134.3847	-64.22208	-41.92973	-31.04847	17.87279

Sigma Amplification Recommendations

- Minimum sample size: 300.
- Maximum amplification factor: 2; it is recommended that you use a sample size and uncertainty value that gives an amplification factor between 1.1 and 1.5.

Application Example for Sigma Amplification

The sample size for MC analysis is limited by the computation power and run time requirements. Computing high sigma quantiles is therefore a massive task. In the example described in this section, you can compare the results between Large Scale MC (LSMC) and the most simple extrapolation method.

Extrapolation assumes that the sigma value is directly proportional to the standard deviation, that is, the third sigma value is three standard-deviations away from the mean and so on. This holds true only when the distribution of the measure results is Gaussian. Because the assumption that all the measure results follow a Gaussian distribution is not always the case, you must compute the high sigma quantiles by using an MC simulation with a large sample size.

The example described in this section computes and compares the quantile values at 1-, 2-, 3-, and 4-sigmas between LSMC and extrapolation, to check if extrapolation would suffice to do the job. Additionally, we explore whether Sigma Amplification can overcome the shortcomings of the extrapolation method and can provide quantile values closer to the real distribution generated using LSMC.

Example

Table 6 Quantile Information for the opampmc.sp example

Measurement: Leakage Current (in uA)	-1 σ	-2 σ	-3 σ	-4 σ
Real distribution (2M samples)	8.87E+00	8.07E+00	7.39E+00	6.74E+00
Estimation assuming a Gaussian distribution ($\mu = 9.890\text{uA}$; $\sigma = 1.045\text{uA}$)	8.845	7.8	6.755	5.71
Estimation from Sigma Amplification	8.707	7.955	7.302	6.67

In the preceding table:

- **Row 1:** Two million sample run to measure 1 σ , 2 σ , 3 σ , and 4 σ values
- **Row 2:** Estimate 1 σ , 2 σ , 3 σ , and 4 σ values from mean and StdDev of the 10K sample run (assuming a Gaussian distribution)

Mean + n*stdDev (9.89 - 2 * 1.045=7.8 uA)

- **Row 3:** Sigma amplification MC to measure 1 σ , 2 σ , 3 σ , and 4 σ values

Note:

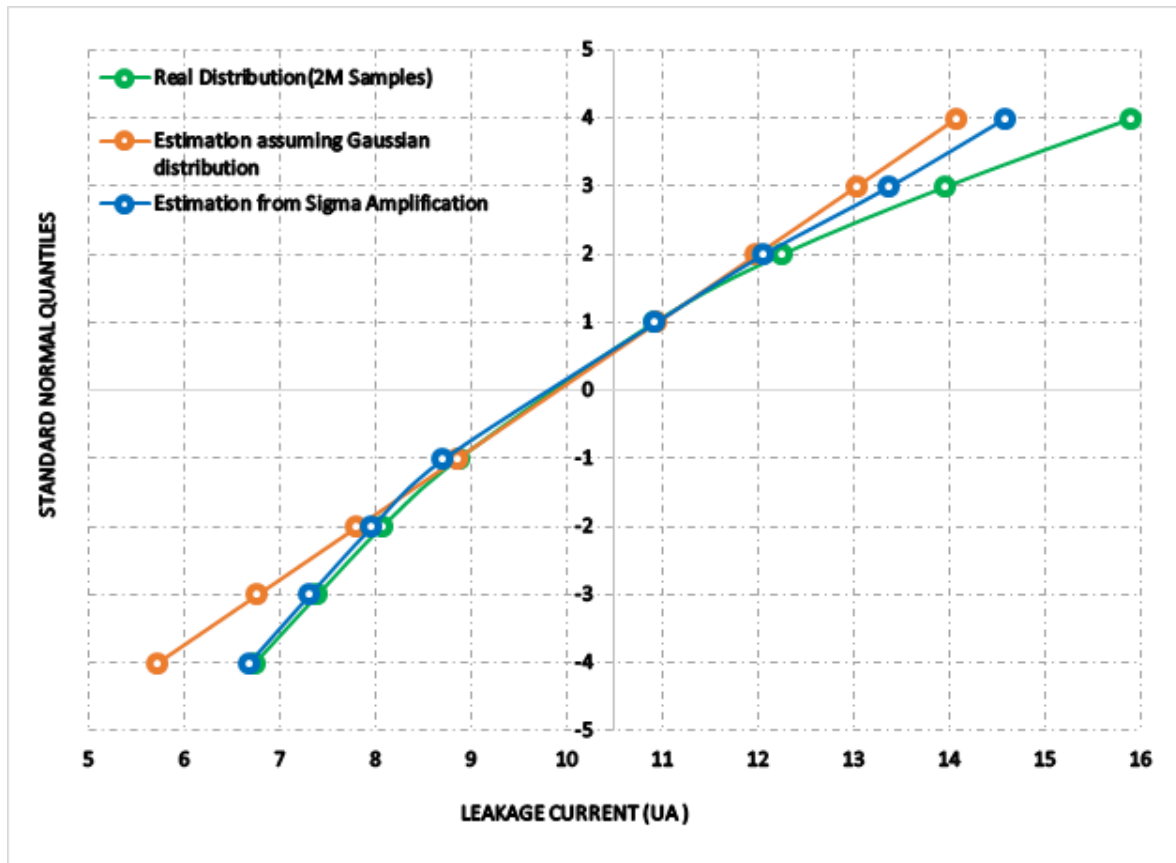
In this example, only the left side quantiles are considered and therefore, they are listed as (-n σ) in [Table 6](#).

Observations

Extrapolation does not predict the higher quantile values as correctly as LSMC. However, using LSMC takes a lot of space and a runtime of 15 minutes for 2 million samples. With Sigma Amplification, the values are closer to LSMC and it takes less than one minute to produce these results.

In the following graph containing the overlay of the Q-Q plots of each experiment, Sigma Amplification matches the real distribution (the LSMC run) better than extrapolation method, even near the tails.

Figure 78 Overlay of LSMC, Extrapolation, and Sigma Amplification Q-Q Plots



Block Specific Sigma Amplification(BSSA)

This chapter describes the principles of setting different amplification factors for different blocks in the design for circuit reliability analysis.

- [Introduction to Block Specific Sigma Amplification](#)
- [Invoking Block Specific Sigma Amplification](#)
- [Configuring Inputs for Block Specific Sigma Amplification](#)
- [Block Specific Sigma Amplification Outputs](#)

Note:

Throughout this chapter, all command-line invocations are denoted with `$SIM`. For the current chapter, `$SIM` refers to PrimeSim HSPICE, PrimeSim XA, PrimeSim SPICE, and PrimeSim Pro.

Introduction to Block Specific Sigma Amplification

In memory applications, each building block of memory such as Bit cells, Local IOs, and Local Control have different high-sigma requirements. Applying sigma amplification over the whole top level netlist using a single scaling factor for all blocks creates pessimistic results. To counteract, Block Specific Sigma Amplification allows you to set different amplification factors for different blocks in the design for circuit reliability analysis

Block Specific Sigma Amplification allows you to set different amplification factors for different blocks in the design for circuit reliability analysis, however the setting is complex as defining the different scaling factor values for memory blocks is difficult.

BSSA now supports setting sigma for each block, the corresponding scale factors are calculated based on the sigma and affordable sample numbers.

For example, applying a sigma amplification factor of 2 on bitcells (with sigma qualification of 6.3) creates an optimal design but applying the same amplification factor of 2 on Local Control (which has been designed only for 4.3-sigma qualification) may show false failures.

The supported simulators are PrimeSim HSPICE, PrimeSim SPICE, PrimeSim Pro, and PrimeSim XA.

Invoking Block Specific Sigma Amplification

Use the following syntax to invoke Block Specific Sigma Amplification through the Advanced Variation Analysis dialog box:

```
$SIM -i netlist.sp -o out/ -adv user.json
```

To invoke the specific simulators, use the appropriate syntax as listed below:

- `hspice` for PrimeSim HSPICE
- `primesim` for PrimeSim Pro
- `primesim -spice` for PrimeSim SPICE
- `xa` for PrimeSim XA

Use the `hspice` in place of `$SIM` to invoke PrimeSim HSPICE. For PrimeSim SPICE, PrimeSim Pro, and PrimeSim XA, refer to the respective chapters.

Inputs

Argument	Description
<code>netlist.sp</code>	Input SPICE netlist.

Argument	Description
-adv	After initial parsing and nominal simulation, invokes thePrimeSim AVA tool.
user.json	Input configuration file. Sample of user.json content for a HSMC simulation: <pre>{ "Block_Specific_Sigma_Amplification": { "N_Afford_Samples": 350, "Block_Sigma": [{ "Sigma": 2.5 }, { "Sigma": 4, "Instances": ["xi0.xrr1"] }, { "Sigma": 5, "Instances": ["xi0.xmm0", "xi0.xmm8", "xi0.xmm3"] }] } }</pre> If this configuration file is not provided, the PrimeSim AVA displays a dialog box to help you create one or browse and open an existing file. See Configuring Inputs for Block Specific Sigma Amplification.

Outputs

The Block Specific Sigma Amplification generates the following output.

File	Description	See Also
out.ava.critical	Contains critical corner sweep indexes and corresponding IRVs in MC0 file format.	*.ava.critical File
out.ava.hm	Health monitor file that lists the resources used and reports the progress of the simulation in real time.	*.ava.hm File
out.ava.log	Run log: memory, elapsed time, and so on.	

File	Description	See Also
<code>out.ava.report</code>	Nominal, minimum, and maximum measure values. Regular AVA outputs such as quantiles, moments, sensitivity analysis, and QQ plots are disabled. Due to different scale factors applied to different circuit blocks, the statistics are less correlated to realistic process variation.	The *.ava.report File

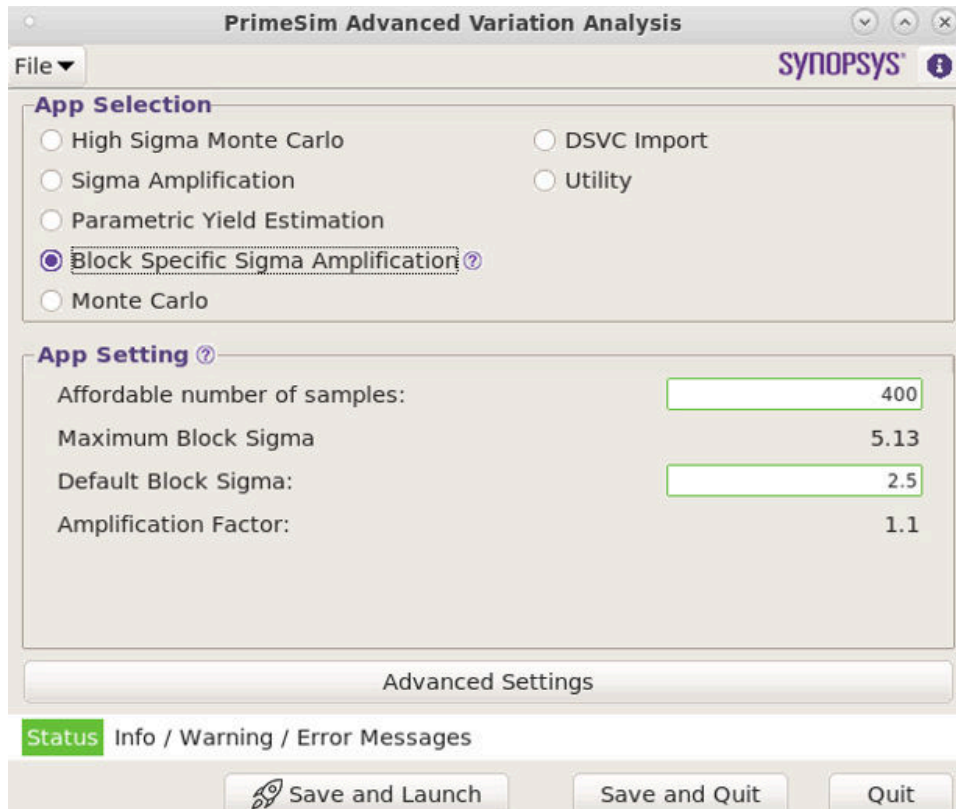
Configuring Inputs for Block Specific Sigma Amplification

Use the command line to launch the PrimeSim AVA simulation, the design hierarchy is read by AVA and displayed after the netlist parsing is completed. Circuit blocks are identified by instance and/or subcircuit.

A global amplification factor is defined for the whole netlist in the **Basic Settings**. You can specify scale factors for global, instances and subcircuits via the `.json` input file or through the dialog box.

1. Use the `$SIM -i netlist.sp -o out/ -adv user.json` syntax to invoke the following screen

Figure 79



2. Choose **Block Specific Sigma Amplification**.
3. Enter the **Affordable number of samples**.

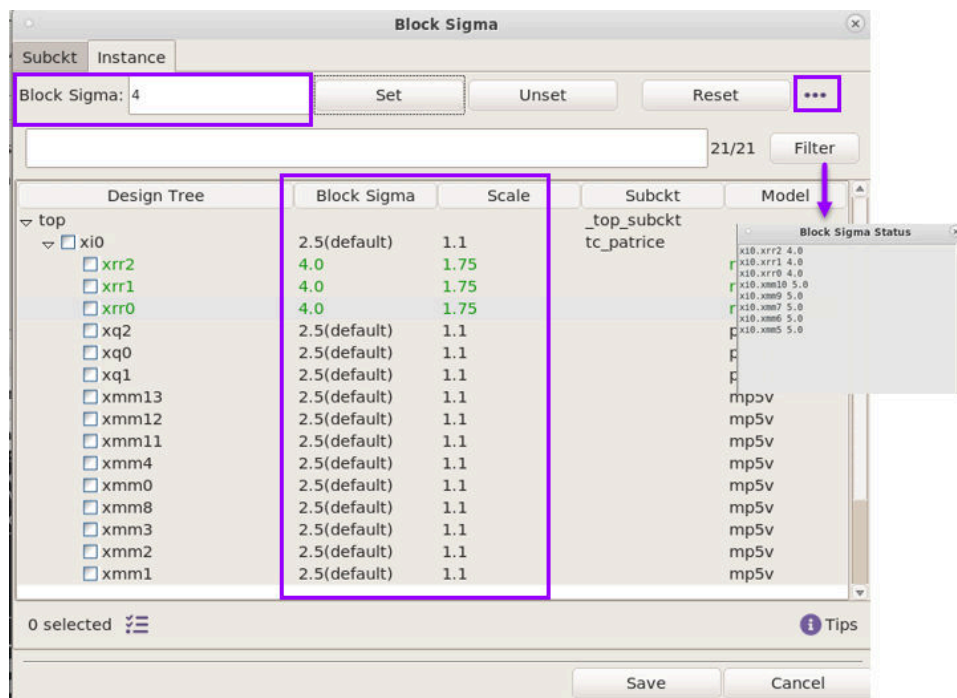
Based on the **Affordable number of samples**, the PrimeSim AVA tool sets the **Maximum Block Sigma** value. You cannot set individual block sigmas larger than this value.

- The max limitation is based on the max scale factor = 2.0
 - You can increase the **Affordable number of samples** if you need a higher block sigma
 - **Maximum Block Sigma** cannot be larger than 6.5
4. Enter the **Default Block Sigma**, which is the global sigma for each block by default
Amplification Factor is the corresponding scale factor of default Block Sigma.

5. To set specific amplification factors for different blocks, click **Advanced Settings > Scale Factor**. The dialog box displays the netlist in the form of a design hierarchy.
6. Select the required instances using the check boxes on the left. For selected instances, you can set the sigma of each block--the scale factor of each block is calculated from the block sigma--then click **Set**.

The block sigmas cannot be larger than the **Maximum Block Sigma**. A warning box appears if you increase the affordable samples to get a larger sigma. Click **OK** to increase the affordable samples or click **Cancel** to select a small sample.

Figure 80 Scale Factor



If the scale factor < 1 , it resets to 1.

7. Click the top-right ... button to display the instance/subckts and their block sigma. This is useful for large netlist, to quickly have an overview of the status.
8. Click the **Save** button to save the settings.
9. (Optional) For more settings, use the other tabs in the **Advanced Settings** option.
10. Click the **Save and Launch** button to invoke the BSSA simulation.

Block Specific Sigma Amplification Outputs

The Block Specific Sigma Amplification generates the following output.

- [.mt# File](#)
- [.ava.report File](#)
- [.ava.critical File](#)

.mt# File

The BSSA is run for the specified number of samples under **Affordable number of samples**. The measure values are listed in the `.mt#` file. This is the Primary output for failure analysis.

.ava.report File

The `.ava.report` file lists the minimum and maximum indexes and corresponding measure values. The `ava.report` file lists nominal, min, and max of measure values as well as number of failures. [The *.ava.report File](#) shows an example of the `.ava.report` outputs.

Regular AVA outputs such as quantiles, moments, sensitivity analysis, and QQ plots are not available in BSSA.

Figure 81 `out.ava.report File`

```
#Number of Samples
499
#Number of Measures
1
t_delay1
Nominal      1.801e-09
Min          1.751e-09
Max          7.256e-09
Number_of_Failures 0
```

.ava.critical File

The `out.ava.critical` file consists of the list of IRVs for the min and max indices listed in the report file. These can be used further for re-simulating. If the **Critical Waveform** option is selected under **Advanced Settings > General**, then corresponding waveforms are also generated. The [*.ava.critical File](#) shows the `out.ava.critical` outputs.

Figure 82 *out.ava.critical File*

```
.data cp_data
index par1:@:random1:@:ignc par2:@:random2:@:ignc par3:@:random3:@:ignc par4:@:random4:@:ignc x
inv1.mpl.temp1:@:snps20p:@:par1:@:ilnc xinv1.mn1.temp1:@:snps20n:@:par1:@:ilnc xinv2.mpl.temp
1:@:snps20p:@:par1:@:ilnc xinv2.mn1.temp1:@:snps20n:@:par1:@:ilnc xinv1.mpl.temp2:@:snps20p:@
:par12:@:ilnc xinv1.mn1.temp2:@:snps20n:@:par12:@:ilnc xinv2.mpl.temp2:@:snps20p:@:par12:@:ilnc
xinv2.mn1.temp2:@:snps20n:@:par12:@:ilnc
* Critical_Point: nominal
1 0 0 0 0 0 0 0 0 0 0 0
* Critical_Point: t_delay1.min
2 3.134988 0.460028 -0.994426 -0.770400 0.794184 -1.214515 -2.161450 0.443523 0.095113 0.849720
1.763177 -1.649364
* Critical_Point: t_delay1.max
```

Sigma Amplification FAQ

- Does Sigma Amplification depend on the foundry or the technology node?

No. It is independent of the foundry and technology node.

- Does Sigma Amplification depend on the number of measures?

It is independent of the number of measures. However, the storage for the Q-Q plots grows linearly with the number of measures.

The postprocessing time and the file size are acceptable even for circuits with ~3K measures.

- Does Sigma Amplification work with all circuit types and measures?

It works very well when the response distribution is close to Gaussian.

It has been found to work well in many cases even when the distribution departs significantly from Gaussian but is continuous.

It currently does not work for discrete measures like pass/fail. As far as possible, use continuous measures.

- Does Sigma Amplification depend on circuit size?

For the most part, it is independent of circuit size and the number of IRVs, but care is required to avoid excessive number of parameter out-of-bound errors.

Also, larger circuits take longer to simulate.

- What is the algorithm associated with the inverse transformation in Sigma Amplification?

This information is Synopsys proprietary.

- How are the confidence intervals created on the Q-Q plot?

The confidence intervals are based on the relation between order statistics and the beta distribution, and are exact.

- Do the confidence intervals depend on the technology node and circuit type?

The confidence intervals depend only on sample size and the amplification factor.

- Can I use a very small sample size, say 30?

Small sample size is not recommended because the confidence intervals will be very wide and the simulation results will not be reliable.

- Can I use a large Sigma Amplification factor to reduce the sample size drastically?

While the Y-axis range on the Q-Q plot increases with a higher amplification factor, the confidence interval width is also a function of the amplification factor and will be wider for a large amplification factor.

10

Design Specific Variation Corners Export/Import

This chapter details the export/import of designs specific variation corners.

- [Principles of Design Specific Variation Corners Export/Import](#)
- [DSVC Export](#)
- [Configuring Inputs for DSVc Export](#)
- [DSVC Import](#)

Note:

Throughout this chapter, all command-line invocations are denoted with `$SIM`. For the current chapter Design Specific Variation Corners Export/Import, `$SIM` refers to PrimeSim HSPICE, PrimeSim XA, PrimeSim SPICE and PrimeSim Pro.

Principles of Design Specific Variation Corners Export/Import

Monte Carlo analysis possesses major challenge in designs like SRAMs, since billions of samples of bitcells must be executed to meet 6-sigma qualification. Additionally, this problem is compounded in cases of RAM generator where the validation is done across PVT corners and multiple cuts. The possible non-Gaussian nature of the measures discourages using simple methods like extrapolation of lower sigma values toward higher sigma or Extreme Value Distributions (EVDs). These methods save time and computational power but cannot meet the scalability across PVT corners and stimulus sets. It also requires extensive data analysis to create such distributions accounting for all probable read/write failures. So, designers often tend to be conservative by leaving additional specification margins resulting in over designing.

To achieve better accuracy with fast runtimes and cover extensive scenarios, the PrimeSim AVA application Design Specific Variation Corners (DSVC) employs weak cell methodology. This consists of two phases:

- **DSVC Export:** Generation of IRVs of selected instances at Design Specific Variation Corner at a given sigma for a specified measure in a standalone simulation. This data forms the input to estimate measure distribution and failures in full-instance simulation.
- **DSVC Import:** Simulate the top netlist with selected cells set in DSVC. This creates and runs only the extreme corners of the top netlist containing full instance without running expensive Monte Carlo simulation. The measure data generated provides overview of the distribution and is helpful in calculating yield.

Figure 83 and Figure 84 illustrates the DSVC.

Figure 83 Design Specific Variation Corners – 1

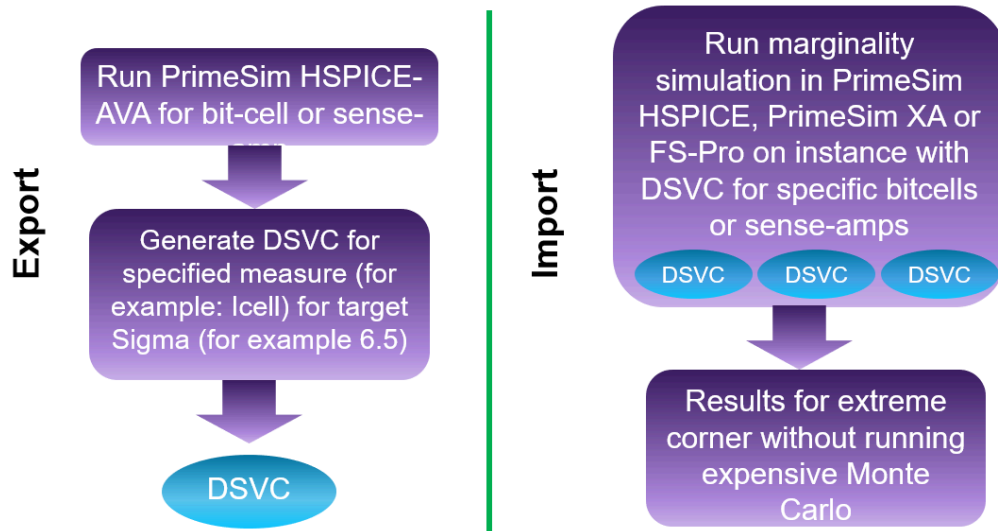
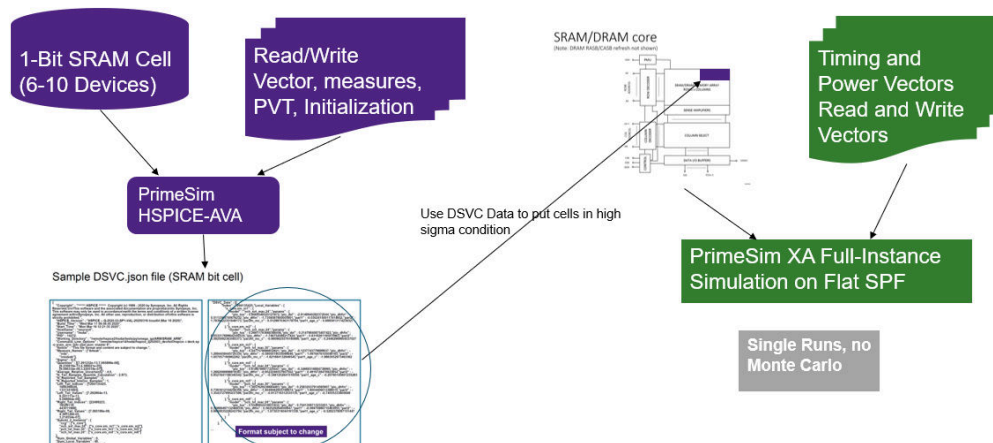


Figure 84 Design Specific Variation Corners – 2



DSVC Export

In a top-level netlist of SRAM or memory cores, the individual blocks like bitcells or sense amplifiers are repeated in multitude. The [System and Component Reliability](#) as the repetition of a cell becomes higher, the Monte Carlo simulation must be performed for achieving a higher target sigma. This could mean billions of samples, high resource and time consumption to estimate the outputs of statistically worst cases.

Design Specific Variation Corners (DSVCs) are the extreme conditions where an individual cell possibly poses a challenge in meeting its specification. These variation corners are defined for each measure defined in the individual cell netlist. Currently, the following variation corners are accounted for by PrimeSim AVA:

- Minimum
- Maximum
- Left most quantile value
- Right most quantile value

The PrimeSim AVA applications High Sigma Monte Carlo or Sigma Amplification are used to create DSVCS. The target sigma of the individual cell is decided first. For this target sigma, High Sigma Monte Carlo analysis or Sigma Amplification must be performed to create the distribution of the output measures. From this simulation, you can opt to create Variation Corners pertaining to each of the measures. The **DSVC Export** tab in Advanced Settings in Advanced Variation Analysis dialog box helps select the required devices that need to be put under extreme conditions while simulating the top netlist in later steps that is DSVCS Import.

See Also

- [Critical Points](#)

Running DSVC Export Using High Sigma Monte Carlo

Use the following syntax to invoke High Sigma Monte Carlo application through Advanced Variation Analysis dialog box:

```
$SIM -i <input> -adv <user.json> -o <out>
```

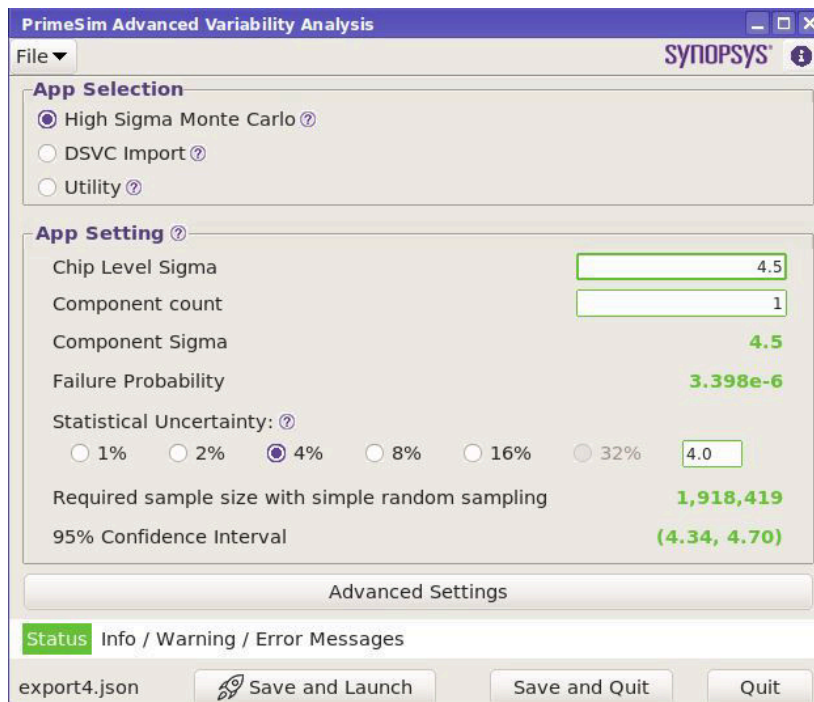
Argument	Description
<code>\$SIM</code>	PrimeSim HSPICE or PrimeSim
<code>-i <input></code>	Input netlist of the individual component like bitcell or sense amplifier
<code>-adv <user.json></code>	Input configuration file. Sample of <code>user.json</code> content for a DSCV Export simulation: <pre>"Active_Application": "High_Sigma_Monte_Carlo", "High_Sigma_Monte_Carlo": { "Chip_Sigma": 3.5, "Component_Count": 1, "Average_Relative_Uncertainty": 4.0 }, "DSCV_Export": { "Instances": ["xi82.mn1", "xi82.mn2"] }</pre>
<code>-o <out></code>	Output file prefix or folder

Configuring Inputs for DSVC Export

Use the export syntax to invoke High Sigma Monte Carlo application through Advanced Variation Analysis dialog box and perform the following:

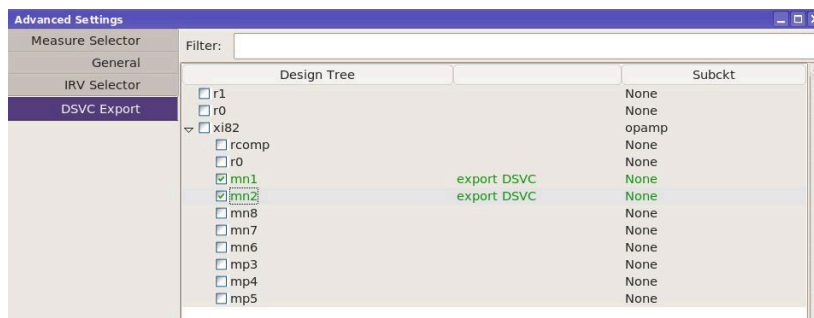
- Choose High Sigma Monte Carlo.
- Select the inputs for **Chip Level Sigma**, **Component count**, and **Statistical Uncertainty** values for achieving the target sigma of the individual cell.

Figure 85 DSVC Import



3. Click **Advanced Settings** to open the **Advanced Settings** dialog box, then click the **DSVC Export** tab.

Figure 86 DSVC Export



4. Check the components of the netlist from the Design Tree to list the instance's IRVs in the output file.
5. Click the **X** icon to save and close the Advanced Settings.
6. Select the **Save and Launch** option to run the HSMC simulation.

Outputs

Once simulation completes, the above DSVC Export step generates an AVA output file in JSON format `out.ava.dsvc.json`. This file consists of the DSVCs per measure. For more details, see [*.ava.dsvc.json File](#).

DSVC Import

Use the information of DSVCs in `*.ava.dsvc.json` file, to import top level netlist using DSVC import application in PrimeSim AVA. The AVA dialog box reads the JSON file and displays possible variation corners for all measures.

For a given measure's variation corner, you can map specific instances of top-level netlist to be set in the selected DSVC making them weak cells. This results in enabling specific n-sigma weak cell analysis for the entire top-level netlist.

Simulation of this top-level netlist with few components set in DSVCs provide results for extreme corners without running Monte Carlo simulation. So, DSVC Import application provides a way to perform design assessment with simulation cost slightly higher than that of a nominal simulation, instead of expensive Monte Carlo simulation.

Running DSVC Import

Use the following syntax to invoke DSVC Import application through the Advanced Variation Analysis dialog box:

```
$SIM -i <input> -adv <user.json> -o <out>
```

Argument	Description
<code>\$SIM</code>	PrimeSim HSPICE or PrimeSim
<code>-i <input></code>	Input netlist of the top-level netlist containing the instances of the individual cells from DSVC Export step

Argument	Description
<code>-adv <user.json></code>	Input configuration file. Sample of <code>user.json</code> content for a DSVCS_Import simulation: <pre>{ "Active_Application": "DSVC_Import", "DSVC_Import": { "Data": [{ "DSVC_File": "<path_to_file>/opampmc.ava.dsvc.json", "Variation_Corners": ["leakpwr.Max"], "Instance_Map": { "xi82.mn1": ["xi82.mn6"], "xi82.mn2": ["xi82.mn8", "xi82.mn7"] } }, "Scale_Factor": 1.0] }, "Dryrun": true, "Nominal_Run": true }</pre>
<code>-o <out></code>	Output file prefix or folder

Configuring Inputs for DSVC Import

Use the import syntax to invoke High Sigma Monte Carlo application through Advanced Variation Analysis dialog box and perform the following:

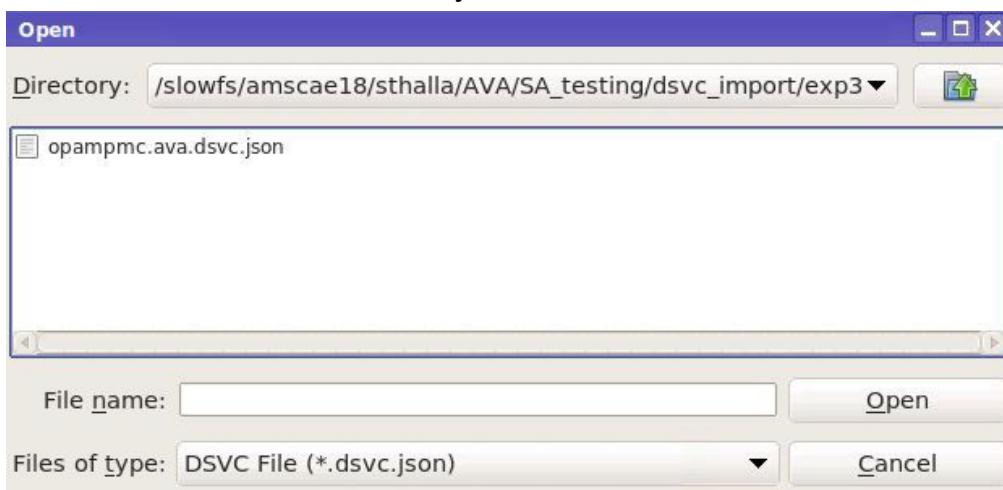
1. a. Select the **DSVC Import** application.

Figure 87 DSVC Import



2. In the **App Setting** area, select the **Import DSVC** option. Select the file browser to select the *.ava.dsvc.json file generated in DSVC Export step.

Figure 88 Selection of the *.ava.dsvc.json File

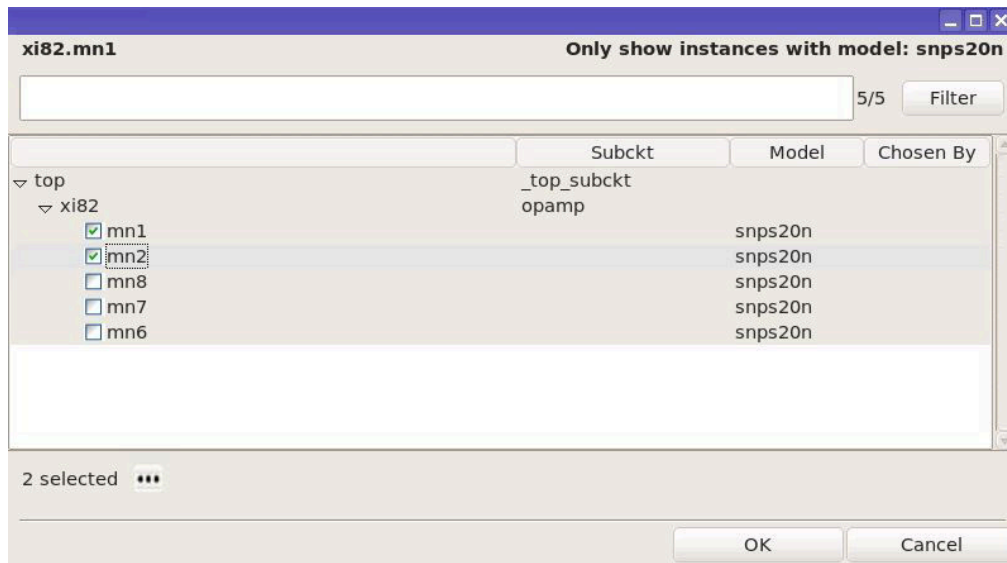


3. The PrimeSim AVA tool imports the measure and DSVC data corresponding to each measure and displays the **DSVC Import Edit** dialog. In **Select Variation Corner**

section, select the target measure and target variation corner (for example: Min, Max, QL, QR). This combination of measure and its specific corner together is called as Design Specific Variation Corner.

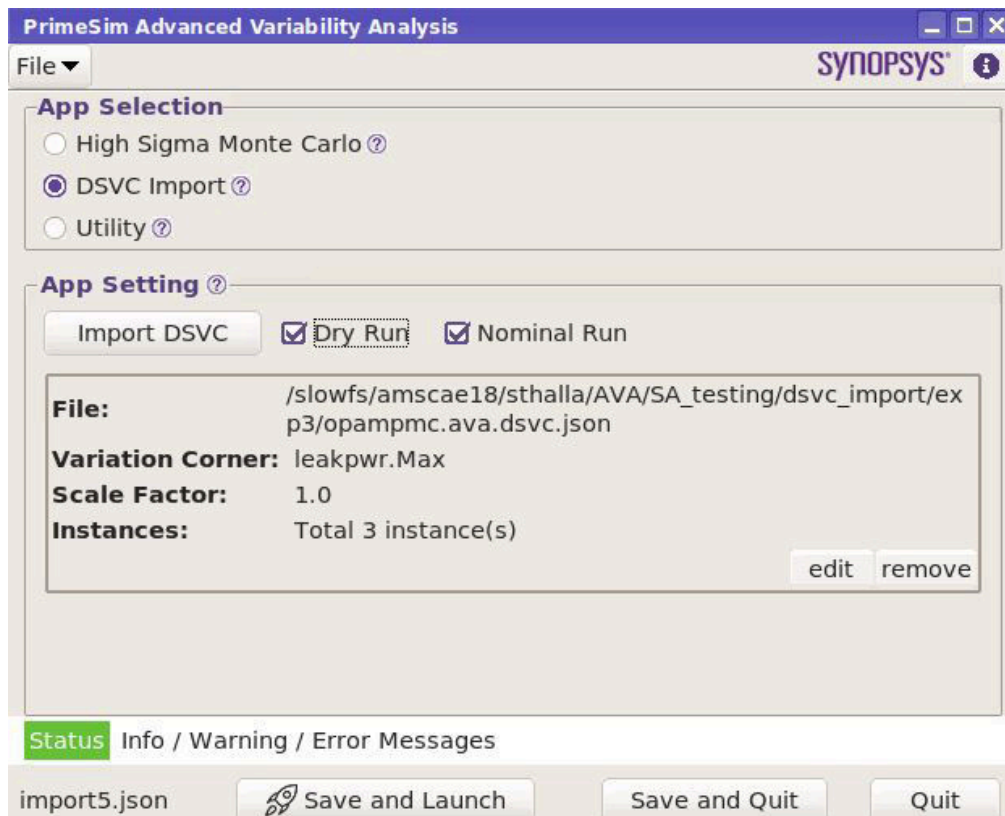
- Once the DSVC is decided, select the instances of the top-level netlist that must be set in above DSVC. This process is called mapping. All the instances exported are listed in the DSVC Import Edit Dialog. In **Setup Target Instances** section, click the **Setup** beside individual cell's instance to bring up the top-level netlist design tree.

Figure 89 Setup Target Instances



- Click the checkbox beside the instances which must be set in DSVC of individual cell's instance.
- Click **OK**. The **DSVC Import Edit** dialog displays the number of top-level instances mapped into design corner of individual cells.
- After mapping all the instances required, click **OK**. The AVA dialog box displays the summary of the instance mapping done in steps 3-6.

Figure 90 Summary of the Instance Mapping



Note:

Use the **Scale Factor** option for the cases where the `out.ava.dsvc.json` file is generated for a target sigma slightly apart from intended target sigma. Using a `.dsvc.json` file generated for 5 sigma and a scale factor of 1.1, the target sigma 5.5 is achieved.

8. By default, Scale Factor is set at 1. The Scale Factor must be non-negative and preferably in range [0.75, 1.25]
9. Select the **Dry Run** check box to skip the simulation and identify variation corners in DSV file and list instances to substitute in AVA log. This performs necessary error check and serves as a check. Additionally, this allows creation of the `.json` file, which must later manually be edited to include cell names which need to be replaced by DSV file.
10. Check the **Nominal run** check box to additionally perform nominal simulation along with DSV simulation.
11. Click the **Save and Launch** option to complete the DSV Import simulation.

Outputs

A single simulation of the top-level netlist is performed with selected cells set in DSVCs, and the measures are populated in the .m(s/t/d)# file. You can verify the top-level netlist for weak cells at targeted sigma using these measures.

If Nominal Simulation is enabled, the measure file consists of both nominal and DSVC results.

Figure 91 *DSVC Output Results*

```
$OPTION MEASFORM=1
$there is no nominal sample in Monte Carlo
.TITLE '*** copyright (c) 2005 synopsys, inc. all rights reserved'
index systoffset1 systoffset2 systoffset3 systoffset4 leakpwr logleakpwr temper alter#
1 1.399701e-03 1.086253e-03 9.046714e-04 7.708879e-04 2.441035e-05 -10.6205034 25.0000000 1
2 1.099966e-03 8.804951e-04 7.420470e-04 6.359534e-04 2.054126e-05 -10.7930749 25.0000000 1
```

11

Advanced Design Exploration With Design Insights

This chapter introduces the concept of design insights and the Design Insights tool.

- [Overview of Design Insights](#)
- [Setting up Design Insights](#)
- [Launching Design Insights](#)
- [Creating diPhysical View](#)
- [Exporting DSPF by Selected Nets](#)
- [Exploring diPhysical view](#)
- [Analyzing Parasitic](#)
- [Histograms for Net Details and Analysis Result](#)
- [Cross Highlighting](#)

Overview of Design Insights

Parasitic plays a crucial role in custom design. It has a wide impact from delay, crosstalk, device operating point, circuit functionality, and AC characteristics. To get better insight on the design it is important to consider interconnect parasitic and device characteristics together. Design Insights offer deeper design insights for debugging and design optimization for simulation infrastructure and design environment.

The following are the important features of Design Insights:

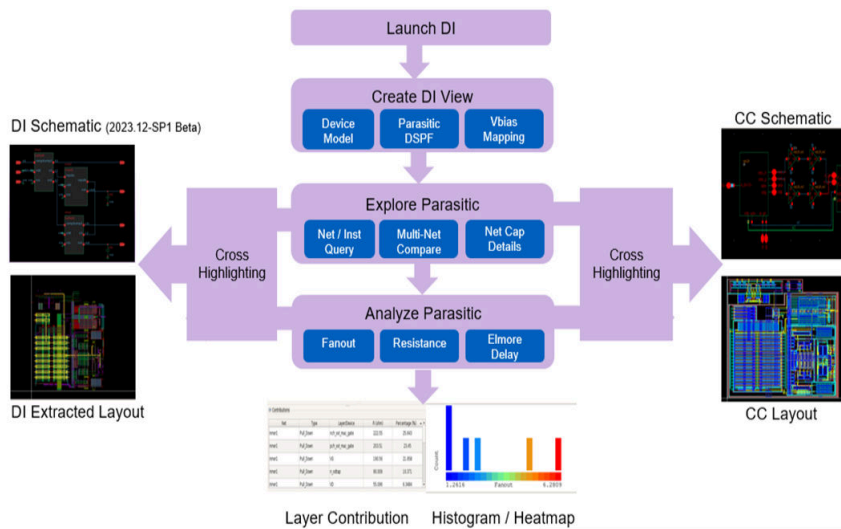
- Perform circuit topology and device aware analyses
- Identify design bottlenecks like fanout and RC contribution before running expensive simulations
- Visualize the post extracted shapes and parasitic with intuitive user interface in the Custom Compiler tool

- Perform interactive and iterative queries to gain better insights on design
- Review multiple variables and its impact on design targets

Design Insights provides a solution to efficiently analyze, visualize parasitics from large DSPF or GPD databases. It provides deep insights into the design and identify bottlenecks in layout before performing expensive simulations. It can also be used along with design and simulation environment to understand simulation failures and perform design exploration.

Design Insights is built on Custom Infrastructure providing intuitive visualizations with cross probing to schematic and layout environments for faster debugging.

Figure 92 Design Insights Flow Diagram



Setting up Design Insights

To run Design Insights with the Custom Compiler tool, the following are the requirements.

License required

- `primewave_re` (UI environment)
- `primesim_ra` or `sim_hsmc` (engine analysis)

Launch flow

1. Add `<your_custominfrastructure_installation_directory>/bin/` to PATH environment variable.
2. Add `<your_custom_compiler_installation_directory>/bin/` to PATH environment variable.
3. Add `<your_primesim_di_installation_directory>/bin/` to PATH environment variable.
4. Run `custom_compiler`.

To run Design Insights with the PrimeWave Design Environment tool, the following are the requirements.

License required

- `primewave_re` (UI environment)
- `primesim_ra` or `sim_hsmc` (engine analysis)

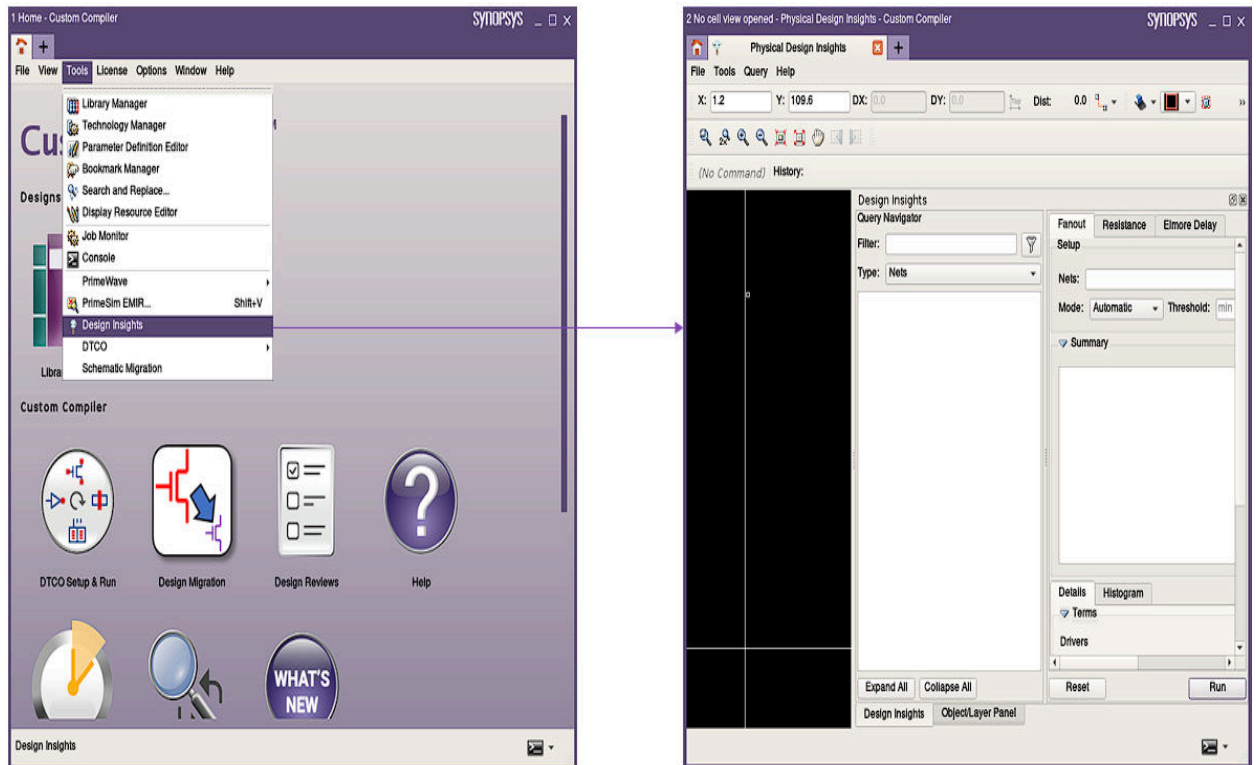
Launch flow

1. Add `<your_custominfrastructure_installation_directory>/bin/` to PATH environment variable.
2. Add `<your_custom_compiler_installation_directory>/bin/` to PATH environment variable.
3. Add `<your_primesim_di_installation_directory>/bin/` to PATH environment variable.
4. Run `primewave_reliability`.

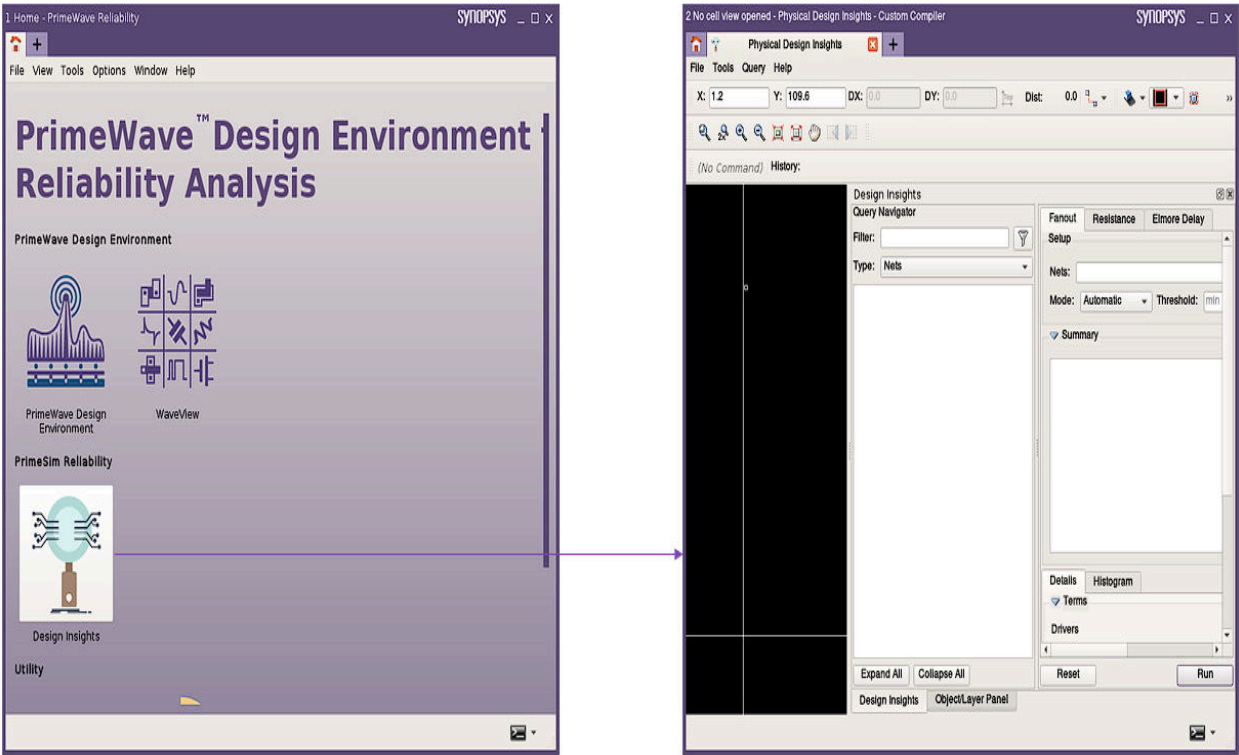
Launching Design Insights

The following figure shows how to launch Design Insights from the **Layout Editor** of the Custom Compiler tool. You can launch Design Insights from Custom Compiler home screen by selecting **Tools > Design Insights**.

Chapter 11: Advanced Design Exploration With Design Insights Launching Design Insights

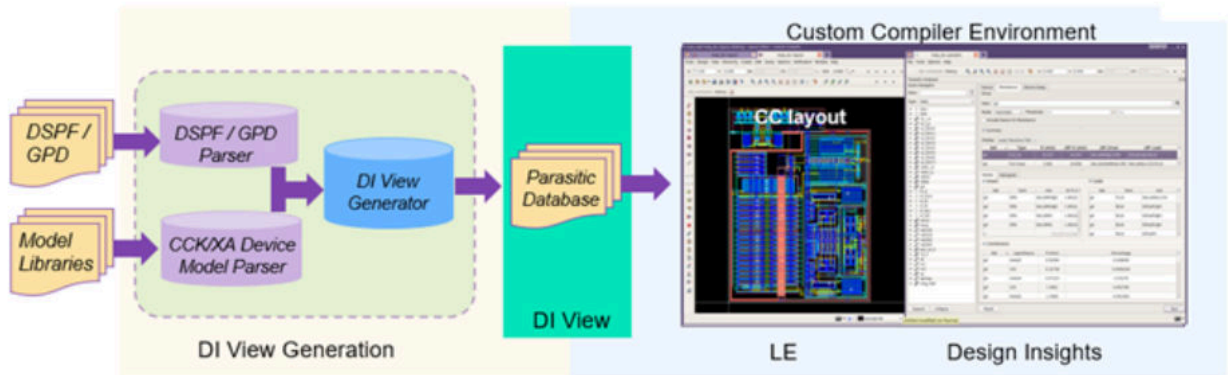


The following figure shows how to launch Design Insights from the PrimeWave Design Environment tool.



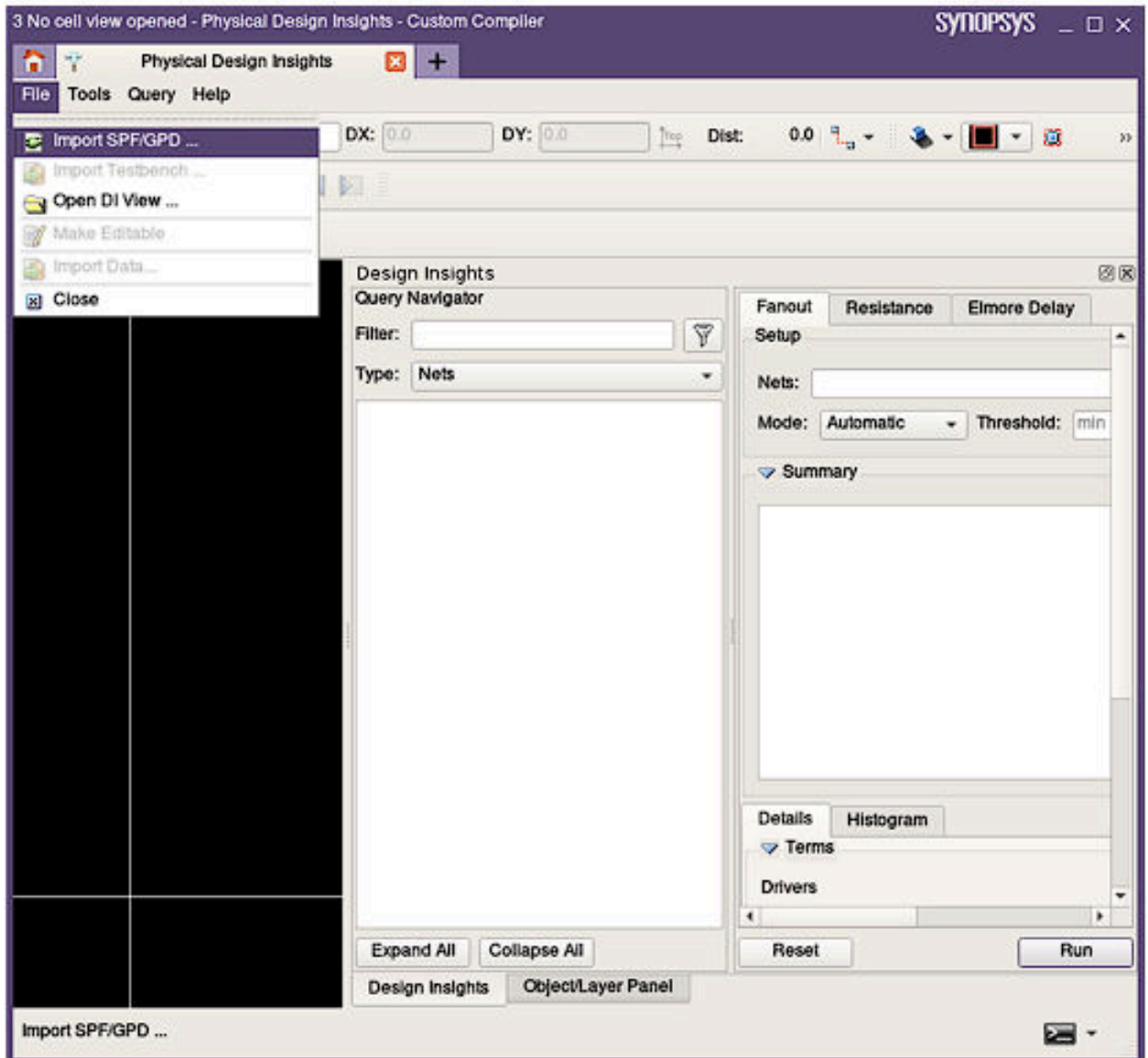
Creating diPhysical View

Design Insights generates design insights view with parasitic data to correlate existing layout.



Complete the following steps to create a design insights view:

1. From the **Design Insights** window, click **File > Import SPF/GPD**.



The **Import SPF/GPD** window appears.

2. In the **Import SPF/GPD** window,
 - **Results Cell View:** Select **Library**, **Cell** and **View** details from the drop-down menu.
 - **Input Data:** Input parasitic types and path.

If you select **DSPF**, it supports flatten DSPF without *include* statement.

- **Vbias Mapping File:** Voltage bias definitions.
- **Default Vbias:** Default voltage bias definitions.

Note:

Either specify a global bias voltage for all devices or set a bias voltage mapping file path.

- **Model Libs:** Specify a model lib path, section, corner (if any) and click to insert a definition item.
- **Skip nets:** Specify names of nets that to be skipped in parasitic view

Ignore Subckt Not Found - Specify if there are any black box cells in the design, enter the cell names in this text box so that the tool can ignore the cells if their subckt definition is not found.

Advanced Options - By default, the SQL and OA database are created within the cell view. You can provide custom paths using the OA Output Root Directory and SQL Output Root Directory widgets.

Import SPF/GPD

Results Cell View

Library:

Cell:

View: diPhysical

Input Data

Type: ☒ DSPF ☐ GPD ☒ Use ITF Layer Name

Path:

Click to add

Vbias Mapping File:

Default Vbias:

Model Libs:

Model Lib	Section	Corner
Click to ...		

Model Syntax:

hspice

Ignore Subckt Not Found:

Nets: ☒ Skip ☐ Select

An empty string mean...

Advanced Options

OA Output Root Directory:

SQL Output Root Directory:

Help

OK

Apply

Close

StarRC Options Required for DSPF Input

The following StarRC options are required for the DSPF input in Design Insights:

StarRC Options	DSPF Information
NETLIST_CONNECT_SECTION: YES	Generates * I and * P in DSPF files
NETLIST_NODE_SECTION: YES	Generates * S in DSPF files
NETLIST_TAIL_COMMENTS: YES	Writes resistors' width, length, and layer information in DSPF files
EXTRA_GEOMETRY_INFO: RES or EXTRA_GEOMETRY_INFO: RES NODE	Writes resistors' bounding box in DSPF files

Device Bias Voltage Definition File

For device evaluations, Design Insights can include device **R_{On}** and terminal cap in analyses to improve accuracy. To do that you must either specify a global bias voltage for all devices or define for specific model or instance in a mapping file. The following is an example of a bias voltage mapping file

```
model      N 1.8
model      P 1.8
model      N25 2.5
model      P25 2.5
instance   ld_l2h|Mpdm<*>|M* 2.5 0.1
```

There are different modes to estimate **R_{On}**. In the following example, **specified** mode means **vgs** and **vds** are specified. For **equivalent**, **min**, or **max** modes, **vdd** is specified and Design Insights estimates **R_{On}** for three predefined scenarios. You can control the mode to be used for a model or instance in bias voltage definition file. The default mode is **equivalent**.

To define mode preference you can also apply the following console command:

```
diPreprocessorRonMode 0|1|2
```

Chapter 11: Advanced Design Exploration With Design Insights Exporting DSPF by Selected Nets

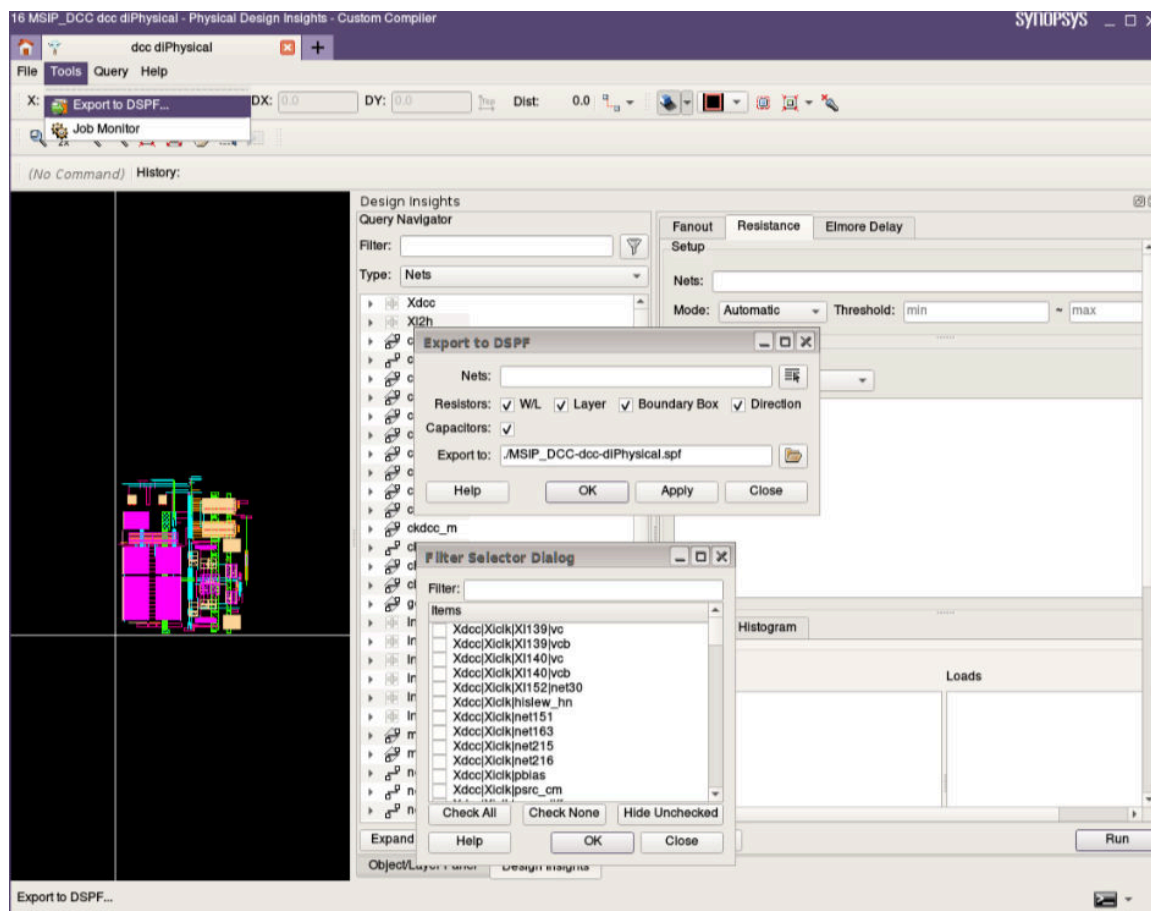
Where,

- 0 (default) : equivalent mode. (average)
- 1 : max mode.
- 2 : min mode.

Example: `db::setPrefValue diPreprocessorRonMode -value 0`

Exporting DSPF by Selected Nets

When the parasitic data is established and the Design Insights View is generated you can export a DSPF file with selected nets. To select one or multiple nets, go to **Tools > Export to DSPF** which exports a DSPF file with parasitic R/C only on the selected nets.

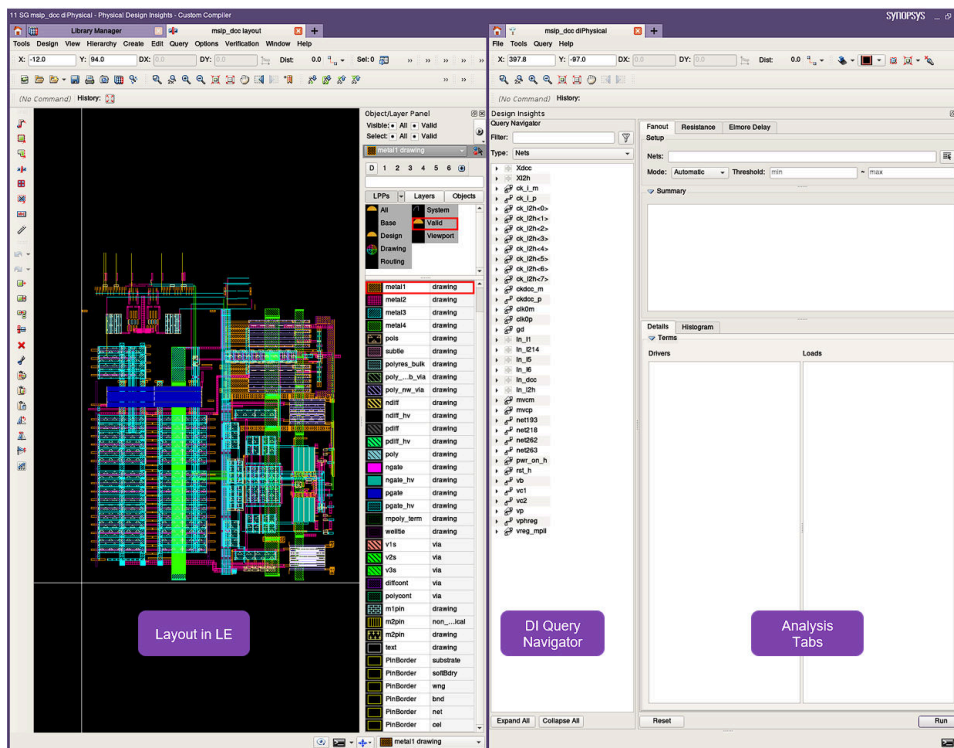


Exploring diPhysical view

When Design Insights generates diPhysical view, designer can use Design Insights interactive and iterative query capability to visualize and analyze parasitic. The following figures show exploring diPhysical view in the Custom Compiler and PrimeWave Design Environment for Reliability Analysis.

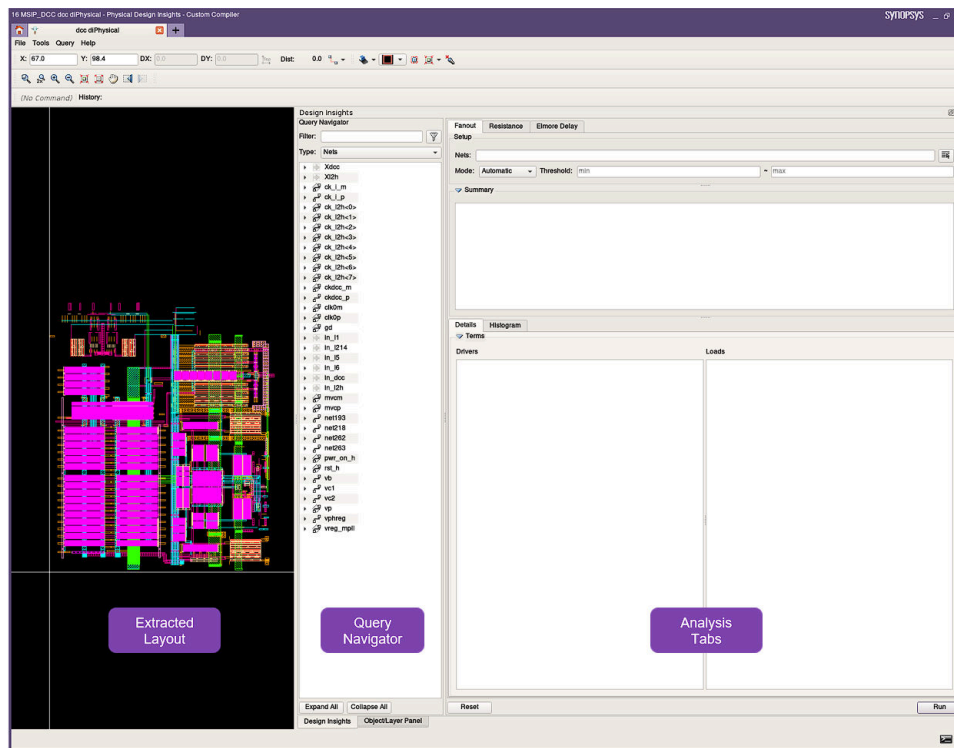
Use Custom Compiler layout to explore parasitic analysis. In Custom Compiler home screen, select **Tools > Design Insights** to bring up Design Insights main window, the parasitic data is imported from **File > Open DView** and selecting the required LCV.

The following figures show how to explore diPhysical views in the Custom Compiler and PrimeWave Design Environment tools respectively.



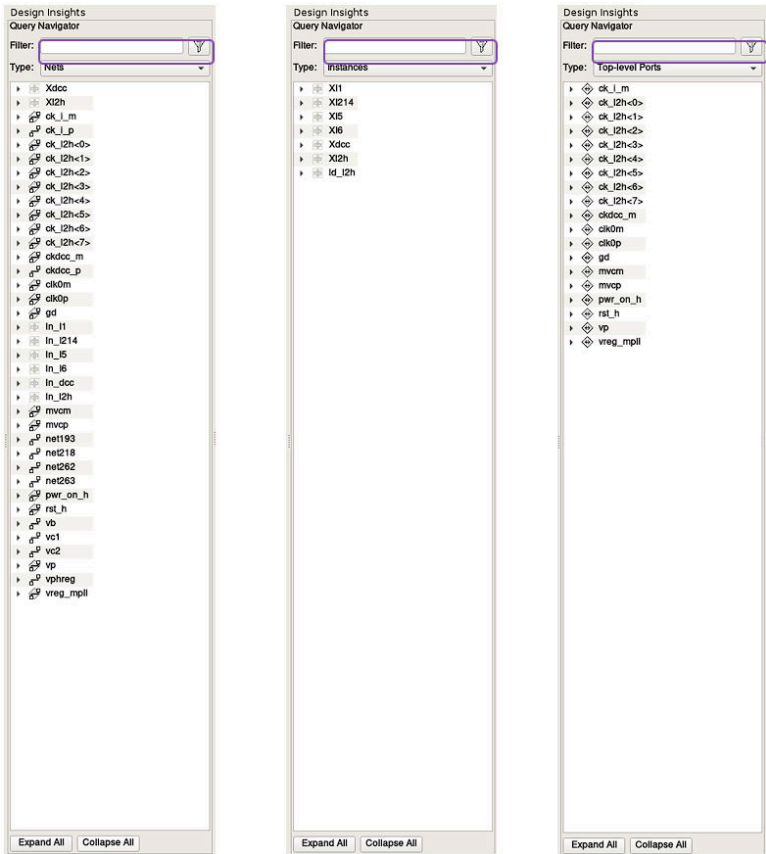
Chapter 11: Advanced Design Exploration With Design Insights

Exploring diPhysical view



The following procedure shows how to open Query Navigator;

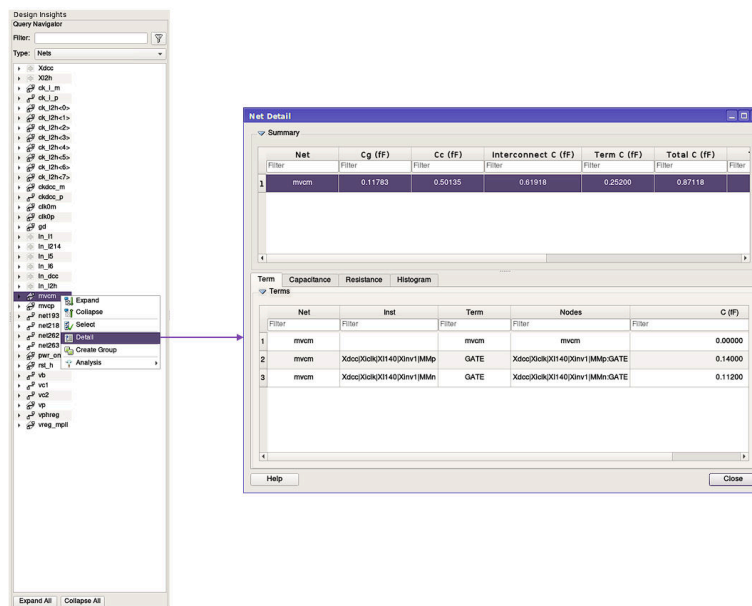
1. Open **Query Navigator** to query **Nets**, **Instance** and **Top-level Ports**.



Chapter 11: Advanced Design Exploration With Design Insights

Exploring diPhysical view

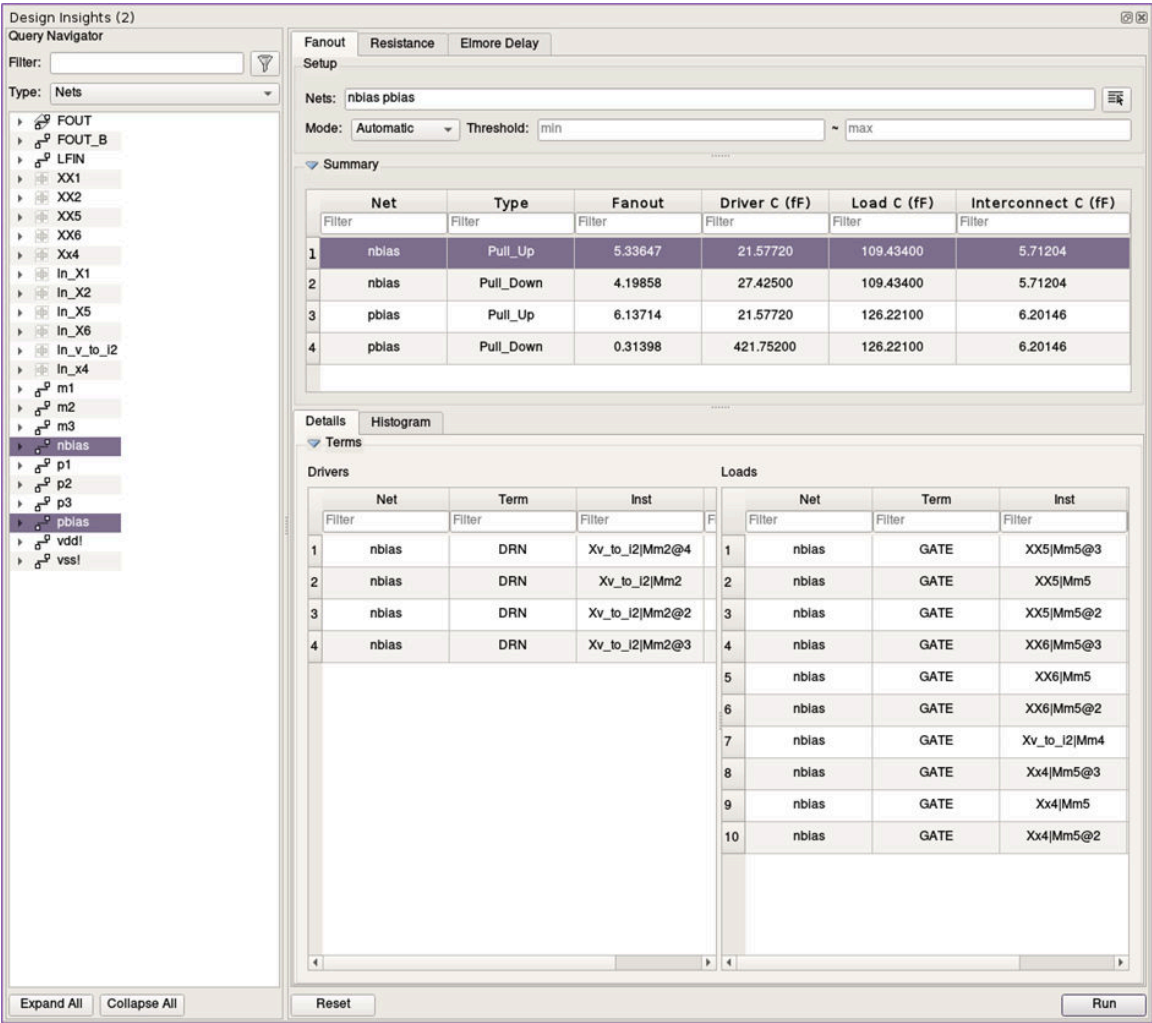
2. Select a specific net and explore parasitic details.



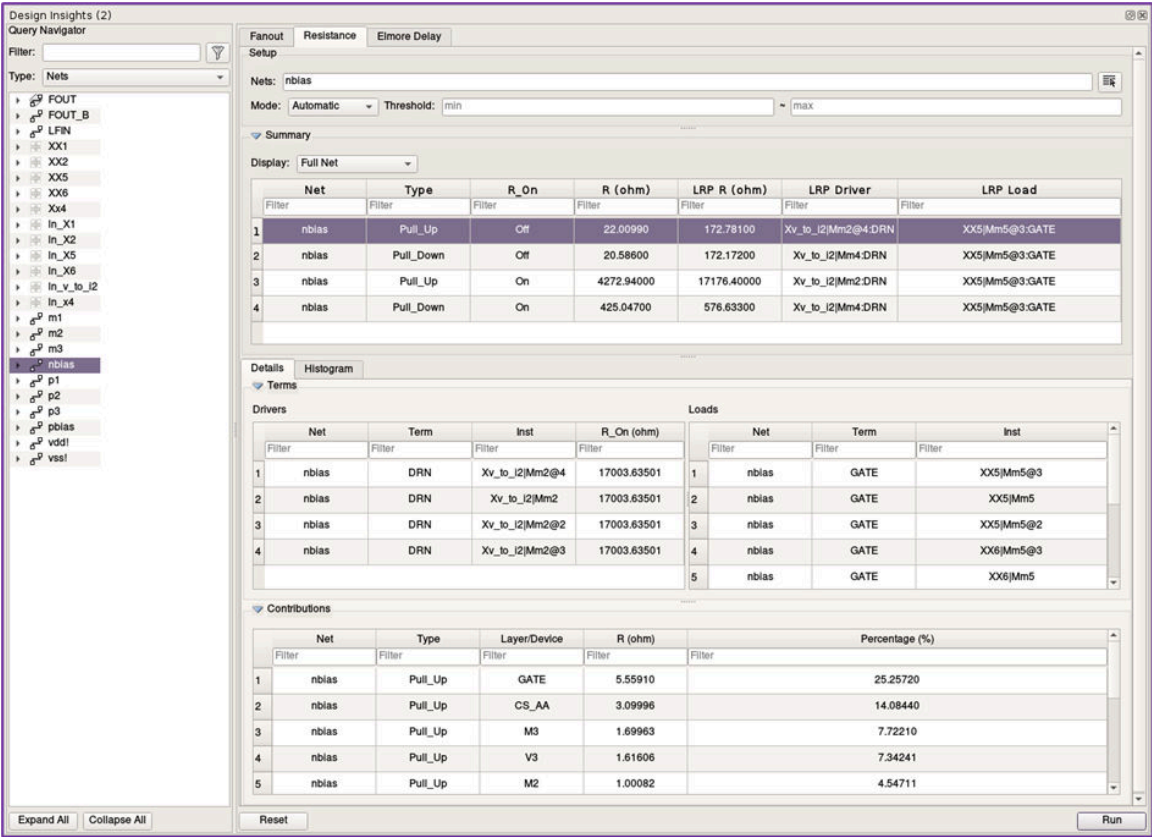
Analyzing Parasitic

Design Insights allows you to run the following analysis:

- Fanout - determines the capability of a device to drive the loads which are connected to it.



- Resistance - calculates the effective resistance of all the paths from drivers to loads of the selected nets.



- Elmore Delay - evaluates the delay and check rise or fall time and bandwidth of all the paths from drivers to loads of the selected nets.

Design Insights (2)
Query Navigator

Filter:

Type: Nets

Net: nbias pbias

Mode: Automatic Threshold: min max

☐ Include Device Terminal Capacitance

Summary

Net	Type	R_On	Elmore Delay (ps)	Rise/Fall Time (ps)	Bandwidth (GHz)
1 nbias	Pull_Up	Off	0.95846	0.12846	2722.28000
2 nbias	Pull_Down	Off	0.05053	0.11102	3149.80000
3 pbias	Pull_Up	Off	0.03876	0.08517	4105.80000
4 pbias	Pull_Down	Off	0.01941	0.04265	8199.07000
5 nbias	Pull_Up	On	24.33990	53.48030	6.53884
6 nbias	Pull_Down	On	2.36083	5.18727	67.41490
7 pbias	Pull_Up	On	26.40060	58.00800	6.02847
8 pbias	Pull_Down	On	5.08999	11.18390	31.26820

Details Histogram

Terms

Drivers

Net	Term	Inst	C (fF)	R_On (ohm)
1 nbias	DRN	Xv_to_i2Mm2@4	2.35429	17003.63501
2 nbias	DRN	Xv_to_i2Mm2	2.35429	17003.63501
3 nbias	DRN	Xv_to_i2Mm2@2	2.35429	17003.63501
4 nbias	DRN	Xv_to_i2Mm2@3	2.35429	17003.63501

Loads

Net	Term	Inst	C (fF)
1 nbias	GATE	XX5Mm5@3	9.11210
2 nbias	GATE	XX5Mm5	9.11210
3 nbias	GATE	XX5Mm5@2	9.11210
4 nbias	GATE	XX5Mm5@3	9.11210
5 nbias	GATE	XX6Mm5	9.11210
6 nbias	GATE	XX6Mm5@2	9.11210
7 nbias	GATE	Xv_to_i2Mm4	27.42496
8 nbias	GATE	Xx4Mm5@3	9.11210
9 nbias	GATE	Xx4Mm5	9.11210

Expand All Collapse All Reset Run

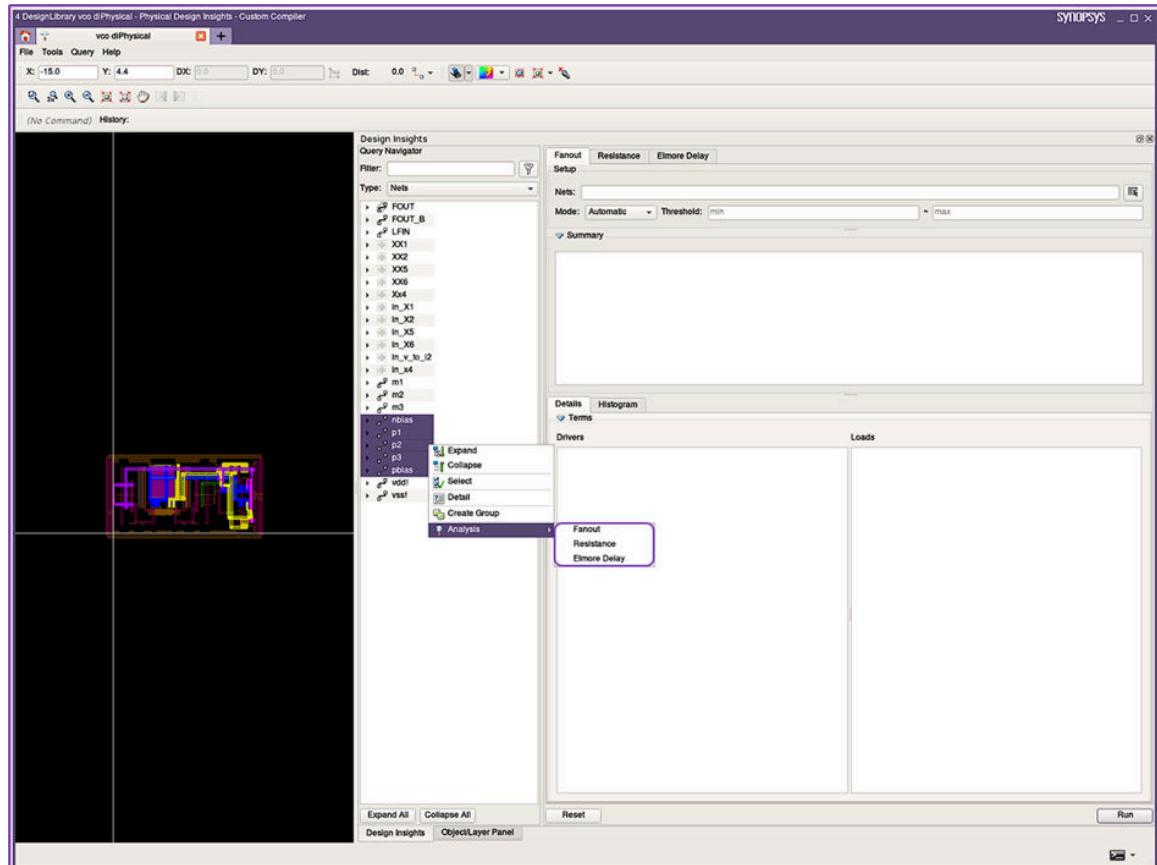
The analysis can be run in the following modes:

- Automatic
- InstPin to Port
- Point-to-Point

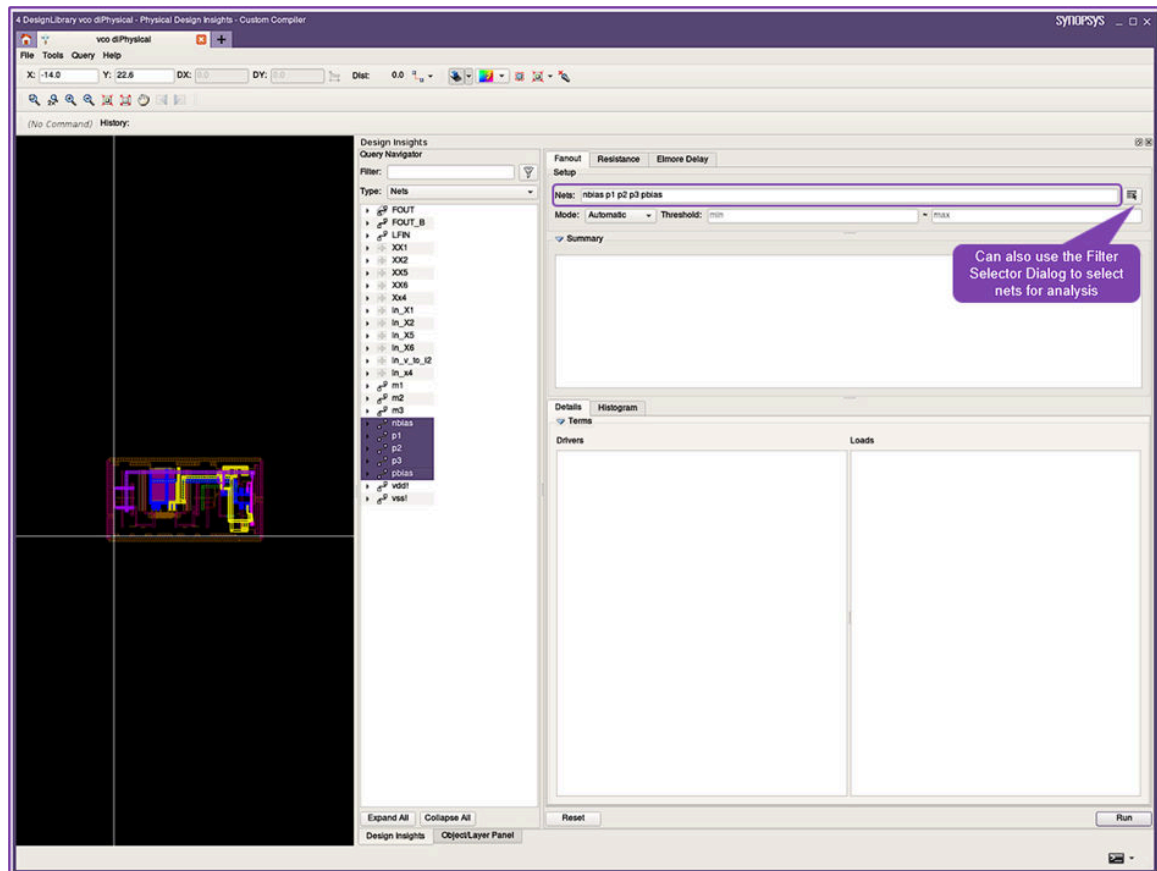
Steps to Run Analysis Using Design Insights

The following steps are common for running any analysis:

1. From **Query Navigator**, select the **Nets**, right-click and select **Analysis**.

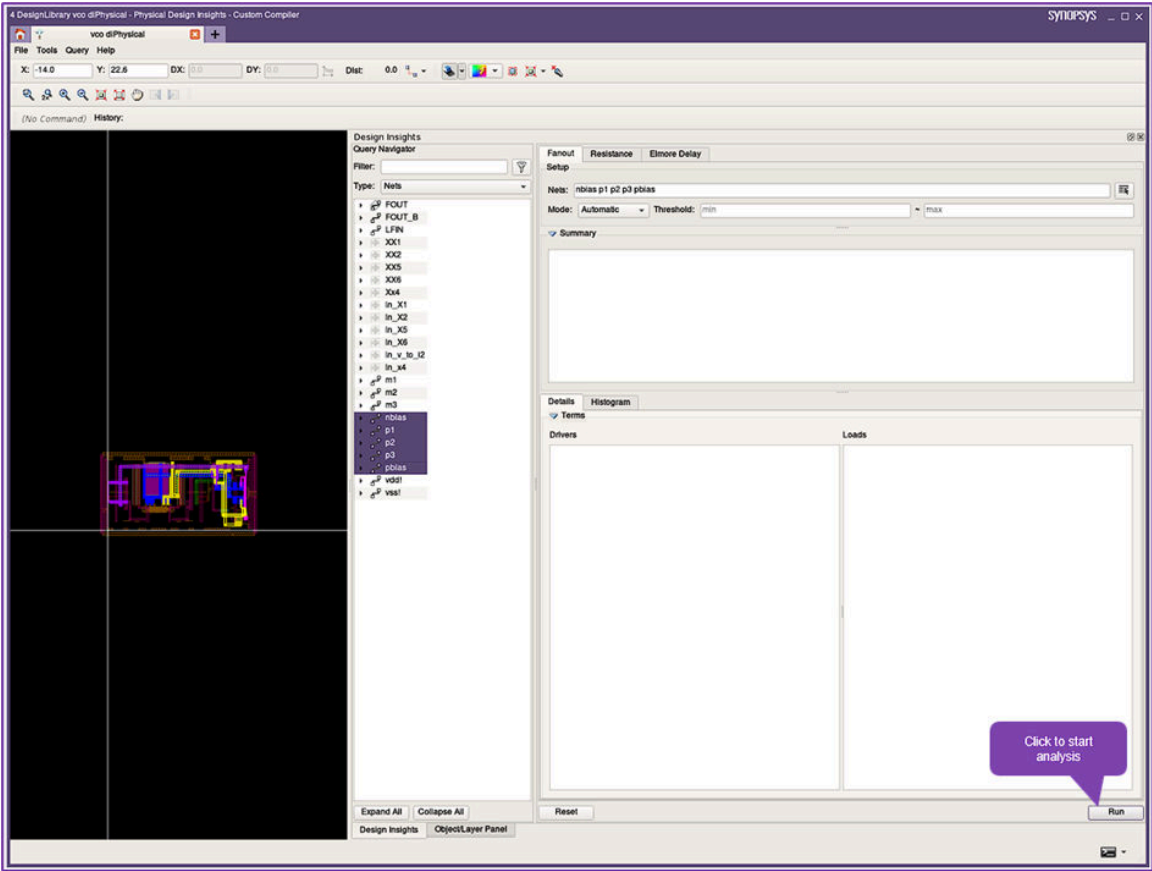


2. Select the required analysis in the context-sensitive menu and observe that the net names are filled in the Nets widget of the selected analysis tab. Here, Fanout analysis is selected.

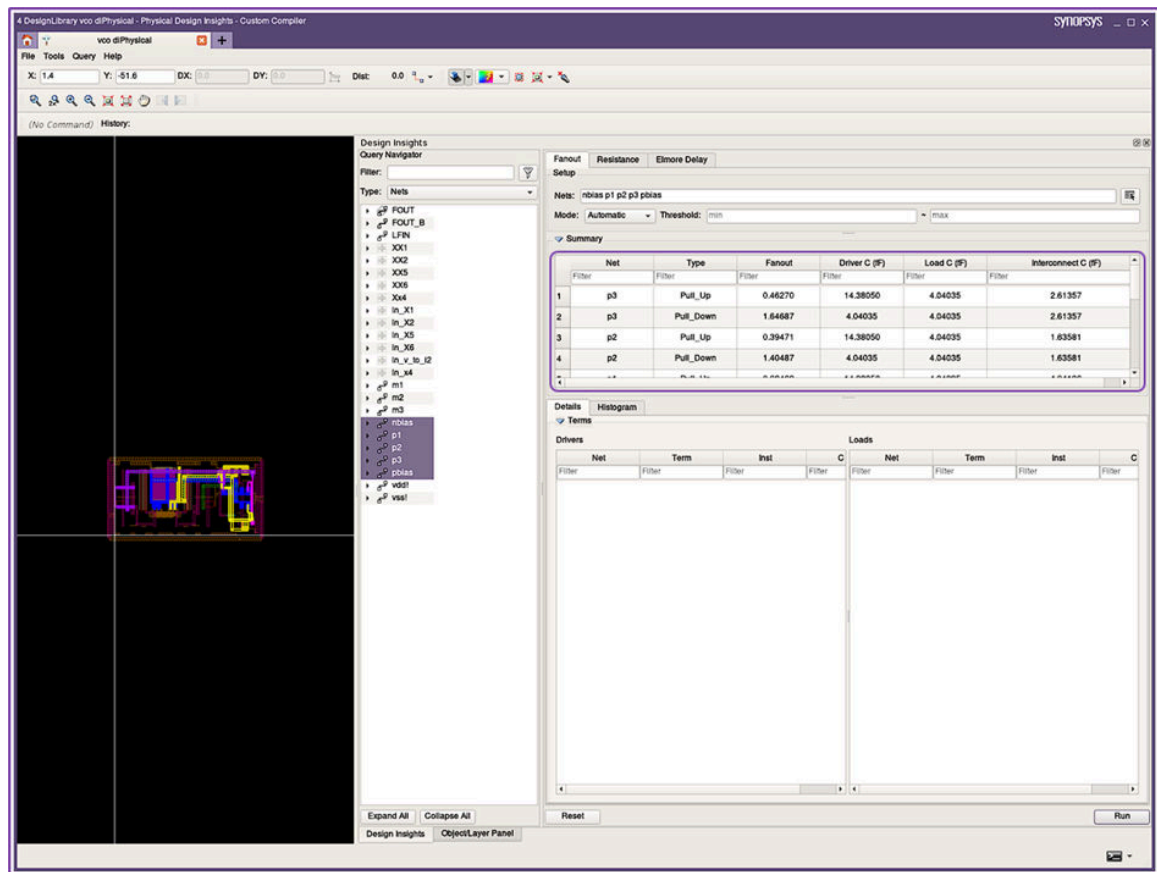


3. Click **Run** to start the analysis.

Chapter 11: Advanced Design Exploration With Design Insights
Analyzing Parasitic



4. After analysis is completed, the results are displayed as shown in the following image:

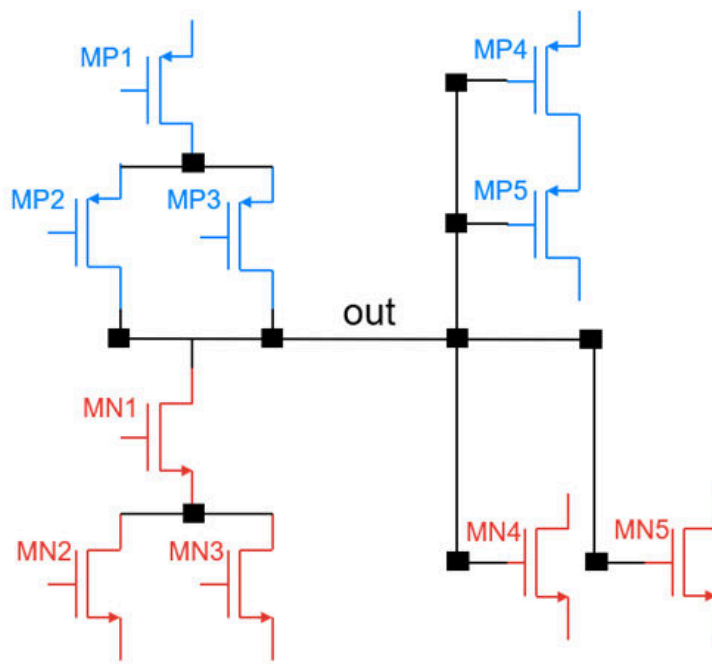


Fanout Analysis

Fanout is estimated based on the total gate size of drivers and loads.

$$Fanout = \frac{C_{total_loads} + C_{parasitic}}{C_{total_drivers}}$$

For example, consider the following circuit:



Here, for net out,

- C_{driver}
 - for pull-up path includes C_{P2} and C_{P3}
 - for pull-down path includes C_{N1}
- C_{load} includes C_{P4} , C_{P5} , C_{N4} and C_{N5}
- $C_{parasitic}$ is equal to C_{out}

Then, for the pull up path,

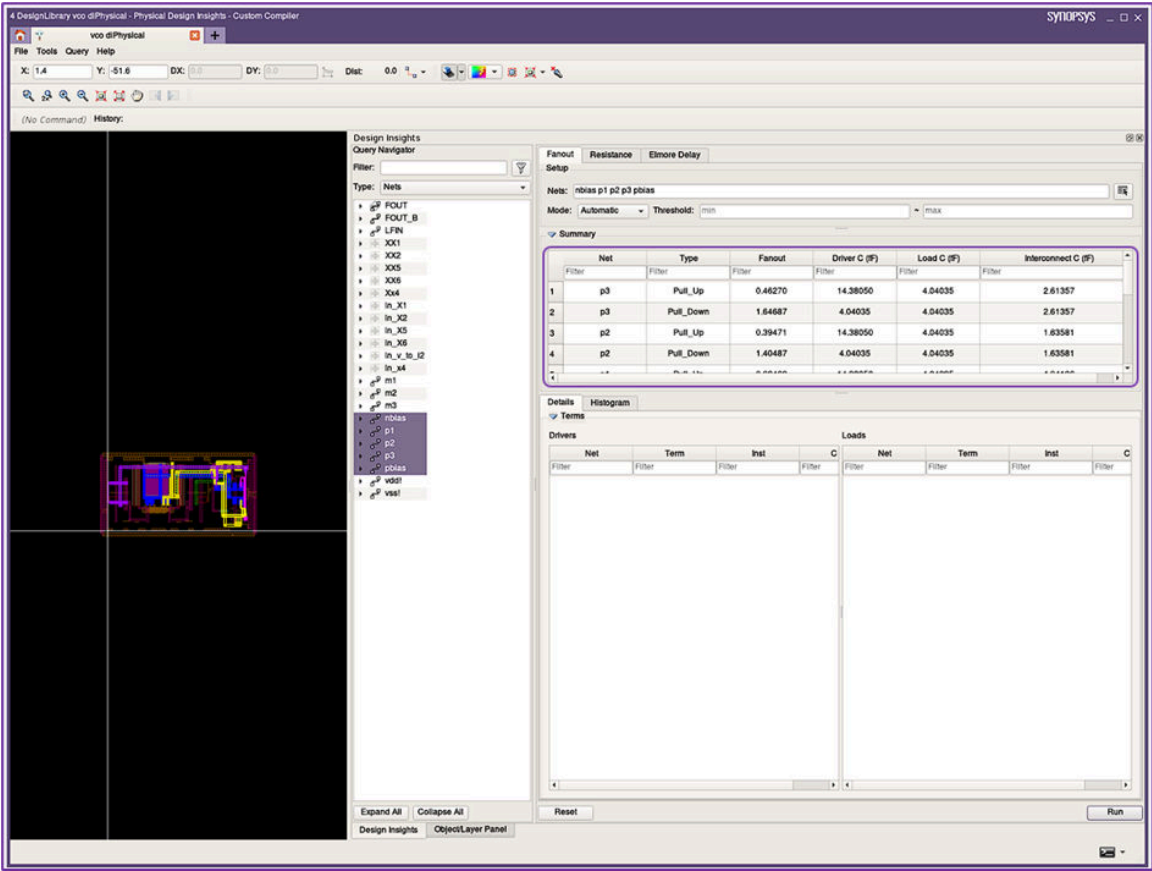
$$Fanout_{pull-up} = \frac{(C_{P4} + C_{P5} + C_{N4} + C_{N5}) + (C_{out})}{(C_{P2} + C_{P3})}$$

For the pull-down path,

$$Fanout_{pull-down} = \frac{(C_{P4} + C_{P5} + C_{N4} + C_{N5}) + (C_{out})}{(C_{N1})}$$

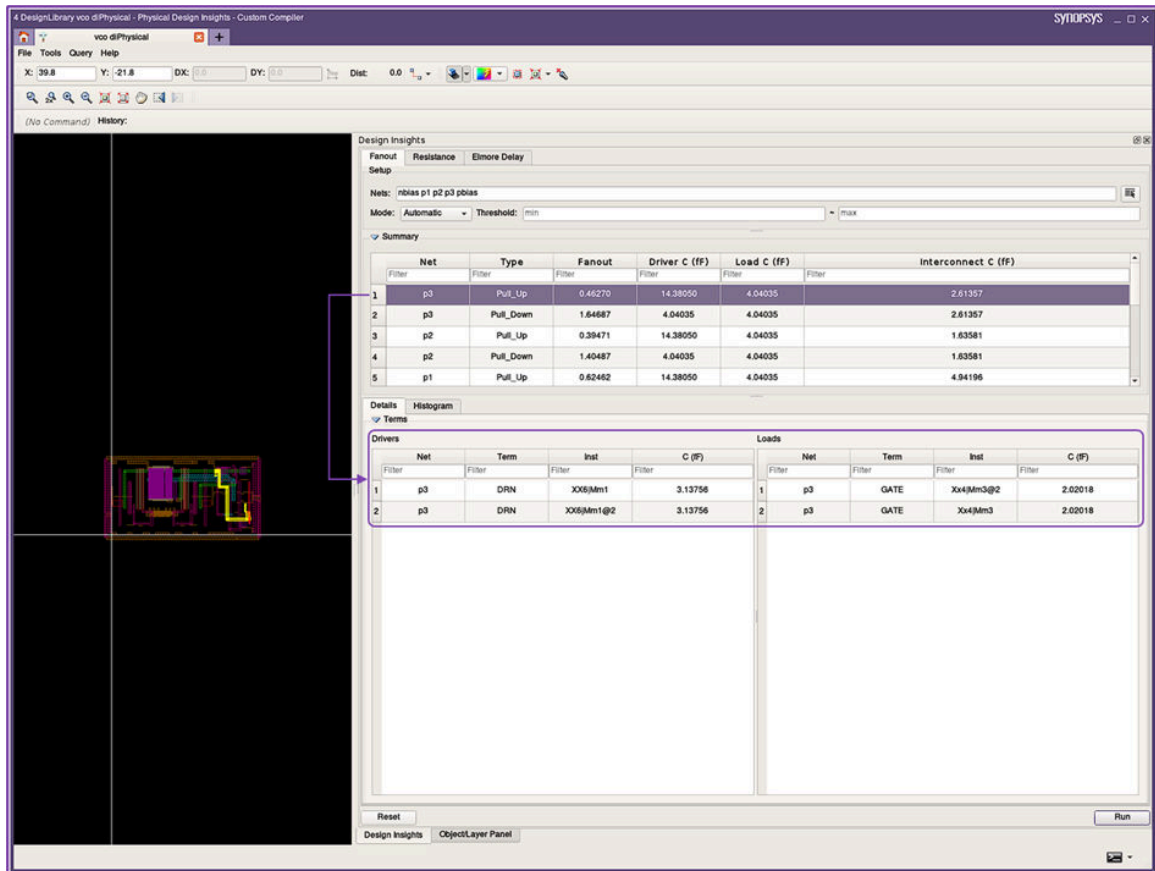
In the Design Insights GUI, the results are organized as:

- **Summary:** This table displays all the paths identified by the tool for the specified nets. For each path, the following information is displayed:
 - **Fanout**
 - **Driver C (fF)**
 - **Load C (fF)**
 - **Interconnect C (fF)**



Note:
Select an entry in the **Summary** table to highlight the net on the canvas.

- **Details:** This tab shows the terminals which are connected to the net selected in the **Summary** table. Each terminal's capacitance value is displayed to the user, and they are categorized into **Drivers** and **Loads**.



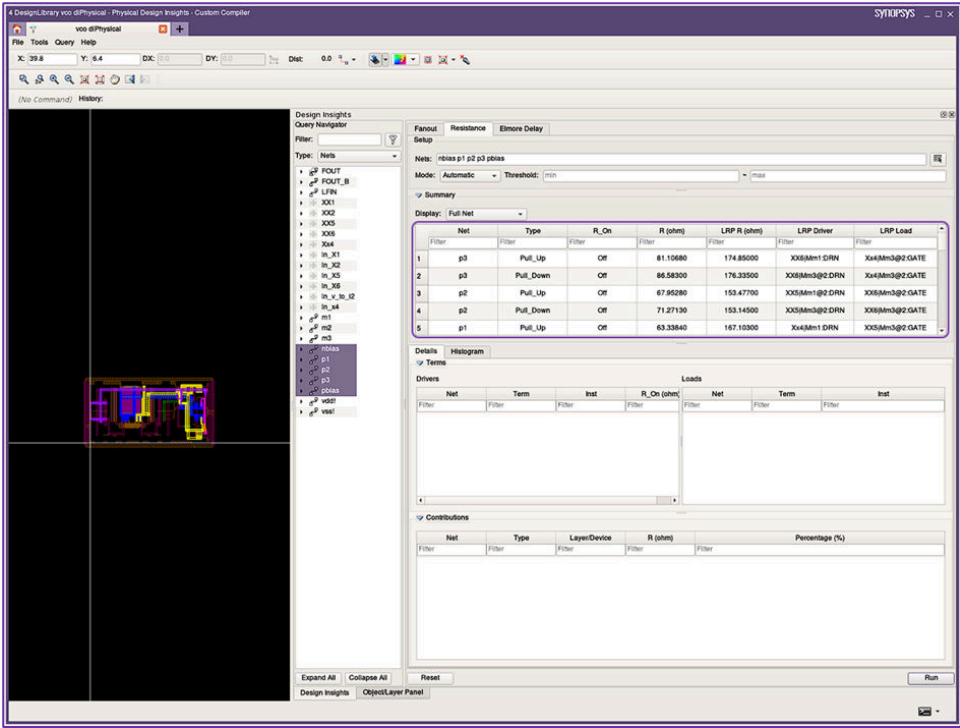
Understanding Resistance Analysis Results

Designers can run resistance analysis in Design Insights to get in-depth parasitic resistance information about each net. The following features are available within the tool to help with parasitic resistance debugging:

- Display Least Resistance Path (LRP)
- Check layer wise resistance contribution
- Calculate effective resistance for a path
- Perform point-to-point resistance checks for a specific net

After running resistance analysis, the **Summary** table shows the following information:

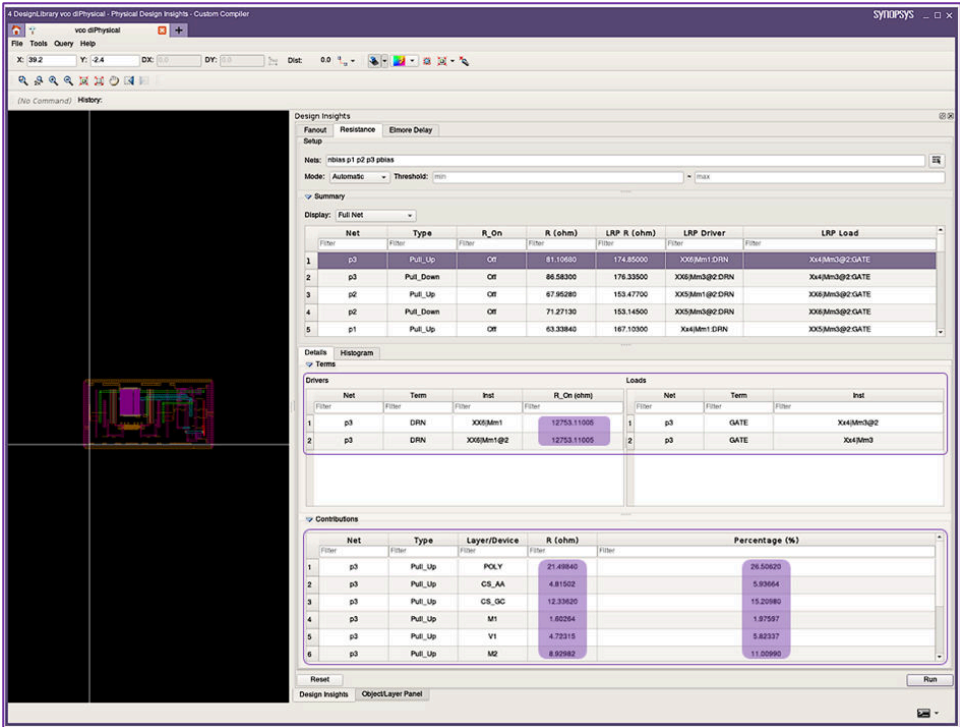
- **R (ohm)** - effective resistance of the path
- **LRP R (ohm)** - resistance of the least resistance path of the selected path
- **LRP Driver** - driver terminal name for the LRP
- **LRP Load** - load terminal name for the LRP



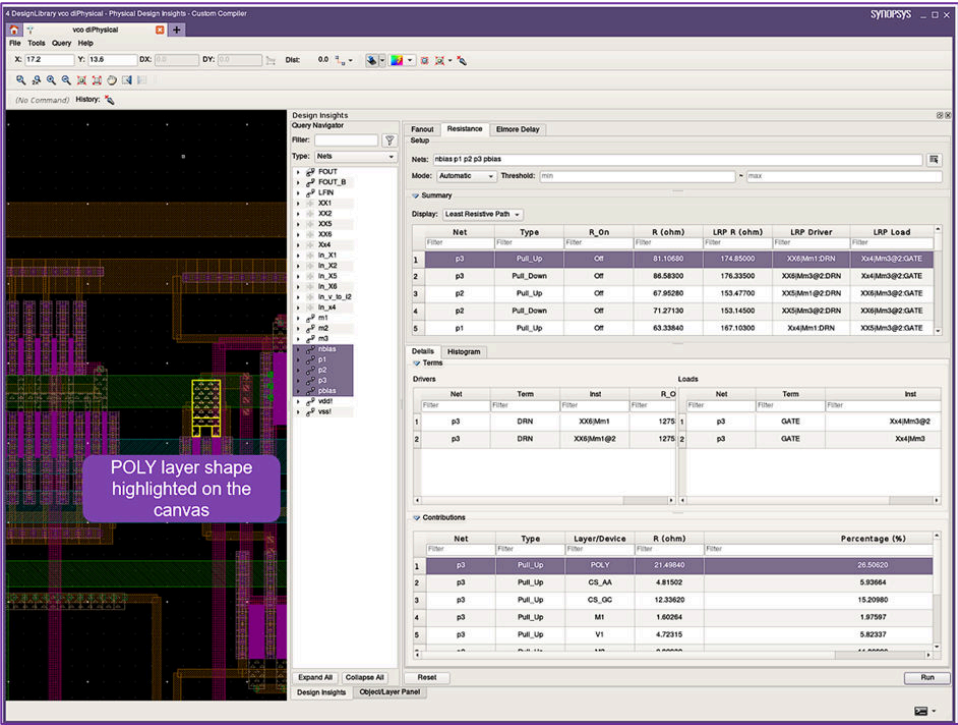
Check The Device ON Resistance (R_{on}) and Layer Wise Contribution

Select an entry in the **Summary** table to fill the tables within **Details** tab with appropriate information. There are three tables in this tab:

- **Drivers** - lists all the driver (drain/source) terminals of the selected path along with the R_{on} resistance of the devices.
- **Loads** - lists all the load (gate) terminals of the selected path.
- **Contributions** - provides the layer-wise resistance contribution of the selected path. This table shows the resistance contributed by a particular layer and also it's percentage contribution.



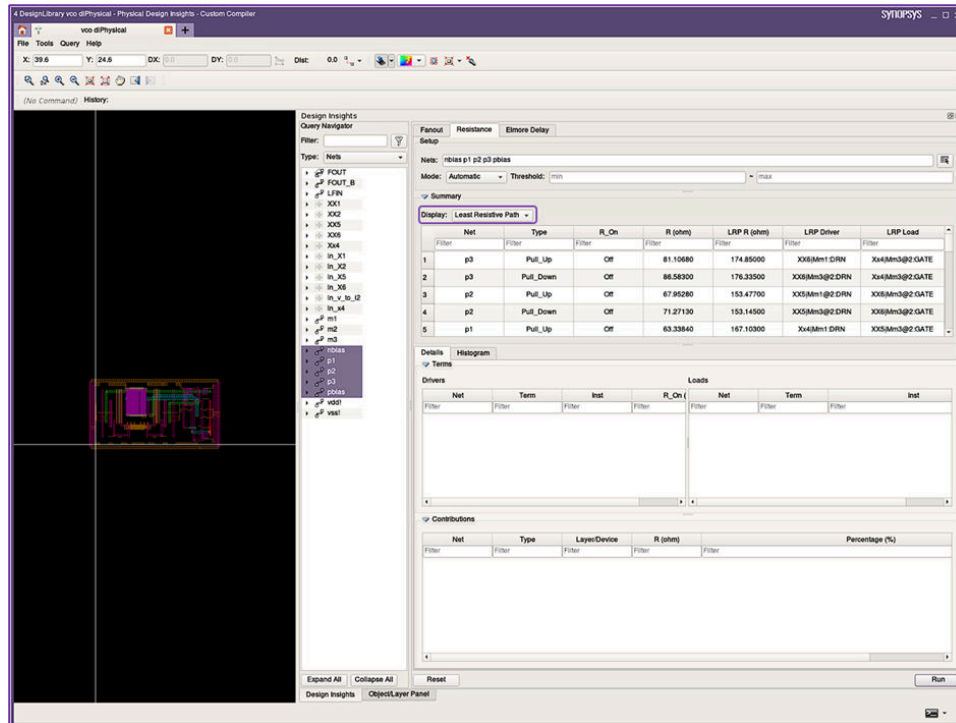
Note:
Click any entry in the **Contributions** table to highlight the corresponding shapes on the canvas.



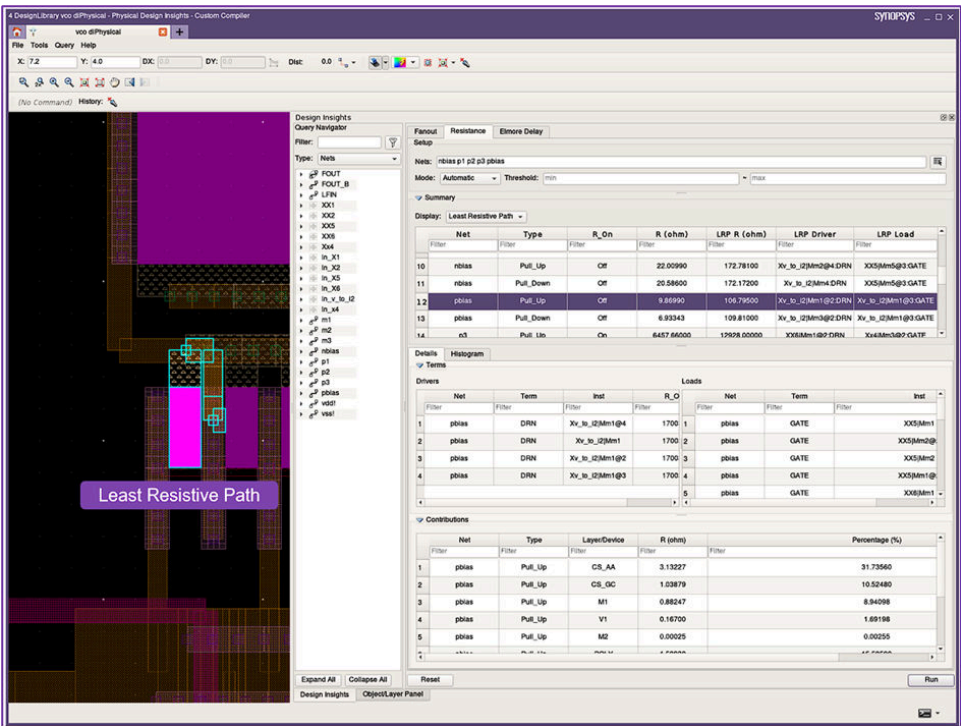
View the LRP Path on Canvas

To view the LRP path on canvas:

1. In the **Summary** section, set **Display:Least Resistive Path**.



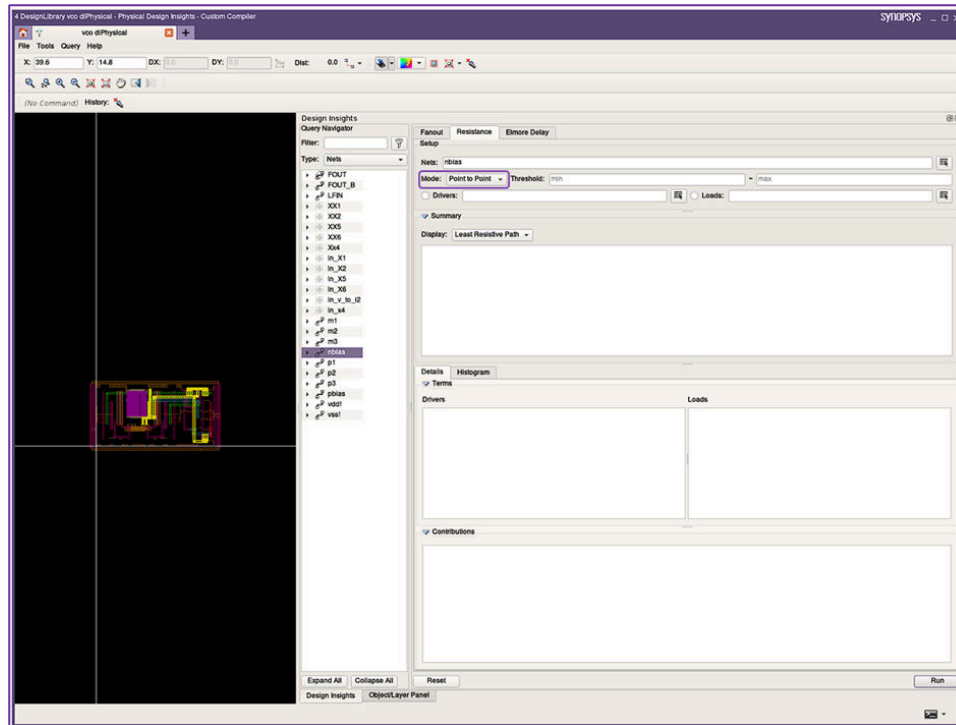
2. Select any entry in the **Summary** table to view the LRP path on canvas.



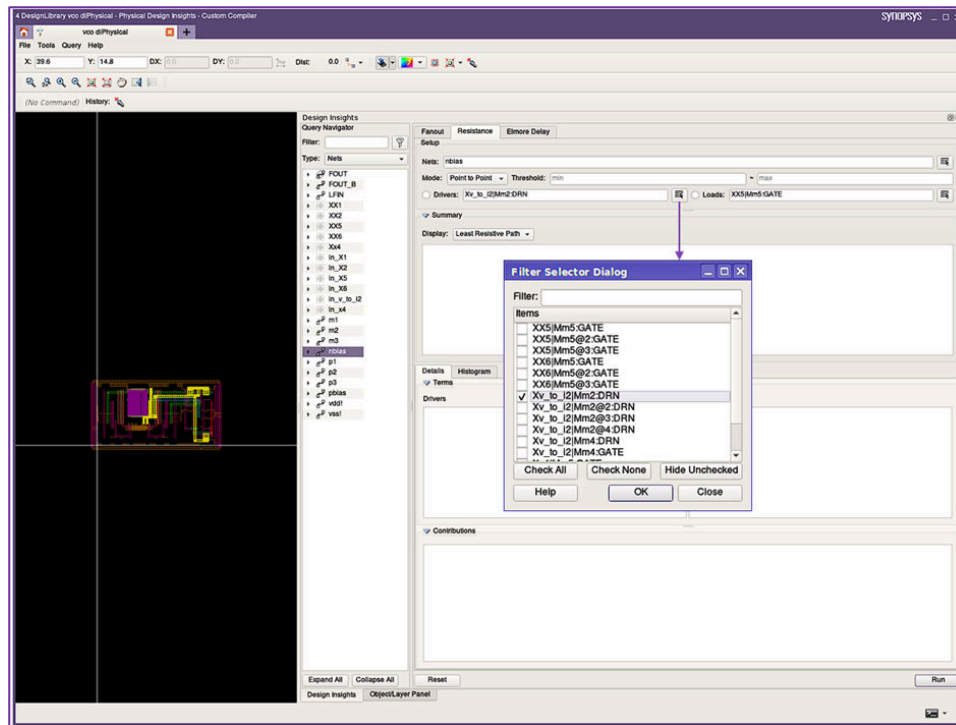
Perform Point-to-Point Resistance Checks

To run point-to-point resistance analysis:

1. Select a net for resistance analysis and set **Mode: Point-to-Point**.



2. Select the **Drivers** and **Loads** points using the **Filter Selector** dialog box.

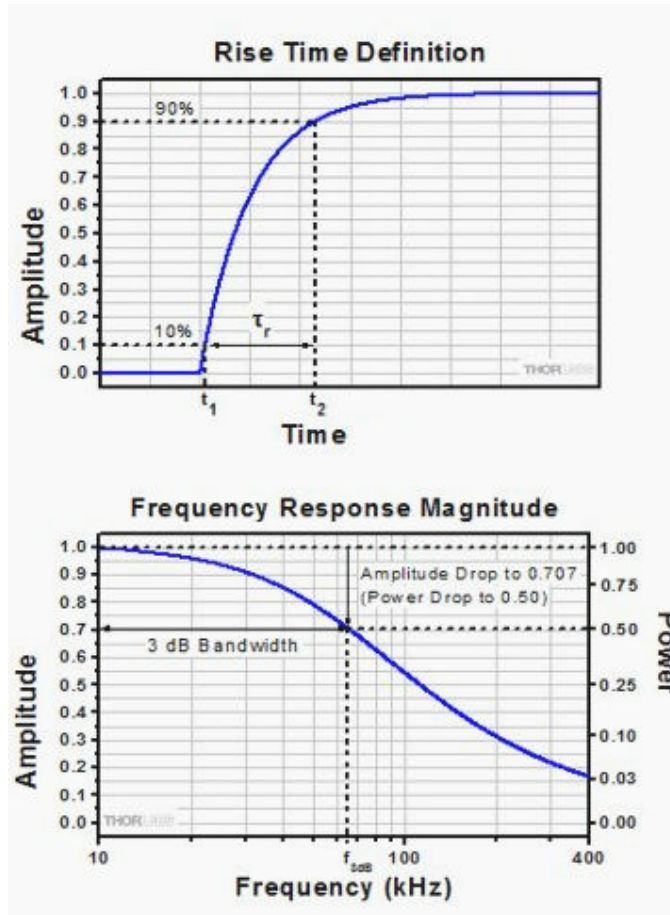


3. Click **Run** to start the analysis and the results can be checked as mentioned in the previous sections.

Understanding Elmore Delay Analysis Results

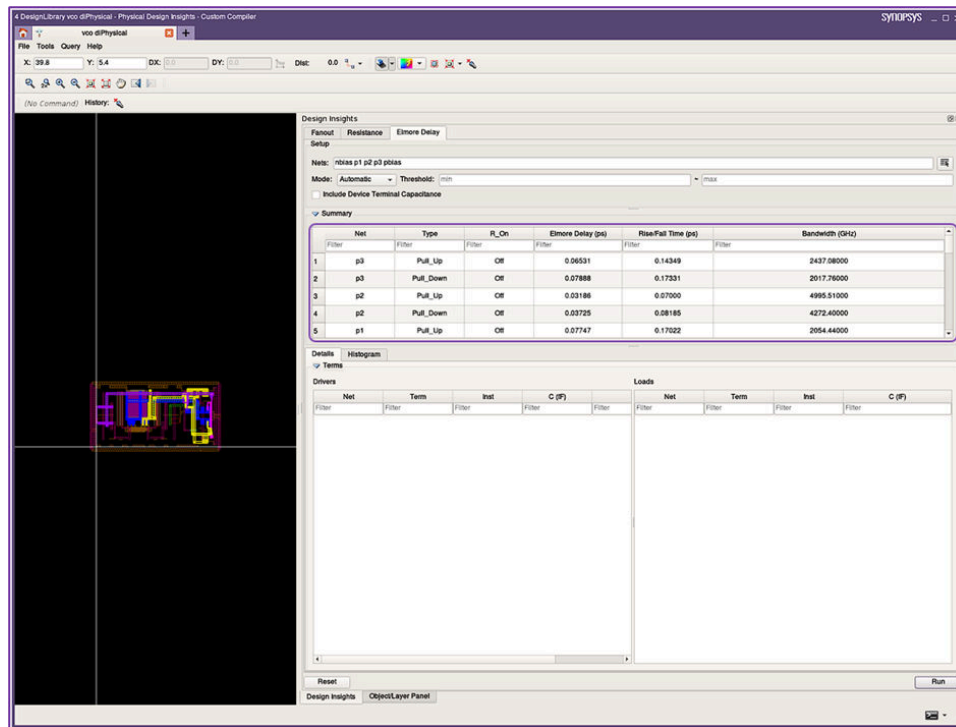
Elmore delay represents the RC time constant of a connection, based on which the temporal and frequency response information is provided.

- Time-domain response: With user-defined start and end amplitude rations, Design Insights provides rise and fall time from Elmore Delay.
- Frequency-domain response: Half-ratio power delivery frequency (-3 dB bandwidth).

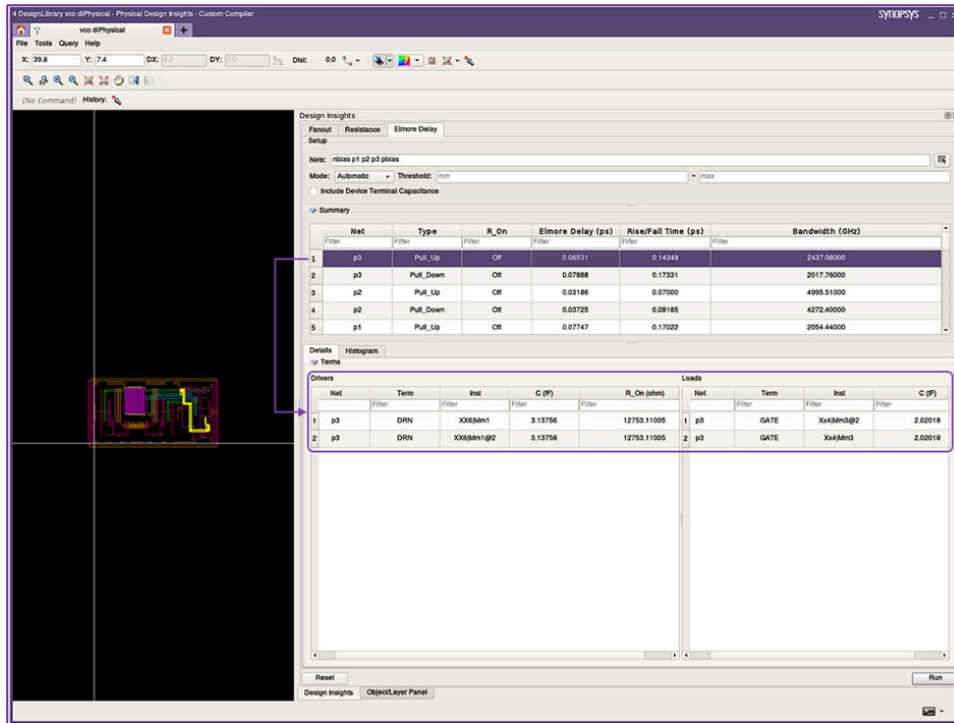


The results of Elmore Delay analysis are organized into,

- **Summary** - This table displays all the paths identified by the tool for the specified nets. For each path, the following information is displayed –
 - **Elmore Delay (ps)**
 - **Rise/Fall Time (ps)**
 - **Bandwidth (GHz)**



- **Details:** This tab shows the terminals which are connected to the net selected in the Summary table. Each terminal's capacitance value is displayed and they are categorized into Drivers and Loads. In addition, the R_{On} value for Drivers is displayed.



Note:

Delay analysis can be performed by enabling the **Include Device Terminal Capacitance** check box to incorporate device terminal capacitances.

To run Elmore Delay analysis in point-to-point mode, set **Mode**: Point-to-Point and follow the same steps as in the section Perform Point-to-Point Resistance checks.

Modes of Analysis in Design Insights

Design Insights analyses can be run in the following modes:

- Automatic - supported for Fanout, Resistance and Elmore Delay analysis
- Point-to-Point - supported for Fanout, Resistance and Elmore Delay analysis
- InstPin to Port - supported for Resistance analysis

Automatic Mode

- Pull-up
- Pull-down
- Port to Gate
- InstPin to Gate
- Port to Port
- InstPin to InstPin

ID	Path Type	Drivers	Loads
1	Pull-up	D1/D3 (Shorted)	G1/G2 (Shorted)
2	Pull-down	D2/D4 (Shorted)	
3	Port to Gate	A	
4		B	

ID	Path Type	Drivers	Loads
5	InstPin to Gate	N1	
6		N2	
7	InstPin to Port	D1/D3 (Shorted)	A
8			B
9		D2/D4 (Shorted)	A
10			B
11		N1	A
12			B
13		N2	A
14			B
15	Port to Port	A	B
16	InstPin to InstPin	D1/D3 (Shorted)	N1
17			N2
18		D2/D4 (Shorted)	N1
19			N2
20		N1	N2

Point to Point Mode

In this mode, analysis is run between two points of a net which are selected by the user.

InstPin to Port Mode

In this mode, analysis is run on a net by identifying all the paths from instance terminals to top-level ports in the design.

Histograms for Net Details and Analysis Result

When you select one or multiple nets in the summary table, a histogram can be displayed in the **Histogram** tab. You can select specific histogram of data to be displayed by changing the **Type** option. You can also adjust **Min**, **Max**, and **Color Level** to optimize histogram display.

Figure 93 shows the histogram for net comparison and Figure 94 shows the histogram for analysis results.

Figure 93 Histogram for Net Comparison

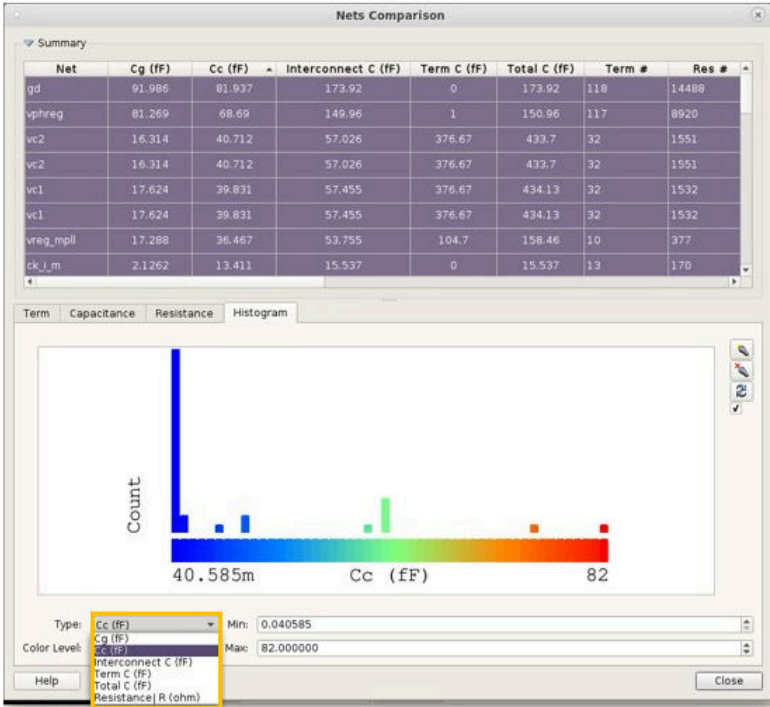
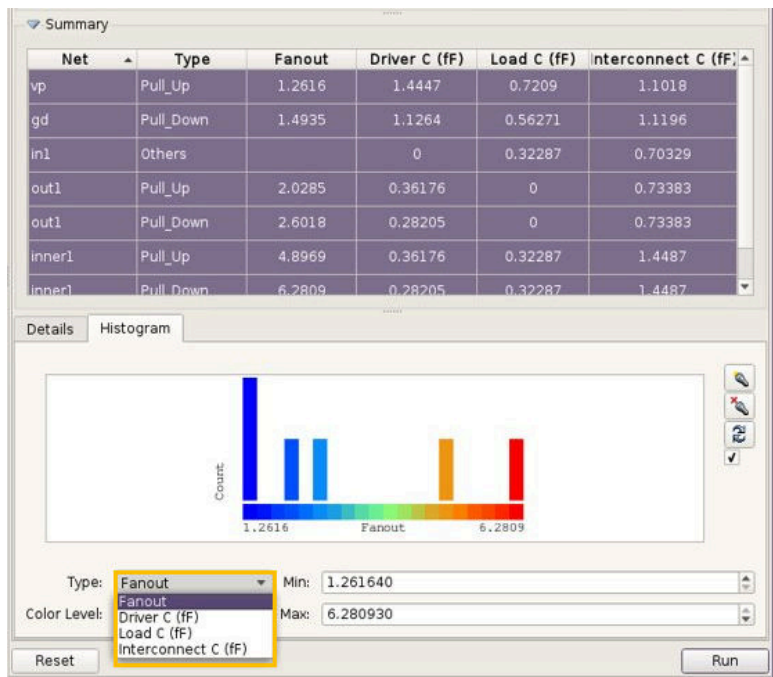
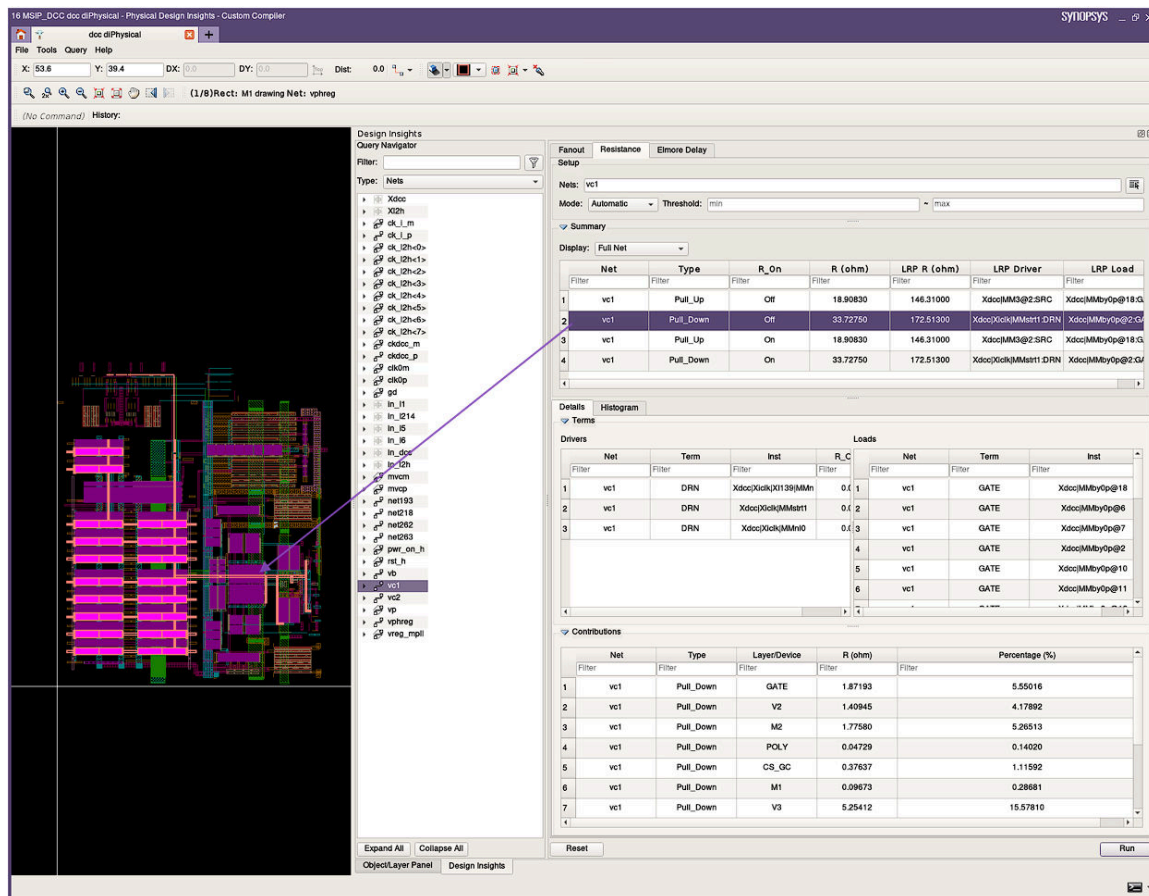


Figure 94 Histogram for Analysis Result

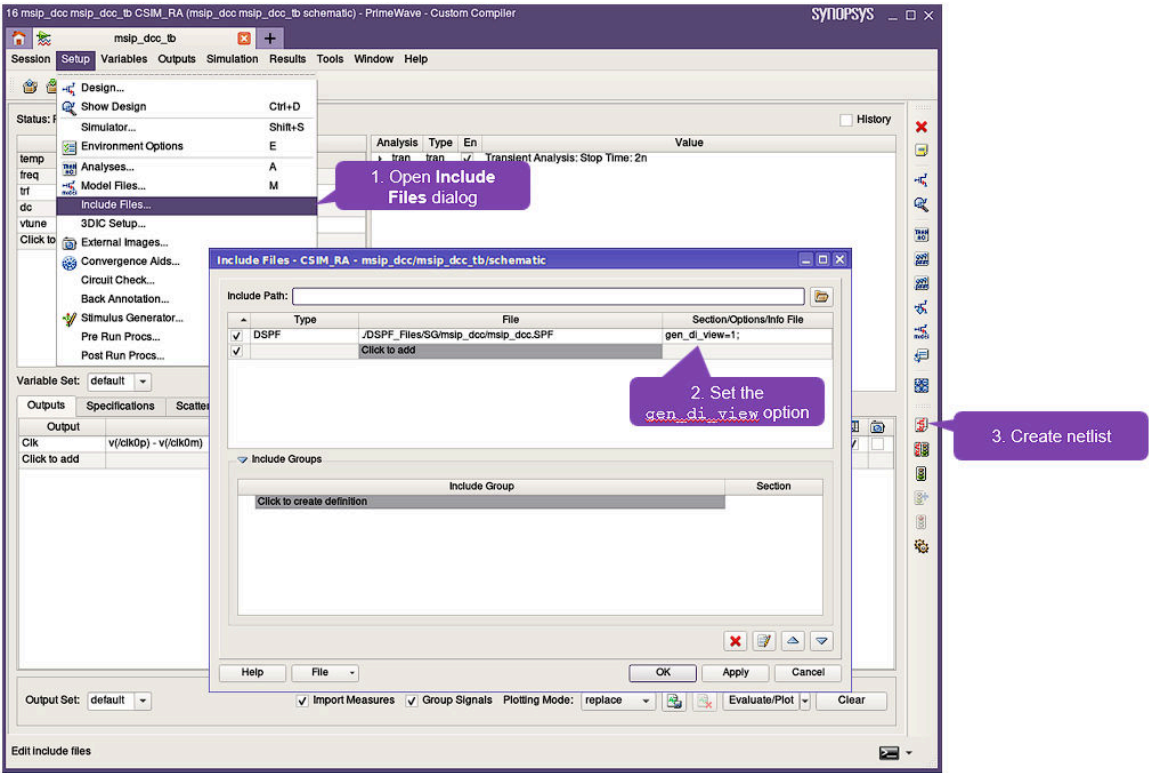


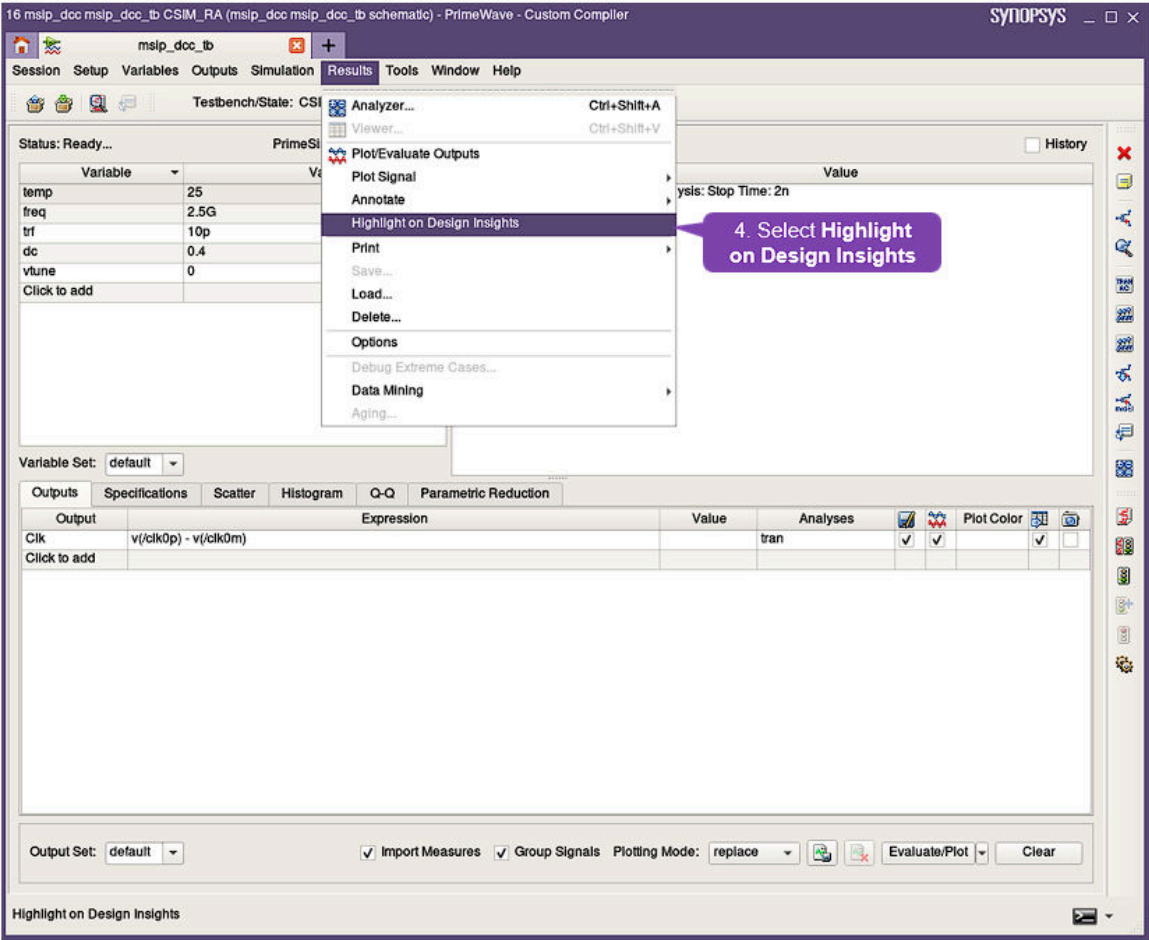
Cross Highlighting

Cross highlighting feature in Design Insights enables automatic highlighting in **Design Insights** view by selecting one or multiple target nets.



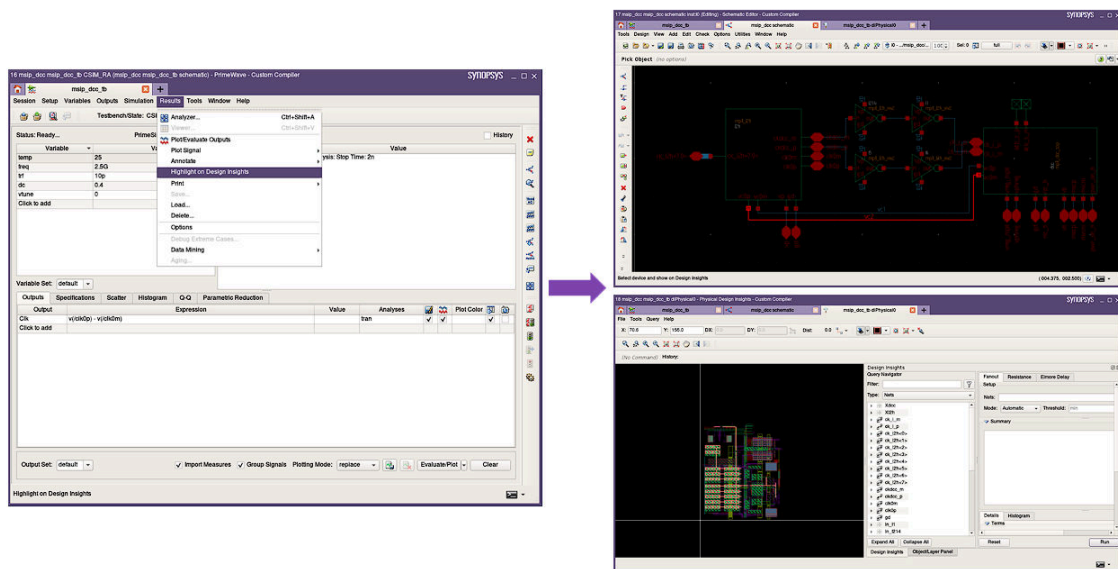
Starting with V-2023.12 release, the cross highlighting to Custom Compiler schematic is available through interfacing with PrimeWave Design Environment, the steps are described in following figures.





Chapter 11: Advanced Design Exploration With Design Insights

Cross Highlighting



Design Insights can generate hierarchical schematic based on DSPF input with logic cone back tracing capabilities to increase user debugging.

12

Appendix

- [Probability and Statistical Concepts](#)
 - [JSON Format and Examples](#)
 - [JSON Entries](#)
 - [</> Marks in AVA Yield Output Reports](#)
 - [Command Reference \(Alphabetical\)](#)
-

Probability and Statistical Concepts

This chapter describes the probability and statistical terms associated with Variation analysis and provides guidelines for selecting the number of samples for Variation analysis.

To perform an advanced variation analysis, you must be familiar with the following concepts:

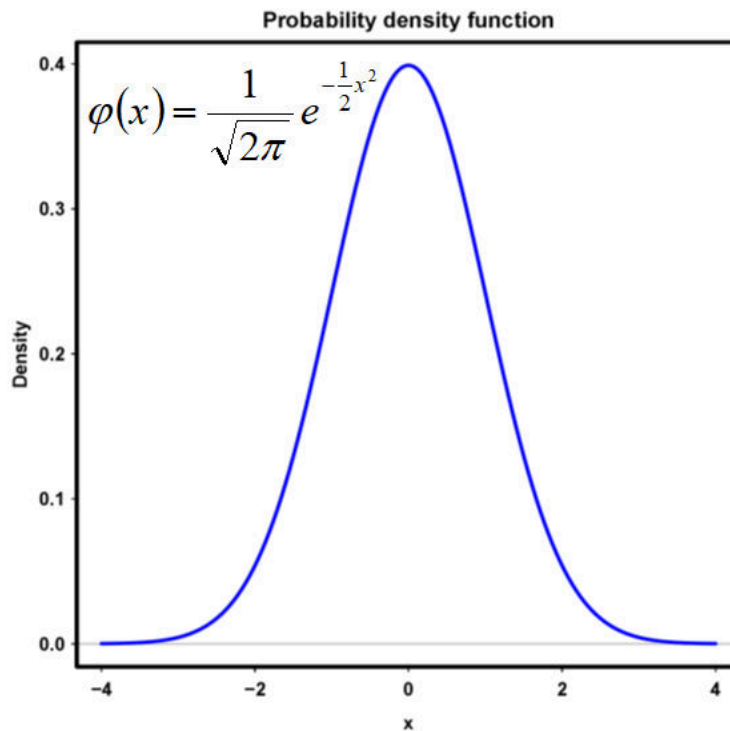
- [Probability Density Function](#)
- [Cumulative Distribution Function](#)
- [Statistical Measures](#)
- [Transformations on the CDF](#)
- [Comparing Distributions](#)
- [From Probabilities to Ordinates](#)
- [Probability and Sigma](#)
- [System and Component Reliability](#)
- [Sampling and Monte Carlo Simulation](#)
- [Q-Q Plot - Confidence Intervals](#)

- [Understanding the PrimeSim AVA Q-Q Plots](#)
- [Sigma, Sample Size, and Statistical Uncertainty](#)
- [One- or Two-Sided Specifications](#)

Probability Density Function

Process variations are often modeled using the Gaussian or normal probability density function (PDF). See [Figure 95](#).

Figure 95 Probability Density Function



All density functions are non-negative at all values of x and the integral under the PDF curve is 1.

Representing process variables by a normal density is an approximation because the probability is non-zero from $-\infty$ to ∞ while physical quantities like channel length have limited range.

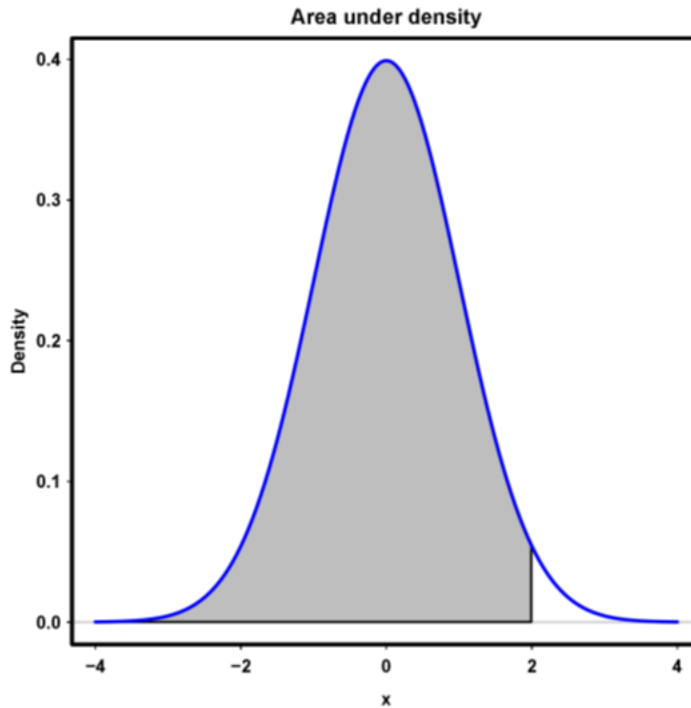
Non-physical values seldom arise in simulation because variation in production processes is small.

You can use the PrimeSim AVA option `Normal_Limit` to bound the range of variables.

Area Under the PDF

Area under the density function to a specified ordinate x gives the probability of an event. For example, in [Figure 96](#), the area of the shaded region is the probability that the value of the random variable x is less than 2.

Figure 96 Area Under the PDF



The area does not have a simple expression but can be determined from tables or from functions in mathematical and statistical software application packages.

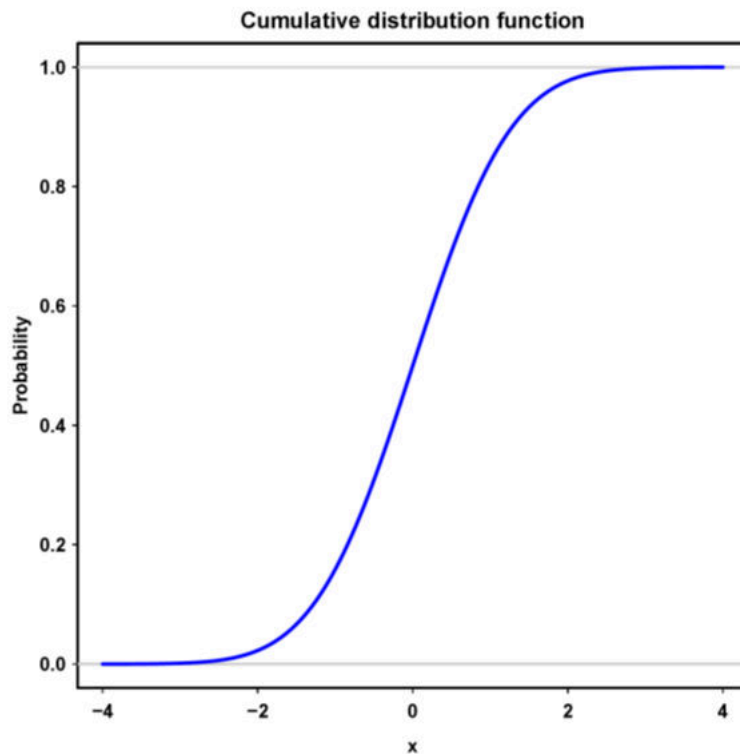
For the normal distribution:

$$p = \Phi(x) = \int_{-\infty}^x \varphi(u) du = \frac{1}{2} \left[1 + \operatorname{erf} \left(\frac{x}{\sqrt{2}} \right) \right]$$

Cumulative Distribution Function

[Figure 97](#) shows the value of the integral under the density curve from $-\infty$ to a particular ordinate x , say $p = j(x)$ for the normal density. This is called the cumulative distribution function (CDF).

Figure 97 Cumulative Distribution Function



It ranges from 0 to 1 and is monotonically non-decreasing. The advantage of the CDF is that the probability of the random variable being less than a certain value can be determined directly from the plot. At times, the ordinate corresponding to a certain probability must be set.

Statistical Measures

In addition to plots for distributions, a few associated quantitative values are also useful. These are the moments namely mean μ , standard deviation σ , skewness γ , and kurtosis κ , and the quantiles.

$$\mu = \int x p(x) dx$$

$$\sigma^2 = \int (x - \mu)^2 p(x) dx$$

$$\gamma = \int \left(\frac{x - \mu}{\sigma} \right)^3 p(x) dx$$

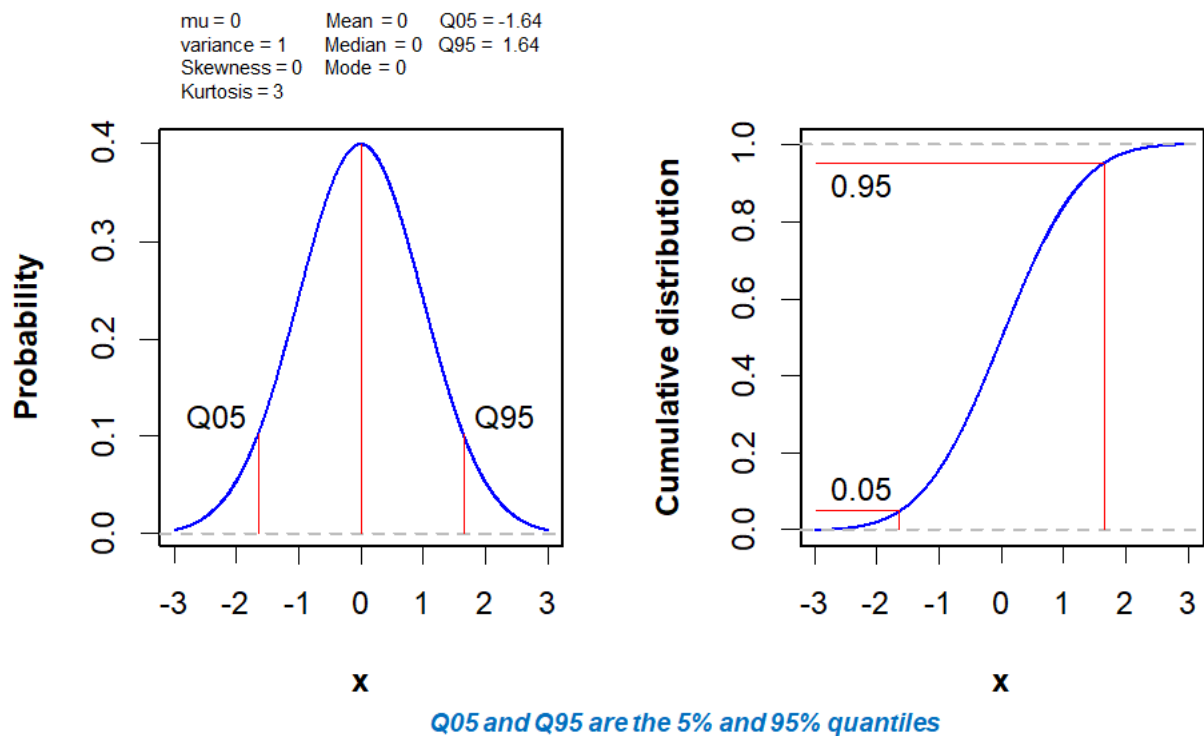
$$\kappa = \int \left(\frac{x - \mu}{\sigma} \right)^4 p(x) dx$$

Skewness is zero for symmetric distributions while kurtosis has value 3 for normal distributions.

Quantiles provide robust estimates of the CDF. Q01 and Q99 estimate the coordinates where the CDF has value 0.01 and 0.99 respectively, while Q50 is the median.

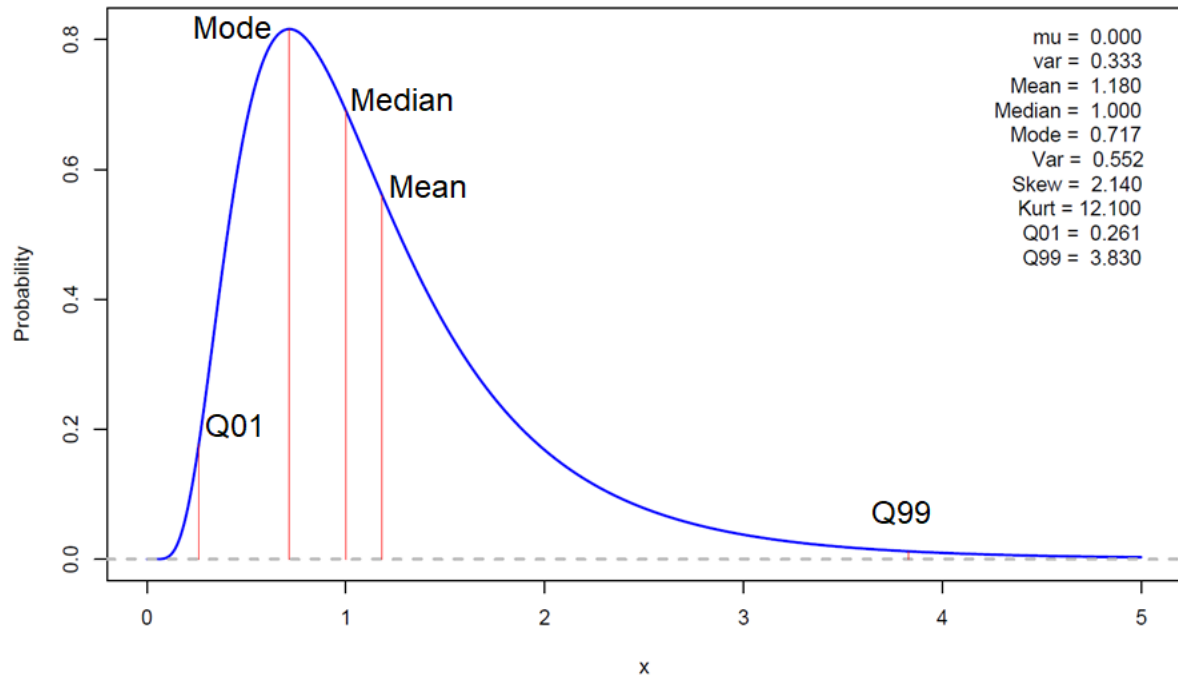
Statistical Measures: Standard Normal Distribution

Figure 98 PDF vs CDF Plot



Statistical Measures: Log Normal Distribution

Figure 99 Log Normal PDF



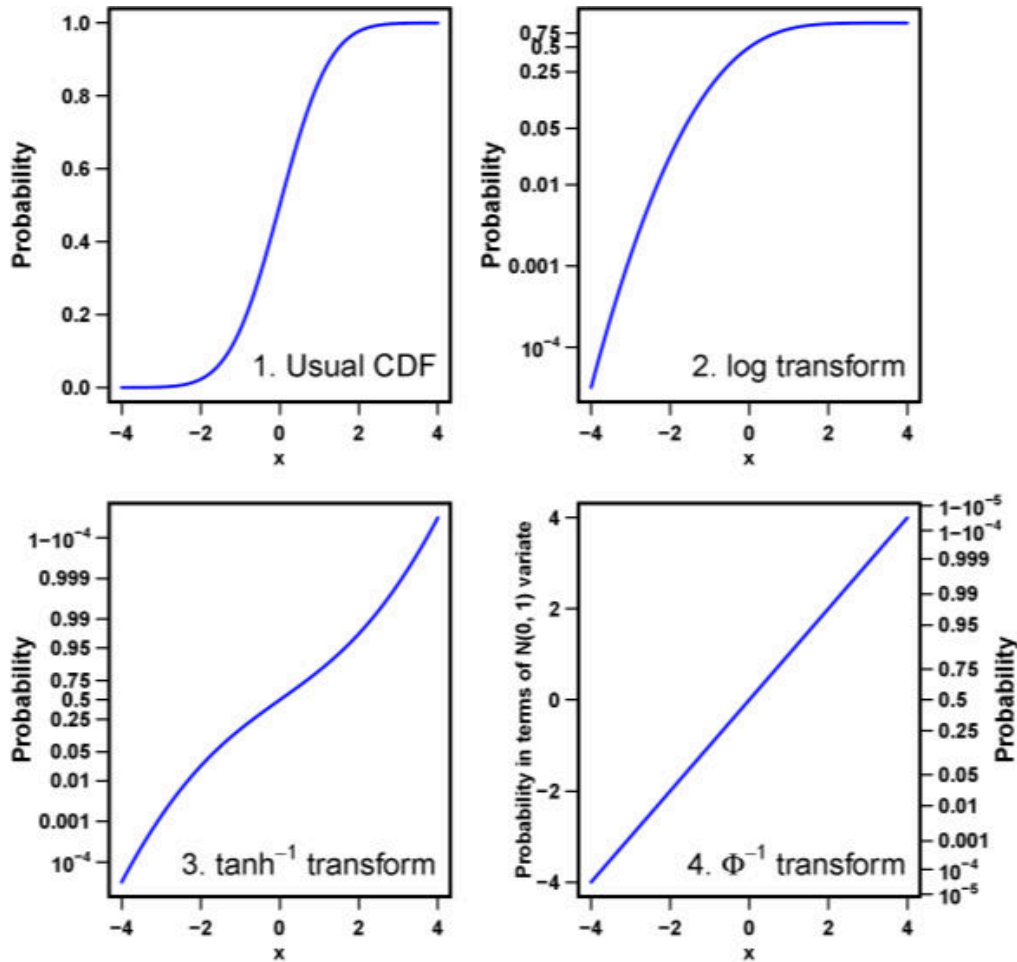
Transformations on the CDF

The cumulative distribution function is repeated in the plot 1(top left) of [Figure 100](#). The range for the y-axis range is between 0 and 1.

While the CDF is for a normal distribution, the shape for other distributions is similar. The CDF is close to 0 or 1 in the tails but the values are hard to resolve on the regular y-axis scale.

Plots 2, 3, and 4 shows some transformations on the y-axis.

Figure 100 CDF Transformations



Plot 2(top right) of Figure 100 shows the CDF with the y-axis on the log scale. In the log transformation, while it is possible to resolve values close to 0, values close to 1 are still compressed.

Consider plot 3 of Figure 100 which shows a scaling with $\tanh^{-1}(2y-1)$. Here the values close to 0 and 1 are both resolved.

Plot 4(bottom right) shows the probit transformation (Φ^{-1}) which changes the y-axis such that the CDF curve becomes a 45° straight line for a standard normal distribution.

In plot 4, the probability axis is shown on the right side and ranges over many orders of magnitude. It is customary to use the equivalent ordinates of the standard normal CDF on the left axis for ease of interpretation. The same transformation is used for Q-Q plots (see Figure 101).

The tail probabilities are resolved well in this plot and departures from a normal distribution are easily detected visually.

Comparing Distributions

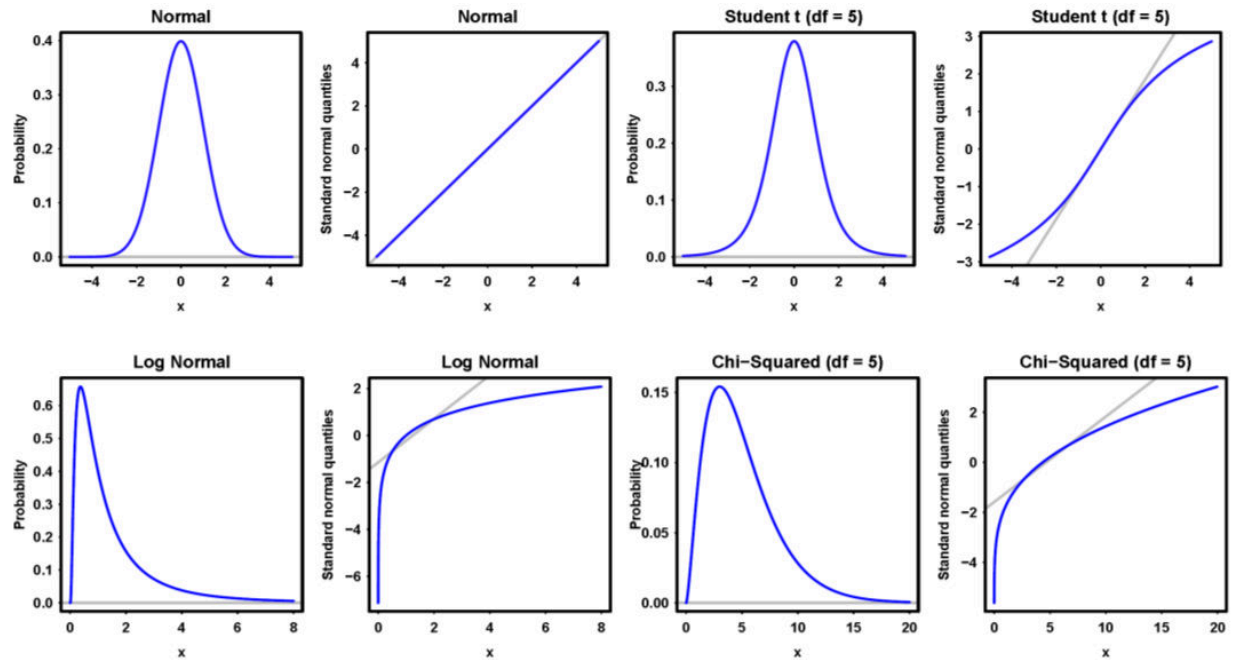
To compare distributions, examine the following:

- Given samples from a distribution, you can determine whether the underlying distribution is a normal distribution by looking at whether:
 - Median $\approx$ Mean
 - Sigma $\approx$ MAD
 - Skewness $\approx$ 0
 - Kurtosis $\approx$ 3
- Given data from two random experiments, you can tell whether they arise from the same distribution by doing the following:
 - Use a formal test of hypothesis, such as the Kolmogorov-Smirnov (K-S) test which is the most common one for arbitrary distributions. This test requires some coding or access to a statistical package like R or Matlab. The results, however, are not intuitive.
 - A quick qualitative idea can be obtained from the Q-Q plot. The data are expected to lie on or close to a 45 degree line if the response is standard normal. However, some departure in the tail regions is typical.

While strongly different distributions are easily identified using Q-Q plots, it is difficult to distinguish between similar distributions.

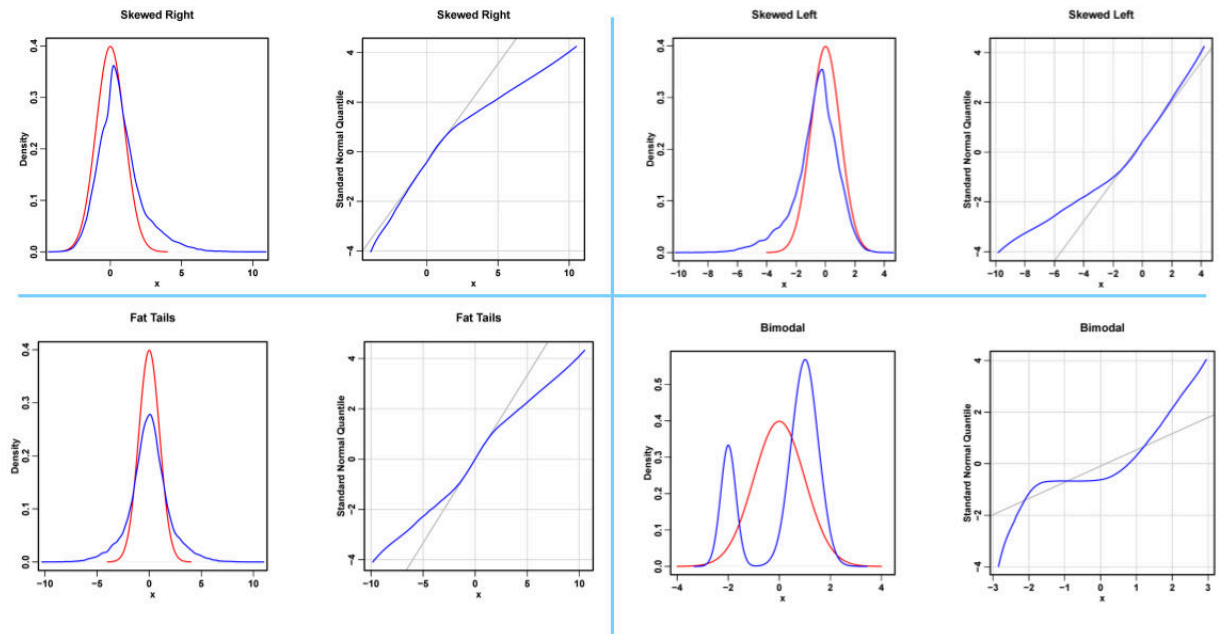
Q-Q plots for some typical response distributions are shown in [Figure 101](#):

Figure 101 Density and Q-Q Plots for Typical Distributions



While the plots for non-normal distributions are no longer straight lines, the tail probabilities can still be resolved easily.

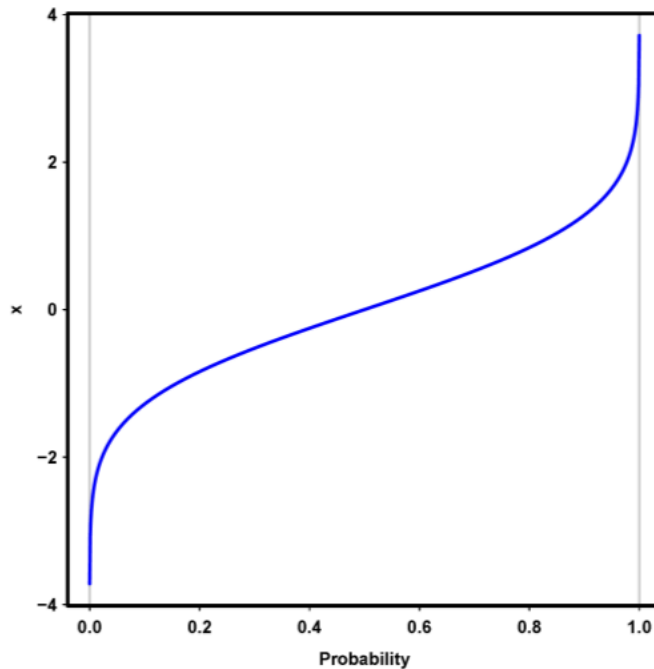
Figure 102 Additional Q-Q Plot Examples



From Probabilities to Ordinates

The ordinate of the standard normal density that corresponds to a specified probability, $x = \phi^{-1}(p)$ must be ascertained. This is done by exchanging the axes on the CDF plot.

Figure 103 Inverse Probability Transformation



This technique is also used to generate random numbers from distributions using the inverse probability integral transform method.

Mathematically, the transformation to go from a probability to a standard normal ordinate is:

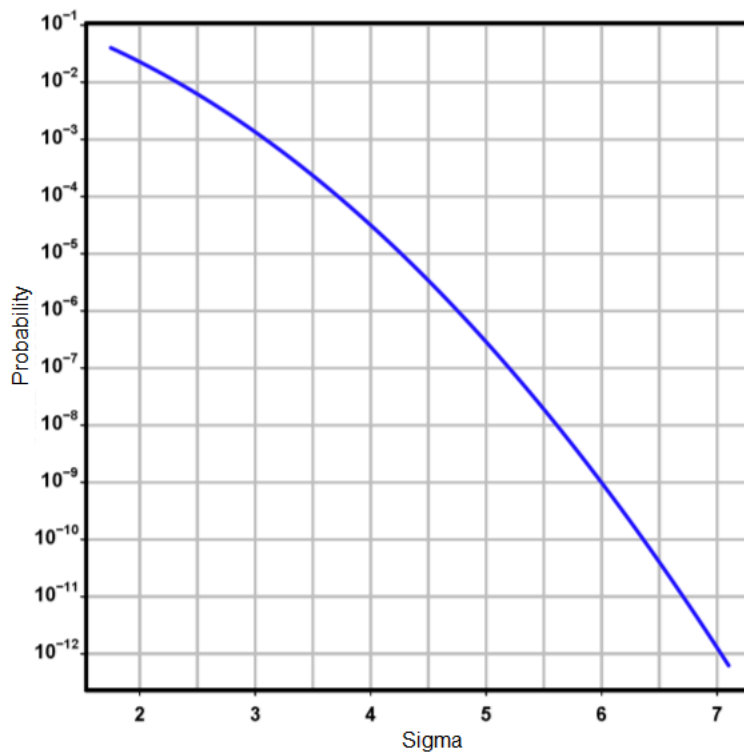
$$x = \Phi^{-1}(p) = \sqrt{2} \operatorname{erfc}(2p - 1), \quad 0 \leq p \leq 1$$

Probability and Sigma

Design engineers often define probability in terms of the sigma of the standard normal distribution even when the distribution is non-normal. Statistical literature uses the term percentiles while reliability engineers use parts per million (ppm) failures. Percentiles can range over many orders of magnitude while sigma values are usually small numbers between -7 and 7.

Figure 104 shows the mapping between sigma and probability.

Figure 104 Probability vs. Sigma



This manual often represents probabilities using the sigma for the standard normal density.

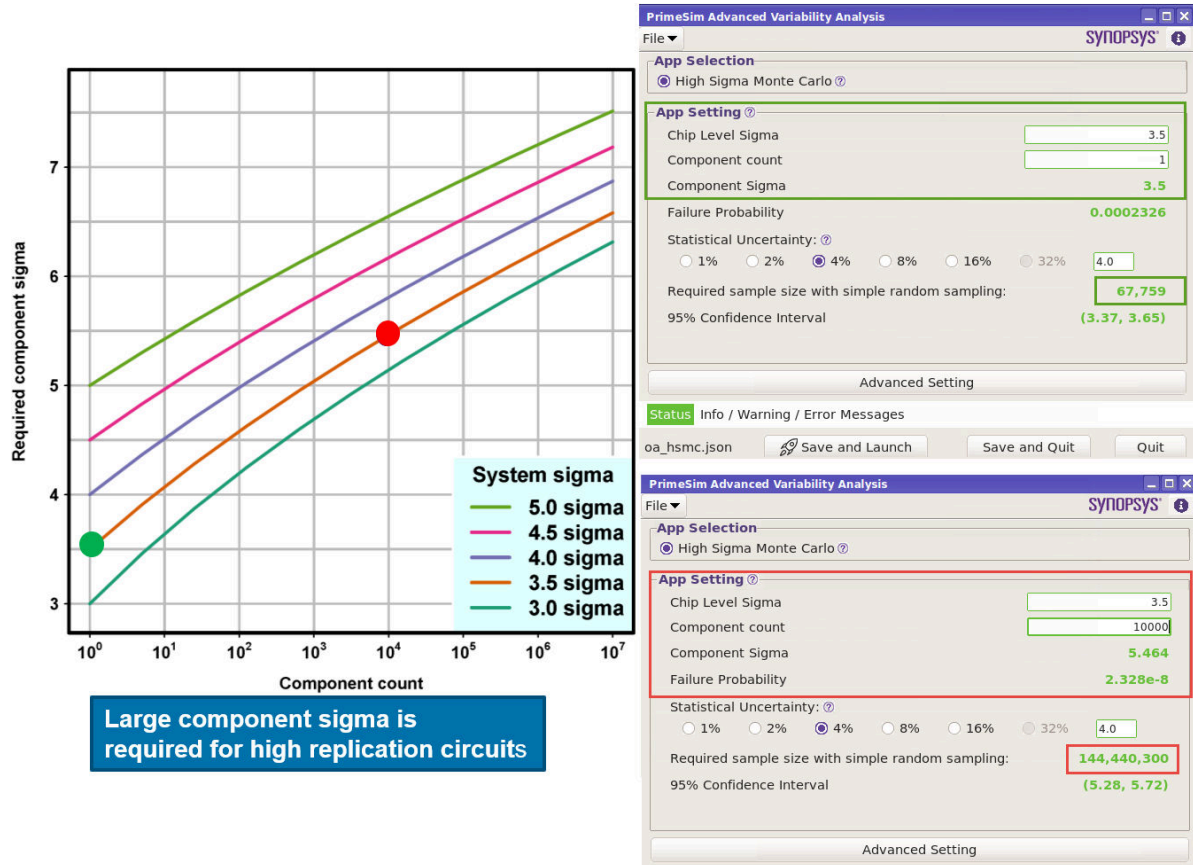
System and Component Reliability

Consider a system with multiple components and that the system functions only if all the components function (assuming there is no redundancy). If component j has probability P_j of functioning correctly, the probability of the system functioning correctly is $\prod_j P_j$.

If the system consists of K identical components, the system failure probability f_s is related to the component failure probability f_c by $f_s = 1 - (1 - f_c)^K$.

The curves in [Figure 105](#) show this equation in chart form.

Figure 105 Chip Level and Component Sigma



Consider this example: Let the system requirement be 3.5 sigma, which corresponds to the orange curve in [Figure 105](#). If it contains only a single component, then it can be denoted by the green dot. A system with a target 3.5 sigma with a single component simply requires a component sigma of only 3.5. The same can be seen from the HSMC computation in the top-right of [Figure 105](#). The verification of such a system by simple random sampling requires ~68K MC samples.

If the component is replicated 10K times in the system, then it can be denoted by the red dot on the orange curve (3.5 sigma) in [Figure 105](#). The component sigma requirement, for a system sigma requirement of 3.5, increases to ~5.5 (see the HSMC computation on the bottom-right) of the figure. The verification in this case requires ~144M MC samples.

Sampling and Monte Carlo Simulation

While the distribution of the process variables is provided by the foundry and is generally assumed to be Gaussian, the distribution of the response is usually not known analytically.

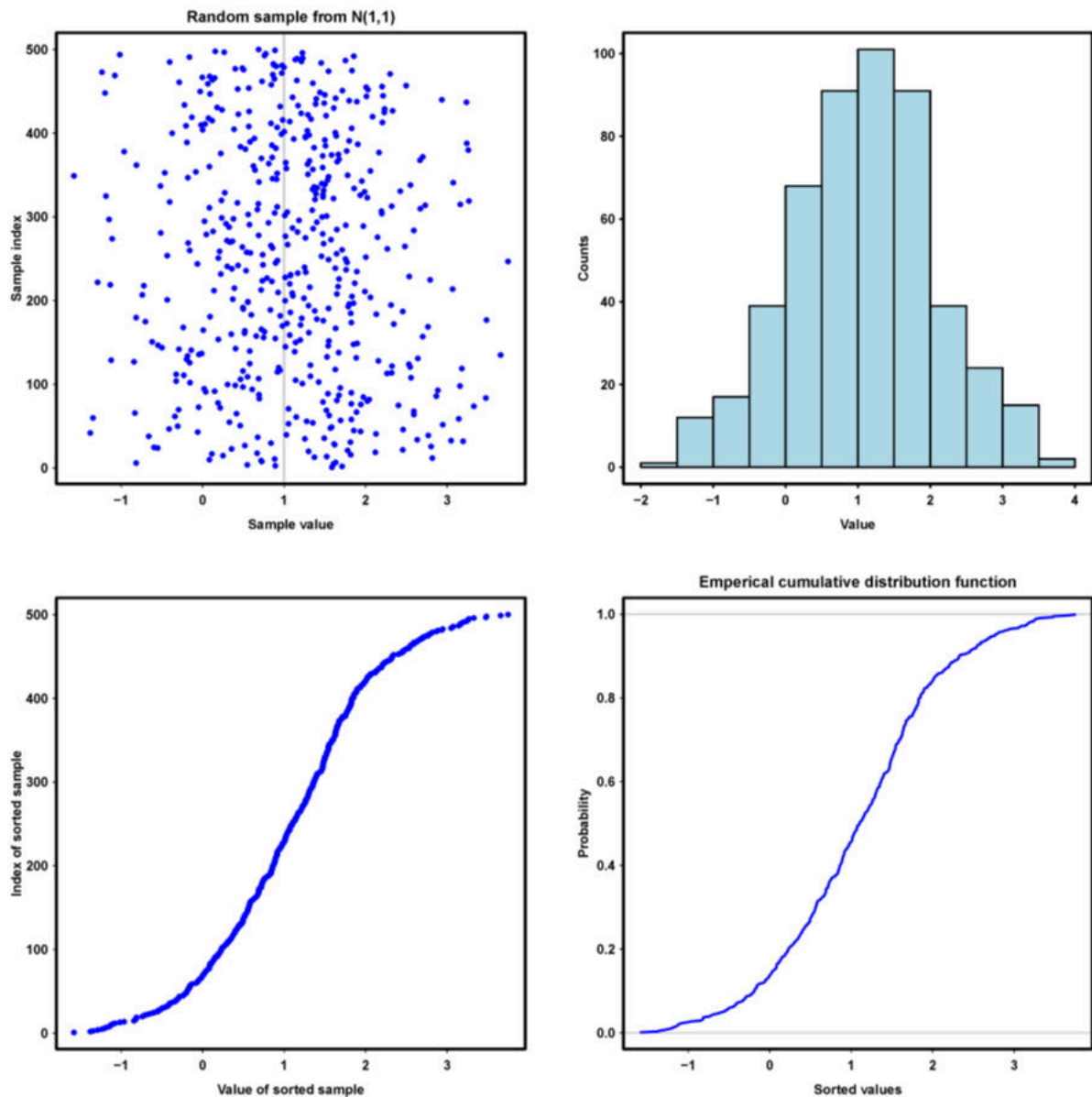
The response distribution and appropriate statistical measures such as moments and percentiles are determined by *Monte Carlo sampling*.

To mimic the manufacturing process on the computer, perform the following steps:

1. Generate random numbers for the process variables.
2. Simulate the circuit with these random variables.
3. Post-process the output once a sufficient number of samples have been simulated.

While the distributions of the independent process random variables (IRVs) are provided by the foundry, the distribution of the circuit response is unknown and may not have an analytic form. Monte Carlo simulation enables you to generate samples from the IRVs, run simulations, and find corresponding values for the response.

Figure 106 Sampling and Monte Carlo



The top-left diagram in [Figure 106](#), shows one such response. The x-axis gives the sample values and the y-axis is the corresponding Monte Carlo sweep index.

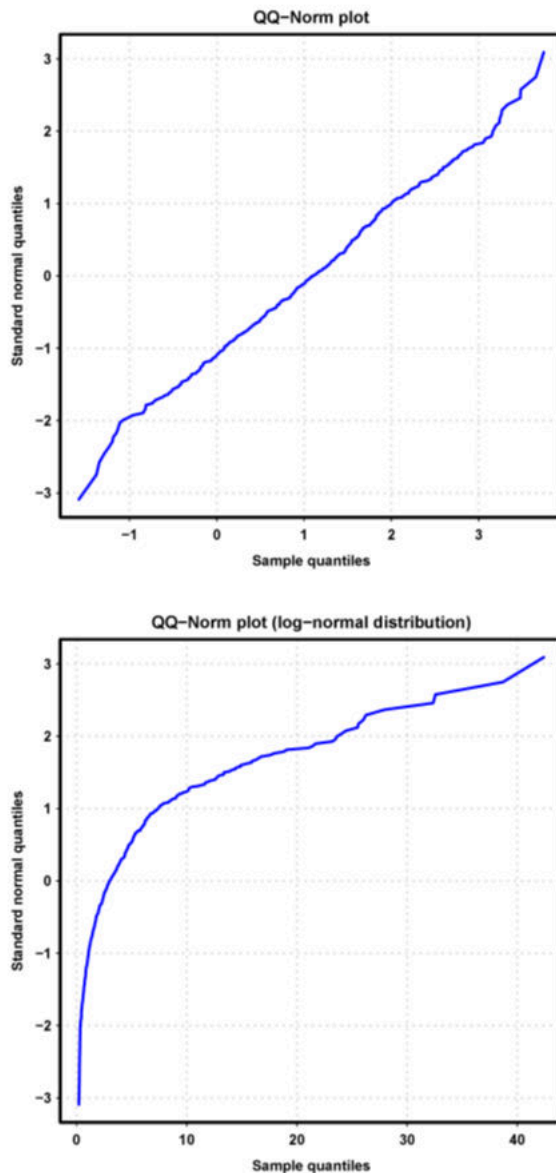
The data summary is shown in the top right as a histogram which is an approximation to the PDF.

A different view can be obtained if we sort the data and permute the associated index, as seen in the bottom-left plot.

Finally, we approximate the CDF by dividing the y-axis by the number of samples, as seen in the bottom-right plot. This is called the Empirical CDF (ECDF) and is an approximation to the CDF for the response density.

As with the cumulative distribution function, it is difficult to read values of small and large probabilities on the ECDF plot. You can apply the probit transformation to the y-axis, mapping probabilities to equivalent ordinates of the standard normal distribution (see the top plot in [Figure 107](#)).

Figure 107 More Q-Q Plot Examples



This stretches the y-axis and leads to near straight lines for samples from normal distributions.

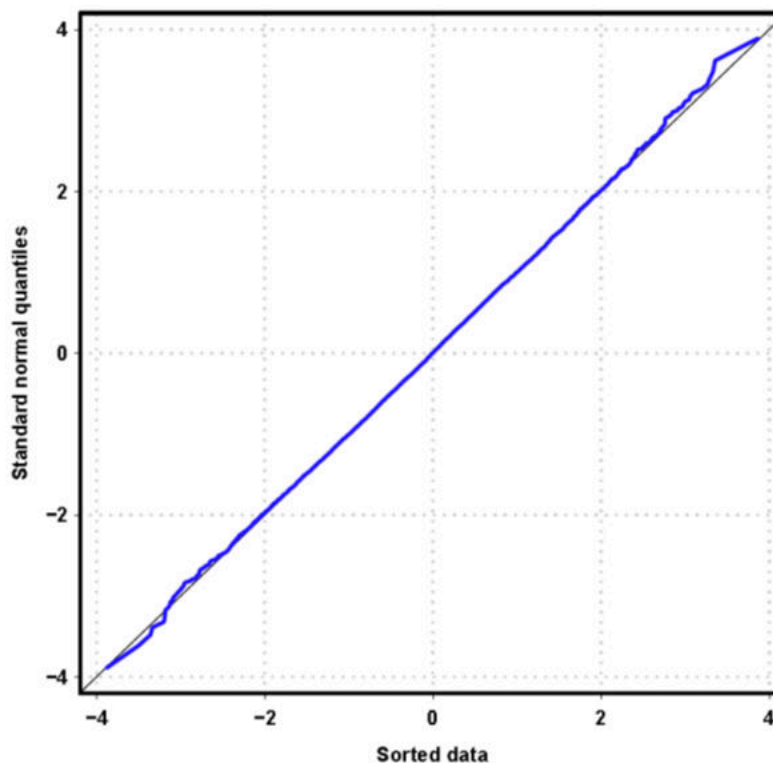
The same transformation can be used with samples from non-normal distributions. In [Figure 107](#), the plot at the bottom is for samples from a log-normal distribution.

The curve no longer looks like a straight line. However, the probit transformation still stretches the y-axis and enables us to easily read small and large probability values. The plot where the sorted sample values form the x-axis and the probit transformation of the sequence $(1:N - 0.5)/N$, where N is the sample size, gives the y-axis is called the sample Q-Q plot.

Q-Q Plot - Confidence Intervals

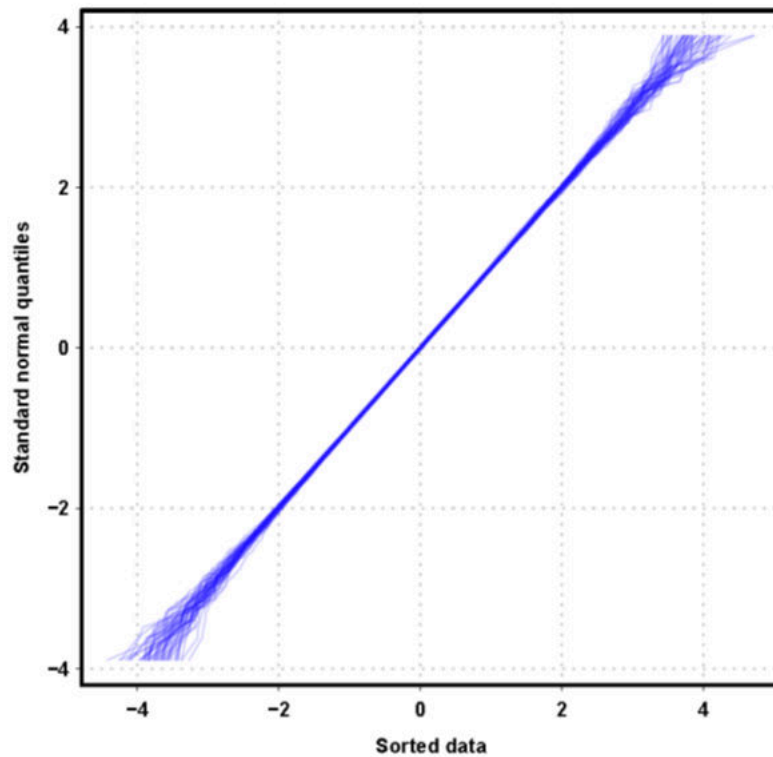
The following example draws a sample from a distribution and creates Q-Q plot for the data. Unlike the ideal probit plot, there is a small departure from a straight line (see [Figure 108](#)), due to the approximation of the distribution by drawing a sample from it.

Figure 108 Q-Q Plot of a Sample Distribution



Repeat the experiment a few more times with different random number seeds and superpose the plots (See [Figure 109](#)).

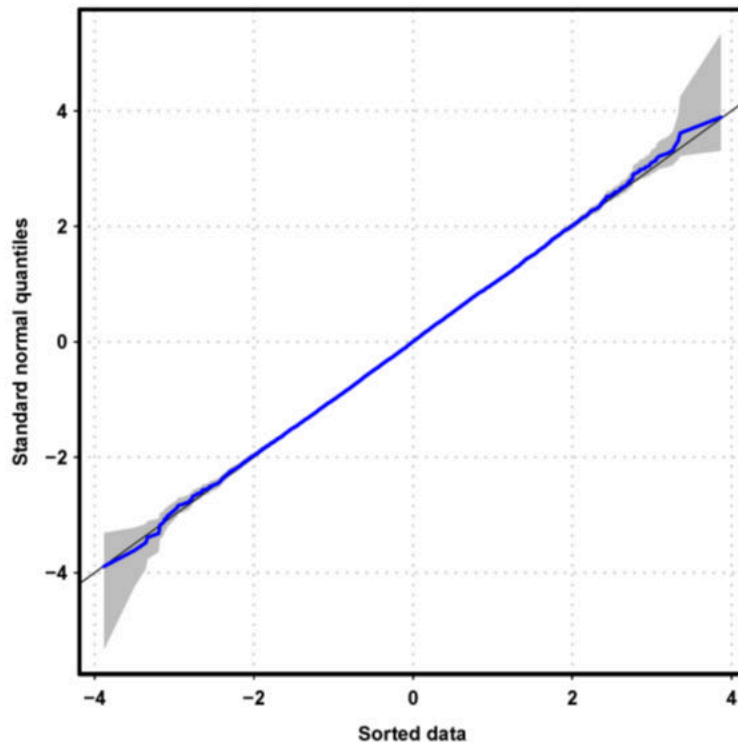
Figure 109 Q-Q Plots for Repetitions of the Experiment



The plots are not identical but are closer together in the middle and spread out towards the ends due to *sampling errors*.

The confidence interval, the grey shaded region in [Figure 110](#), captures the statistical sampling error concept in a formal manner.

Figure 110 *Confidence Intervals*



The different Q-Q plots lie within the grey area 95% of the time when the confidence interval is 95%. The Q-Q plot confidence interval is based on the beta distribution for the order statistics.

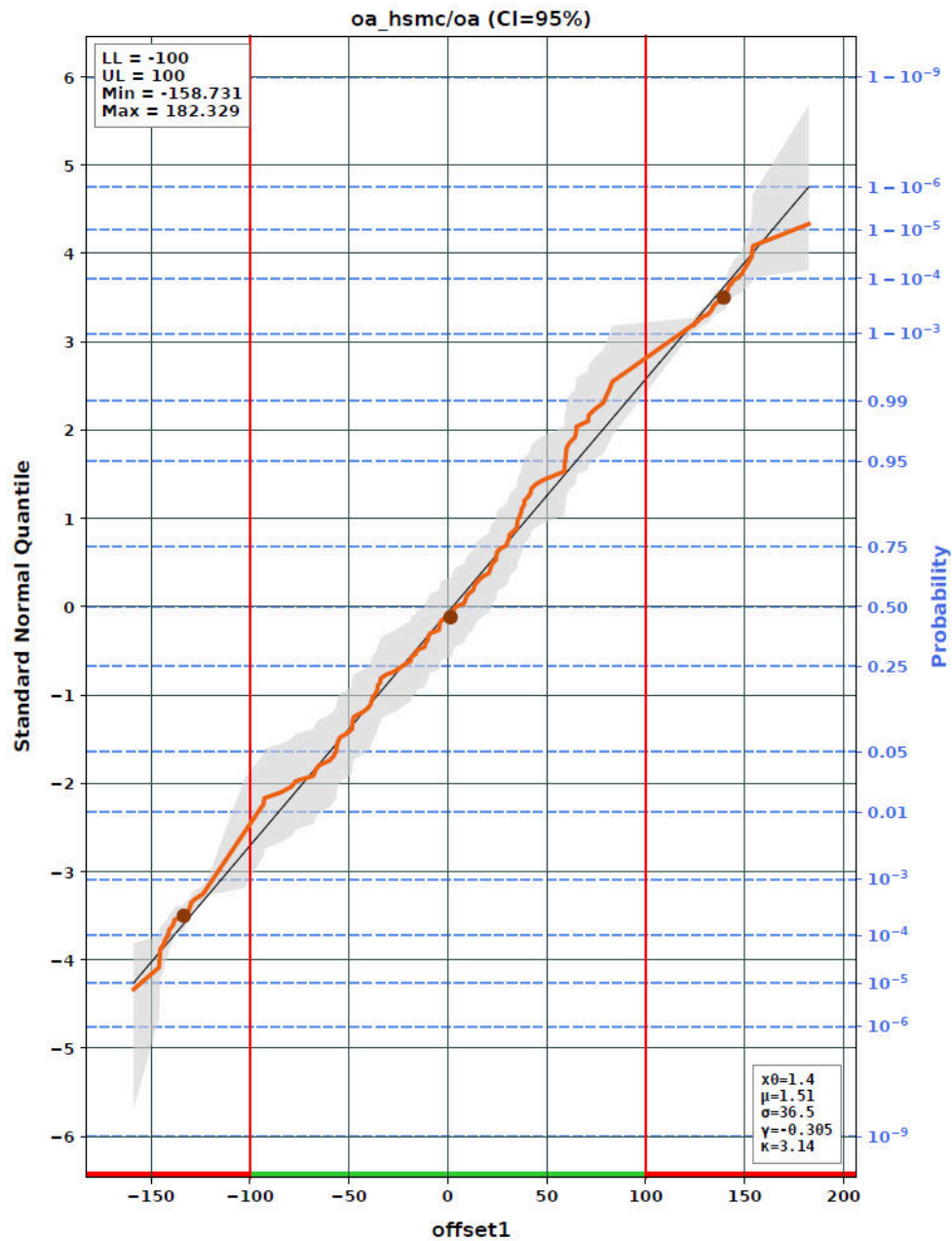
The y-axis range and the confidence interval width depend on the sample size. The y-axis range increases and the confidence interval shrinks with increasing sample size.

Understanding the PrimeSim AVA Q-Q Plots

The PrimeSim Design Robustness tools supports the following types of Q-Q plot outputs:

- PDF
- WaveView
- CSV (comma-separated value)

Figure 111 PrimeSim AVA Q-Q Plot



The x-axis corresponds to a particular measure. The left side y-axis gives probabilities in terms of standard normal quantiles while the right side y-axis gives explicit probabilities. The orange line is the Q-Q plot curve for the data.

The grey shaded area gives the 95% confidence intervals for the Q-Q plot. The black diagonal line is called the Q-Q line and is a visual guide to show where data from a normal distribution would lie.

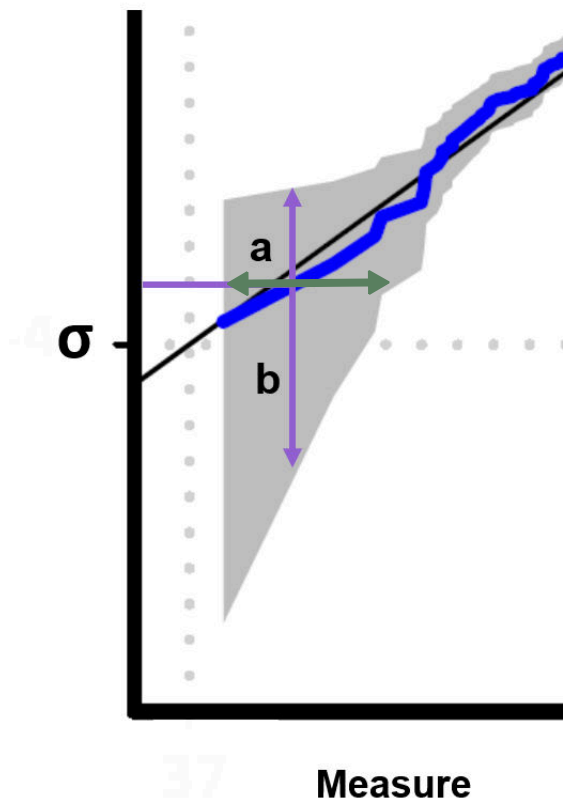
The bottom-right box gives values for nominal run and first four central moments (mean, standard deviation, skewness, and kurtosis). Sigma Amplification gives two more values : sample size and Sigma Amplification factor (if not equal to 1). The green and red bars along the x-axis show the specifications for the measure and the vertical red lines serve as visual guides. The green and red bars are not present in the plot if the measure has no specifications.

Sigma, Sample Size, and Statistical Uncertainty

This topic discusses how to determine the accuracy of your results for a given sample size or what sample size to use for a specified accuracy.

Repeated experiments with different seed values give different Q-Q plots that lie within the grey shaded region in, for example, 95% of the experiments.

Figure 112 Statistical Uncertainty Due to Sampling



The vertical arrow represents the confidence interval for Sigma (U) and the horizontal arrow represents the confidence interval of the measure (U_{measure}).

$$U = \frac{a+b}{2|\sigma|} \times 100\%$$

Figure 113 illustrates graphically the PrimeSim AVA HSMC engine's computation of the confidence intervals for a given chip level sigma and statistical uncertainty.

Partial Q-Q plot of right tail

PrimeSim Advanced Variability Analysis

App Selection

- High Sigma Monte Carlo

App Setting

- Chip Level Sigma: 4.5
- Component count: 1
- Component Sigma: 4.5
- Failure Probability: 3.398e-6
- Statistical Uncertainty: 4.0% (Selected: 4%, 1%, 2%, 8%, 16%, 32%)
- Required sample size with simple random sampling: 1,918,419
- 95% Confidence Interval: (4.34, 4.70)

Status Info / Warning / Error Messages

oa_hsmc.json Save and Launch Save and Quit

Measure

Partial Q-Q plot of right tail

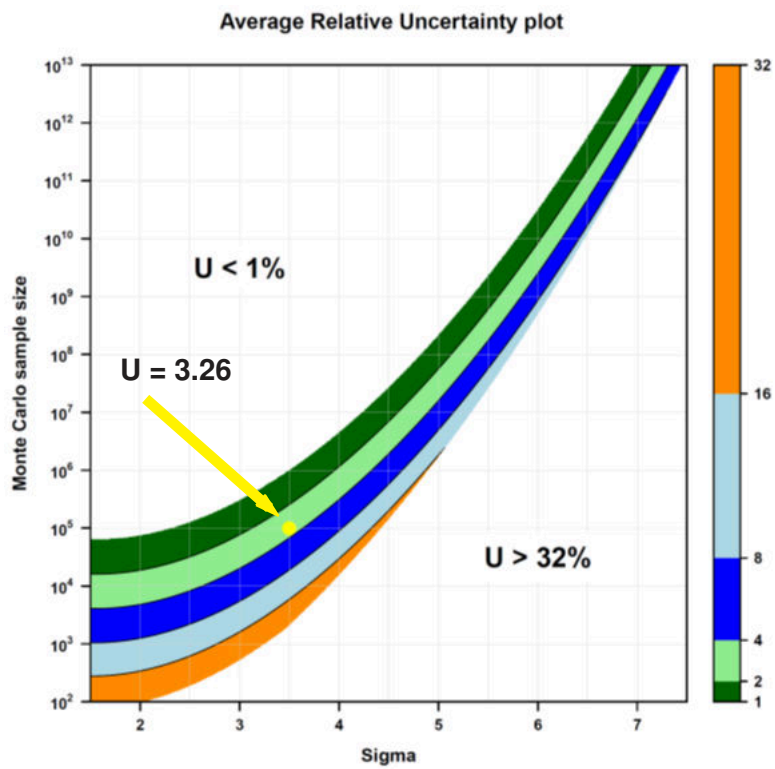
U = 0.5*(4.7-4.34)/4.5*100% = 4%

Here confidence bands:

a = (4.7-4.5) b = (4.5-4.34)

PrimeSim™ Design Robustness User Guide
W-2024.09

Figure 114 Uncertainty, Sigma, and Sample Size



From the plot, for example, U is 3.26% for $\sigma=3.5$ and $N=10^5$.

Figure 115 provides an example of uncertainty as a function of sigma and Monte Carlo sample size.

Figure 115 PrimeSim AVA HSMC: Uncertainty as a Function of Sigma and Sample Size

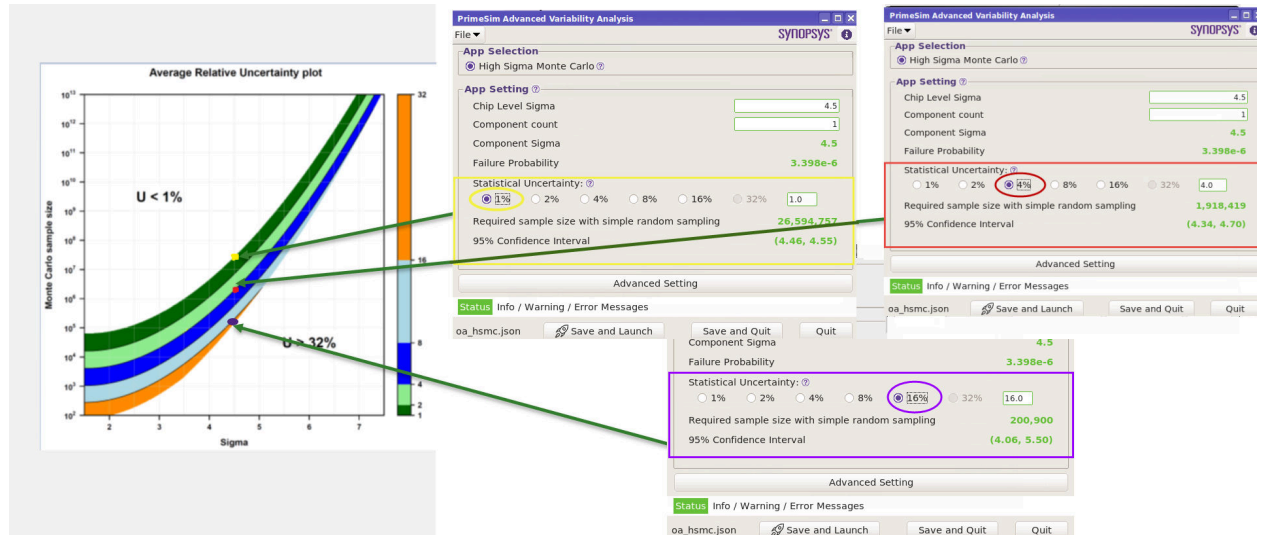


Table 7 UI Details for Figure 115

UI Reference	Component Sigma	Uncertainty	Required Sample Size	95% Confidence Interval
Corresponding to the purple point on the Average Relative Uncertainty plot	4.5	16%	200,900	4.06, 5.50
Corresponding to the red point on the Average Relative Uncertainty plot	4.5	4%	1,918,419	4.34, 4.70
Corresponding to the yellow point on the Average Relative Uncertainty plot	4.5	1%	26,594,757	4.46, 4.55

It can be seen from the preceding figure and table, that ~1.9 M samples are required for sigma= 4.5 and U = 4% and that increases to ~26 M samples for the same sigma for U=1%.

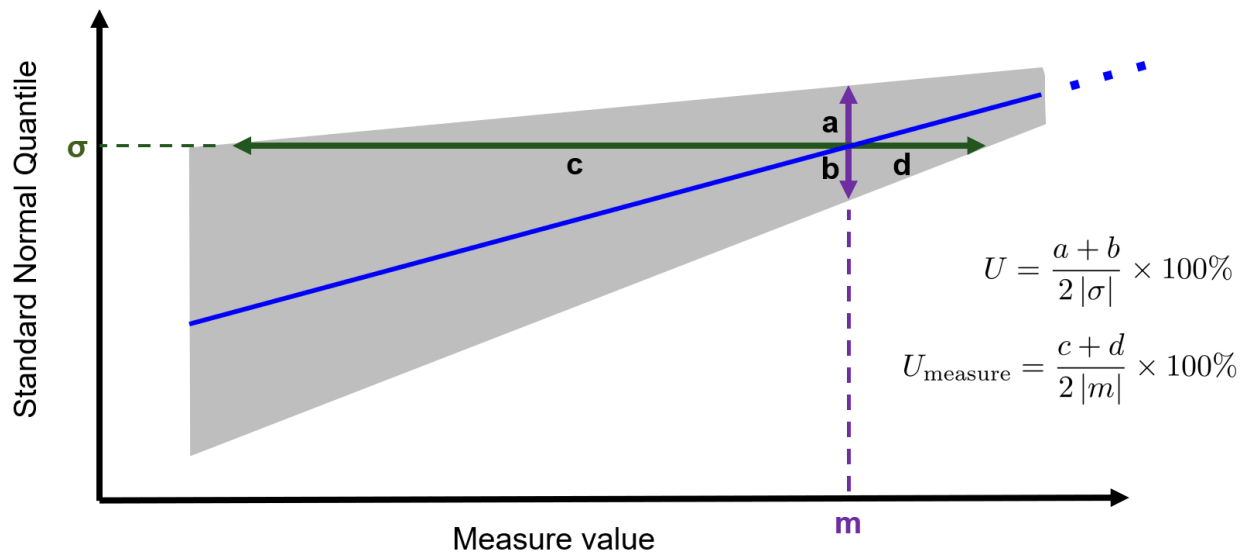
The table in [Figure 116](#) describes quantitatively the increase in the sample size required for standard Monte Carlo as a function of sigma and average relative uncertainty.

Figure 116 Sample Size Required for Standard Monte Carlo

Average Relative Uncertainty						
Sigma		1%	2%	4%	8%	16%
	2.5	125,714	31,985	8,269	2,201	618
	2.75	186,452	47,585	12,373	3,329	954
	3	300,388	76,920	20,125	5,478	1,608
	3.25	524,406	134,769	35,499	9,789	2,950
	3.5	989,944	255,393	67,759	18,955	5,876
	3.75	2,017,082	522,520	139,711	39,704	12,678
	4	4,429,252	1,152,384	310,700	89,833	29,556
	4.25	10,467,733	2,735,990	744,282	219,251	74,284
	4.5	26,594,757	6,984,925	1,918,419	576,528	200,900
	4.75	72,566,762	19,156,633	5,315,636	1,631,491	583,673
	4.8	89,456,711	23,640,209	6,573,777	2,026,389	728,595
	5	212,478,240	56,393,265	15,820,754	4,963,331	1,819,152
	5.25	667,133,580	178,064,310	50,542,823	16,216,300	--
	5.5	2,244,712,400	602,699,820	173,216,190	56,847,730	--
	5.75	8,089,479,800	2,185,585,500	636,470,150	213,638,800	--
	6	31,209,304,000	8,487,394,400	2,506,185,600	860,018,860	--
	6.25	128,845,080,000	35,281,192,000	10,570,487,000	3,705,834,800	--
	6.5	568,996,420,000	156,933,910,000	47,734,955,000	17,142,492,000	--
	6.75	2,686,971,100,000	746,707,780,000	230,707,130,000	84,343,037,000	--

[Figure 117](#) illustrates the uncertainty in sigma and measure values.

Figure 117 Uncertainty in Measure Value for Various Sigma

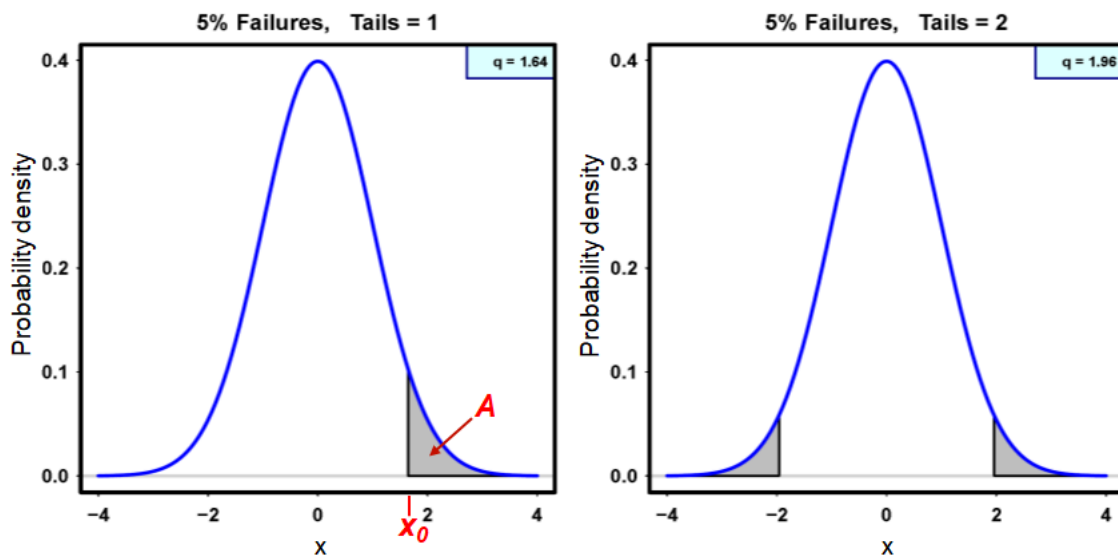


Depending on the distribution of a given measure, you may have $U > U_{\text{measure}}$ or $U < U_{\text{measure}}$.

One- or Two-Sided Specifications

Specifications can be given for one or both tails. In [Figure 118](#), a normal distribution is used for illustration. The concepts, however, apply to all distributions.

Figure 118 One- and Two-Sided Specifications



Two common situations can arise:

- The area A of the grey region is given, determine x_0 (find percentile).
- x_0 is given, determine the area A of grey region (find yield).

JSON Format and Examples

This appendix provides a brief overview of the JSON format.

This sections covers the following topics

- [What is JSON?](#)
- [JSON Examples for MC](#)
- [JSON Examples for HSMC and Sigma Amplification](#)
- [JSON for Exporting DSVs While Running High Sigma Monte Carlo](#)

- [JSON for 10-sigma quantile using EVD in HSMC](#)
- [JSON for Block Specific Sigma Amplification](#)

What is JSON?

JSON stands for JavaScript Object Notation. It is a simple and lightweight data-interchange format. JSON parsers exist for all common languages (Python, R, and C++, for example).

Uses data types such as key/value pairs, arrays (with nesting), Boolean, numeric, and strings.

For a beginners tutorial on JSON, see <https://beginnersbook.com/2015/04/json-tutorial/>.

JSON Examples for MC

This sections covers the following topics:

- [Sample Size Mode Using AVA](#)
- [Sigma/Uncertainty Mode Using AVA](#)

Sample Size Mode Using AVA

Example 7 MC simulation in Sample Size mode using AVA

```
{
  "Active_Application": "Monte_Carlo",
  "Monte_Carlo": {
    "N_Samples": 20000,
    "Sampling_Method": "SRS",
    "Normal_Limit": 20,
    "LSMC": false
  }
}
```

Sigma/Uncertainty Mode Using AVA

Example 8 MC simulation in Sigma/Uncertainty mode using AVA

```
{
  "Active_Application": "Monte_Carlo",
  "Monte_Carlo": {
    "Chip_Sigma": 4.5,
    "Component_Count": 1,
    "Average_Relative_Uncertainty": 4.0,
    "Sampling_Method": "SRS",
    "Normal_Limit": 20,
    "LSMC": true
  }
}
```

```
}  
}
```

JSON Examples for HSMC and Sigma Amplification

This sections covers the following topics

- [JSON for HSMC Basic Run](#)
- [JSON for HSMC With Measure Specification, Do_Not_Vary Control, and AVA Output Settings](#)
- [JSON for Left, Right, and Both Measure Tail Selection](#)
- [JSON to Update Alert Time to 60 Minutes](#)
- [JSON for HSMC When Reference Results are Specified Manually](#)
- [JSON for Sigma Amplification With AVA Output Settings](#)
- [JSON for PYE - Left, Right, and Both Measure Tail Selection](#)

JSON for HSMC Basic Run

Example 9 JSON for HSMC Basic Run

```
{  
  "Active_Application": "High_Sigma_Monte_Carlo",  
  "High_Sigma_Monte_Carlo": {  
    "Chip_Sigma": 3.5,  
    "Component_Count": 1,  
    "Average_Relative_Uncertainty": 4.0  
  }  
}  
  
{  
  "Active_Application": "High_Sigma_Monte_Carlo",  
  "High_Sigma_Monte_Carlo": {  
    "Chip_Sigma": 3.5,  
    "Component_Count": 1,  
    "N_SRS_Samples": 67759  
  }  
}
```

JSON for HSMC With Measure Specification, Do_Not_Vary Control, and AVA Output Settings

Example 10 JSON for HSMC With Measure Specification, Do_Not_Vary Control, and AVA Output Settings

```
{  
  "Active_Application": "High_Sigma_Monte_Carlo",
```



```
"High_Sigma_Monte_Carlo": {
  "Chip_Sigma": 4.0,
  "Component_Count": 1,
  "Average_Relative_Uncertainty": 8.0
},
"Measure_Specification_File": "user_meas_spec.csv",
"Do_Not_Vary": {
  "Instances": [
    "xi6.mb2",
    "xi6.mb1",
    "xi5.mb2",
    "xi5.mb1"
  ]
},
"AVA_Output": {
  "QQ_PDF": true,
  "QQ_WV": true,
  "Nominal": true,
  "Detailed": true,
  "Sensitivity": true,
  "ava.dsvc.json": true
}
```

JSON for Left, Right, and Both Measure Tail Selection

Example 11 JSON for Left, Right, and Both Measure Tail Selection

```
{
  "Active_Application": "High_Sigma_Monte_Carlo",
  "High_Sigma_Monte_Carlo": {
    "Chip_Sigma": 3.5,
    "Component_Count": 1,
    "Average_Relative_Uncertainty": 4.0
  },
  "Measure_Specification_File": "user_meas_spec.csv"
}
user_meas_spec.csv
Name,Include,Tail,Lower_Spec,Nominal,Upper_Spec,Weight
"offset1",1,Left,-inf,NA,inf,1
"offset2",1,Right,-inf,NA,inf,1
"pwr",1,Both,-inf,NA,inf,1
```

Note:

Measure name in spec `csv` is in double quotes, special char in measure name such as `#` is supported.

JSON to Update Alert Time to 60 Minutes

Example 12 JSON to Update Alert Time to 60 minutes

```
{
  "HM_Alert_Time_Limit" : 60,
```

```
"Active_Application" : "High_Sigma_Monte_Carlo",
"High_Sigma_Monte_Carlo" : {
"Measure_Specification_File": "meas.csv",
"Average_Relative_Uncertainty" : 4,
"Component_Sigma" : 5
}
}
```

JSON for HSMC When Reference Results are Specified Manually

Example 13 JSON for HSMC When Reference Results are Specified Manually

```
{
"Active_Application" : "High_Sigma_Monte_Carlo",
"High_Sigma_Monte_Carlo" : {
"Golden_Prefix" : "./golden",
"Average_Relative_Uncertainty" : 4,
"Chip_Sigma" : 6,
"Component_Count" : 1
}
}
```

JSON for Sigma Amplification With AVA Output Settings

Example 14 JSON for Sigma Amplification With AVA Output Settings

```
{
"Active_Application": "Sigma_Amplification",
"Sigma_Amplification": {
"Chip_Sigma": 3.5,
"Component_Count": 1,
"Average_Relative_Uncertainty": 4.0,
"N_Afford_Samples": 700
},
"AVA_Output": {
"QQ_PDF": true,
"QQ_WV": true,
"Nominal": true,
"Detailed": true,
"Sensitivity": true
}
}
```

JSON for PYE - Left, Right, and Both Measure Tail Selection

Example 15 JSON for PYE - Left, Right, and Both Measure Tail Selection

```
{
Active_Application: Parametric_Yield_Estimation,
Parametric_Yield_Estimation: {
Average_Relative_Uncertainty:4.0
Golden_Prefix :gol,
},
}
```

```

        Measure_Specification_File: input_meas_spec.csv,
        ...
    }
Index,Name,Include,Monitor,Tail,Lower_Spec,Nominal,Upper_Spec,Weight,Objective
0,"log_pwr",1,0,Both,3,NA,3.5,1,monitor
1,"offset1",1,0,Right,-inf,NA,100,1,monitor
2,"offset2",0,0,Left,-100,NA,inf,1,monitor

```

Note:

Measure name in spec csv is in double quotes, special char in measure name such as # is supported.

Define the limit spec in input_meas_spec.scv under the same directory. An example of limit spec is as follows.

```

Index,Name,Include,Monitor,Tail,Lower_Spec,Nominal,Upper_Spec,Weight,Objective
0,"log_pwr",1,0,Both,3,NA,3.5,1,monitor
1,"offset1",1,0,Right,-inf,NA,100,1,monitor
2,"offset2",0,0,Left,-100,NA,inf,1,monitor

```

JSON for Exporting DSVCs While Running High Sigma Monte Carlo

Example 16 JSON for Exporting DSVCs While Running High Sigma Monte Carlo

```

{
  "Active_Application": "High_Sigma_Monte_Carlo",
  "High_Sigma_Monte_Carlo": {
    "Chip_Sigma": 3.5,
    "Component_Count": 1,
    "Average_Relative_Uncertainty": 4.0
  },
  "Measure_Specification_File": "exp3/export3_meas_spec.csv",
  "DSVC_Export": {
    "Instances": [
      "xi82.mn1",
      "xi82.mn2"
    ]
  }
}

```

JSON for Importing DSVCs

Example 17 JSON for Importing DSVCs

```

{
  "Active_Application": "DSVC_Import",
  "DSVC_Import": {
    "Data": [

```

```
{
  "DSVC_File": "<path_to_file>/opampmc.ava.dsvc.json",
  "Variation_Corners": [
    "leakpwr.Max"
  ],
  "Instance_Map": {
    "xi82.mn1": [
      "xi82.mn6"
    ],
    "xi82.mn2": [
      "xi82.mn8",
      "xi82.mn7"
    ]
  },
  "Scale_Factor": 1.0
},
{
  "Dryrun": true,
  "Nominal_Run": true
}
}
```

JSON for 10-sigma quantile using EVD in HSMC

Example 18 JSON for 10-sigma quantile using EVD in HSMC

```
{
  "Active_Application": "High_Sigma_Monte_Carlo",
  "High_Sigma_Monte_Carlo": {
    "Chip_Sigma": 10,
    "Component_Count": 1,
    "Average_Relative_Uncertainty": 4.0,
    "Enable_EVD": true
  }
}
```

JSON for Block Specific Sigma Amplification

Example 19 Block Specific Sigma Amplification

```
{
  "Active_Application": "Block_Specific_Sigma_Amplification",
  "Block_Specific_Sigma_Amplification": {
    "N_Afford_Samples": 400,
    "Block_Sigma": [
      {
        "Sigma": 2.5
      },
      {
        "Sigma": 4.0,
      }
    ]
  }
}
```

```
        "Instances": [
            "xi82.r0"
        ]
    },
    {
        "Sigma": 4.5,
        "Instances": [
            "xi82.mn8"
        ]
    },
    {
        "Sigma": 3.5,
        "Instances": [
            "xi82.mp3",
            "xi82.mp4"
        ]
    }
]
},
"Measure_Specification_File": "2303sp1_meas_spec.csv",
"AVA_Outputs": {
    "Nominal": true
}
}
```

JSON Entries

This appendix lists JSON entries for the PrimeSim Design Robustness tool.

This section covers the following topics:

- [JSON Files](#)
- [Application Related JSON Entries](#)
- [Output Format Entries](#)
- [Other Entries](#)

JSON Files

JSON input file gathers the information from the AVA dialog box based on the user input. JSON files are extremely syntax-sensitive. Each dialog box input translates to a JSON field followed by its acceptable argument.

For example: "Sampling_Method": "SRS"

A JSON file can either be created using the dialog box or an existing file. For command-line tool users, invoking the dialog box every time is complex. Therefore, you can intend to bypass the dialog box and initiate AVA simulation directly. In that case, you can conveniently make changes to an existing JSON file using text editor or scripts. This section describes frequently used JSON file entries along with their acceptable arguments to aid in customizing the JSON file.

Note:

To avoid any syntax-prone errors, it is recommended to create a reference JSON file using dialog box and then make required changes using text editors or shell scripts.

JSON file refers to each entry based on the function and places them in separate categories as shown in [Figure 119](#).

Figure 119 JSON File

```
{
  "Active_Application": "Monte_Carlo",
  "Monte_Carlo": {
    "N_Samples": 10000,
    "Sampling_Method": "SRS",
    "Normal_Limit": 20,
    "LSMC": false
  },
  "Measure_Specification_File": "oa_MC1/oa_MC1_meas_spec.csv",
  "AVA_Outputs": {
    "QQ_PDF": true,
    "QQ_WV": true,
    "QQ_CSV": true,
    "Nominal": true,
    "Detailed": true,
    "Sensitivity": true,
    "Critical_Sweep_Ext": true,
    "Critical_Sweep_Waveform": false,
    "Measure_File": true,
    "Sample_File": true
  },
  "Sensitivity_Method": "Classic",
  "Sensitivity_Screening_Method": "Pearson",
  "Sensitivity_Output_CSV": false,
  "Sensitivity_Detailed_File": false,
  "Sensitivity_Hide_IRV_Level": false
}
```

Application related entries

Output format entries

Others

You can classify these entries into three types:

- Application related entries
- Output format entries
- Others entries

Application Related JSON Entries

Several Application related entries are common to multiple applications. These are listed under General. The entries specific to each application are listed under its name.

Application related JSON Fields	Arguments	Description
General		
Chip_Sigma	Scalar value	Target sigma of the whole chip
Component_Count	Scalar value	Number of reputations of the cell within the chip
Average_Relative_Uncertainty	Scalar value	Statistical uncertainty – affects sample size and accuracy
Normal_Limit	Scalar value	Value of Normal Limit
Monte Carlo : Sample Size mode		
N_Samples	Scalar value	Number of samples for simulation
Sampling_Method	SRS/Sobol/OFAT	Sampling method
Monte Carlo : Sigma/Uncertainty Mode		
Sampling_Method	SRS Sobol	Sampling method
Sigma Amplification		
N_Afford_Samples	Integer	Value of affordable number of samples
Postprocessing		
Measure_Files	.m(s/a/t)# files	Pointers to the measure files
Sample_Files	.mc0 files	Pointers to the IRV files
Other_Percentile_Sigma	Numerical	List of user-specified sigma values at which percentiles are printed in the <code>.ava.report</code> file
Other_Percentile_Percentage	Numerical	List of user-specified percentages values at which percentiles are printed in the <code>.ava.report</code> file
DSVC Import		

Application related JSON Fields	Arguments	Description
Nominal_Run	Yes/No	Includes nominal simulation in addition to variation corners
Dryrun	Yes/No	Skips import simulation. Identifies variation corners in DSVC file, lists instances to substitute in the AVA log, and performs necessary error check.
Data	Other JSON sub-entries	List of data elements containing DSVC file, variation corners, and instances to substitute.
DSVC_File	Pointer to file	DSVC file to import,
Variation_Corners	Selected corners in format <measure_name>.<corner_type>	List of variation corners from DSVC file.
Instance_Map	Instances list	Dictionary mapping from exported instances in the DSVC file to instances /substitute in import netlist.
Scale_Factor	Scalar value	IRVs in the DSVC file are multiplied by the scale factor before substitution.
Subckt	Subckt name	All instances of the specified subcircuit to substitute in the import netlist.

Output Format Entries

Output Format JSON Entries (True/False)	Description
QQ_PDF	Generates QQ plot in the pdf format
QQ_WV	Generates QQ plot in the WaveView readable format WaveView Q-Q Plots
QQ_CSV	Generates QQ plot in the CSV Q-Q File format
Nominal	Performs nominal simulation and saves outputs

Output Format JSON Entries (True/False)	Description
Detailed	Creates the *.ava.details.json File containing right/left tail samples data
Sensitivity	Performs sensitivity analysis and creates the <i>Variable screened by importance</i> section in the The *.ava.report File
Critical_Sweep_Ext	Generates critical point external sampling file *.ava.critical File
Critical_Sweep_Waveform	Simulation of critical points to obtain the waveforms *.ava.critical File
Measure_File	Saves measure file
Sample_File	Saves IRV sample file

Other Entries

JSON Entries	Arguments	Description
Active_Application	High_Sigma_Monte_Carlo Sigma_Amplification Parametric_Yield_Estimation Monte_Carlo DSVC_Import Utility	Forms the base for all the remaining json entries
Measure_Specification_File	.csv file	Points to the csv file that contains information on measures such as enable/disable, left/right specification and so on
DSVC_Export	List of instances in the json format	List of selected instances from design tree. The IRVs of these are exported to the <code>out.ava.dsvc.json</code> file
DSVC_Convert_Hier_Delim	Characters like / or:	Replaces specified delimiter with default delimiter for instance names in the DSVC file
Vary/ Do_Not_Vary	List of instances in the json format; contains sub-entries instances, models, and subckts	List of selected instances to be included/excluded from variation definitions

JSON Entries	Arguments	Description
Sensitivity_Method	Classic/Fast	Choose to use the ML-based fast sensitivity analysis for creating sensitivity report
Sensitivity_Screening_Method	Pearson/spearman	Choose screening method
Sensitivity_Output_CSV	True/False	Creates the <code>csv</code> format file for sensitivity report
Sensitivity_Detailed_File	True/False	Creates <code>out.ava.irv_corr.csv</code> with detailed information of the sensitivity report
Sensitivity_Hide_IRV_Level	True/False	Hides parameter level sensitivity report
HM_Monitor_Interval	Numerical	Time interval between updating the health monitor <code>.ava.hm</code> file
Sensitivity_Composite_Device	.json file	Points to the <code>.json</code> file generated by simulator; contains list of all composite devices used in sensitivity analysis
enable_EVD	True	Invokes Extreme Value Distribution methods under HSMC to compute quantiles at very high sigmas.
EVD_sigmas		Lists sigma values (must be greater than 6) ;

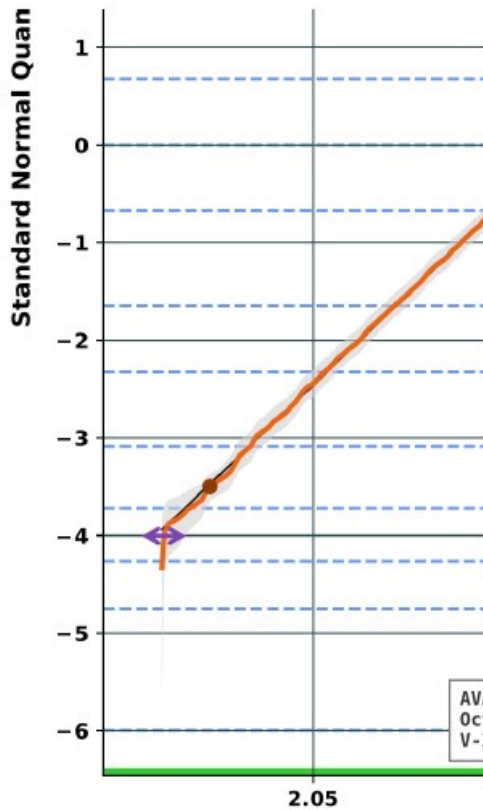
</> Marks in AVA Yield Output Reports

</> Marks in AVA Yield Output

The </> marks indicate that the confidence interval around the quantile extends beyond the limits covered by the QQ plot.

In the following example, `output.qq.csv` has the following record for Q = 4 sigma:

```
tout__q__hl LCL( tout__q__hl UCL( tout__q__hl
2.0149 <2.0146 2.0186
```



The purple arrow on the figure shows the confidence interval. The upper bound of the CI can be estimated. The lower bound extends outside the covered range. It is reported as <2.0146

</> Marks in Yield Value

In some cases, the yield value for some measures is reported with </> signs as in the following example:

Failure Report in Percentage

	Left_Spec	CI_LL%	P_fl%	CI_LU%	Right_Spec	CI_RL%	P_fr%	CI_RU%	CI_ML%
icellmin	-2e-05	<2.67918e-10	<1.05822e-08	<3.90364e-08	-4e-06	2.34566e-07	4.02123e-07	5.39688e-07	<2.67918e-10
Overall	.	.	.	.	.	.	.	.	<2.67918e-10

Yield Report in terms of the Standard Normal Quantile

	Left_Spec	CI_LL	P_yL	CI_LU	Right_Spec	CI_RL	P_yR	CI_RU	CI_ML	P_yM	CI_MU	CI_AL	P_yA	CI_AU
icellmin	-2e-05	>6.149	>6.353	>6.896	-4e-06	5.718	5.768	5.826	>6.149	>6.353	>6.896	5.718	5.768	5.826
Overall	.	.	.	.	.	.	.	.	>6.149	>6.353	>6.896	5.718	5.768	5.826

P_fl is reported as less than 1e-8 % because the right spec in this case is dominant with a failure probability of 4e-7%. The right spec has a failure probability > 40X that of the left spec. when computing yield AVA focuses on the dominant spec.

Possible Scenarios for Yield Reporting

The following are the possible scenarios of the Yield reporting

- No Clear Dominance: More than one spec that all have close probability of failure
- Clear Dominance: The probability of failures for different specs are not close. There exists a dominant spec(or specs)
- All Spec have >6.5 sigma Yield.

No Clear Dominance

When more than one spec that all have close probability of failure is a case of no clear dominance of yield reporting.

An example case:

Failure Report in Percentage

	Left_Spec	CI_LL%	P_fL%	CI_LU%	Right_Spec	CI_RL%	P_fr%	CI_RU%	CI_ML%	
v1	5.5	0.000182462	0.000497195	0.000966904	14.5	9.03127e-05	0.000331463	0.000726506	<2.09798e-06	<
v2	5.5	0.000233215	0.000580061	0.00108218	14.5	0.000134532	0.000414329	0.000848676	<2.09798e-06	<
Overall	.	.	.	.	.	.	.	.	<2.09798e-06	<

Yield Report in terms of the Standard Normal Quantile

	Left_Spec	CI_LL	P_yL	CI_LU	Right_Spec	CI_RL	P_yR	CI_RU	CI_ML	P_yM	CI_MU	CI_AL	P_yA	CI_AU
v1	5.5	4.272	4.418	4.630	14.5	4.336	4.505	4.774	>4.522	>4.791	>5.482	4.187	4.307	4.467
v2	5.5	4.247	4.385	4.579	14.5	4.301	4.458	4.693	>4.522	>4.791	>5.482	4.154	4.266	4.411
Overall	.	.	.	.	.	.	.	.	>4.522	>4.791	>5.482	4.041	4.129	4.235

In this example:

- All specs have yield around 4 sigma
- No dominant spec
- No </> in yield
- P_yM reports the yield based on measurement failure (simulation failure as opposed to spec failure). P_yM > 4.791 indicates that no measurement failures were encountered up to 4.791 sigma

Clear Dominance

If the probability of failures for different specs are not close, a dominant spec(or specs) exist.

An example case:

Chapter 12: Appendix

</> Marks in AVA Yield Output Reports

Failure Report in Percentage

	Left_Spec	CI_LL%	P_fL%	CI_LU%	Right_Spec	CI_RL%	P_fR%	CI_RU%	CI_ML%
icellmin	-2e-05	<2.67918e-10	<1.05822e-08	<3.90364e-08	-4e-06	2.34566e-07	4.02123e-07	5.39688e-07	<2.67918e-10
Overall	.	.	.	.	.	.	.	.	<2.67918e-10

Yield Report in terms of the Standard Normal Quantile

	Left_Spec	CI_LL	P_yL	CI_LU	Right_Spec	CI_RL	P_yR	CI_RU	CI_ML	P_yM	CI_MU	CI_AL	P_yA	CI_AU
icellmin	-2e-05	>6.149	>6.353	>6.896	-4e-06	5.718	5.768	5.826	>6.149	>6.353	>6.896	5.718	5.768	5.826
Overall	.	.	.	.	.	.	.	.	>6.149	>6.353	>6.896	5.718	5.768	5.826

The right spec is dominant in this case, AVA focuses on the sigma range around the dominant spec ~ 5.7 sigma. The right spec has a failure probability > 40X that of the left spec. The left spec has a negligible impact on the overall yield.

All Spec have > 6.5 sigma Yield

The range of operation for PYE is up to 6.5 sigma. If the yield for all specs is > 6.5 sigma. AVA runs up to 6.5 sigma and reports yield values >6.5 sigma for all specs. There is no dominant spec in this case.

Command Reference (Alphabetical)

This section lists the Design Robustness commands:

- [active_device_flow](#)
- [check_tol_sign](#)
- [cck_soa_val_ib](#)
- [cck_soa_val_id](#)
- [cck_soa_val_ig](#)
- [cck_soa_val_is](#)
- [database](#)
- [delta_ids_file](#)
- [delta_ids_pct](#)
- [delta_vth_file](#)
- [delta_vth_val](#)
- [device](#)
- [device_model](#)
- [disable_gate_cap](#)
- [dp_config](#)
- [dp_num](#)
- [dr_setup](#)
- [dr_tag](#)
- [dra](#)
- [drp](#)
- [drp_stage](#)
- [enable](#)
- [enable_lpe](#)
- [exclude_device_filter](#)
- [ids_scale](#)

- [idsat_scale](#)
- [include_device_filter](#)
- [ioff_scale](#)
- [load_lpe_file](#)
- [mc_index](#)
- [mc_sweep](#)
- [meas](#)
- [meas_list_file](#)
- [outdir](#)
- [outprefix](#)
- [search_method](#)
- [set_current_tol_abs](#)
- [set_current_tol_rel](#)
- [set_hier_delimiter](#)
- [set_timing_tol_abs](#)
- [set_timing_tol_rel](#)
- [set_voltage_tol_abs](#)
- [set_voltage_tol_rel](#)
- [sim_cmdline](#)
- [sim_setup](#)
- [stage](#)
- [stress_test](#)
- [temperature](#)
- [top_region](#)
- [meas_type](#)
- [type](#)
- [variation_feature](#)

active_device_flow

Contains all the commands relating to enabling active device flow. Active device flow is only supported for the Design Robust Analyzer (DRA).

Syntax

```
"active_device_flow" : [ {  
    command1,  
    command2,  
    ...  
    commandN  
} ]
```

Description

This is a subcommand of the [dr_setup](#) command. It is set up to be an array which can contain many objects separated by curly braces (`{}`). This is a high-level command to contain all the commands to enable active device flow and specify all the setting to control the active devices.

The Design Robustness tool runs a nominal simulation and uses Dynamic Circuit Check (CCK) to identify the active devices in the circuit. Only the identified active device is applied with variation for simulation.

This command supports the following subcommands:

- [cck_soa_val_ib](#)
- [cck_soa_val_id](#)
- [cck_soa_val_ig](#)
- [cck_soa_val_is](#)
- [enable](#)

check_tol_sign

Specifies the direction of error tolerance for a DRA search. The error tolerance is defined as the difference between variation result to nominal result.

Syntax

```
check_tol_sign : "absolute" | "positive" | "negative"
```

Description

There are three options available, positive, negative, and absolute. Positive reports only positive tolerance for Error/Rel_Error and negative reports only negative tolerance for

Error/Rel_Error. Absolute reports both positive and negative tolerance for Error/Rel_Error and list in absolute values. The default is absolute.

Examples

```
"meas" :  
{ "meas_list_file" : "key_meas_file",  
  "check_tol_sign" : "positive" }
```

cck_soa_val_ib

Specifies the current value of terminal bulk (I_b) of a MOSFET.

Syntax

```
"cck_soa_val_ib" : ib_value
```

Description

This is a subcommand of the [active_device_flow](#) command. It specifies a value of I_b . If the simulated value from the nominal run is greater than *ib_value*, the device is active. The default is 1e-6 amperes.

cck_soa_val_id

Specifies the current value of terminal drain (I_d) of a MOSFET.

Syntax

```
"cck_soa_val_id" : id_value
```

Description

This is a subcommand of the [active_device_flow](#) command. It specifies a value of I_d . If the simulated value from the nominal run is greater than *id_value*, the device is active. The default is 1e-6 amperes.

cck_soa_val_ig

Specifies the current value of terminal gate (I_g) of a MOSFET.

Syntax

```
"cck_soa_val_ig" : ig_value
```

Description

This is a subcommand of the [active_device_flow](#) command. It specifies a value of I_g . If the simulated value from the nominal run is greater than *ig_value*, the device is active. The default is 1e-6 amperes.

cck_soa_val_is

Specifies the current value of terminal source (I_s) of a MOSFET.

Syntax

```
"cck_soa_val_is" : is_value
```

Description

This is a subcommand of the [active_device_flow](#) command. It specifies a value of I_s . If the simulated value from the nominal run is greater than *is_value*, the device is active. The default is 1e-6 amperes.

database

Specifies the name of the database to load for generating graphs.

Syntax

```
"database" : "name_of_database"
```

Description

This is a subcommand of the [drp](#) command. When you specify `"drp_stage" : "graph"`, it enables the Design Robustness tool to generate graphs from a preexisting database file for analysis. You must use this command to provide a database file to generate graphs.

Examples

The following example enables the Design Robustness tool to generate histogram and heatmap graphs for analysis using a preexisting `dr_default.db` database file:

```
"drp" : {  
  "drp_stage"      : "graph",  
  "database"       : "dr_default.db"  
}
```

delta_ids_file

Provides a file to manually override the percentage change in I_{ds} for each device for simulation.

Syntax

```
"delta_ids_file" : "filename"
```

Description

This is a subcommand of the `dra` command. The file contains the list of device names and the `delta_ids` percentage to vary on the specified devices.

- If a device name is specified in the file, it overrides the `delta_ids` percentage specified in the `delta_ids_pct` command.
- If it is not specified in the file, it uses the percentage of the `delta_ids_pct` command in the configuration file.

The following is the format of the file:

```
device_name1, delta_ids_in_percentage_1  
device_name2, delta_ids_in_percentage_2  
...  
device_nameN, delta_ids_in_percentage_N
```

Examples

The following example reads in the `delta_ids_file.txt` file which contains the device names and the `delta_ids` percentages to vary. For devices not specified in the file, they use 20% as the `delta_ids` percentage.

```
"dra" : {  
  "delta_ids_pct" : -20,  
  "delta_ids_file" : "delta_ids_file.txt",  
  ...  
}
```

The content of the file is as follows:

```
xn0.main,-9  
xn1.main,-30  
xn10.main,-7  
xn11.main,-7  
xp5.main,-8  
xp6.main,-7  
xp7.main,-4  
xp8.main,-22  
xp9.main,-17
```

delta_ids_pct

Specifies the percentage change in `ids` value for variation.

Syntax

```
"delta_ids_pct" : "delta_ids_pct"
```

Description

This is a subcommand of the `dra` command. It varies the `Ids` by the specified percentage, as follows:

- NMOS (`Ids` has positive value): $Ids + (Ids * delta_ids_pct)$
- PMOS (`Ids` has negative value): $Ids - (Ids * delta_ids_pct)$

The default percentage is -20%.

delta_vth_file

Provides a file to manually override the change in `vth` value for each device for simulation.

Syntax

```
"delta_vth_file" : "filename"
```

Description

This is a subcommand of the `dra` command. The file contains the list of device names and the change in `vth` values to vary on the specified devices.

- If a device name is specified in the file, it overrides the delta `vth` value specified in the `delta_vth_val` command of the specified device.
- If it is not specified in the file, it uses the value of the `delta_vth_val` command in the configuration file.

The following is the format of the file:

```
device_name1, delta_vth_in_volts_1  
device_name2, delta_vth_in_volts_2  
...  
device_nameN, delta_vth_in_volts_N
```

Examples

The following example reads in the `delta_vth_file.txt` file which contains the device name and the delta `vth` value to vary. For devices not specified in the file, they use 0.07 as the delta `vth` value.

```
"dra" : {  
    "delta_vth_val" : 0.07,  
    "delta_vth_file" : "delta_vth_file.txt",  
    ...  
}
```

The content of the file is as follows:

```
xn0.main,-0.09
xn1.main,-0.03
xn10.main,-0.47
xn11.main,-0.67
xp5.main,-0.08
xp6.main,-0.07
xp7.main,-0.04
xp8.main,-0.072
xp9.main,-0.07
```

delta_vth_val

Specifies the change in `vth` value for variation in volts.

Syntax

```
"delta_vth_val" : "delta_vth_value"
```

Description

This is a subcommand of the `dra` command. It varies the `vth` device model data by the specified value, as follows:

- NMOS (`vth` has positive value): $V_{th} + \text{delta_vth_value}$
- PMOS (`vth` has negative value): $V_{th} - \text{delta_vth_value}$

The default value is 0.05 volts.

device

Contains all the commands relating to scoping. It enables you to apply variation to the circuit scope.

Syntax

```
"device" : {
    command1,
    command2,
    ...
    commandN
}
```

Description

This is a subcommand of the `dr_setup` command. This is a high-level command to contain all the commands to set the circuit scoping where it applies variation.

This command supports the following subcommands:

- `exclude_device_filter`
- `include_device_filter`
- `type`

device_model

Lists the device model to apply scaling of the specified model data for a stress test.

Syntax

```
"device_model" : ["model_name", "model_name", ...]
```

Description

This is a subcommand of the `stress_test` command. The command is set up to be an array. You can apply the list of device models to the scaling of specified model data within an object.

If this command is not specified, it applies to all device models. Wildcard is not supported.

If multiple device models are applied to the same model type for scaling, the last value of the scaling is applied.

Note:

If `ids_scale` is applied to the same device model as `idsat_scale` or `ioff_scale`, an error is printed.

Examples

The following command performs a stress test to run a nominal simulation and a variation simulation with setting `ids_scale` to 0.9 for `pch` and `nch` models, and `idsat_scale` and `ioff_scale` to 1.1 for `psvtlp` and `nsvtlp` models. All other models remain unchanged.

```
"stress_test" : {  
  "set_scale" : [ {  
    "ids_scale" : 0.9, "device_model" : ["pch", "nch"]  
  },  
  {  
    "idsat_scale" : 1.1,  
    "ioff_scale" : 1.1,  
    "device_model" : ["psvtlp", "nsvtlp"]  
  }  
}]
```

disable_gate_cap

Enables the Design Robustness tool to calculate the capacitance on the gate in Design Robustness Profiler (DRP) simulation.

Syntax

```
"disable_gate_cap" : true|false
```

Description

This is a subcommand of the [drp](#) command. The default is disable. It must be used with "disable_gate_cap" : false subcommand to calculate the capacitance on the gate specified in the file and database.

Examples

The following command enables the Design Robustness tool to process The content of the output file in CSV format:

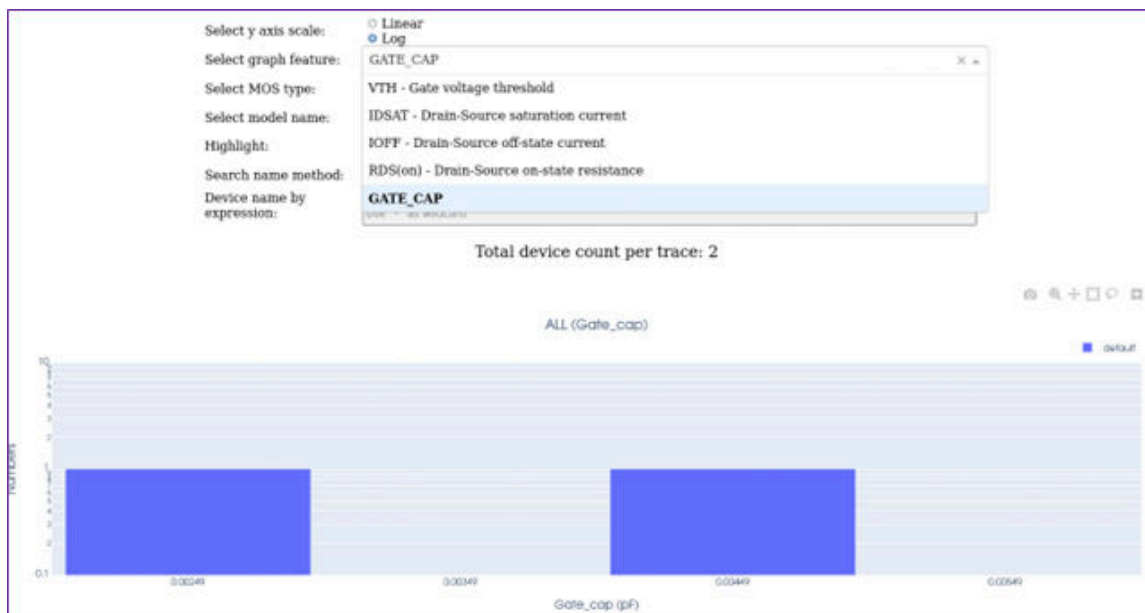
```
"disable_gate_cap" : false
```

The CSV format is as follows:

```
dev_no, dev_name, model_name, nfin, vth, idsat, ioff, ron, gate_cap  
1, mmmn5, nch_svt_mac, 4, 0.409297, 0.000114958, 9.91865e-10, 9951.43,  
2.18115e-15,  
2, mmmn4, nch_svt_mac, 8, 0.405116, 0.000223533, 1.97579e-09, 5117.82,  
4.34069e-15,
```

The graphic feature can be found in the Select graph feature drop-down list:

Figure 120 Graph feature of disable_gate_cap



dp_config

Specifies the hosts information file for distributed parallel processing capability.

Syntax

```
"dp_config" : "hostsfile"
```

Description

Synopsys supports Common Distributed Library (CDPL) to enable distributed processing. CDPL supports the following protocols for launching worker processes on remote nodes/machines:

- RSH – Remote Shell
- SSH – Secure Shell
- SH – Shell process on the local host
- SGE/UGE – Originally Sun Grid Engine, lately Univa Grid Engine
- LSF – Load Sharing Facility, from Platform Computing
- PBS – Portable Batch System

- RTDA – Runtime Design Automation Network Computer
- Netbatch – Intel Netbatch Compute Farm
- SLURM – Simple Linux Utility for Resource Management
- CUSTOM – Customer farm type unknown to CDPL

To enable CDPL, you must provide a hosts information file ("*hostsfile*") to specify machine resources for distributed processing run. If it is not specified, the default is to run the simulation in the local host.

The following are some examples of specifying the hosts information file:

- RSH – Remote Shell

```
1| teralab-x16 | 8 | /remote/users/tmp | RSH | rsh
```

- SSH – Secure Shell

```
1| cdpl_lab-x16 | 8 | /remote/users/tmp | SSH | ssh
```

- SH – Shell process on the local host

```
1| localhost | 8 | /remote/users/tmp | SH | sh
```

- SGE/UGE – Originally Sun Grid Engine, lately Univa Grid Engine

```
1| | 3 | /remote/users/tmp | SGE | qsub -P bnormal -l arch=glinux
```

- LSF – Load Sharing Facility, from Platform Computing

```
1| | -1 | /remote/users/tmp | LSF | bsub -P bnormal
```

- PBS – Portable Batch System

```
1| | 16 | /remote/users/tmp | PBS | qsub -q bnormal -l arch=glinux
```

- RTDA – Runtime Design Automation Network Computer

```
1| | 10 | /remote/users/tmp | RTDA | nc run
```

- Netbatch – Intel Netbatch Compute Farm

```
1| | 8 | /remote/users/tmp | NB | nbjob run
```

- SLURM – Simple Linux Utility for Resource Management

```
1| | -1 | /remote/users/tmp | SLURM | sbatch
```

- CUSTOM – Customer farm type unknown to CDPL

```
1| teralab-x16 | 8 | /remote/users/tmp | RSH  
| /usr/local/bin/custom_cmd
```

Examples

The following is an example of the hosts information file on SGE:

```
# HOSTS INFORMATION FILE
# Each valid host definition occupies an entire line.
# Line with '#' as the first character is a comment line.
# Format of each line:
# flag|hostname|slots|tmpDir|protocol|command
#
# DPCONFIG FILE FORMAT
# FLAG|hostname|slots|tmpdir|protocol|command

# SGE dpconfig file
1||-1|/tmp|SGE|qsub -P bnormal
```

dp_num

Sets the maximum number of parallel simulations that are run.

Syntax

```
"dp_num" : #sims
```

Description

If it is not specified, the default is 4 which allows four parallel simulations to run at one time.

dr_setup

Contains all the design robustness related setup commands.

Syntax

```
"dr_setup" : [ {
    command1,
    command2,
    ...
    commandN
},
{
    command1,
    command2,
    ...
    commandN
},
{
    ...
} ]
```

Description

This is a high-level command to contain all other design robustness related subcommands. It is set up to be an array which can contain many objects separated by curly braces (`{}`). Each object is a group of commands to set up design robustness simulation. Each group of commands is a configuration and is represented with a tag name specified by a `dr_tag` command. You can use the `-t` command-line option to call the specified tag name to select the configuration for design robustness simulation.

dr_tag

Specifies the tag name for the group of the commands.

Syntax

```
"dr_tag" : "tagname"
```

Description

This is a subcommand of the `dr_setup` command. It specifies the tag name of a group of commands inside the curly braces (`{}`). The *tagname* can be specified with the `-t` command line option to select the settings in the group for design robustness simulation. This is a mandatory command.

Examples

The following example creates two sets of configurations with names `default` and `ids_30`. Each configuration contains a set of commands to run design robustness simulation.

```
"dr_setup" : [ {  
    "dr_tag" : "default",  
    ...  
},  
{  
    "dr_tag" : "ids_30m",  
    ...  
} ]
```

dra

Contains all the commands related to the Design Robustness Analyzer (DRA) application.

Syntax

```
"dra" : {  
    command1,  
    command2,  
    ...  
}
```

```
        commandN
    }
```

Description

This is a subcommand of the [dr_setup](#) command. It contains all other DRA-related subcommands. When DRA is enabled, the Design Robustness tool uses the following DRA subcommands for design robustness simulation:

- [delta_ids_file](#)
- [delta_ids_pct](#)
- [delta_vth_file](#)
- [delta_vth_val](#)
- [search_method](#)
- [variation_feature](#)

drp

Contains all the commands related to the Design Robustness Profiler (DRP) application.

Syntax

```
"drp" : {
    command1,
    command2,
    ...
    commandN
}
```

Description

This is a subcommand of the [dr_setup](#) command. It contains all other DRP-related subcommands. When the DRP application is enabled with the [stage](#) command, the Design Robustness tool uses the following group of DRP commands for design robustness simulation:

- [database](#)
- [drp_stage](#)
- [enable_lpe](#)
- [load_lpe_file](#)
- [mc_index](#)
- [mc_sweep](#)

- `temperature`
- `disable_gate_cap`

drp_stage

Selects the stages to execute in Design Robustness Profiler (DRP) simulation.

Syntax

```
"drp_stage" : "stage"
```

Description

This is a subcommand of the `drp` command. The DRP application goes through many stages in a simulation, including Data Generation and Graph Generation. You can also use this command to compare two DRP databases to analyze the differences in the results.

This command selects the type of stages to perform in DRP simulation, as follows:

<i>stage</i>	Description
<code>data_generation</code>	Enables only Data Generation. A database file with device model data is generated. You can generate the graphs using the database file. See Generating Graphs in DRP Using a Database File .
<code>graph</code>	Enables only Graph Generation. A database file must be specified with the <code>database</code> command. You can also enable graph generation using command line options. See Generating Graphs in DRP Using a Database File .
<code>full</code>	(Default) Enables all stages from Data Generation to Graph Generation.

enable

Enables or disables the active device flow.

Syntax

```
"enable" : true | false
```

Description

This is a subcommand of the `active_device_flow` command. By default, it is set to `true` to enable active device flow for DRA. You can set it to `false` to disable active device flow.

enable_lpe

Enables the Design Robustness tool to process a list of standard layout parameters in Design Robustness Profiler (DRP) simulation.

Syntax

```
"enable_lpe" : true|false
```

Description

This is a subcommand of the `drp` command. By default, the Design Robustness tool does not process the layout parameters (`false`). If you want the tool to process layout parameters, you must set it to `true`. The standard layout parameters are L, W, AD, AS, PD, PS, NRD, NRS, SA, and SB.

When this command is used with `"drp_stage" : "full"`, the Design Robustness tool generates a report suffixed `*.lpe.csv.gz`.

Note:

You can override the list of reported layout parameters by using the `load_lpe_file` subcommand.

exclude_device_filter

Provides a file to list all the devices names to exclude from variation for simulation.

Syntax

```
"exclude_device_filter" : "exclude_filename"
```

Description

This is a subcommand of the `device` command. This command provides a file to specify a list of device names to exclude from applying variation. The asterisk (*) wildcard character is supported to specify the device name.

The file format is as follows:

```
device_name1  
device_name2  
...  
device_nameN
```

Examples

This command reads in the `exclude_file.txt` file which contains the device names to exclude from applying variation. The variation is excluded from device `xn0.main` and all devices starting with name `xp` and ending with name `.main`.

```
"device" : {  
    "exclude_device_filter" : "exclude_file.txt",  
    ...  
}
```

The content of the file is as follows:

```
xn0.main  
xp*.main
```

ids_scale

Scales the value of drain-source current (I_{ds}) of a MOSFET.

Syntax

```
"ids_scale" : scale_factor
```

Description

This is a subcommand of the [stress_test](#) command. When this command is set, it scales the drain-source current of a MOSFET in all operating regions.

Note:

`ids_scale` cannot be used with [idsat_scale](#) and [ioff_scale](#), or an error message is printed.

idsat_scale

Scales the value of drain current (I_d) and drain-source current (I_{ds}) of a MOSFET in the linear and saturation region.

Syntax

```
"idsat_scale" : scale_factor
```

Description

This is a subcommand of the [stress_test](#) command. When this command is set, it scales the drain current of a MOSFET in the saturation region.

Note:

`idsat_scale` can be used with [ioff_scale](#) but cannot be used with [ids_scale](#).

include_device_filter

Provides a file to list all the device names to apply the variation for simulation.

Syntax

```
"include_device_filter" : "include_filename"
```

Description

This is a subcommand of the [device](#) command. This command provides a file to specify a list of device names to apply the variation to. The asterisk (*) wildcard character is supported to specify the device name.

The file format is as follows:

```
device_name1  
device_name2  
...  
device_nameN
```

Examples

The following example reads in the `include_file.txt` file which contains the device names to apply variation. The variation applies to device `xn0.main` and all devices starting with name `xp` and ending with name `.main`.

```
"device" : {  
    "include_device_filter" : "include_file.txt",  
    ...  
}
```

The content of the file is as follows:

```
xn0.main  
xp*.main
```

ioff_scale

Scales the value of drain current (I_d) and drain-source current (I_{ds}) of a MOSFET in the cutoff region.

Syntax

```
"ioff_scale" : scale_factor
```

Description

This is a subcommand of the [stress_test](#) command. When this command is set, it scales the drain current of a MOSFET in the cutoff region.

Note:

`ioff_scale` can be used with [idsat_scale](#) but cannot be used with [ids_scale](#).

load_lpe_file

Enables the Design Robustness tool to load a file containing a list of layout parameters to process in Design Robustness Profiler (DRP) simulation.

Syntax

```
"load_lpe_file" : "filename"
```

Description

This is a subcommand of the [drp](#) command. It must be used with `"enable_lpe" : true` subcommand. The Design Robustness tool processes the list of layout parameters specified in the file.

Examples

The following command loads in `lpe_file.txt` which contains a list of layout parameters to process:

```
"load_lpe_file" : "lpe_file.txt"
```

The content of the `lpe_file.txt` file is as follows:

```
L w sa sb  
nrd  
nf lssource lgdrain  
lgsource sc nrs
```

The content of the output file in CSV format is as follows:

```
Dev_name, L, w, sa, sb, nrd, nf, lssource, lgdrain, lgsource, sc, nrs,  
xitop.xidummy_bank.xmm4.xm.m1, 5.4e-08, 1.215e-07, 9.9e-08, 9.9e-08, 0, 1,  
3.339e-07, 1.215e-07, 1.215e-07, 0, 0,  
xitop.xibotbanks_add0.xm.m1, 3.6e-08, 2.7e-07, 2.745e-07, 1.08e-07, 0.15, 1,  
1.35e-07, 2.7e-07, 2.7e-07, 0, 0.15,  
xitop.xibotbanks_add0.xm.m2, 3.6e-08, 2.7e-07, 2.745e-07, 1.08e-07, 0.15, 1,  
1.35e-07, 2.7e-07, 2.7e-07, 0, 0.15,  
xitop.xibotbanks_add0.xm.m3, 3.6e-08, 2.7e-07, 1.035e-07, 2.79e-07, 0.15, 1,  
1.35e-07, 2.7e-07, 2.7e-07, 0, 0.15,  
xitop.xibotbanks_add0.xm.m4, 3.6e-08, 2.7e-07, 1.035e-07, 2.79e-07, 0.15, 1,  
1.35e-07, 2.7e-07, 2.7e-07, 0, 0.15,  
xitop.xibotbanks_add0.xm.m5, 3.6e-08, 2.7e-07, 2.745e-07, 1.08e-07, 0.15, 1,  
1.35e-07, 2.7e-07, 2.7e-07, 0, 0.15,
```

For Monte-Carlo profiling, set up the Monte-Carlo statement in the input netlist. [mc_index](#) and [mc_sweep](#) are the newly added subcommands in the `drp` block of the DR configuration file. These commands can only be used if the netlist contains statements of Monte-Carlo analysis. The `"mc_index"` or `"mc_sweep"` setting in the DR's JSON configuration file ignores and overwrites the Monte-Carlo sweep setting in the netlist.

Netlist.sp:

```
.tran 1n 256n sweep monte=100
.tran 1n 256n sweep monte=list(100)
$ Make sure the sweep is added to the netlist
$ PrimeSim DR will ignore the following monte=100 or
$ monte=list(100)
```

mc_index

Specifies a Monte Carlo index to run in Design Robustness Profiler (DRP) simulation.

Syntax

```
"mc_index" : index
"mc_index": [index, ...]
```

Description

This is a subcommand of the [drp](#) command. This command can only be used if the netlist contains statements for Monte Carlo analysis. You can specify the index number of Monte Carlo to run DRP simulation only on that specified index. The tool profiles the device model data only on the specified index and generates graphs for analysis.

Note:

This command cannot be used with the [temperature](#) command.

Examples

The following example enables the PrimeSim Design Robustness tool to run DRP simulation only on index 10 of a Monte Carlo netlist:

```
"drp" : {
  "drp_stage": "data_generation",
    "mc_index" : 10,
    ...
}
```

The following example enables the PrimeSim DR tool to run DRP simulation only on indexes 34 and 45 of the Monte-Carlo netlist.

```
"drp": {
    "drp_stage": "data_generation",
    "mc_index": [34, 45]
}
```

mc_sweep

Specify the number of Monte-Carlo samples to produce.

Syntax

```
"mc_sweep": val
```

Description

This is a subcommand of the [drp](#) command. val specifies the number of Monte-Carlo samples to produce. This command can only be used if the netlist contains statements for Monte-Carlo analysis. All the Monte-Carlo samples from 1 to val will be profiled and stored in the database.

Note:

~60 Bytes per device for one Monte-Carlo sample is required in the database.

Examples

The following example enables the PrimeSim Design Robustness tool to run DRP simulation only on index 10 of a Monte Carlo netlist:

The following example enables the PrimeSim DR tool to run DRP simulation with 10000 Monte-Carlo samples.

```
"dr_setup": [
  {
    "dr_tag": "default",
    "stage": "drp",
    "drp": {
      "drp_stage": "data_generation",
      "mc_sweep": 10000
    }
  }
]
```

meas

Contains all the commands related to accuracy checking criteria for the Design Robustness Analyzer (DRA) application.

Syntax

```
"meas" : {
  command1,
  command2,
  ...
  commandN
}
```

Description

This is a subcommand of the [dr_setup](#) command. It is a high-level command to contain all the commands to set the accuracy checking criteria for the DRA application. The

accuracy checking criteria helps to determine whether the simulation meets the accuracy requirement.

The Design Robustness tool checks the results of `.MEASURE` statements and Eldo's command of `.EXTRACT` statements against the nominal run with the specified tolerances.

Note:

When the `.EXTRACT` statement is used when running in Eldo netlist format, you must provide a label (`LABEL=`) in the statement.

If the result exceeds the tolerances, the accuracy requirement is not met.

This command supports the following subcommands:

- `meas_list_file`
- `set_current_tol_abs`
- `set_current_tol_rel`
- `set_timing_tol_abs`
- `set_timing_tol_rel`
- `set_voltage_tol_abs`
- `set_voltage_tol_rel`
- `top_region`
- `meas_type`

meas_list_file

Provides a file to list all the key measurements and local tolerance values for each measurement to check for accuracy.

Syntax

```
"meas_list_file" : "filename"
```

Description

This is a subcommand of the `meas` command. This is a mandatory command. The file contains the list of key measurements to check for accuracy. The Design Robustness tool

automatically detects the type of measurement – timing, voltage, current, and others, as follows:

- For timing measurement, it checks the measurement value against the global tolerances set by the `set_timing_tol_abs` and `set_timing_tol_rel` commands.
- For voltage measurement, it checks the measurement values against the global tolerances set by the `set_voltage_tol_abs` and `set_voltage_tol_rel` commands.
- For current measurement, it checks the measurement value against the global tolerances set by the `set_current_tol_abs` and `set_current_tol_rel` commands.
- For other types of measurement, 5% relative tolerance is applied by default.

You can also specify the local absolute and relative tolerance for each measurement in the file. The file format is as follows:

```
measurement_name1[,local_tol_abs1[,local_tol_rel1]]
measurement_name2[,local_tol_abs2[,local_tol_rel2]]
...
measurement_nameN[,local_tol_absN[,local_tol_relN]]
```

Examples

The following example reads in the `key_meas_file.txt` file which contains the key measurement names to check for accuracy.

```
"meas_list_file" : "key_meas_file.txt"
```

The content of `key_meas_file.txt` file is as follows:

```
vdiff
vdiff_s2out,0.5, 4
clk2dout[20],,7
clk2dout[24],1e-5
ids_m1
ids_m2,,5
test_dummy1
test_dummy2,1,2
```

The following commands are specified in the command file:

```
"meas_list_file" : "key_meas_file.txt",
"set_timing_tol_abs" : 1e-9,
"set_timing_tol_rel" : 3,
"set_voltage_tol_abs" : 0.1,
"set_voltage_tol_rel" : 10,
"set_current_tol_abs" : 1e-6,
"set_current_tol_rel" : 15
```

[Table 8](#) shows the absolute and relative tolerances set for each measurement.

Table 8 *Absolute and Relative Tolerances*

Measurement Name	Absolute Tolerance	Relative Tolerance
vdiff	0.1v	10%
vdiff_s2out	0.5v	4%
clk2dout[20]	1e-9s	7%
clk2dout[24]	1e-5s	3%
ids_m1	1e-6A	15%
ids_m2	1e-6A	5%
test_dummy1	N/A	5%
test_dummy2	1	2%

outdir

Specifies the name of output directory to store all the intermediate files and simulation results.

Syntax

```
"outdir" : "outdirname"
```

Description

If it is not specified, the default is equivalent to the current working directory. If the `-o` command line is specified with an output directory name, it overrides the default setting.

outprefix

Specifies the prefix name used as part of the output file name and the intermediate simulation directory name. The intermediate simulation directory contains the intermediate simulation files.

Syntax

```
"outprefix" : "outprefix"
```

Description

If it is not specified, the default is set to `dr`. If the `-o` command line is specified with a prefix name, it overrides the default setting.

The output file name is `outprefix_tagName` and the intermediate simulation directory name is `outprefix_tagName_sim`. `tagName` is the name specified with the `-t` command line option (see [Figure 4](#) for details).

search_method

Specifies the method to search for sensitive devices.

Syntax

```
"search_method" : "method"
```

Description

This is a subcommand of the [dra](#) command. The Design Robustness tool partitions the circuit into many regions when searching for sensitive devices to improve performance.

method	Description
error_tol	Searches the sensitive devices based on the error tolerance. The Design Robustness tool checks the key measurement results with variation applied versus the key measurement results of the nominal run. If the difference of the two measurements is larger than specified error tolerance, it continues the search to find the sensitive devices. You can specify the global absolute error tolerance and global relative error tolerance using the following commands: • set_current_tol_abs • set_current_tol_rel • set_timing_tol_abs • set_timing_tol_rel • set_voltage_tol_abs • set_voltage_tol_rel If a measurement is not detected as timing, voltage, or current type, 5% is applied as the default relative tolerance. You can also specify the local error tolerance for each measurement in the key measurement file using the meas_list_file command.
region	(Default) Searches the sensitive devices only on the region with the largest relative error. When the Design Robustness tool checks the key measurement results with variation applied versus key measurement results of nominal runs, it only continues the search on the region with the largest relative error. The ten most sensitive devices in this region are reported. You can use the top_region command to specify the number of regions to search.

set_current_tol_abs

Specifies the global absolute tolerance value for accuracy checking of the current measurement result.

Syntax

```
"set_current_tol_abs" : value
```

Description

This is a subcommand of the [meas](#) command. The accuracy meets the requirement if the measurement result is within the specified absolute tolerances as follows:

$$|nominal_value - measurement_value| \leq value$$

where *nominal_value* is the measurement result from the nominal simulation, and *measurement_value* is the measurement result from the simulation with variation. The unit is amperes. There is no default value to this command.

The Design Robustness tool detects the type of measurement specified in the key measurement file using the [meas_list_file](#) command. This is a global tolerance that applies to all current measurements. You can specify the local tolerance value for each measurement in the key measurement file (see the [meas_list_file](#) command).

set_current_tol_rel

Specifies the global relative tolerance value for accuracy checking of the current measurement result.

Syntax

```
"set_current_tol_rel" : percentage_value
```

Description

This is a subcommand of the [meas](#) command. The accuracy meets the requirement if the measurement result is within the specified absolute tolerances as follows:

$$\left| \frac{(nominal_value - measurement_value)}{nominal_value} \right| * 100 \leq percentage_value$$

where *nominal_value* is the measurement result from the nominal simulation, and *measurement_value* is the measurement result from the simulation with variation. The unit is percentage. The default percentage value is 5.

The Design Robustness tool detects the type of measurement specified in the key measurement file using the [meas_list_file](#) command. This is a global tolerance that

applies to all current measurements. You can specify the local tolerance value for each measurement in the key measurement file (see the [meas_list_file](#) command).

set_hier_delimiter

Specifies the character used as a hierarchical delimiter in the netlist and the character used as a hierarchical delimiter in the instances during extraction.

Syntax

```
"set_hier_delimiter" : ["char1", "char2", ..., "charN"]
```

Description

If it is not specified, the default hierarchical delimiters supported by the tool are period (.) and slash (/). It includes the hierarchical delimiter used in the netlist and the hierarchical delimiter used to generate a flattened netlist during extraction. Those hierarchical delimiters are often two different characters.

set_timing_tol_abs

Specifies the global absolute tolerance value for accuracy checking of the timing measurement result.

Syntax

```
"set_timing_tol_abs" : value
```

Description

This is a subcommand of the [meas](#) command. The accuracy meets the requirement if the measurement result is within the specified absolute tolerances as follows:

$$|nominal_value - measurement_value| \leq value$$

where *nominal_value* is the measurement result from the nominal simulation, and *measurement_value* is the measurement result from the simulation with variation. The unit is seconds. There is no default value to this command.

The Design Robustness tool detects the type of measurement specified in the key measurement file using the [meas_list_file](#) command. This is a global tolerance that applies to all timing measurements. You can specify the local tolerance value for each measurement in the key measurement file (see the [meas_list_file](#) command).

set_timing_tol_rel

Specifies the global relative tolerance value for accuracy checking of the timing measurement result.

Syntax

```
"set_timing_tol_rel" : percentage_value
```

Description

This is a subcommand of the [meas](#) command. The accuracy meets the requirement if the measurement result is within the specified absolute tolerances as follows:

$$\left| \frac{(nominal_value - measurement_value)}{nominal_value} \right| * 100 \leq percentage_value$$

where *nominal_value* is the measurement result from the nominal simulation, and *measurement_value* is the measurement result from the simulation with variation. The unit is percentage. The default percentage value is 5.

The Design Robustness tool detects the type of measurement specified in the key measurement file using the [meas_list_file](#) command. This is a global tolerance that applies to all timing measurements. You can specify the local tolerance value for each measurement in the key measurement file (see the [meas_list_file](#) command).

set_voltage_tol_abs

Specifies the global absolute tolerance value for accuracy checking of the voltage measurement result.

Syntax

```
"set_voltage_tol_abs" : value
```

Description

This is a subcommand of the [meas](#) command. The accuracy meets the requirement if the measurement result is within the specified absolute tolerances as follows:

$$|nominal_value - measurement_value| \leq value$$

where *nominal_value* is the measurement result from the nominal simulation, and *measurement_value* is the measurement result from the simulation with variation. The unit is volts. There is no default value to this command.

The Design Robustness tool detects the type of measurement specified in the key measurement file using the [meas_list_file](#) command. This is a global tolerance that

applies to all voltage measurements. You can specify the local tolerance value for each measurement in the key measurement file (see the [meas_list_file](#) command).

set_voltage_tol_rel

Specifies the global relative tolerance value for accuracy checking of the voltage measurement result.

Syntax

```
"set_voltage_tol_rel" : percentage_value
```

Description

This is a subcommand of the [meas](#) command. The accuracy meets the requirement if the measurement result is within the specified absolute tolerances as follows:

$$\left| \frac{(\text{nominal_value} - \text{measurement_value})}{\text{nominal_value}} \right| * 100 \leq \text{percentage_value}$$

where *nominal_value* is the measurement result from the nominal simulation, and *measurement_value* is the measurement result from the simulation with variation. The unit is percentage. The default percentage value is 5.

The Design Robustness tool detects the type of measurement specified in the key measurement file using the [meas_list_file](#) command. This is a global tolerance that applies to all voltage measurements. You can specify the local tolerance value for each measurement in the key measurement file (see the [meas_list_file](#) command).

sim_cmdline

Specifies the command line to run transient simulation.

Syntax

```
"sim_cmdline" : "simulator_command_line"
```

Description

This is a subcommand of the [sim_setup](#) command. It must be a valid command line for the supported transient simulator to execute transient simulation.

The command line must include the top-level netlist name and the required command file and appropriate command line option. The netlist must not be an encrypted netlist. This is a mandatory command.

It supports the PrimeSim XA simulator, so you can specify the following command line options:

- `-hspice`
- `-eldo`
- `-spectre`
- `-mt`
- `-ssl2`
- `-c`
- `-wavefmt`
- `-top`

Examples

The following is an example of command line to run the PrimeSim XA simulator:

```
"sim_setup" : {  
    "sim_cmdline" : "xa top.sp -c cmd"  
}
```

sim_setup

Contains all the transient-related setup commands.

Syntax

```
"sim_setup" : {  
    command1,  
    command2,  
    ...  
    commandN  
}
```

Description

This is a high-level command (a command containing other commands) to contain all other transient-related subcommands. This is a mandatory command.

stage

Selects the type of application to execute.

Syntax

```
"stage" : "type_of_application"
```

Description

This is a subcommand of the [dr_setup](#) command. The Design Robustness tool supports many applications, including Design Robustness Analyzer (DRA) and Design Robustness Profiler (DRP).

This command selects the type of application to perform, as follows:

<i>type_of_application</i>	Description
dra	Enables the Design Robustness Analyzer application.
drp	Enables the Design Robustness Profiler application.
nominal	Runs only the nominal run.
stress_test	Enables stress test analysis.

stress_test

Contains all the commands related to enabling a stress test.

Syntax

```
"stress_test" : [ {  
    command1,  
    command2,  
    ...  
    commandN  
},  
{  
    command1,  
    command2,  
    ...  
    commandN  
},  
{  
    ...  
} ]
```

Description

This is a subcommand of the [dr_setup](#) command. It is set up to be an array which can contain many objects separated by curly braces ({}).

It is a high-level command to contain all the commands to set up a stress test analysis. It is used to determine the robustness of the design by running two simulations:

- Nominal simulation
- Simulation by scaling some model data

It can be enabled with "`stage`" : "`stress_test`". See [Stress Test Analysis Output Files](#) for the output files generated by a stress test analysis.

The Design Robustness tool uses the following subcommands for stress test in design robustness simulation:

- `device_model`
- `ids_scale`
- `idsat_scale`
- `ioff_scale`

Note:

Stress test is only supported for HSPICE model level 72.

temperature

Specifies a list of temperatures in Celsius to run in Design Robustness Profiler (DRP) simulation.

Syntax

```
"temperature" : [temp1, temp2, ..., tempN]
```

Description

This is a subcommand of the `drp` command. You can enable temperature sweep in DRP simulation. The DRP application profiles the device model data (`vth` and `Idsat`) in the specified temperatures.

Note:

This command cannot be used with the `mc_index` command.

Examples

The following example enables the Design Robustness tool to run DRP simulation and profiles the device model data in three temperatures: -40°C, 25°C, and 125°C:

```
"drp" : {  
  "temperature" : [-40, 25, 125],  
}
```

```
} ...
```

top_region

Specifies the number of regions with the largest error to search for sensitive devices.

Syntax

```
"top_region" : <value>
```

Description

This is a subcommand of the `meas` command. When `"search_method" : "region"` is specified, the Design Robustness tool searches sensitive devices only on one region with the largest error by default. You can use this command to specify the number of regions with the largest error to search for sensitive devices.

meas_type

Specifies the type of measurements and filters out irrelevant devices to search for sensitive devices effectively.

Syntax

```
"meas_type" : "all" | [{"timing"}, "current"], ["voltage"], ["notype"]]
```

Description

This is a subcommand of the `meas` command. A user can specify the type of measurements to check.

Examples

To specify the measurements of both timing and voltage measurements:

```
"meas_type": ["timing", "voltage"]
```

This configuration specifies the measurements of all types:

```
"meas_type": "all"
```

If `"meas_type"` does not exist in the configuration JSON file, the value of `"all"` is used as default.

type

Scopes the variation to the specified device type.

Syntax

"type" : "device_type"

Description

This is a subcommand of the [device](#) command. This command scopes the variation to only the specified devices, as follows:

<i>device_type</i>	Description
pmos	Scopes the variation to only the PMOS devices.
nmos	Scopes the variation to only the NMOS devices.
all	Scopes the variation to both PMOS and NMOS devices.

variation_feature

Specifies the device model data used for variation.

Syntax

"variation_feature" : "model_feature"

Description

This is a subcommand of the [dra](#) command. You can select the device model feature to apply variation, as follows:

<i>model_feature</i>	Description
vth	(Default, if not specified) Applies variation to threshold voltage (v_{th}) of a MOSFET.
ids	Applies variation to drain-source current (I_{ds}) of a MOSFET.
all	Applies variation to both v_{th} and I_{ds} .

13

References

- R.J. Hyndman and Y. Fan: *Sample quantiles in statistical packages*. American Statistician, pp. 361–365, 1996.
 - C. Clopper and S. Pearson: *The use of confidence or fiducial limits illustrated in the case of the Binomial*. Biometrika, pp. 404–413, 1934.
 - R.S. Woodruff: *Confidence intervals for medians and other position measures*. JASA, pp. 635–646, 1952.
-

AVA Customer Presentations

- (ST) S. Kumar, R. Kumar: Increase Design Robustness and Meet Design Timelines using Fast Monte Carlo Techniques on Full Memory Designs using CustomSim. SNUG India, 2018 ([paper](#), [presentation](#))
- (ST) A. Kumar: *Ensuring Design Robustness on Full Memory IP using Sigma Amplification*. Luncheon [presentation](#), SNUG SV, 2019.
- (Arm) T. Mahatdejkul: *ARM® Physical Design Yield Analysis with HSPICE High Sigma Monte Carlo*. SNUG SV, 2019 ([paper](#), [presentation](#)).
- (ST) A. Kumar: *Ensuring Design Robustness on Full Memory IP using Sigma Amplification*. Poster Paper at VLSI Conference 2020, India.
- (ST) A. Kumar: *Managing Variability in Memory Designs using HSPICE and CustomSim*. AMS SIG, India, 2020. ([presentation](#))
- (Intel) S. Bodapati, J. Fozdar, S. Marella, B. Pothula, H. Sharma: *High Sigma LVF Constraint Characterization with Fast Monte Carlo*. SNUG SV, 2020 ([paper](#), [presentation](#)).
- (GF) X. Cao , O. Jakhete: *Yield Analysis with HSPICE-AVA High Sigma Monte Carlo (HSMC)*. SNUG SV, 2020 ([presentation](#), [session recording](#)).
- (Micron) D. Kao, J. Mullin, R. Sabbah: *Variability Analysis of Sensitive CMOS Circuits using HSPICE High Sigma Monte Carlo Analysis*. SNUG SV, 2020 ([paper](#), [presentation](#), [session recording](#)).

- (ST) A. Kumar, S. Varshney: *Managing Variability in Memory Designs using HSPICE and CustomSim*. SNUG SV, 2020 ([paper](#), [presentation](#), [session recording](#)), **Best Paper Award**.
- (ST) A. Kumar, R. Shenoy, T. McKenzie: Ensuring Design Robustness on Full Memory IP Using Sigma Amplification. Design Automation Conference, July, 2020
- (SNPS) S. Kumar, J. Singh, D. Tiwari Using DSVF flow in HSPICE & XA as an Alternative to Monte-Carlo Simulations for SRAM Designs SNUG South Asia 2021.
- (ST) A. Kumar, S. Varshney, R. Shenoy Memory Yield Estimation Flow using Techniques in PrimeSim HSPICE AVA and XA SNUG South Asia 2021. Best Paper Award.
- (Micron) R. Sabbah: Using PrimeSim Continuum Sigma Amplification Monte Carlo to Perform Variation Analysis on CMOS Circuits. SNUG SV, March, 2022. (session recording)
- Synopsys) [\(361\) SRAM Margin Analysis Workflow | Synopsys – YouTube](#) March 2022.
- (Synopsys) A. Barker et. al.: Simulation Methodology for Timing Analysis and Design Optimization in Digital Superconducting Electronics. ISQED, April 2022.(Session Recording, Paper)
- (ST) A. Kumar, S. Varshney, R. Shenoy: Memory Yield Estimation Flow using High Sigma Monte Carlo. Design Automation Conference, July 2022.
- (Xilinx/AMD) H.B. Dubey, S.C. Nimmagadda, M. Anees, L. Janaswamy: Better Yield Prediction with Faster Run Time using Synopsys Advanced Variation Analysis (AVA) Solution. SNUG India, July 2022. (Presentation, session recording)
- (ST) J.C. Lafont: STMicroelectronics Memory Verification Using PrimeSim XA AVA Applications for Sigma Amplification. SNUG Munich, October 2022.
- (Intel) A. Lau: Speeding up Library Characterization without Further Accuracy Tradeoffs. SNUG SV, March 2023.
- (Synopsys) A. Barker et. Al.: Variation Aware Optimization vs Classical Margin Optimization of Superconducting Electronics. IEEE Trans. on Applied Superconductivity, August 2023 (Paper)

14

AVA Revision History

This appendix lists a summary of changes for PrimeSim Design Robustness .

AVA Change History

Release	Revisions
S-2021.09	• Addition of Monte Carlo and Postprocessing capabilities • Addition of the <code>.ava.sens.csv</code> and <code>.ava.irv_corr.csv</code> files • Addition of Fast Sensitivity method
R-2020.12-SP1	Changes to file name extensions and AVA support for PrimeSim XA
R-2020.12	AVA Output File Extension Updates. For more information, see AVA Output File Extension Updates
Q-2020.03-SP1	• Support for selective variation at model, subcircuit, and instance levels. • Support for one-sided HSMC flow. • The <code>.amp_mpp0</code> file output of Sigma Amplification renamed to <code>.pmpp0</code>.
Q-2020.03-2	• Support for the WaveView file (<code>.ava.qq.wv_plt</code>) and the CSV file (<code>.qq.csv</code>) outputs in HSMC. • The WaveView-readable <code>.json_gz</code> file renamed to <code>.wv_plt</code>.
Q-2020.03	• Support for Sigma Amplification in PrimeSim Design Robustness AVA. • Sigma values added to the quantile table in the <code>.ava.report</code> file.

AVA Output File Extension Updates

Table 9 AVA Output File Extension Updates

S.No	Extensions in 2020.03-SP1	Extensions in 2020.12	Description
1	<code>*.lis/*.log/*.mt0/*.tr0/*.mc0/...</code>	no changes	Any simulator generated output

Table 9 AVA Output File Extension Updates (Continued)

S.No	Extensions in 2020.03-SP1	Extensions in 2020.12	Description
2	*.plog	*.ava.log	AVA log file
3	*.phm	*.ava.hm	AVA health monitor file
4	*.pmpp0	*.ava.report	AVA report output file
5	*.ptail0.json	*.ava.details.json	AVA detailed output file
6	*.ava.qq.csv	*.ava.qq.csv	CSV format QQ plot output File name is agnostic to the sweep type.
7	*.ava.qq.pdf	*.ava.qq.pdf	PDF format QQ plot output File name is agnostic to the sweep type.
8	*.ava.qq.wv_plt	*.ava.qq.wv_plt	WaveView format QQ plot output file name is agnostic to the sweep type.
9	*.DSVC.json	*.ava.dsvc.json	Design Specific Variation Corners file

15

Design Insights Command Reference

This chapter details the Design Insights commands and preferences.

This chapter provides the list of Design Insights commands and preferences and detailed descriptions of the individual commands.

The commands in the Design Insights tool are grouped into the following categories:

- [di::closeResultsCellView](#)
- [di::createResultsCellView](#)
- [di::createParasiticView](#)
- [di::getCapacitors](#)
- [di::getCapAttr](#)
- [di::getCorners](#)
- [di::getElmoreDelayResult](#)
- [di::getElmoreDelayResultAttr](#)
- [di::getFanoutResult](#)
- [di::getFanoutResultAttr](#)
- [di::getInstAttr](#)
- [di::getInsts](#)
- [di::getLayers](#)
- [di::getModels](#)
- [di::getNetAttr](#)
- [di::getNets](#)
- [di::getNodeAttr](#)
- [di::getResAttr](#)

- [di::getResistanceContributionAttr](#)
- [di::getResistanceResult](#)
- [di::getResistanceResultAttr](#)
- [di::getResistors](#)
- [di::getTermAttr](#)
- [di::getTerms](#)
- [di::getViewType](#)
- [di::listAttrs](#)
- [di::loadResultsCellView](#)
- [di::runElmoreDelayAnalysis](#)
- [di::runFanoutAnalysis](#)
- [di::runResistanceAnalysis](#)
- [Preferences](#)

di::closeResultsCellView

Summary: Close the currently open design insights view.

Syntax: `di::closeResultsCellView`

Returns: <none>

Licenses: None.

di::createResultsCellView

Summary: Create design insights view by `rc_preprocessor` with `-path`.

Syntax: `di::createResultCellView[-selectNet <string> | -skipNet <string>]
-lcv<string> [-modelLibs <string>] [-modelSyntax <string>] [-overwrite
<bool>]-paths <string> -type <enum> [-useITFLayerName <bool>] [-vbias
<float>][-vbiasMappingFile <string>]`

Arguments:

`-netID` : Specifies the net id of the capacitors.

`-netName` : Specifies the net name of the capacitors.

-lcv: Target LCV name to create

-modelLibs: Specified the list of the lists to specify the corner, model libs and model sections.

-modelSyntax: Specifies the model syntax type. hspice, eldo, and spectre supported.

-overwrite: Specifies if overwrite parasitic view or not

-paths: Specifies the file paths rc_preprocessorread

-selectNet: Specifies the select net pattern

-skipNet: Specifies the skip net pattern

-type: Specifies the format the rc_preprocessorread. gpd and dspfformats supported. ({ dspf | gpd })

-useITFLayerName: Specify if the layer use ITF layer or not

-vbias: Specifies the vbias number

-vbiasMappingFile: Specified the list of the lists to specify the corner, model libs and model sections.

Returns:<xtJob>: Return the submitted job object

Licenses: None

```
di::createResultsCellView -lcv test2/test3/parasitic -paths p22.gpd -type  
gpd -useITFLayerName true -modelLibs  
{reference40_models.lib TT typical_25}{reference40_models.lib SS  
typical_125}{reference40_models.lib TT typical_M40}} -overwrite 1
```

di::createParasiticView

Summary: Create parasitic view by rc_preprocessor with -path.

Syntax: di::createParasiticView [-selectNet <string> | -skipNet <string>]
-lcv <string> [-modelLibs <string>] [-modelSyntax <string>] [-overwrite
<bool>] -paths <string> -type <enum> [-useITFLayerName <bool>] [-vbias
<float>] [-vbiasMappingFile <string>]

Arguments:

-lcv : Target LCV name to create

-modelLibs : Specified the list of the lists to specify the corner, model libs and model sections.

-modelSyntax : Specifies the model syntax type. hspice, eldo, and spectre supported.

`-overwrite` : Specifies if overwrite parasitic view or not

`-paths` : Specifies the file paths `rc_preprocessor read`

`-selectNet` : Specifies the select net pattern

`-skipNet` : Specifies the skip net pattern

`-type` : Specifies the format the `rc_preprocessor read`. `gpd` and `dspf` formats supported. `( { dspf | gpd } )`

`-useITFLayerName` : Specify if the layer use ITF layer or not

`-vbias` : Specifies the vbias number

`-vbiasMappingFile` : Specified the list of the lists to specify the corner, model libs and model sections.

Returns: `<xtJob>` : Return the submitted job object

Licenses: None

di::getCapacitors

Summary: Report the capacitors' ids from the specified net id/name.

Syntax: `di::getCapacitors [-netID <int> | -netName <string>]`

Arguments:

`-netID` : Specifies the net id of the capacitors.

`-netName` : Specifies the net name of the capacitors.

Returns: `<string>` : Return list of the capacitors' ids.

Licenses: None

di::getCapAttr

Summary: Get capacitor data by specified capacitor id and attribute name.

Syntax: `di::getCapAttr [-attribute <string>] [-corners <string>] -id <int>`

Arguments:

`-attribute` : Specifies the target attribute name.

`-corners` : Specify the list of the corner name to determine the order of the multi-corner attribute value.

-id : Specifies the target id.

Returns: <string> : Return list of the attribute data.

Licenses: None

di::getCorners

Summary: Get corners information

Syntax: di::getCorners

Arguments: Returns: <string> : corners information

Licenses: None

di::getElmoreDelayResult

Summary: Get the Elmore Delay Result for current loaded parasitic view.

Syntax: di::getElmoreDelayResult [-net <string>] -type <enum>

Arguments:

-net : Specifies the target net name.

-type : Specifies the type of the report. ({ all | instpin_to_port | others | point_to_point | pull_down | pull_up })

Returns: <string> : Return the ids of the summary.

Licenses: None

```
di::loadResultsCellView -lcv Test/aaa/parasitic
set net [di::getNets -name "FOUT"]
set job [di::runElmoreDelayAnalysis -mode automatic -nets $net -corners
"typical_M40 typical_125"]
xt::wait $job
set summary_ids [di::getElmoreDelayResult -type all]
foreach summary_id $summary_ids {
  set attrs [di::getElmoreDelayResultAttr -id $summary_id -attribute "net
elmore type bandwidth transitionTime drivers loads"]
  puts "net [lindex $attrs 0]"
  puts "elmore [lindex $attrs 1]"
  puts "type [lindex $attrs 2]"
  puts "bandwidth [lindex $attrs 3]"
  puts "transitionTime [lindex $attrs 4]"
  set drivers [lindex $attrs 5]
  set loads [lindex $attrs 6]
  puts "drivers info"
```

```
foreach driver $drivers {
  puts [di::getTermAttr -id $driver]
}
puts "loads info"
foreach load $loads {
  puts [di::getTermAttr -id $load]
}
}
```

di::getElmoreDelayResultAttr

Summary: Get the Elmore Delay Result for current loaded parasitic view.

Syntax: `di::getElmoreDelayResultAttr [-attribute <string>] [-corners <string>] -id <int>`

Arguments:

`-attribute` : Specifies the target attribute name.

`-corners` : Specify the list of the corner name to determine the order of the multi-corner attribute value.

`-id` : Specifies the target id.

Returns: <string> : Return list of the attribute data.

Licenses: None

di::getFanoutResult

Summary: Get the Fanout Result for current loaded parasitic view.

Syntax: `di::getFanoutResult [-net <string>] -type <enum>`

Arguments:

`-net` : Specifies the target net name.

`-type` : Specifies the type of the report. ({ all | instpin_to_port | others | point_to_point | pull_down | pull_up })

Returns: <string> : Return the ids of the summary.

Licenses: None

```
di::loadResultsCellView -lcv Test/aaa/parasitic
set net [di::getNets -name "FOUT"]
set job [di::runFanoutAnalysis -mode automatic -nets $net]
xt::wait $job
```

```
set summary_ids [di::getFanoutResult -type all]
foreach summary_id $summary_ids {
  set attrs [di::getFanoutResultAttr -id $summary_id -attribute "net
fanout type driver_c interconnect_c load_c drivers loads"]
  puts "net [lindex $attrs 0]"
  puts "fanout [lindex $attrs 1]"
  puts "type [lindex $attrs 2]"
  puts "driver_c [lindex $attrs 3]"
  puts "interconnect_c [lindex $attrs 4]"
  puts "load_c [lindex $attrs 5]"
  set drivers [lindex $attrs 6]
  set loads [lindex $attrs 7]
  puts "drivers info"
  foreach driver $drivers {
    puts [di::getTermAttr -id $driver]
  }
  puts "loads info"
  foreach load $loads {
    puts [di::getTermAttr -id $load]
  }
}
```

di::getFanoutResultAttr

Summary: Get the Fanout Result Attribute data from current fanout result.

Syntax: di::getFanoutResultAttr [-attribute <string>] [-corners <string>]
-id <int>

Arguments:

-attribute : Specifies the target attribute name.

-corners : Specify the list of the corner name to determine the order of the multi-corner attribute value.

-id : Specifies the target id.

Returns: <string> : Return list of the attribute data.

Licenses: None

di::getInstAttr

Summary: Get instance data by specified instance id and attribute name.

Syntax: di::getInstAttr [-attribute <string>] [-corners <string>] -id
<int>

Arguments:

-attribute : Specifies the target attribute name.

-corners : Specify the list of the corner name to determine the order of the multi-corner attribute value.

-id : Specifies the target id.

Returns: <string> : Return list of the attribute data.

Licenses: None

di::getInsts

Summary: Report the list of the instances' ids from the netlist.

Category: Design Insights

Syntax: `di::getInsts [-name <string> | -netId <int> | -netName <string> | -termID <int> | -termName <string>]`

Arguments:

-name : Specifies the name of the instance.

-netId : Specifies net name of the instance.

-netName : Specifies net name of the instance.

-termID : Specifies term id of the net.

-termName : Specifies term name of the instance.

Returns: <string> : Return list of the instances' ids.

Licenses: None

```
di::loadResultsCellView -lcv di_test/di/parasitic
set instances [di::getInsts -name XI1|MM0]
foreach instance $instances {
    set instance_attribute [di::getInstAttr -id $instance -attribute
    "name model type parameters"]
    puts "instance name [lindex $instance_attribute 0] information"
    puts "-----"
    puts "model [lindex $instance_attribute 1]"
    puts "type [lindex $instance_attribute 2]"
    puts "parameters [lindex $instance_attribute 3]"
}
di::closeResultsCellView
```

di::getLayers

Summary: Get layers information

Syntax: `di::getLayers [-layerName <string>]`

Arguments:

`-layerName` : specified layer name

Returns: `<string>` : layers information

Licenses: None

di::getModels

Summary: Get models information

Syntax: `di::getModels [-modelName <string>]`

Arguments:

`-modelName` : specified model name

Returns: `<string>` : models information

Licenses: None

di::getNetAttr

Summary: Get net data by specified net id and attribute name.

Syntax: `di::getNetAttr [-attribute <string>] [-corners <string>] -id <int>`

Arguments:

`-attribute` : Specifies the target attribute name.

`-corners` : Specify the list of the corner name to determine the order of the multi-corner attribute value.

`-id` : Specifies the target id.

Returns: `<string>` : Return list of the attribute data.

Licenses: None

di::getNets

Summary: Report the list of the nets' ids from the netlist.

Syntax: `di::getNets [-name <string> | -termID <int> | -termName <string>]`

Arguments:

`-name` : Specifies the name of the net

`-termID` : Specifies term id of the net.

`-termName` : Specifies term name of the net.

Returns: `<string>` : Return list of the nets' ids.

Licenses: None

```
di::loadResultsCellView -lcv di_test/di/parasitic
set nets [di::getNets -name "ck*"]
foreach net $nets {
    set net_attribute [di::getNetAttr -id $net -attribute "name cg cc
interconnect_c term_c total_c r_count term_count max_r max_r_layer"]
    puts "Net name [lindex $net_attribute 0] information"
    puts "-----"
    puts "cg [lindex $net_attribute 1]"
    puts "cc [lindex $net_attribute 2]"
    puts "interconnect_c [lindex $net_attribute 3]"
    puts "term_c [lindex $net_attribute 4]"
    puts "total_c [lindex $net_attribute 5]"
    puts "r_count [lindex $net_attribute 6]"
    puts "term_count [lindex $net_attribute 7]"
    puts "max_r [lindex $net_attribute 8]"
    puts "max_r_layer [lindex $net_attribute 9]"
    puts "-----"
}
```

di::getNodeAttr

Summary: Get node data by specified node id and attribute name.

Syntax: `di::getNodeAttr [-attribute <string>] [-corners <string>] -id <int>`

Arguments:

`-attribute` : Specifies the target attribute name.

`-corners` : Specify the list of the corner name to determine the order of the multi-corner attribute value.

-id : Specifies the target id.

Returns: <string> : Return list of the attribute data.

Licenses: None

di::getResAttr

Summary: Get resistor data by specified resistor id and attribute name.

Syntax: di::getResAttr [-attribute <string>] [-corners <string>] -id <int>

Arguments:

-attribute : Specifies the target attribute name.

-corners : Specify the list of the corner name to determine the order of the multi-corner attribute value.

-id : Specifies the target id.

Returns: <string> : Return list of the attribute data.

Licenses: None

```
di::loadParasiticDB -lcv di_test/di/parasitic
set resistances [di::getResistors -netName net193]
puts "dump res"
foreach resistance $resistances {
    # just get net node1 node2 position(llx lly urx ury) and value
    set resistance_attribute [di::getResAttr -id $resistance -attribute
"net node1 node2 llx lly urx ury value"]
    puts "net [lindex $resistance_attribute 0]"
    puts "node1 [lindex $resistance_attribute 1]"
    puts "node2 [lindex $resistance_attribute 2]"
    puts "position \{[lindex $resistance_attribute 3] , [lindex
$resistance_attribute 4], [lindex $resistance_attribute 5], [lindex
$resistance_attribute 6]\}"
    puts "value [lindex $resistance_attribute 7]"
}

puts "dump caps"

set capacitances [di::getCapacitors -netName net193]
foreach cap $capacitances {
    set cap_attribute [di::getCapAttr -id $cap -attribute "net1 node1
net2 node2 value"]
    puts "net1 [lindex $cap_attribute 0]"
    puts "node1 [lindex $cap_attribute 1]"
    puts "net2 [lindex $cap_attribute 2]"
    puts "node2 [lindex $cap_attribute 3]"
    puts "value [lindex $cap_attribute 4]"
}
```

```
}  
di::closeParasiticDB
```

di::getResistanceContributionAttr

Summary: Get the Effective Resistance Contribution Attribute data from opened effective resistance result.

Syntax: `di::getResistanceContributionAttr [-attribute <string>] [-corners <string>] -id <int>`

Arguments:

`-attribute` : Specifies the target attribute name.

`-corners` : Specify the list of the corner name to determine the order of the multi-corner attribute value.

`-id` : Specifies the target id.

Returns: <string> : Return list of the attribute data.

Licenses: None

di::getResistanceResult

Summary: Get the Effective Resistance Result for current loaded parasitic view.

Syntax: `di::getResistanceResult [-net <string>] -type <enum>`

Arguments:

`-net` : Specifies the target net name.

`-type` : Specifies the type of the report. ({ all | instpin_to_port | others | point_to_point | pull_down | pull_up })

Returns: <string> : Return the ids of the summary.

Licenses: None

```
di::loadResultsCellView -lcv Test/aaa/parasitic  
set from [di::getTerms -name pfd|inv1|M1:GATE]  
set to [di::getTerms -name pfd|inv1|M3:GATE]  
set job [di::runResistanceAnalysis -mode p2p -drivers $from -loads $to]  
xt::wait $job  
set summary_ids [di::getResistanceResult -type all]  
foreach summary_id $summary_ids {  
  set attrs [di::getResistanceResultAttr -id $summary_id -attribute "net r  
  lrp_load lrp_driver lrp_r type drivers loads contributions"]
```



```
puts "net [lindex $attrs 0]"
puts "r [lindex $attrs 1]"
puts "lrp_load [lindex $attrs 2]"
puts "lrp_driver [lindex $attrs 3]"
puts "lrp_r [lindex $attrs 4]"
puts "type [lindex $attrs 5]"
set drivers [lindex $attrs 6]
set loads [lindex $attrs 7]
set contributions [lindex $attrs 8]
puts "drivers info"
foreach driver $drivers {
    puts [di::getTermAttr -id $driver]
}
puts "loads info"
foreach load $loads {
    puts [di::getTermAttr -id $load]
}
puts "contribution infos"
foreach contribution $contributions {
    puts [di::getResistanceContributionAttr -id $contribution]
}
}
```

di::getResistanceResultAttr

Summary: Get the Effective Resistance Result for current loaded parasitic view.

Syntax: di::getResistanceResultAttr [-attribute <string>] [-corners <string>] -id <int>

Arguments:

-attribute : Specifies the target attribute name.

-corners : Specify the list of the corner name to determine the order of the multi-corner attribute value.

-id : Specifies the target id.

Returns: <string> : Return list of the attribute data.

Licenses: None

di::getResistors

Summary: Report the resistors' ids from the specified net id/name.

Category: Design Insights

Syntax: di::getResistors [-netID <int> | -netName <string>]

Arguments:

-netID : Specifies the net id of the resistors.

-netName : Specifies the net name of the resistors.

Returns: <string> : Return list of the resistors' ids.

Licenses: None

di::getTermAttr

Summary: Get term data by specified term id and attribute name.

Syntax: `di::getTermAttr [-attribute <string>] [-corners <string>] -id <int>`

Arguments:

-attribute : Specifies the target attribute name.

-corners : Specify the list of the corner name to determine the order of the multi-corner attribute value.

-id : Specifies the target id.

Returns: <string> : Return list of the attribute data.

Licenses: None

di::getTerms

Summary: Report the list of the terms' ids from the netlist.

Syntax: `di::getTerms [-instID <int> | -instName <string> | -name <string> | -netID <int> | -netName <string>]`

Arguments:

-instID : Specifies instance id of the terms.

-instName : Specifies instance name of the terms.

-name : Specifies the name of the terms.

-netID : Specifies net id of the terms.

-netName : Specifies net name of the terms.

Returns: <string> : Return list of the terms' ids.

Licenses: None

```
di::loadResultsCellView -lcv di_test/di/parasitic
set terms [di::getTerms -netName net193]
foreach term $terms {
    set term_attribute [di::getTermAttr -id $term -attribute "name nodes
net cap ron type"]
    puts "term name [lindex $term_attribute 0] information"
    puts "-----"
    puts "nodes [lindex $term_attribute 1]"
    puts "net [lindex $term_attribute 2]"
    puts "cap [lindex $term_attribute 3]"
    puts "ron [lindex $term_attribute 4]"
    puts "type [lindex $term_attribute 5]"
}
di::closeResultsCellView
```

di::getViewType

Summary: Get view type.

Syntax: di::getViewType

Returns: <string> : View type

Licenses: None

di::listAttrs

Summary: List all attributes for specified type of the di::get*Attr commands

Syntax: di::listAttrs -type <enum>

Arguments:

-type : List all attributes for specified type of the di::get*Attr commands. ({ cap | elmoreDelayResult | fanoutResult | inst | net | node | res | resistanceContributionResult | resistanceResult | term })

Returns: <string> : The list of the attribute name

Licenses: None

di::loadResultsCellView

Summary: Open a PA view.

Syntax: di::loadResultsCellView -lcv <dmCellView | dmCellViewName>

Arguments:

-lcv : Specify a PA cell view to be opened

Returns: <none>

Licenses: None

di::runElmoreDelayAnalysis

Summary: Get the Elmore delay result for currently loaded parasitic view.

Syntax: di::runElmoreDelayAnalysis [-corners <string>] [-drivers <string>]
[-includeDeviceTermCapacitance <bool>] [-loads <string>] -mode <enum>
[-nets <string>]

Arguments:

-corners : Specify the list of the corner name.

-drivers : Specifies the list of the driver's term id.

-includeDeviceTermCapacitance : Specify if the analysis have to include term cap or not.

-loads : Specifies the list of the load's term id.

-mode : Specifies mode of the command. ({ automatic | p2p })

-nets : Specifies list of the net id.

Returns: <xtJob> : Return the submitted job object

Licenses: None

di::runFanoutAnalysis

Summary: Get the fanout result for currently loaded parasitic view.

Syntax: di::runFanoutAnalysis [-corners <string>] [-drivers <string>]
[-loads <string>] -mode <enum> [-nets <string>]

Arguments:

-corners : Specify the list of the corner name.

-drivers : Specifies the list of the driver's term id.

-loads : Specifies the list of the load's term id.

`-mode` : Specifies mode of the command. ({ automatic | p2p }) `-nets` : Specifies list of the net id.

Returns: `<xtJob>` : Return the submitted job object

Licenses: None

di::runResistanceAnalysis

Summary: Get the effective resistance result for current loaded parasitic view.

Syntax: `di::runResistanceAnalysis [-corners <string>] [-drivers <string>] [-loads <string>] -mode <enum> [-nets <string>]`

Arguments:

`-corners` : Specify the list of the corner name.

`-drivers` : Specifies the list of the driver's term id.

`-loads` : Specifies the list of the load's term id.

`-mode` : Specifies mode of the command. ({ automatic | instP2p | p2p })

`-nets` : Specifies list of the net id.

Returns: `<xtJob>` : Return the submitted job object

Licenses: None

Preferences

The following table provides the preferences supported for Design Insights.

diCanvasClear	int	cellview	0	Clear	
diCanvasCrossSelectMode	bool	cellview	false	Cross select mode	true, false
diCanvasFitMode	bool	cellview	false	Fit mode	true, false
diCanvasShadowMode	bool	cellview	false	Shadow mode	true, false
diHighlightPacket	string	cellview		Highlight packet	

diPreprocessorOaOutputDir	string	cellview		OA output directory	
diPreprocessorRonMode	int	cellview	0	R_on Mode	
diPreprocessorSqlOutputDir	string	cellview		SQL output directory	
diPwdeHighlightToSchematic	bool	cellview	false	Highlight to schematic	true, false
diQueryNavigatorExpandAllWithConnectivity	bool	cellview	true	Expand all with connectivity	true, false
diShowNonPhysicalResistors	bool	cellview	false	Show non-physical resistors	true, false